SWE5001 · NUS-ISS MTECH SE

Architecting Scalable Systems 中英对照复习书

Graduate Certificate · Course Review Book

三门课 · 13 章知识 + 4 章样卷精讲 · 中文讲解，术语中英并列 · 从零学懂，轻松应考

本书覆盖的三门课

ArchSS
Architecting Software Solutions
解决方案架构：需求、逻辑架构、物理架构，以及性能、容量、高可用、可维护、安全等质量属性。
→ **第1、2、12、13章**

PE
Platform Engineering
平台思维、领域驱动设计、可复用服务、API 设计与网关、平台管理与可观测性、数据管理。
→ **第3–6章、第11章**

CNSD
Cloud Native Solution Design
云计算与迁移、云原生架构、架构风格、微服务、Serverless、Kubernetes、云存储。
→ **第7–11章**

书中的标记

（标签上直接写着含义，每页页脚也有图例；正文里 **黄色划线** = 可直接写进答案的重点句）

- ◆ **要点 | 必背结论** — 本节必须掌握的结论
- ■ **定义 | 课件正式说法** — 课件里的正式定义
- ▶ **案例演练 | 演示怎么套用** — 用案例演示知识怎么用
- ✕ **常见误区 | 这样答会丢分** — 容易答错、答偏的地方
- ✎ **考试怎么考 | 题型·分值·套路** — 往年题怎么考这个知识点
- ✎ **小测验 | 先自己答，再看参考答案** — 每章末尾的小测验，答案在题目下方

STUDY PLAN

复习路线：先学哪几章

What to Study First

依据：第14章题型分布表（三套样卷每类题的分值）

目录与侧栏里 ★ 必学 ● 重点

一句话 **开卷考试不用背：先做样卷精讲，碰到不懂的知识点再跳回对应章节查。**

为什么要排优先级

- 按第1章那样的节奏逐章读完全书要 12–16 小时，考前时间不够
- 样卷每套都考的四类题（T1/T2、T3、T4、T5/T6）合计约 31 分，已超过及格线 20 分
- 样卷精讲会反过来告诉你每个知识点怎么考，知识章节只需要读被考到的部分

P.1 四档优先级 Four priority tiers

档	章节	对应题型	样卷分值（PAST / PAST2 / PAST3）
★ 必学	老师划重点（开读前 10 分钟）	全部题型	老师怎么给分、读题注意什么、考场须知；先读这一章再做小测验
	第14章 → 15 → 16 → 17	全部 T1–T12	考场策略 + 三套样卷逐题精讲；第15章 PAST 有官方评分标准，最先做
● 重点	第4章 领域驱动设计	T1/T2 领域模型	9 / 9 / 9
	第2章 逻辑到物理架构	T4 打包/部署找错	6 / 6 / 6
	第5章 API 设计	T3 API 设计	6 / 6 / Q1(c)(ii)
	第8章 + 第10章 云原生架构、K8s	T5 云原生参考架构	10 / 7 / —（单题分值最高）
	第11章 数据与云存储	T6 物理部署/云服务	— / 4+3 / 9
	第6章 平台管理与可观测性	T8 指标设计	4 / 6 / —
中等	第9章 微服务与通信	T12 服务共置、T10	T12 每次 2 分
	第12章 性能与容量	T11 性能/扩展/测试	2+6 / — / —（只有 PAST 考过）
	第1章 质量属性	T7 质量属性	2 / 2 / —
快速过	第3章 平台思维	T9	— / 2 / 2，只抓句式
	第7章 云计算与迁移	T10 的背景	只看结论和对照表
	第13章 高可用与安全	样卷未考（T13–T17）	考到时翻《08 补充卡》

分值来自第14章题型分布表。第4章（每套 9 分）比第2章（每套 6 分）分值更高，所以排在重点第一位。

P.2 怎么用这几章 How to work through them

◆ 要点 | 必背结论

不要按章节顺序读。按第15→16→17章一道题一道题做：

1. **先自己想 2 分钟。**读题卡，判断题型，在草稿上列出要点。
2. **再看精讲。**对照"思考逻辑"和"得分点"，看自己漏了哪一分。
3. **不懂就跳回去查。**题卡上写了对应章节，只读被考到的那一节，读完回到样卷继续。

这样知识章节只读考到的部分，能省掉一半以上的时间。

◆ 要点 | 必背结论

留 3.5 小时做一次限时模拟。正式考试是 18:30–22:00。拿 PAST2 或 PAST3，只用打印好的工具包作答，主要练翻资料的速度：**开卷考试最容易丢分的就是找不到地方。**

打印、装订、给 T1–T17 贴标签另外留约 1 小时。

CONTENTS

目录

Table of Contents

★ 必学 ● 重点 分档理由见前页复习路线（第 2 页）

导读 课程全景与本书用法 7

0.1 这门课在讲什么 7
0.2 贯穿全书的例子：Vote Anywhere 7
0.3 考试题型与本书章节 8
0.4 怎么读这本书 8

老师划重点 评分规则与考场须知 ★ 必学 10

K.1 评分与作答 10
K.2 读题 10
K.3 考场与行政 11
K.4 ArchSS 讲师答疑（机器转写） 11

PART A · 架构基础（ArchSS）

第1章 解决方案架构与质量属性 12

1.1 什么是架构 12
1.2 企业架构、解决方案架构与详细设计 12
1.3 质量属性 13
1.4 让质量属性可度量：场景与效用树 13
1.5 架构资产：风格、模式与参考架构 14
1.6 记录架构：4+1 视图 16
1.7 白话讲解与小测验 16

第2章 架构方法：从逻辑到物理 ● 重点 21

2.1 方法总览 21
2.2 定义需求 21
2.3 逻辑架构：十个任务与两类元素 22
2.4 功能元素：BCE 组件与分层打包 22
2.5 部署元素与打包部署模型 23
2.6 部署模型找错清单 23
2.7 物理架构 24
2.8 白话讲解与小测验 24

PART B · 平台工程（PE）

第3章 平台思维 29

3.1 什么是平台 29
3.2 平台、框架、库与应用 29
3.3 平台思维 30
3.4 平台的工程视角 30
3.5 平台工程与内部开发者平台 31
3.6 小测验 32

第4章 领域驱动设计与可复用资产 ● 重点 34

4.1 可复用资产：服务、框架、共享库 34
4.2 领域驱动设计：战略设计 34
4.3 战术设计：实体、值对象、聚合、领域事件 35
4.4 上下文映射：BC 之间怎么集成 36
4.5 读懂考试中的领域模型图 37
4.6 诊断领域模型问题 37
4.7 画类图与识别共享服务 38
4.8 白话讲解与小测验 39

第5章 API 设计与 API Gateway ● 重点 44

5.1 API 产品思维与设计原则 44
5.2 API 风格 44
5.3 REST 的六个约束 44
5.4 设计最佳实践：资源、动词、状态码 45
5.5 Richardson 成熟度模型 45
5.6 版本管理 46
5.7 API 性能：延迟从哪来、怎么改 47
5.8 API 安全 47
5.9 API 网关与 SDK 47
5.10 白话讲解与小测验 48

第6章 平台管理与可观测性 ● 重点 53

6.1 平台管理工具箱 53
6.2 服务发现 53
6.3 部署策略 54
6.4 熔断、限流与重试 54
6.5 分布式追踪与 Saga 54
6.6 指标 55
6.7 设计一个指标 55
6.8 日志 56
6.9 白话讲解与小测验 57

PART C · 云原生（CNSD）

第7章 云计算基础与云迁移 61

7.1 什么是云计算 61
7.2 服务模型 62
7.3 部署模型 62
7.4 云原生应用与"直接搬家" 62
7.5 云迁移策略 63
7.6 基础设施即代码 63
7.7 白话讲解与小测验 64

第8章 云原生架构与架构风格 ● 重点 67

8.1 什么是云原生 67
8.2 十二要素应用 68
8.3 无状态设计与状态管理 68
8.4 应用架构风格 68
8.5 Serverless 架构 69
8.6 混合云原生参考架构 70
8.7 前端：渐进式 Web 应用还是原生应用 71
8.8 生成式 AI 与云原生应用 71
8.9 白话讲解与小测验 72

第9章 微服务设计与服务间通信 75

9.1 微服务是什么 75
9.2 拆分策略与扩展立方体 75
9.3 服务间通信 76
9.4 消息队列与事件流 76
9.5 API 网关、服务网格与服务发现 77
9.6 编排与协同 77
9.7 微服务的数据 77
9.8 服务共置：哪些服务放进同一个容器 78
9.9 白话讲解与小测验 78

第10章 Kubernetes 与托管服务 ● 重点 83

10.1 容器与虚拟机 83
10.2 Kubernetes 架构 83
10.3 常用 Kubernetes 对象 84
10.4 写一份 K8s 部署计划 85
10.5 托管服务与自建容器 85
10.6 白话讲解与小测验 86

第11章 数据管理与云存储 ● 重点 90

11.1 平台对数据存储的要求 90
11.2 关系型数据库与 NoSQL 90
11.3 多语言持久化与 CQRS 91
11.4 扩展数据库：复制、分片、缓存 91
11.5 CAP 定理与一致性 92
11.6 云上的存储选项 92
11.7 按质量属性选择存储 93
11.8 T6 答法与小测验 94

PART D · 质量专题（ArchSS）

第12章 性能设计与容量规划 97

12.1 性能指标 97
12.2 排队论的直觉 97
12.3 性能设计手段 98
12.4 验证与记录性能设计 98
12.5 性能测试与工作负载模型 99
12.6 容量规划 99
12.7 小测验 100

第13章 高可用、可维护与安全 103

13.1 可用性的基本概念 103
13.2 停机从哪里来 104
13.3 可用性战术与稳定性模式 104
13.4 冗余与备份 105
13.5 可维护性 105
13.6 安全架构（概要） 106
13.7 白话讲解与小测验 107

PART E · 样卷精讲（PAST / PAST2 / PAST3）

第14章 样卷总览与考场策略 ★ 必学 110

14.1 三套样卷长什么样 110
14.2 老师讲过的评分规则 111
14.3 考场时间怎么分 111
14.4 30 分钟阅读时间怎么用 112
14.5 万能答题框架 112
14.6 拿到新案例：去附录哪一节找材料 113
14.7 通用句式库 113

第15章 PAST 精讲：Vote Anywhere 投票平台115 ★ 必学

15.1 案例速读 115
15.2 Q1(a)(i) 性能测试结果不稳定 116
15.3 Q1(a)(ii) 消息队列还是事件流 117
15.4 Q1(b)(i) NotificationManager 难以复用 118
15.5 Q1(b)(ii) Observer 看结果不及时 119
15.6 Q1(c) API 表：RMM、版本、重构 120
15.7 Q1(d) 易用性是不是最重要的非功能需求 121
15.8 Q1(e) "每日处理人数"适合当指标吗 121
15.9 Q2(a) 单体 → 混合云原生参考架构 + K8s 122
15.10 Q2(b) 原生应用还是 PWA 124
15.11 Q3(c) 打包部署模型找错 124
15.12 Q3(d) 数据库写吞吐瓶颈 125
15.13 本卷复盘 126

第16章 PAST2 精讲：MLTTP 公交遥测平台127 ★ 必学

16.1 案例速读 127
16.2 Q1(a) 平台成功的两个关键架构方面 128
16.3 Q1(b)(i) 到站时间计算不够快 128
16.4 Q1(b)(ii) BusNetworkManager 内存占用高 129
16.5 Q1(c) 大批量并发更新时 API 延迟 20–30 秒 130
16.6 Q1(d) 存储最重要的两个质量属性 131
16.7 Q1(e) 两个云数据库服务 132
16.8 Q1(f) 哪些服务放进同一个容器 132
16.9 Q2(a) 本地部署 → 云原生架构 133
16.10 Q2(b) 虚拟机还是容器 134
16.11 Q2(c) 用托管 ML 服务预测到站时间 135
16.12 Q3(a) 第二阶段逻辑架构：找一个错 + 补全打包 135
16.13 Q3(b) 预测准确度指标 136
16.14 本卷复盘 137

第17章 PAST3 精讲：ONE Health 医疗健身平台138

★ 必学

17.1 案例速读 138
17.2 Q1(a) 平台成功的一个关键架构方面 139
17.3 Q1(b)(i) 用 DDD 从用例画 UML 类图 139
17.4 Q1(b)(ii) 推荐的共享服务 140
17.5 Q1(c)(i) 开放哪两个服务的 API，给谁用 140
17.6 Q1(c)(ii) 设计一个 RMM Level 3 API 141
17.7 Q1(d) 通信亲和度高的服务共置 142
17.8 Q2(a) 逻辑详细部署图：找三个错 142
17.9 Q2(b) AWS 物理部署图 + 三个架构决策 143
17.10 本卷复盘 145

附录 云服务对照 · 术语索引 146

A.1 云服务三厂商对照表 146
B.1 中英术语索引 147

INTRODUCTION

导读：课程全景与本书用法

Course Map & How to Use This Book

课件来源：ArchSS / PE / CNSD 三门课的 Course Opening　　建议用时 **15 分钟**

一句话 **这门课教的是：怎样设计一个能撑住大量用户、能持续扩展的软件平台，并把它部署到云上。**

读完导读你能

- 说出三门课各讲什么、彼此怎么衔接
- 认识全书反复使用的例子 Vote Anywhere
- 知道考试的12种题型分别对应本书哪一章
- 按优先级安排自己的阅读顺序

0.1 这门课在讲什么 What the course is about

SWE5001 Architecting Scalable Systems 是 NUS-ISS 软件工程硕士的一个研究生证书，由三门课加一个实践项目组成。三门课看起来分散，其实讲的是同一件事的不同阶段：**先弄清要什么（需求与质量属性），再决定怎么拆（领域模型与服务），然后决定怎么连（API 与通信），最后决定放在哪、怎么跑（云原生部署与运维）。**

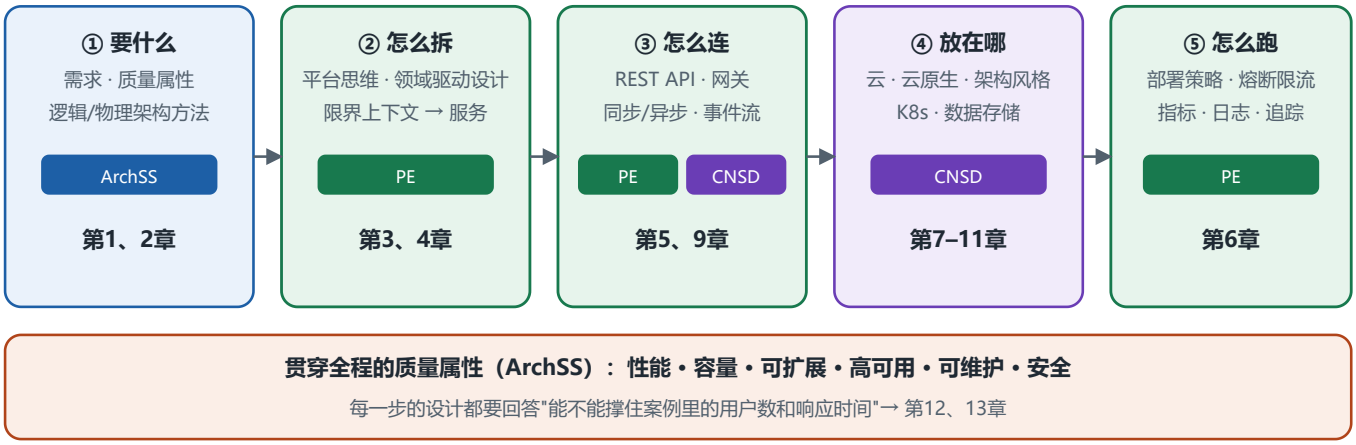


图 0-1 三门课的分工：从需求到上线运行的五个阶段

0.2 贯穿全书的例子：Vote Anywhere The running example

考试的每套卷子都附带一个虚构案例（Appendix A），所有题目都围绕它出。今年样卷用的是电子投票平台 **Vote Anywhere**，你手上有它的官方答案，所以本书讲解时也用它举例。先认识它：

要素	内容（考试时要从案例里照抄的名字）
背景	初创公司 You Vote 做线上投票平台，先在新加坡试点。第一阶段做最小可行产品（MVP），包括近实时更新投票结果；第二阶段加结果趋势预测。你是首席解决方案架构师（Lead Solution Architect）。
角色 Actors	Public User、Platform User、Election Admin（管理投票）、Proctor（视频核验选民身份）、Observer（看详细结果）、Voter（用手机投票）、Platform Admin；外部系统 SingPass Authenticator、Email Gateway
关键用例	Verify Identity → Conduct Vote → Update Current Simplified/Detailed Results；Proctor 侧：Invite Next Voter、Start Vote、Reject Voter；另有 Send Notification

要素	内容（考试时要从案例里照抄的名字）
关键数字（§4）	10万并发选民；交互请求 90% 在 3 秒内响应；简要结果平均 1 分钟内更新，详细结果平均 3 分钟内更新
服务（§7）	UserManager、PollManager、ProctoringManager、VotingManager、ObservationManager、NotificationManager
架构要求（§6）	分层：Presentation / Control / Data Access / External Integration / Shared Library；云原生，服务用微服务实现

0.3 考试题型与本书章节 Question types → chapters

考试共 50 分，开卷，答案必须结合案例。三套往年卷子反复出现 12 种题型（编号 T1–T12，和你的题型卡一致）。下表告诉你：遇到哪种题型，回本书哪一章复习。

题型	考什么	分值	本书章节	案例节
T1	**领域模型诊断：原因、对策、缺点**	9	第4章 4.6	§7
T2	画类图、找共享服务	9	第4章 4.7	§5、§3
T3	**API 设计：RMM、HTTP 动词、版本、延迟**	6	第5章	§7
T4	**逻辑打包/部署模型找错**	6	第2章 2.5–2.6	§6、§8
T5	**云原生参考架构图 + K8s 部署计划**	7–10	第8章 8.6、第10章	§4、§6
T6	物理部署图、选云服务	3–9	第2章 2.7、第11章、附录A	§4、§6
T7	最重要的质量属性	2	第1章 1.3	§4
T8	设计指标：谁采集、多久采、存哪	4–6	第6章 6.7	§1、§4
T9	平台思维：seed、interactions	2	第3章	开头、§1
T10	二选一：队列/事件流、VM/容器、PWA/原生……	2–3	第8、9、10章	§4、§6
T11	性能测试、数据库扩展、分片	2–6	第11章 11.5、第12章	§4
T12	哪些服务放同一个容器	2	第9章 9.8	§7

加粗的 T1、T3、T4、T5 每套必考，合计约 31 分，已超过及格线 20 分。

0.4 怎么读这本书 Reading plan

每章结构相同：**本章导图 → 核心概念 → 对比与选择 → 案例演练 → 常见误区 → 考试怎么考 → 白话讲解与小测验 → 术语表**（小测验的参考答案在题目下方，网页版默认收起）。术语第一次出现写成「中文 English」，章末术语表把本章词汇集中列出，书末附 A–Z 索引。

六种框的标签上直接写着用途（如"× 常见误区 | 这样答会丢分"），每页页脚还有一行图例，读到哪里都不用翻回封面。正文中 **黄色划线的句子是重点句**：可以直接写进答案的结论、判据或官方评分点，每章约 6–8 处，考前快速复习时只看划线即可。

第14–17章是样卷精讲：第14章讲三套样卷的结构、老师的评分规则、时间分配、附录 A 怎么找材料和通用句式；第15–17章逐题讲 PAST、PAST2、PAST3，每题包括题卡（题干要点、分值、原题页码）、思考逻辑、得分点、可照写的英文参考答案（"✍ 参考答案"框）、可用句式（"❝ 可用句式"框）和换案例时怎么迁移。建议先读第1–13章对应知识，再用第15–17章对答案。导读之后的「老师划重点」一章汇总了老师课上说过的评分、读题和考场要点，建议最先读。

优先级	章节	理由	用时
★★★ **先读**	第4章 DDD、第2章 架构方法、第5章 API、第8章 云原生、第10章 K8s	覆盖每套必考的四种题型（约31分）	约 3.5 小时
★★ **再读**	第9章 微服务、第11章 数据、第12章 性能与容量、第6章 平台管理	中等分值题型 T6、T8、T10、T11、T12	约 2.5 小时

优先级	章节	理由	用时
★ 最后	第1章、第3章、第7章、第13章	小题（T7、T9）和背景知识	约 1.5 小时

◆ 要点 | 必背结论 **学习方法**

这是开卷考试，不需要背。读每一章时问自己三个问题：①这个概念用来解决什么问题？②什么情况下用 A、什么情况下用 B？③如果案例换了，我能把它套到新案例上吗？做到这三点，考场上就能快速找到答案并组织成文。

LECTURER'S TIPS

老师划重点：评分规则与考场须知

What the Lecturers Said in Class

来源：Clinic Session 录音校订稿（讲师 Boon Kui HENG，约 61 分钟）· ArchSS 答疑录音　　建议用时 **10 分钟** · 贯穿 **T1–T17**

一句话 **先知道老师怎么给分，再学知识点：同样懂一个概念，答法对了才拿得到分。**

读完这一章你能

- 说出老师评分时看什么：贴合案例、每一部分都答、用完整句子论证
- 知道读题时哪些信息可以忽略、哪些不能当成缺陷
- 提前准备好考场要带的东西和时间安排

据 Clinic Session 录音（讲师 Boon Kui HENG，约 61 分钟）校订稿整理，**中文转述，非逐字原话**；时间为录像定位点，方便回看。K.4 来自 ArchSS 答疑录音的机器转写，可靠性较低。行政安排以考试信为准。

K.1 评分与作答 How Answers Are Marked

要点	时间
开卷不等于照抄：直接抄课件、参考资料，答案不贴合案例，很可能不得分	06:21
题目要求的每一部分都要答；常见错误是找对了原因，却忘了给对策	18:43
要几个就答几个：只问一个原因，不用写十个；问两个缺点，系统里可能有四五个，答对两个即可	17:35 · 48:25
题目给出了现象，就按现象分析，不要答"看不出问题"	20:41
领域模型题要围绕限界上下文、实体归属和服务间通信作答；"加缓存""加负载均衡"属于跑题	21:55
改图只画改动的部分，不用重画整张图；但要补的连线和元素一定要画出来	24:13 · 58:03
题目说刻意省略的部分（如服务器端），指出它缺失不得分	34:23
质量属性、指标等开放题不必与参考答案相同，论证合理就给分；参考答案没列的问题，言之有理也给分	27:54 · 36:52
评分颗粒度：识别 1 + 纠正 1 + 论证 1；部分对、部分错，可能得半分	44:06
不要只写几个词，用完整句子把推理讲清楚	40:00
画图题按要求的元素评分，比较严；但能看出你懂，一般会给分	58:03
推荐云服务时写具体服务名，熟悉 AWS、Azure、Google 任意一家即可，"不熟悉"不是借口	55:22
安全一般不是考点，不建议往这个方向答；但基于安全的论证合理，也会给分	46:27
评分合理，不按钟形曲线；题目偏难或判得过严时，会整体复核调分	40:46 · 59:24

K.2 读题 Reading the Paper

要点	时间
案例里的信息比题目需要的多；Phase 2、安全等内容出现在附录里，不代表一定会出题	07:59
默认按云原生、微服务方向作答，尽量使用云托管服务	11:32
DDD 图要看清层级：附录一般给限界上下文和实体，更细的模型（如聚合）可能出现在题目里	11:59
部署图一般画成逻辑图，不要求懂某种具体技术，但要读清楚图表示的是什么	12:51
一个 section 不一定对应一门课；各 section 分值不固定，按分值分配时间	14:57 · 15:32

要点	时间
PAST 第 3 题原卷编号有误，两小题应为 (a)、(b)	33:30

K.3 考场与行政 Exam Day

要点	时间
开卷，纸质资料不限量；不能用电脑、手机、智能手表等电子设备，需要词典就带纸质版	04:25 · 27:18 · 42:42
30 分钟阅读时间只能读题和提问，不能写字（连答题计划都不行）；之后 3 小时作答	05:17
文字题用笔，不能用红笔；画图可以用铅笔	17:03
不要求计算，不用带计算器	43:44
可以按任意顺序作答，直接写在试卷上；不够写可申请附加答题纸，交卷时订在对应 section 后面	49:31
合理分配时间：在一道题上花太多时间回报会递减，常有人来不及答第 3 题；尽量不留空	38:41
低于 20 分笔试不及格；下学期可补考一次，成绩上限 C+；实践项目和笔试都要通过	47:23
没有额外的考点清单：熟悉课件、实践项目和 workshop 即可，开卷不需要死记	52:48
样卷难度有代表性，每学期尽量保持相近；同样的问法换了案例，答案很可能就变了	51:39 · 54:50

K.4 ArchSS 讲师答疑（机器转写） ArchSS Q&A

要点	出处
调用第三方 API（如外部 LLM）的延迟是不可控约束，要按约束设计；讲师当场提示这类题会考，正式考试确实考到	Day 4 答疑
排队论的数学推导不考	Day 4 答疑
可用性题：不可用大多来自软件更新而不是硬件；串联相乘后往往不达标，要加并联冗余	Day 4 答疑
看 P95 / P99，不看平均值；排队要看峰值	Day 4 答疑
限流和熔断是韧性手段，不是性能优化：把过载挡在外面，代价是部分请求被拒	Day 5 讨论

CHAPTER 01

解决方案架构与质量属性

Solution Architecture & Quality Attributes

课件来源：ArchSS 01 Introduction to Solution Architecture · ArchSS 02 Architect the Solution Architecture (Asset)　　对应题型 T7

一句话 **架构 = 系统由哪些部件组成、部件之间怎样关联、遵循什么原则；判断架构好坏的标准是它能否满足质量属性。**

学完本章你能

- 区分企业架构、解决方案架构、详细设计
- 说出 ISO 25010 的八大质量特性
- 用"质量属性场景"把非功能需求写得可度量
- 认识常见架构风格、参考架构与 4+1 视图

1.1 什么是架构 Architecture

■ 定义｜课件正式说法 **架构（IEEE 1471 / ISO 42010）**

"The fundamental organization of a system embodied in its components, their relationships to each other, and to the environment, and the principles guiding its design and evolution."

> 中文：系统的基本组织方式，体现在它的组件、组件之间以及组件与环境之间的关系，以及指导其设计和演进的原则。

拆开看有四个要素：**部件** components、部件之间的**关系** relationships、与**环境** environment 的关系、指导设计和演进的**原则** principles。

架构不是凭空画出来的。课件引用 Bass 的《Software Architecture in Practice》：架构受四方面影响，即**干系人** stakeholders、开发组织 organization、技术环境 technical environment 和架构师自己的经验 architect's experience；架构建成的系统反过来又会改变这些因素。

为什么要做架构评审 architecture review？调查显示，最主要的收益是**尽早识别架构风险**（88%），其次是**评估质量属性**（如可扩展性、性能）。架构上的错误越晚发现，改起来越贵。

1.2 企业架构、解决方案架构与详细设计 EA vs. SA vs. Design

三者是"上层约束下层"的关系：**企业架构约束解决方案架构，解决方案架构约束软件设计**。

层次	关注范围	主要产出
企业架构 Enterprise Architecture	整个企业：业务流程和 IT 基础设施的组织逻辑，反映企业运营模式的整合与标准化要求	全景图、技术参考模型、架构原则 EA principles
解决方案架构 Solution Architecture	一个项目或系统：各部件在**逻辑和物理层面**如何协同工作；受业务需求、项目约束和企业架构影响	系统上下文、逻辑架构、物理架构、架构决策
详细设计 Detailed Design	每个部件**内部**如何工作（含高层设计 HLD）	类设计、接口细节、数据库表

架构和设计共享一组基本原则：**抽象** abstraction（只关注重要细节）、**封装** encapsulation（隐藏内部细节）、**高内聚** high cohesion（相关的放一起）、**低耦合** low coupling（部件之间少依赖）。这四个词在答题时非常常用，例如"把实体移回所属服务以提高内聚"。

解决方案架构师 Solution Architect（ANSI/IEEE 1471 定义为"负责设计系统架构的个人、团队或组织"）的职责：对方案的可行性负责（能实现、能运行），提供技术领导，协助项目经理交付，与干系人沟通取舍。好的架构师让干系人和团队对架构有信心，能正确定义并记录一个最优方案。

1.3 质量属性 Quality Attributes

质量属性 quality attribute（又叫非功能需求 non-functional requirement, NFR）衡量软件设计得好不好、实现是否符合设计。功能需求说"系统做什么"，质量属性说"做得多好"。课程以 **ISO/IEC 25010 产品质量模型**为基准：

质量特性	含义	子特性
功能适合性 Functional Suitability	功能满足明确和隐含需求的程度	完整性、正确性、适当性
性能效率 Performance Efficiency	相对资源消耗的性能	时间特性 time behaviour、资源利用 resource utilization、容量 capacity
兼容性 Compatibility	与其他系统交换信息、共享环境	共存 co-existence、互操作 interoperability
易用性 Usability	用户有效、高效、满意地达成目标	易识别、易学、易操作、防止用户出错、界面美观、无障碍 accessibility
可靠性 Reliability	在规定条件和时间内完成规定功能	成熟度、**可用性** availability、容错 fault tolerance、可恢复 recoverability
安全性 Security	按授权级别保护信息和数据	保密性、完整性、不可否认 non-repudiation、可追责、真实性
可维护性 Maintainability	修改系统的有效性和效率	模块化、**可复用** reusability、可分析、可修改、可测试
可移植性 Portability	从一个环境迁移到另一个环境	适应性、易安装、可替换

平台和云原生的课程还经常使用三个 25010 没有单列的词：**可扩展性** scalability（负载增长时通过加资源保持性能）、**弹性** elasticity（按负载自动增减资源）、**可演进性** evolvability（能持续发布新功能和修复）。此外还有**业务质量** business qualities：上市时间 time to market、成本收益、预期寿命、与遗留系统集成等。

质量属性之间会相互影响

质量属性之间有**正相关** direct relationship（一个提高另一个也提高）和**负相关** inverse relationship（此消彼长，即权衡 trade-off）。架构决策本质上就是在做取舍：

一对属性	关系	例子
安全性 ↔ 易用性	负相关	多重验证更安全，但用户操作更繁琐
性能 ↔ 可维护性	负相关	把逻辑写进数据库存储过程更快，但更难修改和移植
一致性 ↔ 可用性	负相关	网络分区时只能二选一（CAP 定理，第11章）
可修改性 ↔ 性能	负相关	多加一层抽象便于修改，但多一次调用
可维护性 ↔ 可测试性	正相关	模块化后既好改又好测

1.4 让质量属性可度量：场景与效用树 QA Scenario & Utility Tree

"系统要快""系统要稳定"这种说法没法验证。课程用**质量属性场景** quality attribute scenario 把需求写成六个部分：

组成	含义	例：Vote Anywhere 的性能场景
刺激源 Source	谁触发	选民（10万并发）

组成	含义	例：Vote Anywhere 的性能场景
刺激 Stimulus	发生了什么	发起 Verify Identity 请求
制品 Artifact	作用在哪个部分	身份核验服务
环境 Environment	在什么条件下	投票高峰期、正常运行
响应 Response	系统应该怎么做	处理请求并返回结果
响应度量 Response Measure	怎么量化	90% 的请求在 3 秒内完成

效用树 utility tree 把所有场景组织成一棵树：根是系统的总体效用，第二层是质量属性，第三层是细化，叶子是具体场景。每个叶子标上"重要性/实现难度"（高 H、中 M、低 L），**架构评估时优先看"高/高"的场景**。



图 1-1 Vote Anywhere 的效用树（括号内为 重要性, 实现难度）

✎ 考试怎么考 | 题型·分值·套路

T7 质量属性题（约2分）。PAST Q1(d)：组员说"易用性是最重要的非功能需求"，你是否同意？官方答案：**不同意**，**最重要的是性能/可扩展性**（要支持 10万并发选民）和**安全性**（保证每位选民最多投一票）。每个有说服力的理由 1 分。

答题套路：**选定属性 → 引用案例 §4 的数字或关键用例 → 说明它不满足会怎样 → 说明为什么比其他属性更关键**。

1.5 架构资产：风格、模式与参考架构 Architectural Assets

架构师很少从零开始。课程把可复用的**架构资产** architectural assets 分三类：**参考模式** reference patterns（架构风格、分析模式、设计模式）、**参考架构** reference architectures（某领域的成熟模板）、**架构描述** architectural descriptions（记录架构的标准方法）。系统和以前做过的越相似，越多地复用资产；越创新，越依赖方法（第2章）。

架构风格

类别	风格	要点	日常例子（抢票 / 外卖）
结构型 Structural	**客户端-服务器** Client-Server	服务器通过定义良好的接口提供服务，客户端调用	手机 App 查余票、下单；后台通过接口提供服务
	分层/多层 N-Tier	表现层、业务层、数据层分开部署；每层只调用下一层	界面 → 业务层（校验余票、扣库存、调支付）→ 数据库；界面不能直连数据库
	点对点 Peer-to-Peer	每个节点既是客户端又是服务器，无中心服务器（如 BitTorrent）	BitTorrent 下载大文件，边下边传

类别	风格	要点	日常例子（抢票 / 外卖）
通信型 Communication	**数据流** Data Flow	数据依次流过各组件，适合批量集成（文件 FTP、数据库联邦 federation、数据库复制 replication）	每天凌晨导出订单文件，FTP 传给财务系统对账
	调用-返回 Call and Return	调用方交出控制权，等被调方返回（RPC、SOAP/REST Web 服务）	点"支付"，订单服务调用支付服务，等结果返回
	事件•点对点 Point-to-Point	生产者发消息，**一条消息只被一个消费者消费**，消费后不再可用（消息队列）	抢票高峰下单请求进队列，后台处理程序各取一单
	事件•发布订阅 Publish/Subscribe	订阅者登记感兴趣的事件，发布者发出后**任意多个订阅者都能收到**	支付成功事件 → 短信、推送、积分、邮件各收一份

网络与服务器组件

部署图里常见的组件：**非军事区** DMZ（隔开内网和外网的网段，放对外服务）、**反向代理** reverse proxy（位于互联网用户和内部服务器之间，转发请求、隐藏内部结构）、**防火墙** firewall、**负载均衡器** load balancer；服务器侧有应用服务器（执行服务）、门户服务器（B2C 集成）、**API 网关** API Gateway / **企业服务总线** ESB（服务和 B2B 集成）、数据库服务器。

组件	服务谁	主要做什么	一句话类比
反向代理 Reverse Proxy	外部用户的请求	转发、隐藏内部服务器、负载均衡、按路径分发	服务器的"前台"
防火墙 Firewall	网络流量	按 IP、端口、协议规则放行或拦截，划定区域边界	区域之间的"门卫"
门户服务器 Portal	终端用户（B2C）	把多个系统的界面整合到一个入口页面（常带单点登录）	统一的"网页入口"
API 网关 API Gateway	调用 API 的程序（B2B）	路由、认证、限流、版本、监控	API 的"总入口"
企业服务总线 ESB	企业内部异构系统	协议 / 格式转换、路由、编排	内部系统的"翻译 + 调度"

防火墙只按规则管"谁能访问什么"；识别 SQL 注入这类内容攻击要靠 WAF；"超大文件上传"要在网关和服务端校验，不是防火墙的职责。

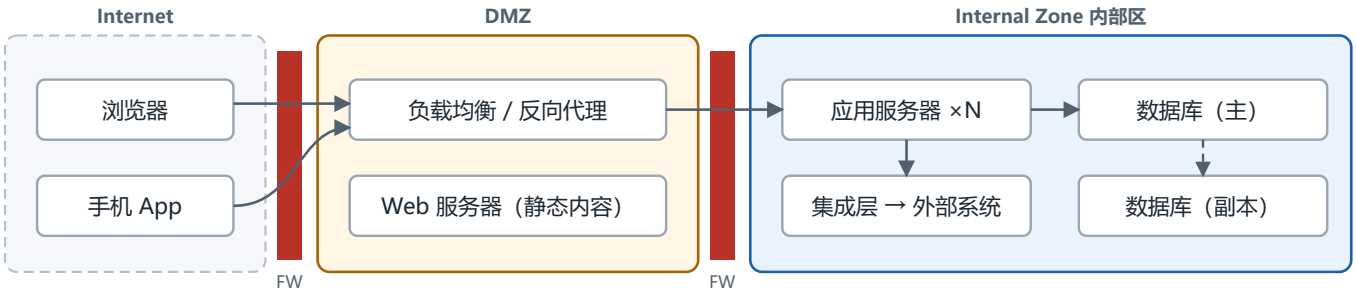


图 1-2 典型 Web 参考架构的三个区：数据库永远放在内部区，DMZ 只放对外的代理和 Web 层

课程介绍过的参考架构

Web 架构、移动架构（含 AWS/IBM 移动参考架构）、以客户端为中心的架构（应用在浏览器里运行，如单页应用）、代理/ESB 架构、SOA（Open Group 分层：消费者层、业务流程层、服务层、组件层、运营系统层）、**微服务架构**（扩展立方体、服务发现、API 网关，第9章）、云架构（服务模型、部署模型、虚拟机与容器、多租户与弹性，第7章）、**Serverless**（第8章），以及**响应式宣言** Reactive Manifesto：系统应当**及时响应** responsive、**有韧性** resilient、**有弹性** elastic、**消息驱动** message driven。

1.6 记录架构：4+1 视图 Architectural Description

IEEE 42010 规定了描述架构的元模型：**干系人**有**关注点** concerns，**视点** viewpoint 规定用什么方法表达某类关注点，按视点画出来的就是**视图** view。最常用的是 Kruchten 的 **4+1 视图模型**：

视图	给谁看 / 关注什么	常用 UML 图
逻辑视图 Logical View	最终用户：系统提供哪些功能	类图、对象图
进程视图 Process View	集成人员：并发、性能、可扩展性	时序图、活动图
开发视图 Development View	程序员：代码模块如何组织	组件图、包图
物理视图 Physical View	系统工程师：部署拓扑、节点	部署图
场景 Scenarios (+1)	所有人：用用例把四个视图串起来验证	用例图

课程还列了其他视点：组织、业务功能、业务流程、应用结构、应用行为、基础设施、部署、信息结构视点。考试中的 §7 领域模型属于逻辑视图，§8 打包部署模型属于部署视点。

✕常见误区｜这样答会丢分

- 写质量属性只说"要快、要稳"，不给数字。**答案里一定要引用案例 §4 的数字**，比如"10万并发""90% 请求 3 秒内"。
- 题目问"最重要的两个属性"，却把五六个都列成重要。**要选两个，并说明它们为什么比其他属性更关键。**
- 把可用性和可靠性混为一谈：可用性是"需要时能不能用"，可靠性是"一段时间内能不能正确工作"（第13章）。

1.7 白话讲解与小测验 Plain-language Notes & Drills

◆要点｜必背结论 **效用树为什么先看"高 / 高"**

每个场景有两个维度：**重要性**（干系人定）和**难度**（架构师判断）。高 / 高 = 业务上很重要、又很难做 = **架构风险最大**，最能左右消息队列、分库、缓存这些架构选择，所以评估时先看。评级本身是主观判断，书里图 1-1 是示例；考试重点是用案例数字把理由讲清楚。

✎ 小测验｜先自己答，再看参考答案 **小测验 1：判断下面 5 个场景各用了哪种架构风格**

候选风格：客户端-服务器、分层（N-Tier）、点对点（Peer-to-Peer）、调用-返回、数据流（批处理）。每个场景选一种，并写出你依据的信号。

1. 手机 App 通过接口**向后台查询余票**，后台集中提供查询服务。
2. 网上银行分成界面、业务服务器、数据库三部分；**界面不能直接查数据库**，三部分**可以分开部署**。
3. 用户确认收货时，订单服务调用资金服务给商家打款，并**等打款结果返回后**再更新订单。
4. **每天深夜**把当天的销售数据导出成文件，**用 FTP 传给总部**做汇总。
5. 下载大文件时，每个下载者在下载的同时，也把自己已有的分块上传给其他下载者，**没有中心服务器**。

▾ 参考答案

#	答案	判断信号
1	客户端-服务器	一方请求、一方集中提供服务
2	分层	每层只调用下一层、可以分开部署（只说"客户端-服务器"不够准确）
3	调用-返回	调用后等待结果再继续
4	数据流（批处理）	批量、定时、以文件形式传递
5	点对点	每个节点既是客户端又是服务器，没有中心

✎ 小测验｜先自己答，再看参考答案 **小测验 2：三项工作分别属于哪个层次？它们是什么关系？**

公司要上线一个抢票系统，手头有三项工作：① 集团规定所有新系统都部署在 AWS、**统一用 OAuth 登录**；② 决定抢票系统**拆成哪些服务**、各自怎么部署、用什么架构风格；③ 设计订单服务内部有**哪些类、哪些表**、接口参数。三项分别属于企业架构（EA）、解决方案架构（SA）、详细设计中的哪一层？三层之间是什么关系？

▾ 参考答案

工作	层次	管什么
① **AWS + OAuth 统一规定**	企业架构 EA	整个企业、跨所有项目的业务和 IT 规定（像城市规划）
② **拆服务、定部署和风格**	解决方案架构 SA	一个系统由哪些部件组成、怎么协同和部署（像这栋楼的设计图）
③ **类、表、接口**	详细设计	每个部件内部怎么实现（像某个房间的施工图）

关系：**上层约束下层**：SA 必须遵守 EA 的规定，详细设计必须落在 SA 定好的边界里。

✎ 小测验｜先自己答，再看参考答案 **小测验 3：把一句模糊的需求改写成六部分质量属性场景**

在线投票平台：选举日投票开放后的**前 10 分钟**，**约 10 万选民**集中提交选票。需求文档里只写了一句"**系统要能应付大量用户**"。请改写成一个完整的质量属性场景，写出 Source、Stimulus、Artifact、Environment、Response、Response Measure 六部分。目标值可以自行假设，但**必须写成具体数字**。

▾ 参考答案

部分	写法	注意
Source（来源）	约 10 万名选民	写具体角色和数量
Stimulus（刺激）	投票开放后前 10 分钟内集中提交约 10 万张选票	写具体的一种请求
Artifact（制品）	投票服务及其数据库	指系统的哪个部分被作用，不是请求本身
Environment（环境）	选举当天高峰期，系统正常运行	和刺激区分：在什么状态下发生
Response（响应）	接收并确认每张选票，不丢票	
Response Measure（度量）	例如：95% 的提交在 3 秒内确认，错误率低于 0.1%	必须有数字

✎ 小测验｜先自己答，再看参考答案 **小测验 4：产品经理说"易用性是最重要的非功能需求"，你同意吗?**

在线投票平台：选举日**约 10 万选民**要在**固定的投票时段**内完成投票，规则要求**每位选民只能投一票**，结果要**公开、可信**。产品经理认为易用性最重要。请表态，选出你认为最关键的两个质量属性，并说明为什么它们比易用性更关键。答案不唯一，但每个理由都要用到题干里的条件。

▾ 参考答案

✍ 参考答案｜可照着写，名字和数字换成新案例的

I disagree. Usability is important, but it is not the most important non-functional requirement here. The two most critical ones are performance/scalability and security.

> 我不同意。易用性重要，但这里不是最重要的非功能需求。最关键的两个是性能 / 可扩展性和安全性。

Performance/scalability: about 100,000 voters must vote within a fixed voting period. If the system becomes slow or crashes at peak time, voters cannot cast their votes and the election cannot proceed normally.

> 性能 / 可扩展性：约 10 万选民要在固定时段内投票。高峰期系统变慢或崩溃，选民就投不了票，选举无法正常进行。

Security: each voter can vote only once. If someone can vote more than once, the result will not be trustworthy. Poor usability only makes voting less convenient, but problems in security or performance make the system fail its main purpose.

> 安全性：每位选民只能投一票。有人能投多次，结果就不可信。易用性差只是不方便，安全或性能出问题会让系统失去存在的意义。

其他说得通的选择：**可用性**（投票时段内不能停机）、**完整性 / 可审计性**（结果要公开可信）。只要绑定题干条件并说明后果，都能得分。

✎ 小测验｜先自己答，再看参考答案 **小测验 5：这两个需求各用点对点队列还是发布订阅?**

两种消息模式：**点对点队列**（Point-to-Point queue：一条消息只交给一个消费者处理；和第 1 题的点对点架构风格 Peer-to-Peer 不是一回事）；**发布订阅**（Pub/Sub：每个订阅者各收一份）。下面两个需求各用哪种?

1. 电商订单支付成功后，要**同时给用户发短信、给会员加积分、通知推荐服务记录这次购买**（用于以后给用户推荐商品），**以后还可能加更多下游功能**。
2. 客服工单进入系统后，**只需要任意一名空闲客服处理一次**，不能被两个人同时处理。

▾ **参考答案**

1. **发布订阅**：一条"支付成功"消息要让短信、积分、推荐各收一份；以后加功能只需新增订阅者，发布方不用改。
2. **点对点（消息队列）**：一个工单只交给一个消费者：取走后对其他客服不可见，处理完确认后才从队列删除（处理失败会重新投递）；多个客服竞争取工单，天然削峰。

判断问句：这条消息需要几个接收者各自处理一遍?

❝ 可用句式｜___ 处填案例内容

- The reverse proxy is placed in the DMZ. External requests first go to the reverse proxy, and then it forwards them to the internal servers, so the internal servers are not directly exposed to the Internet.
  > 反向代理放在 DMZ。外部请求先到反向代理，再由它转发给内部服务器，所以内部服务器不会直接暴露在互联网上。
- ESB centralizes all communication, so it may become a performance bottleneck and a single point of failure. Modern microservices use lightweight messaging and API gateways instead, and put the logic in each service.
  > ESB 把所有通信集中到一点，可能成为性能瓶颈和单点故障。现代微服务改用轻量的消息机制和 API 网关，把逻辑放在各个服务里。

本章术语表 Glossary

中文	English	一句话解释
架构	Architecture	部件、关系、环境与指导原则的总和
干系人	Stakeholder	对系统有利益或关注点的人或组织
企业架构	Enterprise Architecture (EA)	企业级业务流程与 IT 的组织逻辑
解决方案架构	Solution Architecture	单个系统中各部件在逻辑/物理层面如何协同
质量属性	Quality Attribute	衡量系统"做得多好"的特性
非功能需求	Non-Functional Requirement (NFR)	对质量属性和约束的需求
权衡	Trade-off	提高一个属性的代价是降低另一个
可扩展性	Scalability	负载增长时通过加资源保持性能
弹性	Elasticity	按负载自动增减资源
可演进性	Evolvability	能持续发布新功能与修复
质量属性场景	Quality Attribute Scenario	用刺激、响应、响应度量等六部分描述 NFR
效用树	Utility Tree	按属性→细化→场景组织的需求树
内聚	Cohesion	一个模块内部元素的相关程度，越高越好
耦合	Coupling	模块之间的依赖程度，越低越好
非军事区	DMZ (Demilitarized Zone)	隔开内网与外网、放对外服务的网段

中文	English	一句话解释
反向代理	Reverse Proxy	代表内部服务器接收外部请求的中间服务器
发布/订阅	Publish/Subscribe	一个事件可被多个订阅者接收
参考架构	Reference Architecture	某领域可复用的成熟架构模板
4+1 视图	4+1 View Model	逻辑、进程、开发、物理视图加场景

CHAPTER 02

架构方法：从逻辑架构到物理架构

The Method: Requirements → Logical → Physical Architecture

课件来源：ArchSS 03 Architect the Solution Architecture (Method)，及 PAST 样卷 §6、§8 与官方答案

对应题型 **T4**（每套必考）· **T6**

一句话 **先定义需求，再画与技术无关的逻辑架构（把功能元素打包成子系统并部署到节点），最后才选具体产品和机器，做成物理架构。**

学完本章你能

- 说出方法的三项活动和各自产出
- 按"服务 × 分层"把功能元素打包成子系统
- 读懂打包部署模型里的 location、node、manifest、deploy
- 用检查清单找出部署模型中的错误并给出纠正

2.1 方法总览 Fundamentals of the Method

课程的方法来自 Eeles 的《The Process of Software Architecting》，兼容统一过程 Unified Process（RUP、OpenUP 等）。方法用四个问题描述：**何时**（阶段与迭代 phases & iterations）、**谁**（角色 roles，如解决方案架构师、基础设施架构师、数据架构师）、**做什么**（工作产品 work products）、**怎么做**（活动 activities，每项活动由若干任务 tasks 组成）。

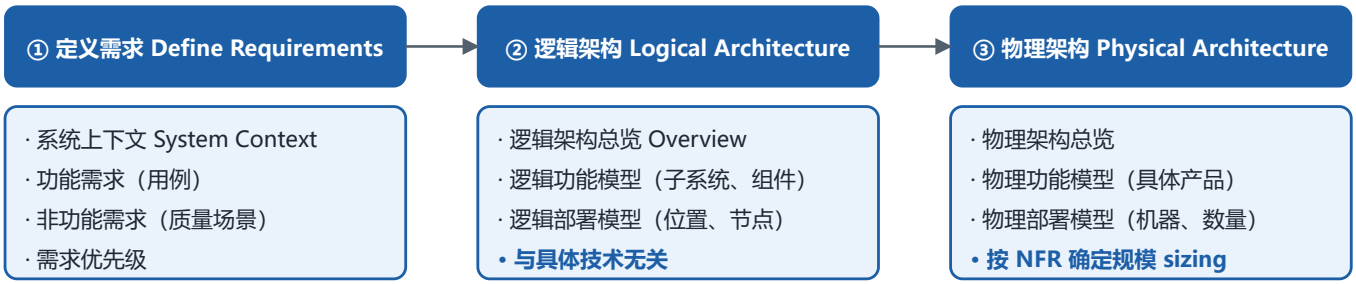


图 2-1 方法的三项活动及其主要产出（每项活动最后都要更新架构文档并与干系人评审）

2.2 定义需求 Define Requirements

定义需求活动有 10 个任务。架构师在每个任务里都有自己要特别留意的事：

任务	架构师要注意
1 收集干系人诉求 Stakeholder Requests	别漏掉技术角色（系统管理员、外部系统的拥有者）；帮业务方理解每项诉求对成本和上市时间的影响
2 建立通用词汇 Glossary	技术术语定义一致（对应 DDD 的通用语言，第4章）
3 定义系统上下文 System Context	找出所有角色和外部系统（如支付网关）、它们所在的**位置**、彼此之间的数据流
4–5 概述功能/非功能需求	包括技术用户的需求；也要明确"不需要什么"
6 需求排序	找出**架构上重要的需求** architecturally significant requirements：关键功能（80/20 规则）、关键质量（如性能）、关键约束（如必须集成某外部系统）、有技术风险的元素
7–8 细化功能/非功能需求	用例要写主流程、备选流程、前置/后置条件；NFR 写成质量属性场景（1.4 节）

任务	架构师要注意
9–10 更新文档、评审	解决方案架构文档 SAD：引言、范围、架构决策、架构机制、架构表示、可追溯性

2.3 逻辑架构：十个任务与两类元素 Logical Architecture

逻辑架构的任务依次是：①调研架构资产 ②定义架构总览（层、位置及其关系）③概述功能元素 ④概述部署元素 ⑤验证架构 ⑥细化功能元素 ⑦细化部署元素 ⑧确认架构（可做概念验证 proof-of-concept）⑨更新架构文档 ⑩与干系人评审。逻辑架构有两类元素，各分"概述"和"细化"两个层次：

元素	概述 Outline	细化 Detailed
功能元素 Functional	子系统 subsystem、组件 component	接口 interface、操作及其签名、前置/后置条件、与数据的映射
部署元素 Deployment	位置 location、节点 node	部署单元 deployment unit、连接 connection

重要的取舍要写成**架构决策** architecture decision，后续阶段都会引用。课件的格式：

字段	例：IROne 度假村系统的规则引擎
问题 Issue	系统有约 30 条需要能修改的业务规则
决策 Decision	自行开发一个简单的规则引擎子系统
假设 Assumptions	规则数量保持在 30 条左右，最多 40 条
备选 Alternatives	①购买商业规则引擎 ②把规则写死在代码里
理由 Justification	规则少，买商业引擎不划算；写死在代码里违反关注点分离和解耦原则

2.4 功能元素：BCE 组件与分层打包 Functional Elements

概述功能元素时，先按用例找出三类组件（**BCE 模式**），再把它们**打包** package 成子系统：

组件类型	职责	Vote Anywhere 例子	外卖 App 白话例子	放进哪层
<<boundary>> 边界	系统与外部的边界：面向用户的界面，或与外部系统收发数据的接口	VoteCastingScreen（界面）；SingPass、Email Gateway 适配器	下单页面、"确认支付"按钮（像餐厅前台）	Presentation（人） External Integration（外部系统）
<<control>> 控制	用例的流程和协调逻辑；一般每个用例一个控制器	VoteController、NotificationController	下单流程：查库存 → 算价格 → 扣款 → 通知商家（像领班）	Control
<<entity>> 实体	业务信息及其持久化	CastVote、Voter、Poll	订单、用户、菜品（像账本和档案）	Data Access

案例 §6.1 规定了分层：**Presentation**（用户交互）、**Control**（用例流程逻辑）、**Data Access**（访问持久存储）、**External Integration**（与外部系统集成）、**Shared Library**（共享/可复用库）。§6.2 规定了分层部署：前端是浏览器和"带本地存储的手机"，后端是 Web Presentation、Service、Storage、Integration 几个层 tier。

> **◆ 要点 | 必背结论　打包规则：服务 × 层**
>
> 子系统的命名和划分 = **所属服务（§7 领域模型里的 <<service>>）+ 所在层**。例如 VotingManager 服务的控制器打包进 "Voting Manager Control"，它的实体打包进 "Voting Manager Data Access"。**一个实体属于哪个服务，就打包进那个服务的 Data Access 子系统**，这是找错时最常用的依据。

2.5 部署元素与打包部署模型 Logical Packaging cum Deployment Model

部署元素有四种：**位置 location**（物理或网络位置，如 Internet、DMZ、Internal Zone）、**节点 node**（能运行软件的设备，如手机、应用服务器）、**部署单元 deployment unit**（打包好的子系统）、**连接 connection**（节点间的通信及协议）。细化部署元素就是"把部署单元分配到节点，并定义组件、节点、位置之间的连接"。

案例 §8 的图把两件事画在一起，所以叫"打包兼部署模型"：虚线 `manifest` 表示"这个子系统包含这个组件"，虚线 `deploy` 表示"这个子系统部署到这个节点"。下图是 PAST 样卷 Figure 6 按官方答案改正后的样子：

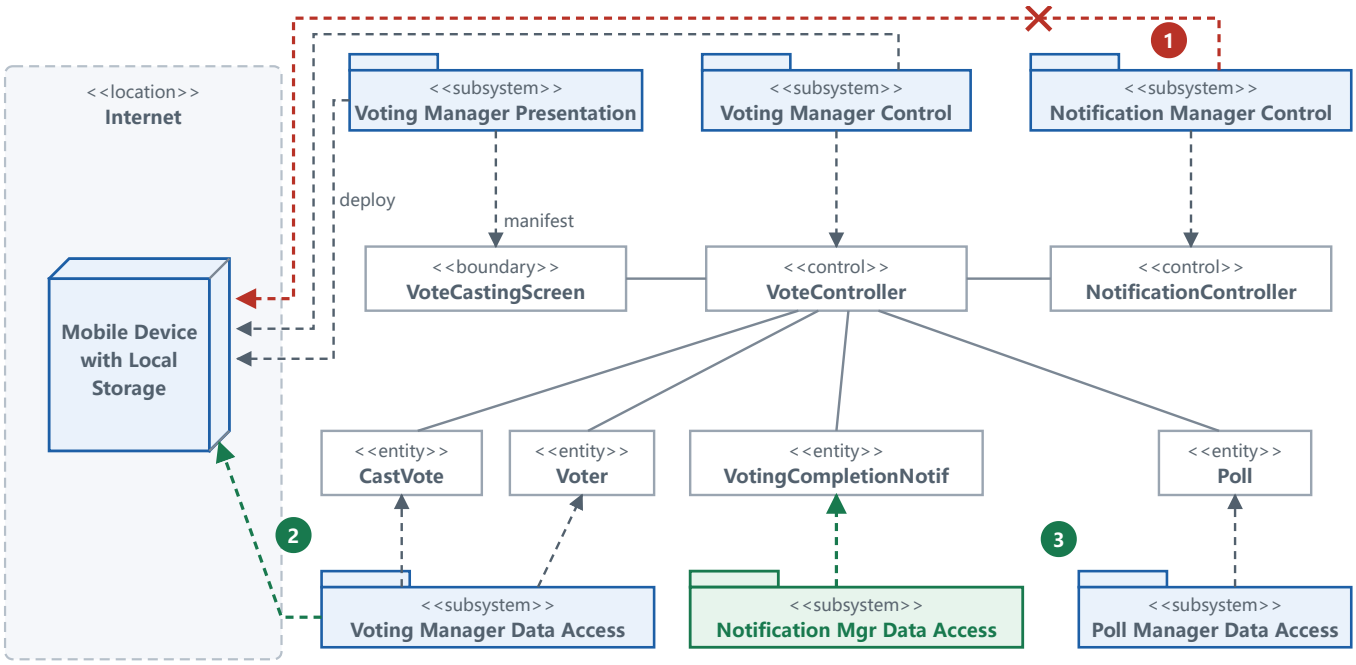


图 2-2 Conduct Vote 用例的打包部署模型（改正后）：红色为应删除的部署，绿色为补上或改正的部分

▶ 案例演练 | 演示怎么套用　PAST Q3(a)（6分）：找出两个不合理的架构决策并纠正

#	不合理的决策（识别 1分）	纠正（1分）	论证（1分）
1	Notification Manager Control 部署到了手机	删除这条部署，留在后端	通知不是从前端发出的，手机端只负责投票界面和本地存储
2	Voting Manager Data Access 没有部署到手机	补上到 Mobile Device 的部署	Conduct Vote 要求把投票细节（CastVote）存在选民设备本地，作为投票凭证
3	实体 VotingCompletionNotif 被打包进 Voting Manager Data Access	改打包进 Notification Manager Data Access	§7 领域模型中 VotingCompletionNotif 属于 NotificationManager 服务

评分：每个决策 **识别 1 分 + 纠正 1 分 + 论证 1 分**；题目要两个，答对任意两个即可满分。题目说"刻意省略了后端部署"，**所以不要把"没画后端"当成错误**。

2.6 部署模型找错清单 Checklist for Undesirable Decisions

考试的 Figure 6 每次都"不完整且可能有错"。**按下面的顺序逐项检查，几乎能找到所有错误**：

检查项	判断依据	典型纠正
① 实体打包对不对	对照 §7：实体属于哪个 <<service>>	改打包到所属服务的 Data Access 子系统
② 组件放对层没有	boundary→Presentation/External Integration，control→Control，entity→Data Access	移到对应层的子系统

检查项	判断依据	典型纠正
③ **子系统是否混了多个服务**	一个子系统只属于一个服务（限界上下文）	按服务拆分
④ **部署到前端的东西该不该在前端**	只放用户交互、该设备必须的本地逻辑和本地存储	把后端职责（发通知、调外部系统、共享数据）移回后端
⑤ **前端是否缺了必须的部署**	用例是否要求离线、本地存储、本地凭证	补上 Data Access 等子系统的部署
⑥ **外部系统集成是否经过集成层**	§6.1 规定 External Integration 层负责对接外部系统	加 External Integration 子系统，部署到 Integration 层
⑦ **位置/区域是否合理**	存储在内部区；对外入口经过 DMZ 的反向代理	把数据库移到 Internal Zone
⑧ **还没打包的元素**	题目要求"complete the diagram"	按"服务名 + 层名"新建子系统并画出 manifest

✎ 考试怎么考 | 题型·分值·套路

T4 每套必考，6 分。三种问法：PAST 找 2 个错误；PAST2 找 1 个错误并补全打包；PAST3 在逻辑部署图上找 3 个错误。**画图时只画改动的部分，不用重画全图**（老师明确说过）。每个错误按"**Undesirable decision → Corrective action → Justification**"三段写。

2.7 物理架构 Physical Architecture

物理架构活动的任务清单和逻辑架构一样，但每一步都落到具体技术上：

- **逻辑元素到物理元素的映射**：一对一（一个物理产品就能满足规格）；一对多（如为了可用性部署多个实例）；多对一（如一个套装软件实现多个逻辑元素）。
- **部署元素**：每个逻辑位置有几个物理实例；每个逻辑节点对应哪些物理节点；每个物理节点的软硬件规格；**按非功能需求确定节点规模** sizing（第12章容量规划）。
- **集成端点表**：列出每条连接的源、目标、协议（HTTP/HTTPS、JMS）、格式（HTML、XML、JSON）、是否需要转换。
- 物理决策同样写成架构决策，例如"用 IBM WebSphere，因为团队已有相关技能"。

传统 Java EE 的映射示例：面向人的边界组件 → JSP/Servlet；面向系统的边界组件 → JMS；控制组件 → Session EJB；实体组件 → JPA。云原生时代对应的是：前端框架、API 网关、容器中的微服务、托管数据库（第8、10、11章）。**考试的物理部署题（T6）就是把逻辑节点换成具体云服务**，例如逻辑的"Storage tier"→ Amazon RDS + ElastiCache。

2.8 白话讲解与小测验 Plain-language Notes & Drills

◆ 要点 | 必背结论 为什么要打包：三句话

几百个类直接扔到服务器上会有三个麻烦：不知道谁放哪台机器、改一处影响一大片、团队难分工。**打包 = 把相关的类归进一个盒子（子系统），每个盒子整体决定放哪台机器。**界面、流程、数据三类东西分开放，因为它们要放的位置不同：界面在手机，流程和数据在后端。

	Presentation	Control	Data Access
订单服务	订单页面	下单流程控制器	订单（实体）
通知服务	—	通知流程控制器	通知记录（实体）
商家服务	商家页面	商家流程控制器	商家、菜品（实体）

每个格子就是一个子系统，名字 = 服务名 + 层名。

■ 定义｜课件正式说法 **整盒搬：deploy 线只能画在子系统上**

- **manifest**：子系统 → 组件 / 实体，表示盒子里装什么（打包）；**deploy**：子系统 → 节点，表示盒子装到哪台机器（部署）。
- 盒子部署到哪，里面的组件和实体就**跟到哪**。想把某个实体单独放到别的节点，必须先把它拆成独立的子系统。
- 推论：实体被打包进别的服务的盒子，那个盒子部署到哪它就跟到哪，所以找错要同时查"盒子里装的对不对"和"盒子放的位置对不对"。
- 官方答案图里，红色 = 应删除，绿色 = 应补上，灰色 = 原本就对；红框只圈出错误位置，不代表框里每条线都错。

✎ 小测验｜先自己答，再看参考答案 **小测验 1：优惠券实体被打包进了订单服务的数据层**

本章约定：打包部署图里，每个服务（如 Order Manager）按层分成几个子系统，命名为"服务名 + 层名"：Presentation（界面）、Control（流程控制）、Data Access（数据存取）。实体属于哪个服务以领域模型为准。找错题按 ① 识别（哪里不合理）② 纠正（怎么改）③ 论证（为什么）三段式作答。

外卖平台：领域模型规定 Coupon（优惠券）**属于 Marketing Manager**（营销服务），但打包图里 Coupon 被打包进了 **Order Manager Data Access** 子系统。请按三段式作答。

▾ **参考答案**

识别：实体 Coupon 被打包进了 Order Manager Data Access。**纠正：**改为打包进 Marketing Manager Data Access。**论证：**领域模型中 Coupon 属于 Marketing Manager；实体放进所属服务自己的子系统，两个服务各管各的数据，改动互不影响，符合高内聚、低耦合。

注意写"营销服务"而不是"营销服务器"：打包按服务（逻辑划分）分，部署才落到服务器 / 节点（物理机器）。

✎ 小测验｜先自己答，再看参考答案 **小测验 2：投票 App 的通知控制子系统被部署到了选民手机上**

延续第 1 题的约定和三段式。投票平台的打包部署图里，Notification Manager Control 子系统负责在投票完成后，**通过外部的 Email Gateway** 给选民发确认邮件。图上它有一条 deploy 线**连到了"选民手机"节点**。请按三段式回答：这是打包错还是部署错？这个子系统应该部署在哪里？

▾ **参考答案**

识别：Notification Manager Control 被部署到了选民手机。这是**部署**错，不是打包错。**纠正：**删掉它到手机的 deploy 线，让它留在后端。**论证：**发邮件是平台的职责，由后端统一调用 Email Gateway；如果部署到手机，调用外部网关的凭据就要下发到每台手机，既不安全也无法统一管理；手机只需要放投票界面和必须的本地存储。

❝ 可用句式｜___ 处填案例内容

- The Notification Manager Control subsystem is deployed to the mobile device. It should be removed from the device and stay on the backend, because notifications are sent by the platform through the Email Gateway, not by the phone.

  > Notification Manager Control 子系统部署到了手机。应删除这条部署、留在后端，因为通知由平台通过 Email Gateway 发送，不是手机发出的。

✎ 小测验｜先自己答，再看参考答案 **小测验 3：外卖"下单"的打包部署图里找 3 个错**

延续第 1 题的约定和三段式。子系统里的东西可以移动，但不新增服务；表里没画出的层不算错。请找出 3 个错，按三段式作答。

需求：**购物车保存在手机本地**，App 重启后仍在；**支付由后端对接外部支付网关**。领域模型：Order Manager 有 Order、Cart；Payment Manager 有 Payment；**Notification Manager 有 OrderNotif**。

子系统	里面的东西	部署到手机?
Order Manager Presentation	OrderScreen	是
Order Manager Control	OrderController	否
Order Manager Data Access	Order、Cart、OrderNotif	否
Payment Manager Control	PaymentController	是
Payment Manager Data Access	Payment	否
Notification Manager Data Access	（空）	否

▾ **参考答案**

#	识别	纠正	论证
①	Order Manager Data Access 没有部署到手机	补上它到手机的 deploy 线	需求要求购物车保存在手机本地；Cart 在这个子系统里，子系统是整个部署的（Order 会跟着去，本题可接受）
②	Payment Manager Control 部署到了手机	删除这条部署，留在后端	支付由后端对接外部网关，手机只放交互界面和本地存储
③	OrderNotif 打包进了 Order Manager Data Access	改打包进 Notification Manager Data Access	领域模型中 OrderNotif 属于 Notification Manager

论证只用题目给的依据；不需要为 Cart 另建子系统，补上整个子系统的部署即可。

✎ 小测验 | 先自己答，再看参考答案 **小测验 4：延续第 3 题：加上区和节点后，再找 2 个错**

延续第 3 题，假设第 3 题的三个错都已改正。现在每个服务除了 Presentation、Control、Data Access，还可以有 **External Integration** 子系统，专门放对接外部系统的类。

部署情况：网络区域有 Internet、DMZ、Internal zone；Mobile device 在 Internet，Application server 在 Internal zone，**Database server 放在 DMZ**，上面部署了 Payment Manager Data Access 和 Notification Manager Data Access；外部有 Payment gateway。Payment Manager Control 里装了 PaymentController 和 **PaymentGatewayAdapter**（负责调用外部支付网关）。**入口的反向代理刻意省略**，不算错。请找出这张部署图里另外 2 个错，按三段式作答。

▾ 参考答案

错	纠正	论证
Database server 放在 DMZ	把整个 Database server 节点移到 Internal zone，上面两个子系统一起移过去	DMZ 只放对外的反向代理和 Web 层，数据库不能直接暴露给互联网
PaymentGatewayAdapter 打包在 Control 子系统里	新建 Payment Manager External Integration，把 Adapter 打包进去，部署在内部区的后端	对接外部系统的类属于 External Integration 层；外部接口变化只影响这一层，不影响支付流程

支付网关是外部系统，不是我们能"迁移"的；集成层是后端调用外部系统的出口，放内部区，不放 DMZ。

✎ 小测验 | 先自己答，再看参考答案 **小测验 5：两张部署视图，哪张是逻辑架构、哪张是物理架构？**

团队交来两张部署视图，请按"**是否绑定具体技术和规格**"来判断。图 A："订单服务部署在应用服务器节点，订单数据库在数据库节点，两者都在内部区"。图 B："订单服务跑在 **3 台 EKS 工作节点（m5.large）**，数据库是 RDS PostgreSQL **db.r5.xlarge**、多可用区"。哪张是逻辑架构、哪张是物理架构？两者的根本区别是什么？物理架构的规模依据什么来定？

▾ 参考答案

图 A 是**逻辑架构**：只说有哪些功能元素（子系统、组件）和部署元素（位置、节点），与具体技术无关。图 B 是**物理架构**：给逻辑节点选定具体产品、机型、数量和规格。规模要按非功能需求（并发量、响应时间、可用性目标）来定（sizing）。先有逻辑、再落到物理，换云厂商时只改物理那一层。

本章术语表 Glossary

中文	English	一句话解释
统一过程	Unified Process (UP)	以阶段、迭代、角色、工作产品、活动组织的开发过程
工作产品	Work Product	任务产出或使用的信息，如系统上下文、部署模型
系统上下文	System Context	系统与角色、外部系统之间的边界和数据流
架构上重要的需求	Architecturally Significant Requirement	对架构影响大的功能、质量、约束或风险
逻辑架构	Logical Architecture	与具体技术无关的功能元素和部署元素
物理架构	Physical Architecture	确定具体产品、机器数量和规格的架构
架构决策	Architecture Decision	记录问题、决策、假设、备选和理由
子系统	Subsystem	打包在一起的一组组件，可作为部署单元

中文	English	一句话解释
边界/控制/实体	Boundary / Control / Entity	界面或接口 / 用例流程 / 业务信息
位置	Location	部署的物理或网络区域，如 Internet、DMZ
节点	Node	能运行软件的设备或服务器
部署单元	Deployment Unit	被部署到节点上的打包好的子系统
体现	Manifest	子系统包含（实现）某个组件
部署	Deploy	子系统被安装到某个节点上
层 / 层级	Layer / Tier	逻辑分层 / 物理部署的分层
概念验证	Proof-of-Concept (PoC)	用原型降低架构风险
规模估算	Sizing	按非功能需求确定节点的数量和规格

CHAPTER 03

平台思维

Platforms & Platform Thinking

课件来源：PE 01 Introduction to Platforms · PE 05 Platform Management（开篇） 对应题型 **T9**

一句话 **平台的价值不在功能本身，而在于让生产者和消费者围绕"种子"高效地交互；工程上要保证这些交互能扩展、不中断、能持续演进。**

学完本章你能

- 说出平台的定义、特征和收益
- 区分平台、框架、库和应用
- 用 seed / producer / consumer / interaction 分析一个平台
- 把平台要素对应到工程要素，回答"哪些架构方面对平台成功最关键"

3.1 什么是平台 What is a Platform

课件给出了三种互补的定义：

- **产品视角**：软件平台是一个软件产品或服务，作为基础让**外部各方**在上面构建互补的产品或服务；它是可扩展的系统，提供被多个"应用"共享的核心功能。
- **技术视角**：平台是一组服务，应用构建并运行在其上；现代平台模糊了基础设施架构和应用架构的界线，要处理服务发现、资源协调、容器编排、用量统计等问题。
- **商业视角**：平台是一种商业模式，通过促成两个或多个相互依赖的群体（通常是**消费者** consumers 和**生产者** producers）之间的交换来创造价值。

平台类型	例子	平台类型	例子
服务平台	Airbnb、Uber、Grab	**社交平台**	Facebook、LinkedIn
交易市场	eBay、Amazon、Carousell	**通讯平台**	WhatsApp、WeChat
支付平台	PayPal、Stripe、PayNow	**内容平台**	YouTube

平台的特征：可组合 composable（由能独立使用的组件和服务组成）；上手快、成本低（快速入门、文档、示例代码）；有活跃的用户社区；促进合作伙伴之间的交互；保持数据驱动、持续相关；安全合规 secure and compliant；提升业务敏捷性 business agility。以 Google Pay 为例：容易集成进电商应用（敏捷）、连接商户和买家（交互）、安卓用户采用率高（社区）、用机器学习识别欺诈并使用虚拟卡号（安全合规）。

建平台的收益：与异构系统互操作；更快响应商机；服务可以嵌入价值链而不必按顺序串行；数据可支持决策；平台**内部**的变化可以和平台**之间**的变化以不同节奏进行，互不拖累。

3.2 平台、框架、库与应用 Platforms vs Frameworks vs Libraries

概念	关系	例子
平台 Platform	软件运行的环境；提供服务和可复用组件；通过消费和对外暴露服务来扩展功能。"你在平台上工作"	AWS、Android、Salesforce

概念	关系	例子
框架 Framework	提供某领域的部分实现，由框架调用你的代码。"你使用框架"	Spring、Django、ReactJS、.NET
库 Library	你的代码主动调用它。"你调用库"	NumPy、jQuery、PyTorch
应用 Application	面向最终用户、构建在平台之上的软件	Grab 里的 GrabFood

3.3 平台思维 Platform Thinking

传统**线性业务模式** linear business model（生产 → 组装 → 分销）关注的是**流程**；平台的**网络业务模式** network business model 关注的是**交互**。平台是一种基础设施，让生产者和消费者通过信息（**种子** seed）进行交互。课件强调"想角色，不想市场" think roles, not markets：同一个人可以既是生产者又是消费者。



图 3-1 平台模型：生产者与消费者围绕种子交互，平台用三个组件支撑交互（以 YouTube 为例）

组件	作用
磁铁 Magnet	把两种角色吸引进平台；随时间把消费者转化为生产者（省下营销费用）；让生产者产出更多种子
工具箱 Toolbox	让交互能发生的工具：给生产者创作的工具、给消费者使用的工具（Web、App、API、集成）
撮合者 Matchmaker	用数据收集和分析，把合适的生产者和消费者配对（如推荐算法）

扩展平台业务：**一次只建一种交互** build one interaction at a time。LinkedIn 先做"专业人士之间的交互"，再加招聘者与求职者、讨论组和帖子、关注意见领袖，最后才变现。平台还要决定谁来做**内容审核** moderation：员工、算法，还是用户自己。

商业生态系统 business ecosystem 是参与交付某产品或服务的组织网络（供应商、分销商、客户、竞争者、政府机构等），通过竞争与合作共同运转。平台生态系统由**外部生产的互补品**构成，它们扩展了平台的能力。

3.4 平台的工程视角 Engineering Perspective

这张表是回答平台题的关键：**把平台的商业要素翻译成架构师要设计的东西**。

平台要素	对应的工程要素
种子 Seed	领域实体 domain entities
生产者与消费者	领域实体、**可扩展性、性能**
交互 Interactions	服务设计、架构、**对外暴露服务（API）**、业务流程、可扩展性、性能
磁铁 Magnet	服务的易用性设计、激励机制设计
工具箱 Toolbox	Web 应用、移动应用、API、集成
撮合者 Matchmaker	数据收集与分析

◆ 要点｜必背结论 **平台工程师最关心的四件事（PE 05）**

可扩展性（既能扩大交互量，也能限制交互量）、**可用性**（无缝的平台体验）、**可演进性**（持续发布修复和新功能）、**数据**。课件原话："*Consistent bad experience would kill your platform.*"（**持续的糟糕体验会毁掉平台**。）

3.5 平台工程与内部开发者平台 Platform Engineering

平台工程 platform engineering 是设计和构建工具链与工作流，让软件工程组织能够**自助服务** self-service 的实践。它的起因是：缺乏自动化、DevOps 配置不统一、开发者认知负担过重、只能靠提工单。它和 DevOps（文化与实践）、SRE（可靠性工程）互补而不是替代。

术语	含义
IDP Internal Developer Platform	为开发者构建的整套自助式基础设施平台，是平台团队交付的"产品"
DevEx Developer Experience	开发者工作是否容易、快速、愉快
黄金路径 Golden Path	平台推荐的、预先铺好的标准做法
IaC / GitOps	基础设施即代码；以 Git 作为应用代码和基础设施状态的唯一事实来源
DORA 指标	部署频率、变更前置时间、平均恢复时间 MTTR、变更失败率

课件的**五平面参考模型**：①开发者控制平面（门户如 Backstage、IDE、CLI）②集成与交付平面（CI 流水线、镜像仓库、平台编排器、CD 如 ArgoCD）③资源平面（Kubernetes 集群、数据库、存储、消息队列）④监控与日志平面（Prometheus、Grafana、OpenTelemetry）⑤安全平面（密钥管理、身份与访问控制）。

▶ 案例演练｜演示怎么套用 **Vote Anywhere 的平台分析**

种子：投票（Poll）及其结果。**生产者**：Election Admin 创建投票，Voter 产生选票。**消费者**：Public User 和 Observer 查看结果。**交互**：Conduct Vote、View Current Simplified/Detailed Results。

对平台成功最关键的架构方面：①**可扩展性与性能**：10万并发选民同时投票，交互请求 90% 在 3 秒内响应，否则选民排长队、体验崩溃；②**结果更新的及时性和确定性**：简要结果 1 分钟内更新，需要事件流等机制可靠地汇总选票；③**可扩展的服务与 API**：§3 要求用公共服务和库支持未来扩展新服务、新角色。

✎ 考试怎么考｜题型·分值·套路

T9（约2分/题）。PAST2 Q1(a)、PAST3 Q1(a)：给出种子和交互，问"哪一两个架构方面对平台成功最关键"。答题套路：**点名种子和交互 → 选一个工程要素（可扩展性/性能/可用性/API/数据分析）→ 引用 §4 的数字说明为什么关键**。PAST3 Q1(c)(i) 还问"把哪些共享服务作为 API 暴露给哪些生产者/消费者"：**要点名具体客户（如药品供应商、医院）并说明他们为什么需要这个 API**。

3.6 小测验 Drills

✎ 小测验 | 先自己答，再看参考答案 **小测验 1：以 Grab 为例，说出种子、生产者、消费者和交互**

种子是平台上双方交换的核心价值单元，**只写一个**；**生产者**提供种子，**消费者**使用种子；**交互**是双方围绕种子做的动作。以 Grab 的外卖或打车业务（任选一条）为例，写出种子、生产者、消费者，并写出**生产者和消费者各 2–3 个交互动作**。

▾ **参考答案**

	外卖	打车
种子（只选一个）	一份外卖菜品	一次行程
生产者	餐厅商家	司机
消费者	食客	乘客
生产者侧交互	上架菜品、接单、出餐	接单、接客、送达
消费者侧交互	浏览、下单、支付、评价	叫车、上车、支付、评价

加分：配送员可视为另一类生产者；平台的撮合者（推荐排序、派单算法）把双方配对。

✎ 小测验 | 先自己答，再看参考答案 **小测验 2：晚高峰 5 万人同时叫车：选两个最关键的工程要素并给出设计**

平台的每种交互都对应六个工程要素：服务设计、架构、对外暴露服务（API）、业务流程、可扩展性、性能。某打车平台晚高峰**约有 5 万人同时叫车**。从六个要素里选出对"叫车"这个交互**最关键的两个**，各写一句具体设计，并用上 5 万这个数字。

▾ **参考答案**

参考选择：**可扩展性**：叫车请求先进入消息队列削峰，派单服务做成无状态、按队列长度自动扩容，撑住 5 万人同时叫车。**性能**：5 万人同时叫车时，90% 的叫车请求在 3 秒内返回"已受理"（目标值为假设），派单结果再推送给乘客。

要素	打车里是什么
服务设计	订单、派单、定位、支付服务各管一块
架构	请求先进队列削峰，派单服务水平扩展
对外暴露服务（API）	把"叫车"开放给企业用车系统、地图 App
业务流程	叫车 → 派单 → 接单 → 行程 → 支付 → 评价
可扩展性	高峰自动加机器，平时缩回
性能	90% 的叫车请求 3 秒内有回应

✎ 小测验 | 先自己答，再看参考答案 **小测验 3：创业团队想一次上线"二手交易 + 跑腿 + 拼车"三合一平台，你怎么建议？**

一家校园创业公司计划**第一版就同时上线**二手交易、跑腿代购、拼车三种功能，理由是**"功能多才吸引人"**。请用平台思维给出建议，并写出**至少 3 条理由**。

▾ **参考答案**

建议**一次只建一种交互**，先选一种（如二手交易）跑通。理由：① 平台要先把一种交互的买卖双方聚够，才有吸引力；再加新交互时已有用户基础；② 像 MVP，先在小范围验证这种交互真的有人用；③ 控制风险和成本，资源集中；④ 降低用户学习成本；⑤ 更容易看清哪种交互能带来收入；每加一种交互就要加服务、API 和扩展要求，逐步加能让架构边做边演进。

本章术语表 Glossary

中文	English	一句话解释
平台	Platform	让外部各方构建产品、并促成生产者与消费者交换的基础
生产者 / 消费者	Producer / Consumer	在平台上提供种子 / 使用种子的角色
种子	Seed	用户来平台想要的核心信息或价值
交互	Interaction	生产者与消费者围绕种子发生的动作
磁铁	Magnet	吸引并留住生产者和消费者的机制
工具箱	Toolbox	让交互发生的工具：App、Web、API、集成
撮合者	Matchmaker	用数据分析配对生产者与消费者
网络业务模式	Network Business Model	以交互而非流程为核心的业务模式
商业生态系统	Business Ecosystem	通过竞争与合作共同交付价值的组织网络
平台工程	Platform Engineering	构建自助式工具链和工作流的实践
内部开发者平台	Internal Developer Platform (IDP)	平台团队为开发者提供的自助平台
开发者体验	Developer Experience (DevEx)	开发者工作的顺畅程度
黄金路径	Golden Path	平台预设的标准开发与部署路径
DORA 指标	DORA Metrics	衡量交付效能的四个 DevOps 指标

CHAPTER 04

领域驱动设计与可复用资产

Domain-Driven Design & Reusable Assets

课件来源：PE 02 Reusable Assets and Frameworks · PE 04 Reusable Services and API Gateway（聚合、上下文映射、领域事件）

对应题型 **T1**（每套必考）· **T2**

一句话 **用限界上下文把领域拆成内聚的服务，用小聚合守住一致性；领域模型的问题几乎都出在"实体放错了服务"或"聚合太大"。**

学完本章你能

- 区分服务、框架、共享库三种可复用资产
- 解释限界上下文、通用语言、聚合、领域事件
- 背出聚合设计的 7 条经验法则
- 诊断领域模型问题：原因 → 对策 → 缺点
- 从用例描述中画出类图、找出共享服务

4.1 可复用资产：服务、框架、共享库 Reusable Assets

平台的价值很大程度来自复用。课件把可复用资产分成三类，复用规模从大到小：

资产	含义与例子	注意	白话 · 谁能用它
可复用服务 Reusable Services	可被多个功能或用例复用，如 Customer 服务同时被 Purchase 和 Delivery 服务使用	复用规模最大；边界要用 DDD 来划分	像前台部门：自己独立运行，任何语言都能调它的接口
可复用框架 Reusable Frameworks	用来构建多个客户端和服务，如用 Spring Boot 构建 Purchase 和 Delivery 服务	应简单、借鉴成熟设计、一致、易于集成	像施工模板：只有同一技术栈的项目能用
共享库 Shared Libraries	在服务之间、客户端之间共享的类（状态规范、行为规范）	小规模、临时性复用；容易造成耦合	像每人一本手册：复制进同一语言的程序，库一改所有人都要跟着改、重新测

可复用服务的特征：长期使用；可维护、可扩展、有韧性 resilient；功能稳定且内聚；职责单一（单一职责原则 SRP）；与其他服务松耦合 loosely coupled；可扩展。

共享库的缺点：失去技术异构性（绑定语言和运行平台）；如果几个部分通过库里的静态变量、互斥锁等绑在一起，就难以独立扩展；"不要重复 DRY"原则在**跨服务**时很危险：把公共领域对象做成所有服务共享的库，库一改所有服务都受影响。客户端库则要防止服务端逻辑渗进客户端，并让客户端自己决定何时升级。

4.2 领域驱动设计：战略设计 DDD Strategic Design

领域驱动设计 Domain-Driven Design, DDD 把项目重心放在核心领域和领域逻辑上，用领域模型驱动复杂设计，并要求技术人员和领域专家紧密协作。与传统面向对象分析（用例 → 边界/控制/实体对象）不同，DDD 直接聚焦领域对象。DDD 分两个层次：

层次	处理什么	工具
战略设计 Strategic Design	粗粒度：怎么划分领域	限界上下文、通用语言、子领域、上下文映射
战术设计 Tactical Design	细粒度：一个上下文内部怎么建模	聚合、实体、值对象、领域事件、仓储

■ 定义｜课件正式说法 **限界上下文与通用语言**

限界上下文 Bounded Context, BC：一个明确的边界，边界内的领域模型和术语有唯一、一致的含义。**通用语言** Ubiquitous Language, UL：团队成员在某个限界上下文内共同使用、并体现在代码里的语言。同名实体在不同上下文里可以有不同含义（例如"Account"在银行业务和用户登录里含义不同）。

- **一个 BC 只分配给一个团队**（一个团队可以负责多个 BC）；每个 BC 用独立的代码仓库和独立的**数据库 schema**。在实现上，**一个 BC 通常就是一个微服务**，考试图里的每个 `<<service>>` 框就是一个 BC。
- 没有 BC 的系统会随着功能增加，把越来越多实体堆进同一个模型，变成**大泥球** Big Ball of Mud。
- BC 的好处：BC 内实体功能内聚，不内聚的被推到别的 BC；一个 BC 只有一套通用语言；避免设计出单体应用；每个 BC 的测试用例更少。
- **子领域** subdomain 分三种：**核心域** core（竞争优势所在，如投票平台的计票）、**支撑域** supporting、**通用域** generic（各行业通用，如通知、支付、身份认证）。**通用域最适合做成可复用的共享服务**。

◆ 要点｜必背结论 **划分限界上下文的三条规则（PE 02）**

1. **按领域实体的功能内聚** functional cohesion 划分 BC。
2. 把每个 BC 归类为核心域或子领域。
3. 如果拆分后的 BC 在其他系统中**更容易复用**，就继续拆分。

识别 BC 需要开发者和领域专家一起，用**具体场景** concrete scenarios 描述模型如何工作：场景里的名词是候选实体，动词是行为和关系。

4.3 战术设计：实体、值对象、聚合、领域事件 DDD Tactical Design

构件	含义	例子
实体 Entity	有唯一标识、可随时间变化的个体	Poll、Voter、Order
值对象 Value Object	没有标识、不可变的概念整体，只看值	Address、Money、DateRange
聚合 Aggregate	同一 BC 内一组必须作为整体保持一致的实体，是**事务一致性边界**；作为一个整体加载和保存	Poll + Candidate
聚合根 Aggregate Root	聚合对外的唯一入口，外部只能通过它访问内部	Poll
仓储 Repository	负责一个聚合的持久化和读取	PollRepository
领域事件 Domain Event	BC 内发生的、业务上有意义的事情的记录	VoteCast、PollClosed

聚合设计的 7 条经验法则

#	法则 Aggregate Rules of Thumb	解决什么问题
1	**在聚合边界内保护业务不变量** Protect business invariants inside aggregate boundaries。例：BacklogItem 下所有 Task 剩余工时为 0 时，状态必须为 DONE	一致性
2	**设计小聚合** Design small aggregates。大聚合加载慢、占内存多、垃圾回收慢、事务更容易冲突失败	**内存、性能**
3	**只通过标识引用其他聚合** Reference other aggregates by identity only：持有 ProductId 而不是 Product 对象	内存、耦合
4	**用最终一致性更新其他聚合** Update other aggregates using eventual consistency：发布领域事件，订阅方在自己的事务里更新	事务范围
5	但如果业务要求两个聚合**必须立即一起更新**，就考虑把它们合并成一个聚合	强一致
6	**避免贫血模型** Avoid an anemic domain model：业务逻辑放在聚合里，而不是只有 getter/setter	内聚

#	法则 Aggregate Rules of Thumb	解决什么问题
7	**一个聚合一个仓储** One repository per aggregate	持久化边界清晰

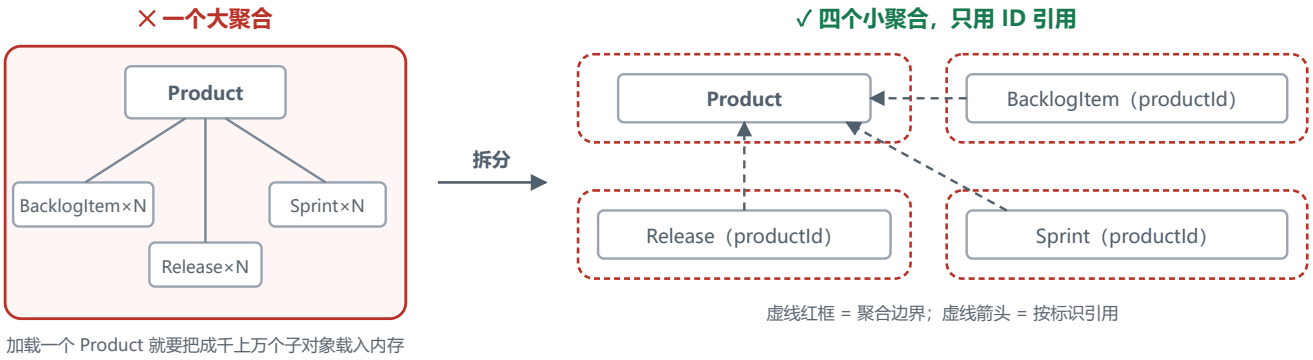


图 4-1 "设计小聚合"与"只通过标识引用"（课件 Scrum 工具例子）：解决内存占用过高的问题

领域事件

- **命名用过去时**：事件是已经发生的事。由命令 CreateProduct 触发的事件叫 ProductCreated。
- **属性**：包含触发它的命令的全部属性，让所有订阅者准确知道发生了什么。
- **顺序很重要**：领域事件的顺序保证因果一致性 causal consistency，即有因果关系的操作在所有依赖方看来顺序相同。
- **如何识别**：对每一个"消息"类型的上下文映射，找出依赖它的业务功能，再找出每个功能需要的事件。
- 课件的租车例子：Rental 聚合内存占用过大，于是拆成 RentCtrl 和 RentExt 两个聚合；RentalManager 与 VehicleManager 两个 BC 之间用 VehicleRented、VehicleReturned 事件同步。

4.4 上下文映射：BC 之间怎么集成 Context Mapping

拆出来的 BC 不是完全独立的，一个 BC 可能要用另一个 BC 的实体。两个 BC 之间的集成关系叫**上下文映射** context mapping（U = 上游提供方，D = 下游使用方）：

映射类型	含义
合作关系 Partnership	两个团队紧密协作，持续对齐集成
共享内核 Shared Kernel	两个团队共享一个小而公共的模型
客户-供应商 Customer-Supplier	上游供应方按下游客户的需要提供功能
遵奉者 Conformist	下游只能照搬上游的模型（上游不迁就下游）
防腐层 Anticorruption Layer	下游加一层翻译，把上游的语言转成自己的通用语言，防止被污染
开放主机服务 Open Host Service	上游定义一套协议/接口，供所有下游访问
发布语言 Published Language	上游用有良好文档的信息交换格式发布接口（如 JSON Schema）
各行其道 Separate Ways	集成收益太小，干脆不集成

集成方式：常用的是 RPC（SOAP）、RESTful HTTP、**消息** messaging；**应避免通过共享数据库或共享文件系统集成**。消息方式最健壮：下游订阅上游发布的领域事件，上游暂时不可用也不影响下游。要求消息机制支持**至少一次投递** at-least-once delivery，接收方实现为**幂等接收者** idempotent receiver（同一消息收到多次，结果相同）。

4.5 读懂考试中的领域模型图 Reading the Domain Model (§7)

图中元素	含义
<<service>> 框	一个服务 = 一个限界上下文 = 将来的一个微服务
框里的小方块	领域实体
实线	关联：两个实体之间有引用关系（线上的文字如 manages、votes 表示关系含义）
空心菱形	聚合关系（整体-部分，如 Poll 有 PollReport、Candidate）
空心三角箭头	泛化/继承（如 ElectionAdmin、Proctor、Voter 都是 User；各类 Notif 都是 NotificationEmail）
红色虚线圈（部分卷子）	聚合边界
跨框的连线	**跨服务引用，运行时需要进程间通信（同步调用或事件），比进程内调用慢得多。**这是诊断题最主要的线索

4.6 诊断领域模型问题 Diagnosing the Domain Model

T1 题给出一个"问题现象"，要求写出**最可能的原因** most likely cause、**有效对策** effective countermeasure（必须通过**修改领域模型**）和**对策的主要缺点** drawback。先从题目关键词判断问题类型：

问题类型（关键词）	最可能的原因	对策（改领域模型）	缺点	例子与易错
慢、不及时 promptly / speedy / latency	一起使用的实体分散在不同服务，每次都要**跨进程通信**	把相关实体迁到同一个服务；或在使用方保存只读副本，用领域事件同步	服务变大、内聚下降、事务范围变大（可用多个聚合缓解）；副本方案有数据冗余、最终一致	电商商品页：Price、Stock 迁入 ProductManager；**库存不能用副本**（晚几秒会超卖）。把商品图片拆出去是答非所问
难以复用 reuse / other platforms	通用服务里混入了大量**业务特有的实体** specificities	把特有实体迁回各自的业务服务，通用服务只保留通用部分	通常没有大缺点；各类实体还能随所属服务独立扩展	外卖通知：问"它在说怎么发，还是发什么业务的事"，前者留、后者搬；缺点写"没有大缺点 + 原因"
内存占用高 runtime memory / footprint	**聚合太大**，一个聚合挂着大量子对象，加载一个就载入整个对象图	按"设计小聚合"拆分聚合，聚合间只用 ID 引用，按需加载	读取关联数据要多次查询；跨聚合更新变成最终一致	在线学习 Course 拆出 Enrollment、Submission、ViewRecord，引用**父聚合 courseId**；网银"收款人不超过 5 个"是不变量，Beneficiary 留在 Account 里
无法独立扩展 scale independently	负载差别很大的功能放在同一个服务里	把高负载的实体和功能拆成独立服务	服务间通信增多，一致性更难保证	直播：**只拆高负载的** Stream，StreamerProfile 留原服务
数据不一致 consistency / invariant	必须同时更新的实体分在不同聚合或服务里	合并成一个聚合来保护不变量	聚合变大，并发冲突增加	优惠券 100 次上限：Redemption 迁入 CouponManager 与 Coupon 同一聚合；缺点是热门时冲突和重试（吞吐下降，不是数据出错）

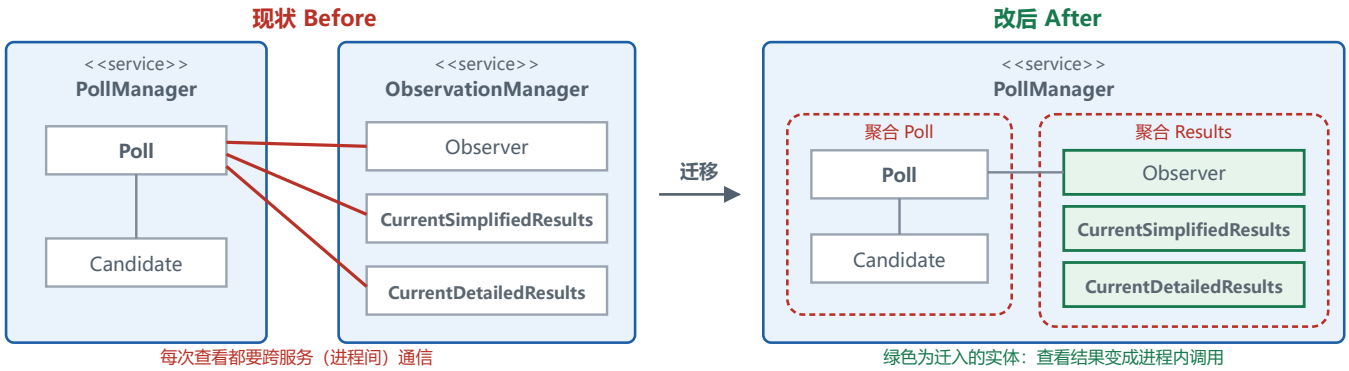


图 4-2 PAST Q1(b)(ii) 的对策：把 ObservationManager 的实体迁入 PollManager（只画改动部分即可）

▶ 案例演练｜演示怎么套用 **PAST Q1(b)：两道诊断题的官方答案（共9分）**

(i) NotificationManager 难以复用到其他平台（5分）。

Cause：现有设计把所有通知都集中在 NotificationManager，它包含了各类通知的业务特有内容（PollingNotif、ProctoringNotif、VotingCompletionNotif 等），这些特有内容在别的平台上用不到。（2分）

Countermeasure：把各类特有通知实体迁回各自的业务服务（如 VotingCompletionNotif 迁到 VotingManager，ProctoringNotif 迁到 ProctoringManager），NotificationManager 只保留通用的 NotificationEmail。（按"特有功能分离"1.5分 + "通用功能内聚"1.5分）

Drawback：没有大的缺点；按类别分开后，各自所属服务还能独立扩展。（0分，但要写）

(ii) Observer 无法及时看到 CurrentSimplifiedResults（4分）。

Cause：查看结果需要 PollManager 和 ObservationManager 协作，两个服务之间的交互必须通过进程间通信（同步或事件），比进程内通信慢得多。（1.5分）

Countermeasure：把 ObservationManager 的实体迁入 PollManager（见图 4-2）。（1.5分）

Drawback：PollManager 变成一个较大的服务（限界上下文），事务范围变大；可以在服务内划分多个聚合来缓解。（1分）

✕常见误区｜这样答会丢分

- **答"加缓存""加负载均衡""加机器"**：老师明确说这类答案偏题。题目要求"通过修改领域模型"，**原因和对策都必须围绕服务边界、实体归属、聚合大小、跨服务通信**。
- 只写对策不写原因，或者不写缺点。**缺点真的没有，也要写**"没有大的缺点"并说明原因。
- 原因写得太笼统，如"设计不合理"。**必须点名：哪个实体在哪个服务里，所以导致什么**。

4.7 画类图与识别共享服务 UML Class Diagram & Shared Services

T2 题（PAST3 Q1(b)）要求用 DDD 方法从用例流程中找出领域对象和关系，画成 **UML 类图** class diagram，再找出可共享的服务。类图常用符号：

符号	含义	写法
类 Class	方框，上格类名，下格属性（考试一般只写类名和关键属性）	Order: orderId, status
关联 Association	实线，两端标**多重性** multiplicity	1、0..1、*、1..*
聚合 Aggregation	空心菱形：整体-部分，部分可独立存在	Poll ◇— Candidate
组合 Composition	实心菱形：部分随整体创建和销毁	Order ◆— OrderLine
泛化 Generalization	空心三角箭头指向父类	Patient —▷ User

◆ 要点 | 必背结论 **从用例描述到类图和共享服务：五步**

1. 逐条读用例流程，圈出**名词**（候选实体）和**动词**（候选关系）。
2. 删掉只是属性的名词（如"日期""数量"）和外部系统；合并同义词，统一命名（通用语言）。
3. 画出实体和关系，标上多重性；有"是一种"关系的用泛化。
4. 按功能内聚把实体分组，每组画成一个 `<<service>>` 框（限界上下文）。
5. **共享服务 = 被多个应用或用例共同使用的服务**，通常是通用域和支撑域，如用户与身份、通知、支付、订单、目录、配送跟踪、报表。

以课程工作坊 Maid2Order（连接需要家政服务的客户与家政中介）为例：实体有 Customer、Agency、AgentAdmin、Maid、Order、Bill、Payment；可分为 UserManager（Customer、AgentAdmin）、AgencyManager（Agency、Maid）、OrderManager（Order）、BillingManager（Bill、Payment）；其中 UserManager、BillingManager 和通知服务可以复用到其他平台。

4.8 白话讲解与小测验 Plain-language Notes & Drills

◆ 要点 | 必背结论 **几个难词的白话版**

- **耦合**：A 改了，B 是不是也得跟着改、跟着重新测。要动的越多，耦合越高（串联彩灯一坏全灭，并联互不影响）。复用规模看"谁能用它"，不是看耦合。
- **限界上下文 BC**：带边界的语境。边界内一个词只有一个意思："订单"在下单这边是"买什么、花多少钱"，在配送那边是"从哪取、送到哪"。考试图里每个 <<service>> 框就是一个 BC。
- **通用域**：各行各业都要、和具体业务无关（通知、支付、身份认证）。理由不要写"简单"，支付一点都不简单。

■ 定义 | 课件正式说法 **"慢"类选对策：搬到一起，还是留一份副本**

问自己	偏向"搬到一起"	偏向"留副本"
这个数据晚几秒更新会不会出事?	会：库存、余额、座位	不会：商品描述、统计数、推荐
有没有"必须同时成立"的规则?	有：库存不能超卖	没有
数据是只读的吗?	要被改：别用副本	只读：适合副本
搬过来后服务会不会杂得离谱?	不会才适合搬	会，就用副本

选副本的前提是规则允许短暂延迟。反例：读者刚欠款，副本没更新，他还能借到新书，规则被绕过。题目没说清时，开头写一句前提（Assume the page only displays data.），选一个主答案，另一种做法最多补一句。

✎ 小测验｜先自己答，再看参考答案 **小测验 1：四个领域模型问题：各属于哪一类？写出原因、对策、缺点**

问题类别：A 一起使用的实体散在不同服务，每次都要跨服务调用（慢）；B 聚合太大，加载一个就要载入大量子对象（内存高）；C 负载差别很大的功能挤在同一个服务（无法独立扩展）；D 必须同时满足的规则跨了服务或聚合（数据不一致）。每个场景写出类别、原因、对策（改领域模型）、缺点。

1. 电商商品页要同时显示商品、价格、库存，三者分别在 ProductManager、PricingManager、InventoryManager，**打开很慢**。规定：**页面显示的库存必须实时准确**。
2. 网银的 Account 聚合下挂着几百万条交易流水、上千份月度账单，以及常用收款人（**规定不超过 5 个**），**加载一个账户内存很高**。
3. 直播平台的 LiveManager 里同时有 Stream（开播高峰负载极高）和 StreamerProfile（主播资料，很少变），**开播时没法单独给 Stream 扩容**。
4. 优惠券规定**最多使用 100 次**；Coupon 在 CouponManager，使用记录 Redemption 在 OrderManager，活动时**出现超发**。

▾ **参考答案**

#	类别	原因	对策	缺点
1	A 慢	显示一页要跨三个服务调用	把总是一起读的 Price、Stock 迁入 ProductManager；库存要求实时准确，不能用晚几秒的副本	ProductManager 变大、内聚下降、事务范围变大
2	B 内存高	一个聚合挂着海量子对象	Transaction、Statement 拆成独立聚合，只用 accountId 引用；Beneficiary 留在 Account（"不超过 5 个"是必须随时成立的规则）	读关联数据要多次查询，跨聚合更新只能最终一致
3	C 无法独立扩展	高负载和低负载功能在同一服务	只把高负载的 Stream 拆成 StreamManager	服务间通信增多，一致性更难保证
4	D 数据不一致	"最多 100 次"要同时看 Coupon 和 Redemption，却分在两个服务	Redemption 迁入 CouponManager，与 Coupon 同一聚合，由聚合根检查上限	聚合变大，热门时并发冲突和重试增多

✎ 小测验｜先自己答，再看参考答案 **小测验 2：订单跟踪页打开慢、高峰超时：位置数据在另一个服务里**

外卖平台：OrderTracking 是一个独立服务，负责订单跟踪页，它**只读取骑手位置 Location**，不修改。Location 由骑手 App **每 5 秒上报到 DeliveryManager** 服务。现在跟踪页每次打开都要跨服务读取 Location，高峰时**打开慢、经常超时**。**位置晚几秒显示业务上可以接受**。写出原因、对策（改领域模型）、缺点。

▾ 参考答案

✍ 参考答案｜可照着写，名字和数字换成新案例的

Cause: OrderTracking reads Location, which is in DeliveryManager, on every page load. Each read is an inter-process call, which is much slower than an in-process call and queues up at peak time.

> 原因：OrderTracking 每次打开页面都要读取 DeliveryManager 里的 Location，每次都是跨进程调用，比进程内调用慢得多，高峰时排队。

Countermeasure: keep a read-only copy of Location in OrderTracking, updated by a LocationUpdated domain event published by DeliveryManager. Moving Location itself is not better, because DeliveryManager writes it every 5 seconds.

> 对策：在 OrderTracking 里保存 Location 的只读副本，由 DeliveryManager 发布 LocationUpdated 领域事件来更新。把 Location 整个搬过去并不更好，因为 DeliveryManager 每 5 秒都要写它。

Drawback: the data is duplicated, and the copy is only eventually consistent, so the displayed position may be a few seconds old, which the business accepts.

> 缺点：数据冗余，副本只是最终一致，显示的位置可能晚几秒，这在业务上可以接受。

✎ 小测验｜先自己答，再看参考答案 **小测验 3：宠物寄养 App：画类图并找出共享服务**

用例：主人登记宠物，选寄养店和日期，系统**检查有没有空笼位**，生成寄养订单，主人在线**支付押金**；寄养期间店员每天**上传宠物照片**，系统推送通知；结束后主人给寄养店**写评价**。

要求：① 列出候选实体，说明删掉了哪些、为什么；② 写出至少 4 条关系和多重性；③ 分成几个服务及依据；④ 哪些做成共享服务（共享服务 = **换一个产品**，比如外卖平台，也能原样复用的服务）。

▾ 参考答案

① 候选实体：主人、宠物、寄养店、日期、笼位、寄养订单、押金、照片、店员、通知、评价、支付。删掉：日期、押金（是属性，押金金额放进订单）；通知（交给通知服务，不画实体）。保留：Owner、Pet、Shop、Kennel（笼位）、BoardingOrder、Photo、Staff、Review、Payment。

② 关系：Owner 1—* Pet；BoardingOrder *—1 Pet；BoardingOrder *—1 Shop；BoardingOrder 1—* Photo；BoardingOrder 1—* Payment；Shop 1—* Kennel；Shop 1—1..* Staff；Shop 1—* Review。宠物和店通过订单联系。"订单 *—1 宠物"读作：一只宠物可以有很多张订单，一张订单只对应一只宠物。

服务	实体	依据
UserManager（共享）	Owner	用户与身份，各行业都要
BookingManager	BoardingOrder、Pet、Photo	围绕一次寄养，总一起用
ShopManager	Shop、Kennel、Staff、Review	围绕店的运营和口碑
PaymentManager（共享）	Payment	支付，各行业都要
NotificationManager（共享）	不画实体	通知交给它

④ 判断共享：用户、支付、通知换到外卖平台也能原样用；带笼位、宠物的不能。

✎ 小测验｜先自己答，再看参考答案 **小测验 4：新订单服务要接入一个模型很乱的老会员系统：照用还是加一层转换？**

新的订单服务要读取公司 **15 年前的老会员系统**。老系统的"会员"**模型很乱**：一个字段既存会员等级又存账号状态，**字段名也和订单团队的叫法不一样**。订单服务应该直接照用老模型，还是在自己这边加一层转换？两种做法分别叫什么，各有什么后果？

▾ **参考答案**

直接照用 = 遵奉者（Conformist）：省事，但老系统的混乱概念会"污染"订单服务，老系统一改，订单服务跟着改。**加一层转换** = 防腐层（Anticorruption Layer）：在订单服务这一侧把老模型翻译成自己的模型，保护自己的通用语言，老系统变化只影响这一层；代价是多写、多维护一层。本例老模型很乱，推荐防腐层。两者都是**下游**这一侧的做法。

❝ 可用句式｜___ 处填案例内容

- ___ is in ___Manager and ___ is in ___Manager. Showing one page needs inter-process calls, which are much slower than in-process calls.
  > ___ 在 ___Manager，___ 在 ___Manager。显示一个页面就要跨进程调用，比进程内调用慢得多。
- The aggregate is too large, so loading one loads the whole object graph into memory. Split it into small aggregates that reference each other only by ID.
  > 聚合太大，加载一个就把整个对象图载入内存。拆成小聚合，彼此只用 ID 引用。
- The rule "___" is an invariant that must always hold, so these entities must stay in the same aggregate.
  > "___" 这条规则是任何时候都必须成立的不变量，所以这些实体必须留在同一个聚合里。
- No major drawback. Each entity can now evolve and scale with its own service.
  > 没有大的缺点。每个实体现在都能随所属服务独立演进和扩展。

本章术语表 Glossary

中文	English	一句话解释
领域驱动设计	Domain-Driven Design (DDD)	以领域模型驱动设计、与领域专家协作的方法
限界上下文	Bounded Context (BC)	领域模型和术语保持一致的明确边界
通用语言	Ubiquitous Language (UL)	一个 BC 内业务和技术人员共用的语言
子领域	Subdomain (Core / Supporting / Generic)	领域的一部分：核心域 / 支撑域 / 通用域
大泥球	Big Ball of Mud	没有边界、模型互相纠缠的系统
功能内聚	Functional Cohesion	按功能把相关实体放在一起
实体	Entity	有唯一标识、可变的领域对象
值对象	Value Object	无标识、不可变、只看值的对象
聚合 / 聚合根	Aggregate / Aggregate Root	一致性边界内的一组实体 / 其唯一入口
业务不变量	Business Invariant	任何时候都必须成立的业务规则
最终一致性	Eventual Consistency	数据暂时不一致，经过一段时间后一致
贫血模型	Anemic Domain Model	只有 getter/setter、没有业务行为的模型
仓储	Repository	负责一个聚合持久化的组件
领域事件	Domain Event	业务上有意义的已发生事件，过去时命名
上下文映射	Context Mapping	两个限界上下文之间的集成关系

中文	English	一句话解释
防腐层	Anticorruption Layer	下游把上游模型翻译成自己的模型
开放主机服务	Open Host Service	上游为所有下游提供的统一接口协议
幂等接收者	Idempotent Receiver	重复收到同一消息结果不变的接收方
至少一次投递	At-Least-Once Delivery	保证消息最终送达，可能重复
共享服务	Shared Service	被多个应用复用的服务
多重性	Multiplicity	关联两端的数量，如 1、0..*

CHAPTER 05

API 设计与 API Gateway

API Design, Versioning & API Gateway

课件来源：PE 03 API Design · PE 04 Reusable Services and API Gateway · CNSD 07（网关与版本）　　对应题型 **T3**（每套必考）

一句话 **好的 REST API 用名词表示资源、用 HTTP 动词表示动作、版本不写死在 URI 里，并且从调用方的需要出发设计（outside-in）。**

学完本章你能

- 说出 API 设计原则和 REST 约束
- 为每个操作选对 HTTP 动词和状态码
- 判断一组 API 处于 RMM 第几级并改到第2、3级
- 比较三种版本管理方法
- 分析 API 延迟原因并给出重构方案
- 说明 API 网关的作用

5.1 API 产品思维与设计原则 API Product Mindset

API（应用程序编程接口 Application Programming Interface）让软件之间相互通信。对平台来说 API 本身就是产品：Salesforce 约 50%、eBay 约 60% 的收入来自 API 生态。课件强调**由外而内** outside-in 的 API 策略：从 API 的使用者（开发者）和最终用户的需要出发设计，而不是把内部实现直接暴露出去（inside-out）。

原则	要点
开发者同理心 Developer Empathy	以用户为中心、简单、适合用途、根据反馈改进
粒度 Granularity	可复用；识别影响服务的关键实体；在最低的实用层次设计；遵循 SOLID 中的单一职责 SRP 和接口隔离 ISP
端点稳定 Endpoint Stability	API 可用性与平台一致；版本必须向后兼容；各版本服务水平要明确
安全 Security	安全级别与数据敏感度相符
文档 Documentation	遵循行业标准，如 **OpenAPI 规范**（Swagger）

5.2 API 风格 API Architectural Styles

风格	特点	适合
REST	资源 + HTTP 动词，无状态，可缓存	绝大多数 Web/移动 API
SOAP	XML 消息、严格契约、内置安全标准	银行、金融等企业级安全场景
GraphQL	**一个端点**，客户端用查询语言精确指定要哪些字段；客户端驱动	数据关系复杂、前端需求多变
gRPC	基于 Protocol Buffers 的二进制 RPC，性能高	微服务之间的低延迟调用
WebSocket	双向长连接，服务器可主动推送	聊天、实时更新、多人游戏
Webhook	事件发生时服务器回调客户端提供的 URL	自动化任务、事件通知

5.3 REST 的六个约束 RESTful Guiding Principles

1. **客户端-服务器** Client-Server：关注点分离，两边可独立演进。
2. **无状态** Stateless：每个请求都包含所需的全部信息，服务器不保存客户端的会话状态（应用状态由客户端持有）。好处：可以把 API 部署到多台服务器上，支撑海量并发用户。

3. **可缓存 Cacheable**：响应要标明能否缓存。缓存可以放在多个层次：客户端、CDN（如 CloudFront）、API 网关、应用层、ElastiCache/DAX。
4. **统一接口 Uniform Interface**：用 URI 标识资源；通过表述 representation（如 JSON）操作资源；消息自描述；**超媒体作为应用状态引擎 HATEOAS**，即响应里带链接，告诉客户端下一步能做什么。
5. **分层系统 Layered System**：客户端不知道中间有没有代理、网关、负载均衡。
6. **按需代码 Code-On-Demand**（可选）：服务器可以下发脚本给客户端执行。

5.4 设计最佳实践：资源、动词、状态码 Best Practices

- **资源用名词，不用动词**：`/customers` 而不是 `/getCustomers`；名词用**复数**（单例资源除外）。
- **用路径表示层级关系**：`/polls/{poll-id}/candidates`；路径用来标识资源，**查询参数**用来过滤、排序、分页：`/users?role=admin`、`/products?sort=price&order=desc`、`/products?page=2&limit=50`。
- 已经在路径里的信息（如 `{voter-id}`），**请求体里不要再重复传**。

方法	用途	安全	幂等	书店例子（方法 + URI + 返回码）
GET	读取资源的表述，**不能改变服务器状态**	是	是	`GET /books/7` → 200 看书详情
POST	创建新资源或子资源；或触发一个**会改变状态的动作**（提交、邀请、开始投票）	否	否	`POST /orders` → 201 新建订单
PUT	整体替换已有资源（不存在时可创建）	否	是	`PUT /parcels/P1` 要传全部字段；只改一个字段别用 PUT
PATCH	部分更新资源（只传要改的字段）	否	否*	`PATCH /orders/5` 体 {"phone": "..."} 只改电话（电话不放 URL）
DELETE	删除资源	否	是	`DELETE /orders/5` → 204；要保留取消记录就用 PATCH 改 status

安全 safe = 不产生副作用；幂等 idempotent = 重复多次和执行一次结果相同。*PATCH 可以设计成幂等。支付这类 POST 接口要防止重复扣款，常用做法是客户端带一个**幂等键 idempotency key**，服务器据此识别重复请求。

状态码	含义	状态码	含义
200 OK	成功	**400 Bad Request**	请求格式或参数错（客户端错误 4xx）
201 Created	已创建，响应带新资源位置	**401 / 403**	未认证 / 无权限
202 Accepted	**已接收，稍后异步处理**	**404 / 409**	资源不存在 / 冲突
204 No Content	成功但无返回内容	**429 Too Many Requests**	超过限流
1xx / 3xx	信息 / 重定向（302 临时移动）	**500 / 503**	服务器错误（5xx）/ 暂不可用

5.5 Richardson 成熟度模型 Richardson Maturity Model (RMM)

RMM 用四个等级衡量一组 API 有多"RESTful"。课件用"点外卖"打比方：

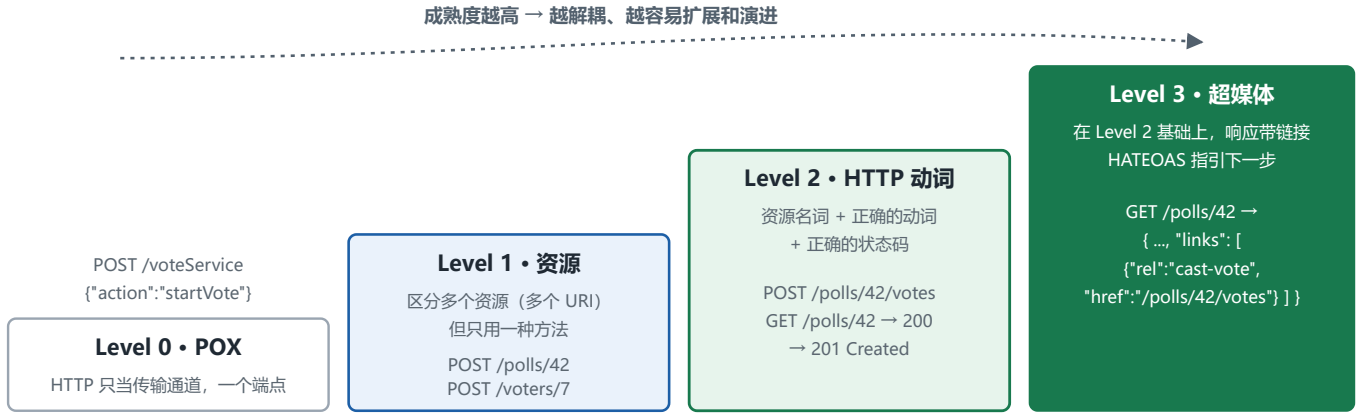


图 5-1 RMM 四个等级（以投票 API 为例）

等级	判断标准	点外卖比喻	看到这样的 API 就是这一级
Level 0	只有一个 URI、一种方法（像 SOAP / XML-RPC），HTTP 只是传输工具	镇上只有一家店，什么都跟它点	`POST /api`：一个地址做所有事
Level 1	有多个资源 URI，但所有操作都用同一个方法（全用 POST 或全用 GET）	去不同的店点不同的菜，但只会"点"这一个动作	`POST /orders/get`、`POST /orders/cancel`：多个地址、只用 POST
Level 2	资源名词 + 正确使用 GET/POST/PUT/DELETE + 正确的状态码。**GET 只读，改变状态用 POST/PUT/PATCH/DELETE**	能指定配料、能取消订单	`GET /orders/5`、`DELETE /orders/5`、`PATCH /customers/9`
Level 3	Level 2 + 响应里返回超媒体链接，客户端按链接导航	点完餐，服务员告诉你接下来可以加饮料或结账	响应里带 {"rel": "cancel-order", "href": "/orders/5"}，且随状态变化

5.6 版本管理 API Versioning

版本号用**语义化版本** Semantic Versioning：**MAJOR**（不兼容的修改）. **MINOR**（向后兼容的新功能）. **PATCH**（向后兼容的修复）。三条规则：①不要删掉已有的东西；②不要改变已有的处理规则和含义；③新增的内容必须是可选的。

方法	写法	优点	缺点
URI 版本	`/v1/votes`	简单直观	版本废弃时 URI 跟着变，管理不好**客户端代码容易出错**
自定义请求头	`Accept-version: 1.1`	**URI 保持干净稳定**，版本协商放在请求头	不能在浏览器地址栏直接看出版本
Accept 头（媒体类型）	`Accept: application/vnd.app.v1+json`	标准的内容协商方式，URI 稳定	略复杂

▶ 案例演练 | 演示怎么套用 **PAST Q1(c)（6分）：Invite Next Voter 与 Start Vote 两个 API**

原设计：`GET /v1.1/votes/{voter-id}/invite`（输入 Voter identifier，输出 List of voters in the polling room）和 `GET /v1.1/votes/{voter-id}/start`（输出候选人列表和房间里的选民列表）。三个小问各 2 分：

1. 是否符合 RMM Level 2? 不符合。两个操作都会**改变服务器上资源的状态**（邀请选民、让选民进入投票模式），却用了 GET。Level 2 要求按语义使用 HTTP 方法，**这里应该用 POST**。

2. 更好的版本策略? URI 版本在版本废弃时会让客户端代码出错；改用**自定义请求头**（如 `Accept-version`），**保持 URI 干净，版本协商在请求头里完成**。

3. 改正后的接口:

URI	Service	Inputs	Outputs	Explanation	HTTP
/votes/invite	ProctoringManager	（无：由系统从队列取下一位）	Voter-id	邀请下一位选民进入房间核验	**POST**
/votes/{voter-id}/start	ProctoringManager	Voter identifier	List of candidates of the poll	让选民进入投票模式	**POST**

改动要点：**去掉 URI 里的版本号；GET → POST；删掉多余的输入**（invite 不需要传 voter-id）和多余的输出（选民列表）。

5.7 API 性能：延迟从哪来、怎么改 API Performance

大量客户端同时调用时，API 常见的延迟原因和对应的重构手段：

延迟原因	重构手段
同步处理：每个请求都等后端处理完才返回	**异步接收**：接口收到后放入消息队列或事件流，立即返回 `202 Accepted`，由后台消费者处理
请求太碎：每次更新一个请求，连接和协议开销大	**批量接口**：一次请求提交多条更新；或改用事件流（Kafka、Kinesis）持续摄取
每次都整体替换资源（PUT 整个对象）	用 **PATCH 部分更新**，或只返回部分字段 partial response
大量并发写同一行/表，锁竞争	按键分区、写入追加日志再汇总（第11章）
响应体太大、重复读	分页（offset 或 cursor）、gzip 压缩（小响应不值得压）、各层缓存
下游慢或失败拖垮调用方	超时、带指数退避的重试 retry with exponential backoff、熔断、舱壁、降级（第6、13章）

5.8 API 安全 API Security

校验参数和请求体（大小、JSON Schema）、威胁检测、**全程 TLS** SSL everywhere、限流和配额 rate limiting and quotas。认证与授权用成熟方案：**OAuth 2.0** 是授权的行业标准（框架）；**JWT** 是 JSON 格式的令牌；**SAML** 是单点登录标准；浏览器跨域调用要配置 **CORS**（跨域资源共享）。

5.9 API 网关与 SDK API Gateway & SDKs

拆成微服务后会出现两个问题：①客户端要同时调用多个服务才能完成原来一次调用就能做的事；②客户端要知道每个服务的位置。**API 网关** API Gateway 同时解决这两个问题：它是所有外部调用的统一入口，接收请求、调用所需的微服务、把结果聚合后返回。

API 网关的职责

- 路由：服务端发现的"路由器"，查服务注册中心
- 聚合多个服务的结果，减少往返
- 协议和消息格式转换
- 认证授权、限流、配额
- 缓存、监控、日志
- 为不同客户端提供最合适的 API（BFF）

常见产品

- Amazon API Gateway（创建、发布、维护、监控、保护任意规模的 API）
- Azure API Management、Google Apigee
- Kong、Spring Cloud Gateway、Netflix Zuul

网关管**南北向**流量（用户↔平台）；服务网格管**东西向**流量（服务↔服务），见第9章。

SDK 与客户端库：平台常为 API 提供各语言的 SDK（如 AWS SDK for Java/Python/JavaScript），让调用方像调本地函数一样使用服务。Spring Cloud 的 **Feign** 能根据接口声明自动生成 REST 客户端。指导原则：把处理传输协议的代码和目标服务本身的逻辑分开；由客户端决定何时升级客户端库。

✎ 考试怎么考｜题型•分值•套路

T3 每套必考，6 分，三种问法：①给出一张有错的 API 表，判断是否符合 RMM Level 2、提出版本策略、重画正确的表（PAST）；②解释大批量并发更新时的延迟原因，重构 API 并举具体例子（PAST2）；③从共享服务中选一个，设计 RMM **Level 3** 的 API，写出 URI、HTTP 动词、输入输出和每个操作的用途（PAST3）。**Level 3 一定要在输出里写出链接（rel + href）**。

✕ 常见误区｜这样答会丢分

- 把动词写进 URI，如 `/getVoter`、`/createOrder`。动作用 HTTP 方法表达，URI 只放名词（`/votes/{id}/start` 这类子资源动作是可接受的折中）。
- 会改变状态的操作用了 GET。**GET 必须是安全、幂等、可缓存的**。
- 改表时只改动词，忘了删多余的输入输出、忘了去掉 URI 里的版本号。

5.10 白话讲解与小测验 Plain-language Notes & Drills

■ 定义｜课件正式说法 **rel 和 href 是什么、links 写什么**

- **rel** = relation：这条链接和当前资源是什么关系，也就是"能拿它干什么"（cancel-order、ship）；**href** = hypertext reference：链接地址。
- Level 3 的响应就是"**当前可以访问的链接目录**"，像命令行的 help。客户端按链接走，不用把地址写死，服务端改地址也不会弄坏客户端。
- links 三步：① 每个响应都加 self；② 其他操作逐个问"当前状态允许吗"，允许才加；③ 加相关资源（事件指回 order、列表加 next）。
- 链接只暴露客户端本来就能调用、且该用户有权限做的操作；请求仍经网关路由，不是额外暴露内部结构。

✎ 小测验｜先自己答，再看参考答案 **小测验 1：书店 API：写出方法和 URI**

为网上书店写出下面每个操作的 HTTP 方法和 URI：

1. 看 7 号书的详情。
2. 新建一个订单。
3. **只修改 5 号订单的收货电话**。
4. 取消 5 号订单。
5. 有人把"取消订单"设计成 `GET /orders/5/cancel`，错在哪？**至少说两点**。

▾ **参考答案**

1. `GET /books/7` → 200（复数名词、以 / 开头）
2. `POST /orders` → 201
3. `PATCH /orders/5`，请求体 {"phone": "..."}；电话属于个人信息，不放在 URL 里
4. `DELETE /orders/5` → 204；要保留取消记录就用 `PATCH /orders/5` 把 status 改成 cancelled
5. ① 会改变状态却用了 GET（GET 必须安全、只读，可能被缓存或预取误触发）；② URI 里带了动词 cancel，动作应交给 HTTP 方法

✎ 小测验｜先自己答，再看参考答案 **小测验 2：这组订单 API 处于 RMM 第几级？怎么改到 Level 2？**

Richardson 成熟度模型（RMM）：Level 0 = 所有操作走同一个入口（一个 URI）；Level 1 = 按资源分出不同的 URI；Level 2 = 正确使用 HTTP 方法和状态码；Level 3 = 响应里带上下一步的链接（HATEOAS）。某团队的订单 API 是这样设计的：`POST /orders/get` 查订单、`POST /orders/cancel` 取消订单、`POST /customers/update` 改客户资料，**所有请求都返回 200**。判断它在第几级，并改到 Level 2（写出方法、URI 和状态码）。

▾ **参考答案**

Level 1：已经按资源分了不同的地址（/orders、/customers），所以不是 Level 0（Level 0 是所有操作走同一个入口）；但所有操作都用 POST、都回 200，没用好 HTTP 方法和状态码，所以到不了 Level 2。

操作	Level 2 写法	状态码
查订单	`GET /orders/5`	200；不存在 404
取消订单	`DELETE /orders/5`，或 `PATCH /orders/5` 改 status	204 / 200；已发货不能取消 409
改客户资料	`PATCH /customers/9`	200；参数错 400

✎ 小测验｜先自己答，再看参考答案 **小测验 3：订单 API 的五个改动，版本号各怎么升？**

团队按语义化版本（MAJOR.MINOR.PATCH）维护订单 API，外部合作方依赖它。下面五个改动彼此独立，分别判断改动后的新版本号：

1. 当前 1.0.0，给订单响应**新增一个可选字段**。
2. 当前 1.0.0，修复一个金额舍入 bug，**接口格式不变**。
3. 当前 1.0.0，**删除响应里的一个旧字段**。
4. 当前 1.1.0，下单接口**新增一个必填参数**。
5. 当前 1.1.0，**新增一个"查询物流"接口**。

▾ **参考答案**

1. 1.1.0（向后兼容的新功能升 MINOR）
2. 1.0.1（兼容的修复升 PATCH）
3. 2.0.0（删字段会让旧客户端出错，升 MAJOR）
4. 2.0.0（旧请求缺这个参数会失败，不兼容）
5. 1.2.0（在当前 1.1.0 上加新功能）

✎ 小测验｜先自己答，再看参考答案 **小测验 4：为书店订单设计 RMM Level 3 的 API**

书店订单的状态：**已付款** → **已发货** → **已签收**；已付款的订单也可以变成**已取消**。规则：**只有"已付款、未发货"的订单能取消、能改收货地址**。管理后台需要五个操作：订单列表、订单详情、改收货地址、取消订单、记录物流事件（如已发货、已签收）。

每个操作写出 URI、HTTP 方法、输入、输出状态码，并说明响应里的 links（rel + href）**在不同订单状态下有哪些**。

▾ **参考答案**

操作	URI 与方法	输入 / 状态码	links
列表	`GET /orders?status=paid`	200	self、每张订单的详情、分页 next
详情	`GET /orders/{no}`	200；不存在 404	随状态变：已付款 → self、cancel、update-address、ship；已发货 → self、deliver（记录签收的物流事件）；已签收 / 已取消 → self
改地址	`PATCH /orders/{no}`	address → 200；已发货 409	与"已付款"相同：self、cancel、update-address、ship
取消	`PATCH /orders/{no}` 把 status 改成 cancelled（或 DELETE）	200 返回已取消的订单；已发货 409	self（若用 DELETE 返回 204，则没有响应体、不带 links）
物流事件	`POST /orders/{no}/events`	eventType、timestamp、trackingNo → 201	self、order

要点：链接只给出"当前状态允许"的操作，客户端照着链接走，不用自己写死状态规则。PUT 是整体替换，只改地址要用 PATCH。

◆ 要点｜必背结论 **事件流、分区、分区键：车道比喻**

事件流是只追加的日志，读过不删、按顺序排着，多个消费者可以各读一遍。**分区**像几条固定的车道；**分区键**（如包裹 ID）决定事件进哪条车道：算哈希再对车道数取余，所以很多个键共用一条车道，不会"分得太细"。同一车道同一时间只有一个 worker 读，所以有序；不同车道并行。分区键选"顺序必须一致的那个对象"的 ID。

✎ 小测验｜先自己答，再看参考答案 **小测验 5：几百把扫码枪更新包裹，高峰要等 20 多秒：原因和重构**

仓库里几百把扫码枪调用 `PUT /parcels/{parcel-id}`，**请求体是包裹的完整信息**（十几个字段）。实际上**每次扫码只是更新包裹所在环节**（入库、分拣、装车），其余字段不变。服务器收到请求后**先重算整个仓库的库存统计**，再返回结果，**高峰时要等 20 多秒**。写出原因、重构方案，并给一个具体请求的例子（方法、URI、请求体、返回码）。

▾ 参考答案

✍ 参考答案｜可照着写，名字和数字换成新案例的

Reasons: (1) the API is synchronous: it recomputes the inventory statistics of the whole warehouse before replying, so concurrent scans queue up. (2) PUT replaces the whole parcel with all its fields on every scan, although only the stage changes.

> 原因：(1) 同步：回复前要重算整个仓库的库存统计，并发扫码排队；(2) 每次扫码都用 PUT 整体替换包裹所有字段，其实只变了环节。

Refactoring: make it resource-oriented and asynchronous. The scanner POSTs each scan as an event under the parcel; the API validates it, publishes it to an event stream partitioned by parcel ID and returns 202 Accepted; one worker per partition updates the statistics incrementally (only the counts of the old and new stage), instead of recomputing the whole warehouse.

> 重构：面向资源 + 异步。扫码枪把每次扫码作为包裹下的事件 POST 上来；API 校验后发布到按包裹 ID 分区的事件流，立即返回 202；每个分区一个 worker 增量更新统计（只改动前后两个环节的计数），不再重算整个仓库。

Example: `POST /parcels/P8841/scan-events` with {"scannerId": "G17", "stage": "loaded", "timestamp": "09:15:30"} → **202 Accepted**.

> 例子：方法 + URI + 请求体 + 返回码四项写全。

按包裹 ID 分区（不是扫码枪 ID）：同一个包裹会被不同的枪扫到，按包裹分区才能保证它的事件按顺序处理。只改环节也可以用 PATCH，但根本原因是同步重算。

✎ 小测验｜先自己答，再看参考答案 **小测验 6：200 个合作方在调用 /v1/orders，v1 半年后下线：怎么做?**

公司的订单 API 被 **200 个外部合作方**调用，**版本写在 URI 里**（`/v1/orders`），**v1 计划半年后下线**。请从两方面回答：① 下线那天如果什么都不做会发生什么，应该走怎样的下线流程；② 版本标识写在 URI 和写在请求头，各有什么优缺点。

▾ 参考答案

① **下线流程：**直接停掉 v1，仍在用的合作方会全部报错。应：v1、v2 并行运行一段时间；提前公告弃用日期，并在 v1 响应里加 Deprecation / Sunset 头；按合作方统计 v1 调用量，逐个通知、提供迁移指南；调用量降到接近零、过了约定日期再下线。

② **标识方式：**URI 版本（`/v1/orders`）直观、好调试、好缓存，但版本一变地址就变；请求头版本（如 `Accept-version: 1.1`）让 URI 保持稳定，但调试不直观、要约定默认版本。无论哪种，下线都要靠上面的流程，换成请求头本身不能避免旧客户端出错。

❝ 可用句式｜___ 处填案例内容

- The API is synchronous, so each request waits for ___ to finish and requests queue up under high load.
  > 这个 API 是同步的，每个请求都要等 ___ 完成，高负载时请求排起长队。
- Make the API resource-oriented and asynchronous. Each ___ sends POST /___/{id}/___; the API validates the request, publishes it to an event stream partitioned by ___, and returns 202 Accepted immediately.
  > 把 API 改成面向资源和异步。每个 ___ 发送 POST /___/{id}/___；API 校验后发布到按 ___ 分区的事件流，立即返回 202 Accepted。

本章术语表 Glossary

中文	English	一句话解释
由外而内	Outside-in Design	从 API 使用者的需要出发设计
开发者同理心	Developer Empathy	站在调用方角度让 API 简单好用
粒度	Granularity	一个 API 承担职责的大小
无状态	Stateless	服务器不保存客户端会话状态
统一接口	Uniform Interface	用 URI 标识资源、用表述操作资源
超媒体作为应用状态引擎	HATEOAS	响应中带链接指引下一步操作
幂等	Idempotent	执行多次与执行一次结果相同
安全方法	Safe Method	不改变服务器状态的方法，如 GET
幂等键	Idempotency Key	客户端生成、用来识别重复请求的唯一键
理查森成熟度模型	Richardson Maturity Model (RMM)	衡量 RESTful 程度的 0–3 级模型
语义化版本	Semantic Versioning	MAJOR.MINOR.PATCH 的版本规则
向后兼容	Backward Compatible	新版本不破坏旧客户端
OpenAPI 规范	OpenAPI Specification (OAS)	描述 REST API 的厂商中立标准
部分响应 / 部分更新	Partial Response / PATCH	只返回 / 只修改需要的字段
指数退避重试	Retry with Exponential Backoff	失败后等待时间成倍增加再重试
授权框架	OAuth 2.0	授权的行业标准
JSON 网络令牌	JWT (JSON Web Token)	JSON 格式的签名令牌
跨域资源共享	CORS	允许浏览器跨域请求资源的机制
API 网关	API Gateway	外部调用的统一入口：路由、聚合、安全、限流
面向前端的后端	Backend for Frontend (BFF)	为某类客户端定制的网关/后端
软件开发工具包	SDK	封装 API 调用的客户端库

CHAPTER 06

平台管理与可观测性

Platform Management & Observability

课件来源：PE 05 Platform Management · PE 05b Logging & Monitoring Workshop · PE 05c Deployment Case Study · ArchSS 07（指标采集）

对应题型 **T8**

一句话 **平台运行起来之后，要靠一套工具箱让交互"扩得开、限得住、不中断、看得见"：服务发现、部署策略、熔断限流、分布式追踪、指标和日志。**

学完本章你能

- 说出平台工具箱每个工具对应的业务目的
- 比较蓝绿、滚动、金丝雀等部署策略
- 解释熔断器的三种状态和 Saga 的两种实现
- 区分基础设施指标和应用指标
- 完整设计一个指标：谁采集、多久一次、存多久、存哪里

6.1 平台管理工具箱 The Platform Toolbox

平台工程师关心的是可扩展性、可用性、可演进性和数据（第3章）。课件把工具箱和业务目的一一对应起来：

工具	解决的问题	对应的业务目的
服务发现 Service Discovery	服务之间怎么找到对方的位置	**扩大交互**（自动扩缩容时新实例可用）
部署策略 Deployment Strategies	怎样发布新版本、在线试验新功能	**无缝体验**、试验与流量管理
熔断与限流 Circuit Breaker & Rate Limiting	怎样优雅地处理错误、防止被流量冲垮	**限制交互**
分布式追踪 Distributed Tracing	怎样跟踪一个请求经过的所有服务	快速排障，降低恢复时间
指标采集 Metrics Collection	收集交互的度量数据	**数据驱动**的决策
日志聚合 Log Aggregation	集中查询所有组件的日志	在增强功能的同时保持体验

为什么需要这些工具？因为平台从单体走向了微服务。课件的对比：**单体**性能好、运维开销小，但紧耦合、不能按功能细粒度扩展；**SOA** 复用性好，但服务总线会成为瓶颈、数据集中在一个库；**微服务**可极致复用、独立扩展和部署、技术栈多样、边界清晰（"谁开发谁运维"），代价是**显著的运维开销**，而工具箱正是用来应对这份开销的。

6.2 服务发现 Service Discovery

在云环境里，自动扩缩容会不断产生新实例，节点和容器会重启，IP 地址随时在变，硬编码 IP 行不通。解决思路借鉴互联网的 DNS：每个服务有别名，调用方按别名查询位置。**服务注册中心** service registry 本质上就是内部 DNS。

- **三步**：注册 register（服务启动时登记自己）→ 查找 lookup（调用方按名字查位置）→ 连接 connect。
- **东西向流量** East-West：平台内部服务之间；**南北向流量** North-South：用户与平台之间。服务发现主要解决东西向。
- 注册中心自身要高可用：通常是 3、5 个（奇数）节点的集群，用共识协议（Paxos、Raft、ZAB）保持一致，如 **ZooKeeper、etcd、Consul**。Spring Cloud 用 **Eureka**；**Kubernetes 自带 DNS**，按 Service 名就能找到服务，几乎不用写代码。
- **客户端发现** client-side discovery：客户端查注册中心并自己做负载均衡；**服务端发现** server-side discovery：客户端请求负载均衡器（或 API 网关），由它查注册中心并转发。

6.3 部署策略 Deployment Strategies

选部署策略前先问：客户能否容忍几分钟停机（银行、电商不能）？用户是否遍布全球？多久发布一次？能多快回滚？要不要在生产环境做实验（A/B 测试、机器学习模型的被动模式）？

策略	做法	适合	注意
蓝绿部署 Blue/Green	新版本与旧版本同时在线，确认新版本正常后再切流量、关掉旧版本	需要一键切换、快速回滚	同时运行两套，资源翻倍
滚动更新 Rolling Update	逐台用新版本替换旧版本，流量逐步转到新版本	**修复补丁**；无停机、无额外资源	新旧版本须**向后兼容**，便于回滚
金丝雀发布 Canary Release	新版本只处理一小部分请求（常见 95/5、90/10）	**实验新功能、评估新 ML 模型**、观察特定用户群行为；低风险	须增量、向后兼容
影子流量 Shadowing	请求同时发给新旧版本，只返回旧版本的结果	用真实流量测试新版本	新版本不能产生副作用

▶ 案例演练 | 演示怎么套用 **课件部署案例：拆分 Payment 表（不兼容的数据变更怎么上线）**

Payment 服务 V1 把所有支付类型写进一张 Payment 表，V2 要拆成 ChequePayment 和 ApplePayPayment 两张表，这一改动**不向后兼容**。做法是分步小升级，**每步都能回滚**：

1. 只运行建表脚本，新建两张表（滚动更新，无代码变更，回滚只需删表）。
2. 发布 V1.1：新交易写新表，查不到的再读旧表；删除改为打标记 tombstone（金丝雀发布，影响面 5–10%，然后全量）。
3. 把旧数据迁移到新表（V1.1 仍正常工作）。
4. 发布 V1.2：只读新表（金丝雀后全量），最后把 V1.2 标为 2.0。

6.4 熔断、限流与重试 Circuit Breaker, Rate Limiting, Retry

问题：Payment 服务宕机或变得非常慢，调用它的 Agency 服务不断发请求、不断超时，线程池被占满，连调用其他服务都做不到；Agency 的调用方也跟着卡住，形成**级联故障 cascading failure**。



图 6-1 熔断器是一个状态机

- **熔断器**：时间窗口内失败超过阈值就"跳闸"，在一段时间内不再调用下游，直接返回错误（或上次的缓存值、通用答案）；时间到后放少量请求试探。**通过拦截器实现，业务代码几乎不用改**。工具：Resilience4J、Sentinel、Istio 服务网格（Hystrix 已不再维护）。
- **限流 rate limiting**：用户并不平等（可按套餐设置条件限流）；服务资源（CPU/内存/IOPS）和预算有限；不限流等于欢迎 DDoS。拦截器从请求中取出用户名、来源 IP、用户套餐等，查询一个快速的**内存配额库**校验并更新用量。
- **重试**：失败后按**指数退避**（如 50ms、100ms、200ms……）重试，**只对部分请求重试**，避免"重试风暴"。

6.5 分布式追踪与 Saga Distributed Tracing & Saga

一个用户请求要经过很多服务，出了问题很难定位，平均恢复时间 MTTR 会显著增加。**分布式追踪**的做法：在入口（根服务）生成一个 UUID 作为**跟踪 ID**，通过 HTTP 头传给下游所有服务；已有该头就不再生成。术语（OpenTracing 标

准）：**Trace** 是跨多个服务的一次事务，**Span** 是其中一个工作单元（通常对应一个服务）。工具：**Jaeger**（CNCF）、**Zipkin**、Spring Sleuth、Datadog。

分布式事务：一个业务事务常跨多个服务。传统**两阶段提交** 2PC（准备、提交）在平台上不适用：很多数据库不支持 ACID 事务，平台用多种数据库，2PC 要求同步通信，而现代应用以异步为主。替代方案是 **Saga 模式**：

Saga	做法	缺点
核心思想	把业务事务拆成一串本地事务 T1…Tn，**每个都有补偿事务** C1…Cn（如 CreateOrder / CancelOrder、MakePayment / CancelPayment）；某步失败就倒序执行补偿；调用都是异步的	
编排式 Orchestration	编排器掌握全部流程，向各服务发送事务事件和补偿事件	编排器是单点故障和争用点
协同式 Choreography	每个服务自己知道收到某事件后该做什么	逻辑分散，难调试

Saga 很依赖带共享跟踪 ID 的分布式追踪；**它没有读隔离，用户可能读到未完成或已被取消的事务数据**。

6.6 指标 Metrics

"*If you can't measure it, you can't improve it.*"（Peter Drucker）平台要降低成本、让 API 更快、留住客户、满足 SLA，都离不开度量。

类别	例子
基础设施指标 Infra Metrics	CPU 和内存使用率；请求量（IOPS）；应用可用性；高峰时段；平均响应时间（要看离群值和**百分位**）；请求成功率/失败率；请求和响应的大小；垃圾回收时间
应用指标 Application Metrics	**任务成功** Task success：交易是成功完成还是用户中途放弃？**清晰度** Clarity：用户学会和使用产品有多容易（如网页热力图）？**满意度** Happiness：回访用户、访问频率
容量规划指标（ArchSS 07）	系统：磁盘利用率、I/O 等待、内存、CPU、已用存储；Web/应用服务器：每秒请求、活跃会话、繁忙线程；数据库：每秒查询、打开的连接、缓存命中率、主从延迟；业务：如 Flickr 每小时上传照片数、平均照片大小

可用时间 Uptime

(MTBF − MTTR) / MTBF，用"几个 9"衡量：5个9 ≈ 每年停机 5 分钟，4个9 ≈ 50 分钟，3个9 ≈ 9 小时

产出率 Yield

成功请求数 / 总请求数。**比 uptime 更重要**：没有请求时停机不影响任何人

收获率 Harvest

可用数据 / 全部数据：部分数据不可用时，返回的结果有多完整

工具：**Prometheus**（时序数据库）+ **Grafana**（仪表盘）；TICK 栈（Telegraf、InfluxDB、Chronograf、Kapacitor）；Splunk；云厂商的 Amazon CloudWatch、Azure Monitor、Google Cloud Monitoring。

6.7 设计一个指标 Designing a Metric

T8 题常要求你提出指标并说明一整套设计决策。**用下面七个字段逐项回答，不会漏分**：

字段	要回答什么	例：平均身份核验时长	打车例：平均接单等待时间	常见错写法 → 改成
① 名称与定义	测什么、怎么算	从选民进入核验队列到 Proctor 完成核验的平均时间	乘客下单到司机接单的平均时间，按城市、每 5 分钟统计	"每天总订单数"只是使用量 → 测结果（耗时、成功率）
② 理由	它反映哪个质量属性或业务目标	反映可扩展性（核验越快，同一时段能服务越多选民）和易用性	性能和可扩展性：高峰期匹配越快越好	理由空着 → 必须写反映哪个质量属性

字段	要回答什么	例：平均身份核验时长	打车例：平均接单等待时间	常见错写法 → 改成
③ 直接采集还是计算得出	派生指标要说明依赖哪些基础指标	派生：依赖"进入队列时间戳"和"核验完成时间戳"两个事件	派生：依赖"下单时间戳"和"接单时间戳"	"依赖的两个指标"就是公式里的两个原始数据
④ 负责的组件	谁产生原始数据，谁计算和上报	ProctoringManager 发出事件；流处理任务（或定时任务）计算平均值	OrderManager 发下单事件、DispatchManager 发接单事件；流处理任务（Kinesis + Lambda）按订单号关联计算	"日志组件" → 日志只负责记录，不是采集指标的组件
⑤ 频率	多久采集或计算一次	每个事件实时采集；每 1 分钟汇总一次	每个事件实时采集；每 1 分钟汇总	"按需" → 写具体频率
⑥ 保存时长	数据留多久	细粒度数据保留到投票结束后 30 天；汇总值随投票报告长期保存	原始事件 90 天；分钟汇总 90 天；日汇总 3 年	"按需"、"RDS 存全部历史" → 写数字
⑦ 存储位置	存在哪种存储	时序数据库（Prometheus / Amazon Timestream），历史数据转存数据仓库或 S3	Amazon Timestream（汇总）+ S3（原始）；历史分析 Redshift	RDS / ElastiCache → 指标首选时序库；ElastiCache 只放最新值给仪表盘

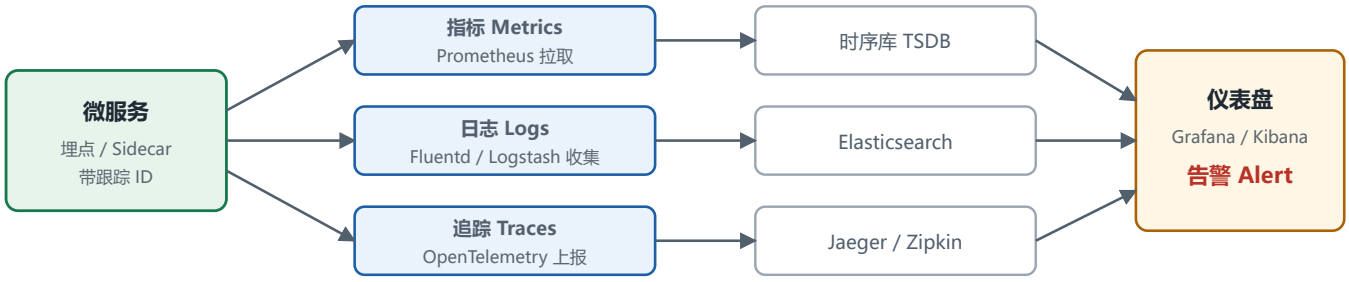


图 6-2 可观测性的三条数据流：指标、日志、追踪

▶ 案例演练 | 演示怎么套用 **PAST Q1(e)（4分）：用"每日处理的选民数"评估可扩展性和易用性，你同意吗?**

不同意。每日处理人数只反映使用量，既不反映高峰时的并发承载能力（可扩展性），也不反映选民完成投票是否顺利（易用性），人数高也可能是排队很久才处理完。**官方认可的指标必须衡量系统的结果（准确投票和使用情况），并论证它的好处**，例如：**平均身份核验时长**（越快越能扩展、越友好）；**每位选民平均投票用时**（衡量主用例的易用性和性能）；**每位选民平均排队时间**（衡量选民体验和投票时的可扩展性）。每个指标连同理由 2 分。

6.8 日志 Logging & Monitoring

工程师大量时间花在调试、分析和性能调优上，日志是基础。**应该记录**：认证/授权的成功和失败；数据实体的变更；访问外部资源（数据库、文件、消息队列）；配置；应用错误；用户的功能使用情况。

组件一多，必须**集中式日志管理**，否则要登录每台机器、每个容器去找日志。三种聚合模式：①应用把格式化好的日志直接发到中心日志服务器；②应用发原始日志给转换服务器，解析后再送中心；③应用写本地文件，轻量代理（如 Filebeat）把原始日志发给重量级转换器，再存入中心。工具：**ELK**（Elasticsearch + Logstash + Kibana）、Kubernetes 中常用的 **EFK**（Fluentd 收集、Elasticsearch 存储、Kibana 展示）、Splunk、AWS 集中日志、Google Cloud Logging。微服务里每条日志都要带**关联 ID** correlation ID，才能把一个请求的日志串起来。

6.9 白话讲解与小测验 Plain-language Notes & Drills

◆ 要点 | 必背结论 **三条数据流各走各的路（图 6-2 的白话版）**

数据	谁收集	存在哪	看什么
指标 metrics	Prometheus 拉取 / CloudWatch	时序库（Prometheus、Timestream）	数字随时间的变化：耗时、成功率
日志 logs	Fluentd / Logstash 收集	Elasticsearch（ELK / EFK），每条带关联 ID	某一次发生了什么
追踪 traces	OpenTelemetry 上报	Jaeger、Zipkin	一个请求经过了哪些服务、卡在哪

日志组件只负责记录；指标要有专门的采集、计算、存储链路。平均值容易被离群值带偏，要看百分位（P90 / P99）。

✎ 小测验 | 先自己答，再看参考答案 **小测验 1：外卖平台想知道"下单流程是否顺畅"，设计一个指标**

假设平台部署在 AWS，可用的组件有：手机 App、各业务服务、流处理（Kinesis + Lambda）、时序数据库（Amazon Timestream）、对象存储（S3）。

外卖平台想知道"**下单流程是否顺畅**"。请设计**一个主指标**（可以另附一个辅助指标），写出：名称与定义、理由（反映哪个质量属性）、产生数据和负责计算的组件、采集和计算频率、保存时长、存储位置。

▾ **参考答案**

主指标：从选好餐厅到下单成功的耗时（看 P90）。**辅助指标：**下单放弃率及放弃在哪一步（漏斗）。**理由：**反映易用性（步骤少、放弃少）和性能（慢也会导致放弃）。**组件：**手机 App 发"开始下单"事件，订单服务发"下单成功"事件；流处理（Kinesis + Lambda）按订单关联两个事件、计算耗时。**频率：**每个事件实时采集，每 1 分钟汇总一次。**保存：**原始事件 30 天，存 S3；分钟汇总 90 天、日汇总 3 年，存 Timestream。

✎ 小测验 | 先自己答，再看参考答案 **小测验 2：打车平台"平均接单等待时间"：按七个字段设计**

假设平台部署在 AWS，可用的组件有：手机 App、各业务服务、流处理（Kinesis + Lambda）、时序数据库（Amazon Timestream）、对象存储（S3）。

打车平台有订单服务 OrderService（乘客下单时创建订单）和派单服务 DispatchService（司机接单时记录）。请为"**平均接单等待时间**"逐项写出**七个字段**：① 名称与定义；② 理由；③ 直接采集还是计算得出（计算得出的，写出它依赖的**两个底层数据：由谁、在什么时候产生**）；④ 负责的组件；⑤ 频率；⑥ 保存时长；⑦ 存储位置。

▾ **参考答案**

字段	参考写法
① 名称与定义	平均接单等待时间 = 司机接单时间 − 乘客下单时间，按城市、每 5 分钟统计；被取消、无人接单的订单单独统计，不计入平均；同时看 P90
② 理由	反映高峰期的性能和可扩展性：匹配越快，乘客体验越好
③ 计算得出	依赖：下单时间戳（OrderService 在乘客下单时产生）、接单时间戳（DispatchService 在司机接单时产生）
④ 组件	两个服务把事件发到 Kinesis；Lambda 按订单号关联两个时间戳并计算
⑤ 频率	每个事件实时采集；每 1 分钟计算一次最近 5 分钟（滑动窗口）的平均值
⑥ 保存时长	原始事件 90 天；分钟汇总 90 天；日汇总 3 年
⑦ 存储位置	原始事件放 S3，汇总值放 Timestream

✎ 小测验｜先自己答，再看参考答案 **小测验 3：平台接入外部第三方 LLM API，对方慢或不稳定：怎么设计、怎么保证响应时间?**

场景（数字为示例）：App 里新增一个问答助手，后台调用外部 LLM API。LLM 有时要 **20 秒以上才返回完整答案**，**约 5% 的调用失败**。需求是"90% 的提问在 **3 秒内让用户看到回应（确认收到或答案的开头）**"。请提出设计，并说明怎么保证这个响应时间。

▾ **参考答案**

先澄清需求：完整答案 3 秒内做不到（LLM 本身就要 20 秒），所以"回应"要定义为确认收到或答案开头，这是设计的前提。外部 API 的延迟是**不可控的约束**，要按约束设计。

手段	做法
流式返回	LLM 分段返回（streaming），用户先看到前几句
异步化	请求先回 202 和 questionId，后台调用 LLM，结果推送（WebSocket / 通知）
超时 + 重试	设超时；指数退避重试；只重试幂等请求
熔断 + 降级	失败率高时熔断，返回缓存的答案或"请稍后"提示
缓存	常见问题的答案缓存在 Redis，命中时不调外部
队列 + 限流	进队列削峰；遵守外部 API 的速率限制，避免 429
外部集成适配层	外部调用集中在一层，供应商 API 变化不影响其他服务
度量	单独监控外部调用的 P95 / P99 和失败率

✍ 参考答案｜可照着写，名字和数字换成新案例的

The LLM API is outside our control, so its latency is a constraint. The app sends POST /assistant/questions and gets 202 Accepted with a questionId at once, and the answer is streamed or pushed when ready, so 90% of questions get a visible response within 3 seconds.

> LLM API 不在我们控制范围内，它的延迟是一个约束。App 发送 POST /assistant/questions，立即收到 202 Accepted 和 questionId；答案以流式或推送的方式返回，所以 90% 的提问能在 3 秒内看到回应。

A background worker calls the LLM with streaming, with a timeout of about 5 seconds for the first part of the answer and about 30 seconds for the whole answer (assumed values), and at most one retry.

> 后台 worker 以流式调用 LLM：首段超时约 5 秒、完整答案超时约 30 秒（数值为假设），最多重试一次。

Normally about 5% of calls fail and are retried once. If the failure rate rises above a threshold (for example 20% within one minute, an assumed value), a circuit breaker opens and the app gets a cached or generic answer, so it stays responsive.

> 平时约 5% 的调用失败，重试一次即可；失败率超过阈值（例如 1 分钟内超过 20%，数值为假设）时熔断器打开，App 拿到缓存或通用答案，保持响应。

✎ 小测验｜先自己答，再看参考答案 **小测验 4：新推荐模型想先在少量用户身上试：选哪种部署策略?**

电商平台训练了一个新的商品推荐算法，不确定它会不会降低购买转化率。希望先让**一小部分用户**用新算法、**对比效果**，**出问题能马上撤回**。从**蓝绿部署、滚动更新、金丝雀发布**中选一种，并说明为什么不选另外两种。

▾ 参考答案

选**金丝雀发布**：只让 5–10% 的流量走新算法，对比转化率，风险低，出问题立刻把流量切回旧版本。**蓝绿**是整体一键切换，做不到只给一部分用户试；**滚动更新**是逐台替换，目的在发布，不是对比实验。补充：如果重点是严格的效果对比，可以叠加 A/B 测试；金丝雀重在控制风险，A/B 重在做实验，两者常一起用。

✎ 小测验｜先自己答，再看参考答案 **小测验 5：uptime 99.95%，但晚高峰 8% 的下单失败：老板说可用性很好，你怎么回应?**

上周的监控报表显示：系统 **uptime 99.95%**，但**晚高峰有 8% 的下单请求失败**。老板据此认为"可用性很好"。你怎么回应？应该看哪个指标?

▾ 参考答案

不同意。Uptime 只看系统在不在线，停机若发生在没人用的时段，不影响任何人；用户真正感受到的是 **Yield = 成功请求数 / 总请求数**（即请求成功率）。本例高峰 Yield 只有约 92%，偏偏在用户最多的时候失败。应把高峰时段的 Yield（下单成功率）作为可用性指标，并配合 P90 / P99 响应时间。

❝ 可用句式｜___ 处填案例内容

- The proposed metric only measures usage, not how well the system performs, so I disagree. A better metric is the average ___ time, because ___.
  > 这个指标只衡量使用量，不衡量系统表现得好不好，所以我不同意。更好的指标是平均 ___ 时间，因为 ___。
- It is a derived metric that depends on two base metrics: ___ and ___.
  > 这是一个派生指标，依赖两个基础指标：___ 和 ___。

本章术语表 Glossary

中文	English	一句话解释
服务发现	Service Discovery	服务按名字找到对方位置的机制
服务注册中心	Service Registry	记录所有服务实例位置的"内部 DNS"
东西向 / 南北向流量	East-West / North-South Traffic	服务之间 / 用户与平台之间的流量
蓝绿部署	Blue/Green Deployment	新旧两套同时在线，确认后切换
滚动更新	Rolling Update	逐个实例替换为新版本，无停机
金丝雀发布	Canary Release	新版本先处理一小部分流量
影子流量	Shadowing	同时发给新旧版本，只采用旧版本结果
熔断器	Circuit Breaker	下游持续失败时暂停调用、快速失败
级联故障	Cascading Failure	一个服务故障沿调用链拖垮其他服务
限流	Rate Limiting	限制单位时间内的请求数
分布式追踪	Distributed Tracing	用跟踪 ID 串起一个请求经过的所有服务
跟踪 / 跨度	Trace / Span	一次跨服务事务 / 其中一个工作单元
两阶段提交	Two-Phase Commit (2PC)	准备+提交两步的分布式事务协议

中文	English	一句话解释
Saga 模式	Saga Pattern	本地事务序列 + 补偿事务实现跨服务事务
补偿事务	Compensating Transaction	撤销前一步效果的事务
编排 / 协同	Orchestration / Choreography	中心协调者驱动 / 各服务靠事件自行协作
平均故障间隔 / 平均恢复时间	MTBF / MTTR	两次故障之间的平均时间 / 平均修复时间
产出率	Yield	成功请求数 / 总请求数
收获率	Harvest	可用数据 / 全部数据
百分位	Percentile (P90/P99)	90%/99% 的请求不超过的值
时序数据库	Time-Series Database (TSDB)	按时间存储指标的数据库
集中式日志	Centralized Logging (ELK/EFK)	把各组件日志汇总到一处查询
关联 ID	Correlation ID	串联一个请求所有日志的标识

CHAPTER 07

云计算基础与云迁移

Cloud Computing Fundamentals & Cloud Migration

课件来源：CNSD 01 Cloud Overview · CNSD 02 Cloud Reference Architecture · CNSD 02a IaC · CNSD 03 Cloud Migration

为 **T5、T6、T10** 打基础

一句话 **云是按需取用、按量付费的共享资源池；上云不是简单搬家，要按每个应用的情况选择迁移策略，能利用云原生特性才是真正的收益。**

学完本章你能

- 说出 NIST 定义的云计算五个基本特征
- 区分 IaaS、PaaS、SaaS、CaaS、FaaS 的责任划分
- 区分公有云、私有云、混合云、多云
- 为一个应用选择合适的迁移策略（7R）
- 说明基础设施即代码的好处

7.1 什么是云计算 Cloud Computing

■ 定义｜课件正式说法 **云计算（NIST）**

一种模式：通过网络方便地按需访问一个**共享的可配置计算资源池**（网络、服务器、存储、应用和服务），这些资源能以最少的管理工作或服务商交互被**快速供给和释放**。

五个基本特征	含义
按需自助 On-demand Self-service	用户自己开通资源，不需要人工审批
广泛网络访问 Broad Network Access	通过标准网络和各种终端访问
资源池化 Resource Pooling	多租户共享物理资源，用户不知道资源的具体位置
快速弹性 Rapid Elasticity	资源可快速扩缩，看起来无限
可计量服务 Measured Service	按使用量计量和计费

为什么云更便宜高效：标准化的工作负载（虚拟机、容器、Serverless）运行在高度集成、可大规模复制的基础设施上，硬件批量采购、软件精简，形成**规模经济** economy of scale。**收益**：经济上入门成本和运营成本低；运营上上市更快、扩展和测试方便、自助服务。**企业上云的障碍**：监管、治理、安全、隐私、迁移、集成、与现有系统共存。

✕ 常见误区｜这样答会丢分 **课件列出的"云的误区"（节选）**

上云不一定省钱；不是所有东西都要上云；"CEO 说要上云"不算云战略；不一定只用一家云；**不要一开始就迁移最关键、最复杂、最大的系统**（如 ERP）。

7.2 服务模型 Cloud Service Models

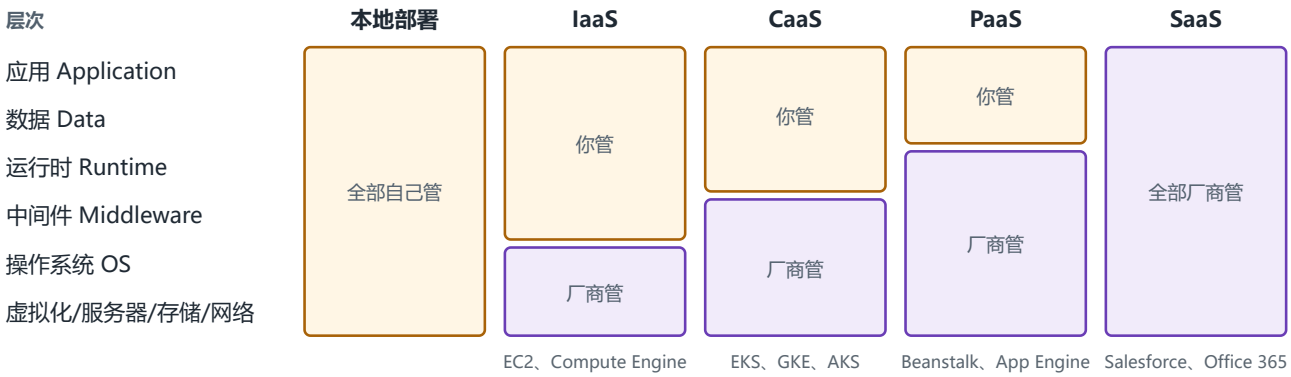


图 7-1 各服务模型中谁管理哪一层（黄=用户管，紫=云厂商管）；FaaS 比 PaaS 更进一步，用户只写函数代码

模型	你得到什么	例子
IaaS 基础设施即服务	租用计算、存储、网络，按用量付费；自己管操作系统往上的一切	Amazon EC2、Azure VM、Google Compute Engine
CaaS 容器即服务	托管的容器编排平台，你只管容器和应用	Amazon EKS/ECS、Azure AKS、Google GKE
PaaS 平台即服务	完整的开发部署栈，开发和部署都在云上完成	AWS Elastic Beanstalk、Google App Engine、Heroku、Cloud Foundry
FaaS 函数即服务	只上传函数，由事件触发，按调用计费（Serverless，第8章）	AWS Lambda、Azure Functions、Google Cloud Functions
SaaS 软件即服务	直接使用完整功能的软件	Salesforce、Microsoft 365

托管云服务 managed cloud services：由厂商或服务商部分或全部管理客户的云资源（迁移、配置、优化、安全、维护）。课程原则："优先使用托管服务" favor managed services（第8章）。

7.3 部署模型 Cloud Deployment Models

模型	含义
公有云 Public Cloud	向任何组织或公众开放，基础设施归厂商所有
私有云 Private Cloud	只为一个组织运营，可自建（on-site）或外包（outsourced）
社区云 Community Cloud	几个有共同需求（如合规）的组织共享
混合云 Hybrid Cloud	两种以上云（如私有+公有）组合，彼此独立但互联，数据和应用可移动
多云 Multi-cloud	同时使用多家云厂商（通常是多个公有云），避免锁定、取各家所长
虚拟私有云 VPC	在公有云里划出逻辑隔离的私有网络

多云环境下要额外考虑：**跨云的身份统一、网络互联、统一监控、可移植的部署方式（容器 + Kubernetes + IaC）**，这正是 PAST Q2(a)(i) 问"多云环境需要哪些额外组件"的考点。

7.4 云原生应用与"直接搬家" Cloud Native vs Cloud Enabled

云原生应用 cloud native 从一开始就按云的原则设计：多租户、弹性扩展、易于集成和管理。**云就绪/云赋能应用** cloud enabled 则是原本为本地环境开发、后来搬上云的应用。**企业第一波上云通常用 IaaS 做直接搬迁** lift and shift，因为资源可随时增减、业务更敏捷。

方式	优点	缺点
直接搬迁 Lift and Shift	工作量最小，迁移最快	通常用不上云平台的原生特性；在云上运行可能更贵

方式	优点	缺点
部分重构 Partial Refactor	只改部分代码，迁移较快	只用到部分云特性；可能比完全重构的运行成本高
完全重构 Complete Refactor	性能更高，可优化到更低运行成本	成本高得多（大部分代码要改），上线慢

7.5 云迁移策略 Cloud Migration Options (7R)

课件按 Gartner 的方法列出应用现代化和迁移的决策结果（常称 7R）：

策略	别名	做法	适合
保留 Retain	—	暂时保持原样，迁移不值得或应用快退役	风险/成本太高
退役 Retire	Decommission	下线，常是替换方案的一部分	已无业务价值
重新托管 Rehost	Lift and Shift	不改代码、不改功能，直接换到云基础设施	快速迁移、减少机房
更换平台 Replatform	Lift, Shape and Shift	换到新运行平台，只做兼容性的少量修改（如改用托管数据库）	想用部分云服务，不改结构
重新架构 Rearchitect	Refactor, Re-engineer	不改外部行为，重构代码结构、改进非功能属性，可转向新架构（如微服务、容器）	需要可扩展、敏捷
重建 Rebuild	Rewrite, Redesign	按原范围和规格重写为云原生应用	旧技术已无法演进
替换 Replace	Repurchase, Drop and Shop	换成商用现成软件 COTS 或 SaaS	非核心、通用功能
封装 Encapsulate	—	把应用的数据和功能包装成 API 服务对外提供	延续旧系统价值

现代化的六个驱动因素

业务视角：业务契合度（不再满足需求）、创新（阻碍抓住新机会）、敏捷（跟不上变化速度）。**IT 视角**：成本（运营维护太贵）、复杂度（难以修改）、风险（安全、合规、可支持性、扩展性、技能短缺）。

评估上云候选的八个标准

架构、安全、可用性、性能、可扩展性、战略重要性、未来计划、团队意愿。另需做财务评估：**总拥有成本** TCO，以及迁移期间的"迁移泡沫"成本（双环境、培训、咨询、租约违约金）。

迁移阶段：选择"直接搬迁"还是"搬迁并优化"；权衡锁定风险与云特有能力带来的价值；建立可行的商业论证；设计好**着陆区** landing zone 架构、转向云架构模式、引入迁移伙伴、过渡到云运维模式；尽早使用云原生服务。**采购模式**：内部管理或合作方管理（托管服务提供商）；**越是核心、差异化、复杂的能力越倾向内部管理**。

7.6 基础设施即代码 Infrastructure as Code

基础设施即代码 Infrastructure as Code, IaC：用机器可读的定义文件（代码）来管理和供给数据中心（服务器、网络、存储），而不是手工配置。好处：

- **速度与一致性**：开发、测试、生产环境部署快，而且每次都完全一样。
- **版本控制与协作**：基础设施像代码一样存进 Git、追踪变更、评审。
- **可见性与文档**：代码本身就是实时的基础设施文档。
- **节省成本**：减少人为错误，不用的环境随时拆掉。
- **灾难恢复**：**灾难发生时可在新区域从代码快速重建整套基础设施**。

工具：Terraform（跨云）、AWS CloudFormation、Azure Bicep/ARM、Ansible（配置管理、编排）、Pulumi。**容器镜像最佳实践**：用精简基础镜像（如 Alpine）、官方或认证镜像、及时打补丁、以非 root 用户运行、对镜像签名、使用 distroless 镜像、不可变镜像、符合 CIS 等安全基线。

7.7 白话讲解与小测验 Plain-language Notes & Drills

◆ 要点 | 必背结论 **服务模型 = 你管到哪一层（以"把到站计算服务部署上云"为例）**

模型	你管	厂商管	AWS	你要做的事
本地	全部	—	自建机房	买服务器、装系统、全部自己管
IaaS	操作系统往上	硬件、虚拟化、网络	EC2	自己装 OS、打补丁
CaaS	容器镜像、K8s 配置	容器编排平台	EKS、ECS	打容器镜像、写 K8s 配置
PaaS	代码和配置	运行时、中间件、OS	Elastic Beanstalk	只上传代码
FaaS	函数代码	所有服务器	Lambda	每条事件触发一次函数，按调用付费
SaaS	直接用	全部	Microsoft 365	买现成软件

越往下：运维越少、控制越少、锁定越强。PaaS 和 FaaS 管的层差不多，区别在粒度：PaaS 部署整个应用，FaaS 只部署一个函数，空闲不花钱但有冷启动。

✎ 小测验 | 先自己答，再看参考答案 **小测验 1：3 人小团队把后端从自管的 EC2 搬到 EKS + Fargate：少管了什么、代价是什么?**

一个**只有 3 名开发、没有专职运维**的团队，原来把后端服务装在**自己维护的 EC2 虚拟机**上（自己装系统、打补丁、扩容）。现在打算改用 Amazon EKS 运行容器，**工作节点用 AWS 托管的 Fargate**。用服务模型解释：他们少管了哪些东西？还要管什么？代价是什么？

▾ **参考答案**

少管：Kubernetes 控制平面、工作节点的操作系统补丁和节点扩缩（Fargate 托管），从 IaaS 变成 CaaS。**还要管：**构建和维护容器镜像、写 K8s 配置（Deployment、Service、HPA 等）、跟进 K8s 版本升级、监控和成本。**代价：**K8s 本身的学习和维护复杂度，对 3 人团队不小；控制平面和 Fargate 的费用；对 AWS 的依赖（锁定）加深。补充：如果团队只想交代码、不碰容器，PaaS（如 Elastic Beanstalk）或更简单的 ECS + Fargate 可能更合适。

✎ 小测验 | 先自己答，再看参考答案 **小测验 2：两个组织各该用哪种云部署模型?**

1. 医院：病历按法规**必须留在本院机房**，但预约挂号在**每天早上 8 点有高峰**，需要弹性扩展。
2. 一家公司**担心被单一云厂商锁定**，也想用不同厂商最擅长的服务。

▾ **参考答案**

1. **混合云**：病历留在本地私有环境满足法规，预约挂号这类弹性负载放公有云，两边互联。
2. **多云**：同时用多家公有云，避免锁定、取各家所长；要额外准备 Kubernetes、IaC、全局 DNS、统一身份和统一监控，保证可移植。

✎ 小测验 | 先自己答，再看参考答案 **小测验 3：工资系统要最快上云，以后还要接新的自助 App：怎么迁移?**

一家公司的工资系统**运行良好**，但机房租约**三个月后到期**，必须尽快上云。迁移策略候选：退役 Retire、重新托管 Rehost、更换平台 Replatform、重新架构 Rearchitect、重建 Rebuild、替换 Replace、封装 Encapsulate。① 选哪种迁移策略？缺点是什么？② 两年后公司想让员工用新的自助 App 查工资，但**近期没有预算改造老系统**。有什么做法能让新 App 用上老系统的数据和功能?

▾ 参考答案

① **重新托管（Rehost / Lift and Shift）**：不改代码，原样搬到云上虚拟机，最快、风险最低。缺点：用不上自动扩缩、托管服务等云原生特性，在云上运行可能比本地更贵。② **封装（Encapsulate）**：不改老系统内部，把它的数据和功能包成 API（例如经 API 网关对外提供），新 App 调这些 API；等有预算再逐步重构。

✎ 小测验 | 先自己答，再看参考答案 **小测验 4：两个团队各该选 PaaS 还是 CaaS?**

1. 团队 A **只想上传代码**，不想碰容器和集群配置。
2. 团队 B **已经有 Docker 镜像**，想**自己控制部署方式、副本数和扩缩规则**。

▾ 参考答案

1. **PaaS**（如 Elastic Beanstalk）：只交代码和配置，运行时、中间件、操作系统都由厂商管；代价是可定制的东西少。
2. **CaaS**（如 Amazon EKS / ECS）：厂商管容器编排平台，团队自己管镜像、部署配置和扩缩策略；控制更多，但要懂容器编排。

✎ 小测验 | 先自己答，再看参考答案 **小测验 5：测试环境能跑、生产出错，机房出事重建花了两周：怎么从根上解决?**

排查发现：测试和生产的服务器都被人**手工改过配置**，**两边已经不一样**，所以"测试能跑、生产出错"。之前机房出事，**重建整套环境花了两周**。用什么做法从根上解决？说出至少三个好处。

▾ 参考答案

用**基础设施即代码（IaC）**，如 Terraform、CloudFormation：所有环境都从版本控制里的模板创建，禁止手工改。好处：① 测试和生产完全一致，部署快；② 变更像代码一样走版本控制和评审，可追溯；③ 代码本身就是最新的基础设施文档；④ 减少手工误操作，不用的环境随时拆掉省钱；⑤ 灾难后能在新区域快速重建整套系统。

本章术语表 Glossary

中文	English	一句话解释
按需自助	On-demand Self-service	用户自行开通资源
资源池化	Resource Pooling	多租户共享物理资源
快速弹性	Rapid Elasticity	资源可快速扩缩
可计量服务	Measured Service	按用量计量计费
规模经济	Economy of Scale	规模越大单位成本越低
基础设施即服务	IaaS	租用计算、存储、网络
平台即服务	PaaS	托管的开发部署平台
容器即服务	CaaS	托管的容器编排平台
函数即服务	FaaS	事件触发、按调用计费的函数平台
软件即服务	SaaS	直接使用的完整软件

中文	English	一句话解释
混合云 / 多云	Hybrid Cloud / Multi-cloud	多种云组合 / 多家厂商组合
虚拟私有云	Virtual Private Cloud (VPC)	公有云中逻辑隔离的私有网络
云原生	Cloud Native	按云的原则从头设计的应用
直接搬迁	Lift and Shift (Rehost)	不改代码直接迁上云
更换平台	Replatform	换运行平台，只做少量兼容修改
重新架构	Rearchitect / Refactor	不改外部行为，重构结构与非功能属性
替换	Replace / Repurchase	改用 COTS 或 SaaS
总拥有成本	Total Cost of Ownership (TCO)	整个生命周期的全部成本
着陆区	Landing Zone	预先设计好的云账号、网络、安全基础环境
基础设施即代码	Infrastructure as Code (IaC)	用代码定义和供给基础设施
厂商锁定	Vendor Lock-in	难以离开某个厂商的依赖

CHAPTER 08

云原生架构与架构风格

Cloud Native Architecture & Application Architecture Styles

课件来源：CNSD 02 · CNSD 05 Architecting a Cloud Native Application · CNSD 06 Architecture Styles · CNSD 08 Serverless · CNSD 09 GenAI · PAST Q2 官方答案

对应题型 **T5**（每套必考）· **T10**

一句话 **云原生应用由无状态的容器化服务组成，状态交给托管存储，按负载自动扩缩；后端常用"混合架构"：核心交互用微服务，突发事件用 Serverless，批处理用定时任务，实时汇总用事件流。**

学完本章你能

- 说出云原生的定义、五条原则和十二要素
- 解释无状态设计和四种状态管理方式
- 比较单体、微服务、Serverless、事件驱动等风格
- 画出混合云原生参考架构并说明每个组件
- 在 PWA 和原生应用之间做选择

8.1 什么是云原生 What is Cloud Native

■ 定义｜课件正式说法 **云原生（CNCF 定义，课件引用）**

云原生技术让组织能在公有云、私有云和混合云这类现代动态环境中构建和运行可扩展的应用。**容器、服务网格、微服务、不可变基础设施和声明式 API** 是这种方法的代表。它们让系统松耦合、有韧性、易管理、可观测；配合强大的自动化，工程师能频繁、可预期地做出重大变更。

云原生是一组能力和实践：应用和基础设施具有**韧性、可扩展、可观测、高度自动化、易于更新。**课件用"三个 P"概括：运行在动态分布式环境上的**平台** Platform（订阅的平台、托管服务）、要具备的**属性** Properties、采用的**实践** Practices（DevOps、持续交付）。

◆ 要点｜必背结论 **云原生架构五原则（Google，CNSD 05）**

1. **为自动化而设计** Design for automation：构建、测试、部署、扩缩、恢复都自动化。
2. **聪明地处理状态** Be smart with state：**尽量无状态，状态集中放在少数专门的存储里。**
3. **优先使用托管服务** Favor managed services：数据库、消息、监控尽量用云厂商托管的。
4. **纵深防御** Practice defense in depth：每一层都设防，不只靠边界。
5. **持续做架构** Always be architecting：架构随业务和技术不断演进。

方面	传统应用（宠物 Pets）	云原生应用（牛群 Cattle）
开发目标	长期稳定	上市速度
方法与团队	瀑布；开发、运维、测试、安全各自独立	敏捷、DevOps；协作团队
交付周期	长	短而持续
架构	紧耦合、单体	松耦合、基于服务、API 通信
基础设施	以服务器为中心，依赖特定基础设施，垂直扩展，按峰值预先采购	以容器为中心，可移植，**水平扩展**，按需扩容
会话	粘性会话 sticky session，坏一台就丢状态	共享状态存储，任意实例可替换

8.2 十二要素应用 The Twelve-Factor App

要素	要求	要素	要求
1 代码库 Codebase	一个代码库，纳入版本控制，多处部署	**7 端口绑定 Port binding**	通过端口绑定对外提供服务
2 依赖 Dependencies	显式声明并隔离依赖	**8 并发 Concurrency**	通过进程模型水平扩展
3 配置 Config	**配置放在环境变量里**	**9 易处置 Disposability**	快速启动、优雅关闭
4 后端服务 Backing services	数据库、队列等当作附加资源	**10 环境等价 Dev/prod parity**	开发、预发、生产尽量一致
5 构建/发布/运行	三个阶段严格分开	**11 日志 Logs**	把日志当作事件流输出
6 进程 Processes	**以一个或多个无状态进程运行**	**12 管理进程 Admin**	管理任务作为一次性进程运行

相关实践：**外部化配置** external configuration（配置与代码分离，改配置不用重新打包）；**服务网格** service mesh（管理服务间通信的流量、韧性、可观测和安全，由控制平面和数据平面组成，第9章）。

8.3 无状态设计与状态管理 Stateless & State Management

无状态应用 stateless application 有三部分：应用本身、存放领域状态的数据库、各自持有会话状态的客户端。应用仍然处理领域状态，但把状态存在数据库里而不是自己内存里。好处：**性能**（不花资源维护缓存）、**可扩展**（不在内存里重复数据库已有的数据）、**一致**（不会与数据库不同步）、**优雅关闭**（任何业务事务之间都能干净地停）、**可恢复**（崩溃时只丢失未完成的事务）。

状态管理方式	做法	适合	白话
粘性会话 Sticky Session	负载均衡总把同一客户端发到同一实例	不推荐：影响可用性和负载均衡	总找同一个服务员；他请假了你点的菜就没了，还会忙闲不均、没法缩容
共享状态存储 Shared State Store	会话、计数等放进分布式缓存（Redis）或数据库	会话、购物车、实时计数	订单写在公共系统里，任何服务员一查就知道
StatefulSet + PV/PVC	Kubernetes 给有状态应用稳定的名字、有序部署和持久卷	在集群内自运行数据库、Kafka	在集群里自己跑数据库，每个 Pod 有固定名字和自己的盘
Operator	用自定义资源把运维知识编码，自动管理有状态应用	复杂有状态中间件	把运维老手的经验写成程序，自动管复杂中间件
外部状态存储 External State Store	有状态服务（数据库、消息队列）放在集群外或用托管服务	生产环境首选（托管 RDS、ElastiCache）	状态交给集群外的托管服务（RDS、ElastiCache），T5 默认写这个
客户端状态	cookies、local storage、session storage	快，但受设备容量和安全限制	存在用户自己的浏览器或手机里，快但容量和安全受限

8.4 应用架构风格 Architecture Styles (Archetypes)

选择架构要综合考虑：领域、影响结构的架构特性（质量属性）、数据架构、组织因素、对流程的了解等。课件介绍的风格：

风格	要点	优点	缺点 / 注意	什么时候选（看负载特点）
单体 Monolithic	UI、业务逻辑、数据访问构建成一个整体单元	开发部署简单，性能好	紧耦合；只能整体扩展；发布和团队协调难	规模小、团队小、要快速上线
分层 Layered	按表现、业务、数据分层	关注点分离	层间调用增加延迟	代码内部结构，不按负载选
微服务 Microservices	拆成小的、松耦合、可独立部署的服务（第9章）	独立扩展和部署、技术多样	运维复杂、网络延迟、分布式数据	核心交互、持续使用、要独立扩缩
六边形 Hexagonal	又叫端口与适配器：领域核心在中间，外部通过端口和适配器接入	领域与技术解耦，易测试	结构更复杂	代码内部结构（端口与适配器），**不是部署风格**
Serverless	由云函数组成，事件触发，厂商管基础设施	按用量付费、自动扩展、少运维	厂商锁定、冷启动、控制力弱	突发、短小、事件触发的任务
事件驱动 Event-driven	围绕"状态变化的事件"构建，组件间通过事件间接通信	松耦合、可扩展、实时响应	难追踪、最终一致	部件之间解耦通信，可和其他风格一起用
Lambda 架构	批处理层 + 速度层 + 服务层	同时处理实时和历史数据；批处理层提供准确结果	两套处理逻辑，复杂	既要实时又要准确的历史结果
Kappa 架构	只用一条流处理管道处理实时和持续数据	简单、低延迟	状态管理和重处理有挑战	只想用一条流水线做实时处理

新型组合风格（课件附录）：**单元化** cell-based（整套栈复制成独立单元，限制故障影响范围）、**边缘原生**（计算推到离用户近的节点）、**微前端与 BFF**、**Sidecar/多运行时**、**数据网格** data mesh、**本地优先/离线优先**、**MACH**（Microservices、API-first、Cloud-native SaaS、Headless）、内部开发者平台。

◆ 要点｜必背结论 **各风格的关系（CNSD 06）**

单体适合小而简单的应用；微服务适合需要可扩展和灵活的大型复杂应用；Serverless 适合事件驱动或偶发的负载，性价比高；Lambda/Kappa 是以数据为中心的架构；**事件驱动是一种可用于所有风格的设计原则**；微服务可以用 Serverless 函数实现。**把这些风格组合起来就是混合架构** hybrid architecture。

8.5 Serverless 架构 Serverless Computing

Serverless 是一种依赖**运行在短暂服务器上的小型独立函数**、在弹性运行时环境中执行的架构方法。不是没有服务器，而是开发者不用管服务器。原则：用计算服务按需执行代码；写单一用途的无状态函数；设计基于推送的事件驱动管道；做更强大的前端；拥抱第三方服务（如认证、数据库）。

方面	内容
优点	只为实际计算时间付费；自动扩展应对波动和突发峰值；无需管理服务器；资源实时快速供给
缺点	厂商锁定；**冷启动** cold start（首次调用慢）；对底层控制少；执行时间有上限；大规模 Serverless 结构设计起来很费力
典型用例	Web/移动应用后端；认证授权；数据处理（上传图片后生成缩略图）；实时流处理（Kinesis + Lambda）；把旧 SOAP 服务包装成 REST 的 **API 代理**；定时任务
AWS 产品	Lambda（函数）、API Gateway（REST 入口）、S3、DynamoDB、SNS、SQS、Kinesis、Step Functions、Fargate
Azure / Google	Azure Functions、Event Grid、Cosmos DB；Google Cloud Functions、Cloud Run、Pub/Sub、Firestore

8.6 混合云原生参考架构 Hybrid Cloud Native Reference Architecture

T5 题通常给一张单体或本地部署的架构图，要求改造成云原生的**混合参考架构**。下图是一个可以直接套用的模板（以 Vote Anywhere 为例，产品名以 AWS 为准）：

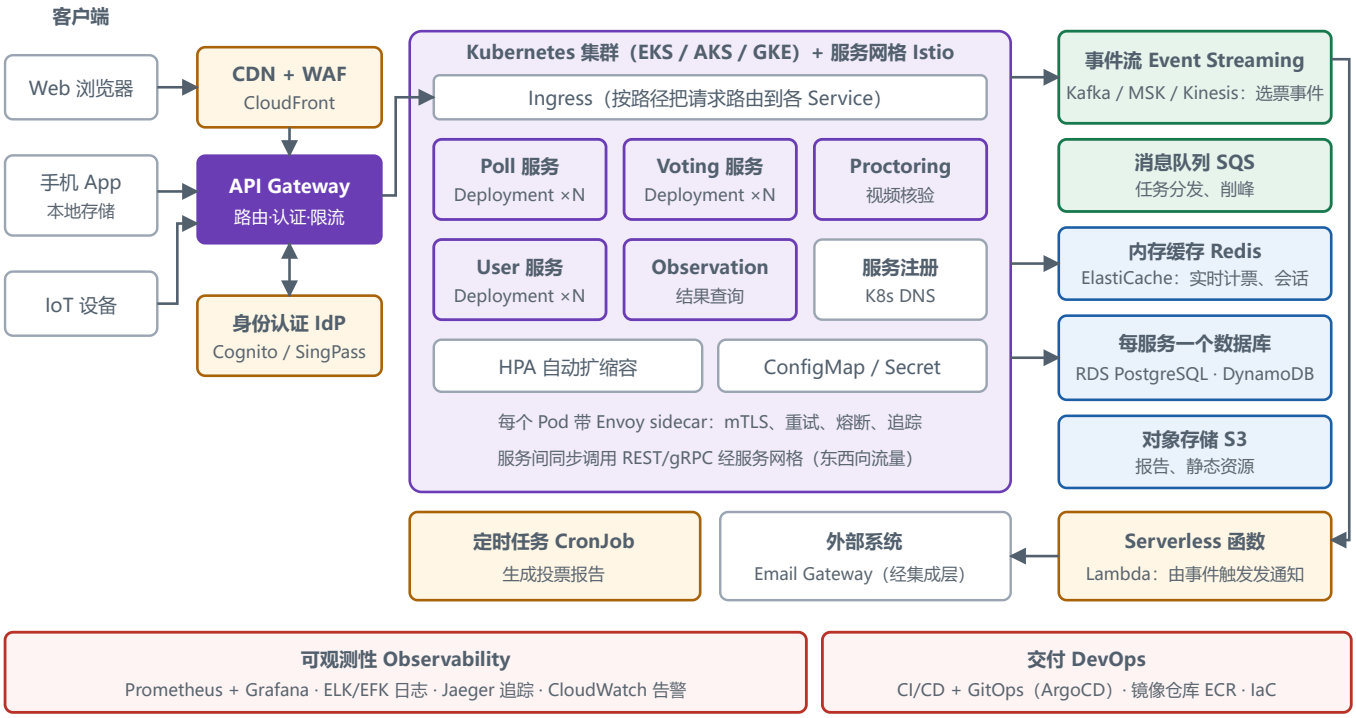


图 8-1 混合云原生参考架构模板：微服务 + Serverless + 定时任务 + 事件流，状态交给托管存储

组件	为什么需要
CDN + WAF	静态内容就近分发、降低延迟；Web 应用防火墙挡住恶意流量
API Gateway	外部请求的统一入口：路由、聚合、认证、限流、监控（第5章）
K8s 集群 + HPA	容器化微服务独立部署、按负载自动扩缩容，应对 10万并发
服务注册 / 服务网格	服务间互相发现；统一做 mTLS、重试、熔断、流量管理和追踪
事件流（Kafka）	选票事件**有序、持久、可重放**，多个消费者（简要结果、详细结果）可各自读取
消息队列 / Serverless	通知等突发、短小任务异步处理，按调用付费，不占集群资源
定时任务	周期性批处理，如投票结束后生成报告
缓存 Redis	内存级读写：实时计票、会话、热点数据
每服务一个库	服务之间数据解耦；按数据特点选型（多语言持久化，第11章）
可观测性、CI/CD	运行起来后能看见问题、能自动且安全地发布

▶ 案例演练 | 演示怎么套用 **PAST Q2(a)（10分）官方答案要点与评分**

后端由**微服务、定时任务和事件驱动架构**组成；§4 的用户量足以证明采用微服务的必要性。官方例子：选票先收集到**内存 Redis**，再持久化到 **PostgreSQL**；Poll 服务用内存数据存储跟踪投票状态；Observation 服务和定时任务汇总所有选票、给出最终结果。支撑这套微服务生态需要 **GitOps 工具链和容器镜像仓库**；API 工程方面需要**服务注册中心、负载均衡器、Ingress、Istio 服务网格。**

评分：**参考架构 4 分（识别服务、注册中心、状态机制和存储）**；额外的通信组件 2 分（包括事件服务和通知服务）；缓存或内存服务 1 分；部署计划 3 分（识别 Pod、Deployment 和 Service，第10章）。

◆ 要点｜必背结论 **T5 四个小问的答题骨架**

1. **组件**：照图 8-1 画，标出每个组件的托管服务名；多云时加上统一身份、跨云网络、统一监控、IaC/容器保证可移植。
2. **内部通信**：同步查询用 REST/gRPC，经服务网格和服务发现；需要解耦、削峰、多消费者的用异步，**选票这类要求有序可重放的用事件流，一次性任务用消息队列**。在图上标出这些组件。
3. **有状态的服务**：点名一个（如 Voting 服务要暂存投票会话和实时计数），说明机制：外部状态存储（ElastiCache Redis）或 StatefulSet + PV/PVC。
4. **K8s 部署计划**：命名空间、每个服务的 Deployment（副本数）+ Service、Ingress、HPA、有状态组件用 StatefulSet 或托管服务、定时任务用 CronJob（第10章模板）。

只用微服务还是混合？（PAST2 Q2a）答混合并说理由：**不同负载用最合适的风格**，核心交互服务用微服务保证稳定低延迟；突发的通知、文件处理用 Serverless 省成本、自动扩展；批处理用定时任务；实时计算用事件流。整体更省钱、更好扩展。

8.7 前端：渐进式 Web 应用还是原生应用 PWA vs Native App

对比	原生应用 Native App	渐进式 Web 应用 PWA
安装	应用商店安装，桌面有图标	浏览器访问即可，可"安装"到主屏，无需应用商店
技术	Swift、Kotlin、Java；跨平台 Flutter、React Native	Angular、Ionic、ReactJS；可结合 WebAssembly
设备能力	**深度访问硬件**：传感器、蓝牙、摄像头、IoT 设备、本地存储	有限：离线访问、推送通知、后台同步
性能与成本	性能最好；每个平台都要开发（跨平台框架可缓解）	接近原生；一套代码覆盖所有平台，开发快、成本低、覆盖广
适合	功能丰富、要求高性能、需要硬件集成	简单应用、电商、新闻，需要快速广泛覆盖（Starbucks、Twitter Lite）

▶ 案例演练｜演示怎么套用 **PAST Q2(b)（3分）：手机端要本地存储，未来还要接 IoT 传感器，选 PWA 还是原生?**

①**存储方面两者都行**：客户端状态管理（cookies、local storage、session storage）两者都支持，**从存储角度没有区别**（1分）。②但未来要在投票站接入 **IoT 传感器和执行器**（为有特殊需要的选民服务），**需要深度访问设备硬件，因此选原生应用**（1分）。③对后端的影响：采用**客户端编排** client-side orchestration，客户端路由、在客户端管理共享状态、注册多个 API 依赖即可，**后端服务不需要改动**（1分）。

8.8 生成式 AI 与云原生应用 GenAI in Cloud Native Apps

CNSD 09 简要介绍了把生成式 AI 集成进云原生应用：Transformer 和自注意力机制是大语言模型 LLM 的基础；**RAG**（检索增强生成）先从企业知识库检索相关内容再让模型回答，减少编造；**智能体** agents 以 LLM 为核心，调用工具自主完成多步任务。实际集成通常用云厂商托管的模型服务（Amazon Bedrock、Azure OpenAI、Google Vertex AI），像其他托管服务一样通过 API 调用。往年样卷没有考过这部分。

8.9 白话讲解与小测验 Plain-language Notes & Drills

■ 定义｜课件正式说法 **状态和无状态**

状态 = 系统记住的、会影响下一次请求结果的信息（答到第几题、剩余时间、是否已登录）。**无状态** = 处理完一个请求，不在自己身上留下需要记住的东西，下一个请求去共享的地方（数据库、缓存）重新查。无状态不是不处理数据，而是数据存在数据库里，不在自己内存里。

◆ 要点｜必背结论 **8.4 的"风格"不在同一个维度上，不是八选一**

维度	回答的问题	包含
部署和运行	这块功能怎么跑、怎么扩展	单体、微服务、Serverless、定时任务
通信	部件之间怎么传话	事件驱动、同步调用
数据处理	数据怎么汇总	Lambda、Kappa、事件流
代码内部结构	一个服务里怎么组织代码	分层、六边形

答案几乎从来不是"选一种"，而是不同功能用不同风格（混合架构）。容易混的两对：微服务一直在跑、Serverless 有事才跑（问"持续的还是偶发的"）；要保序、多方读同一批、能回溯 → 事件流，一件事一人处理一次 → 消息队列。

✎ 小测验｜先自己答，再看参考答案 **小测验 1：在线考试平台改造成 AWS 上的云原生架构**

在线考试平台原来是本地部署：硬件负载均衡器 → Web 服务器 → 应用服务器 → 数据库加备库。现在要改造成部署在 AWS 上的云原生架构。

三类工作：① 答题和交卷：**开考瞬间流量暴涨十倍，平时很低**，**开考时间提前一周公布**；② **交卷后给考生发确认邮件**；③ **每晚统计成绩**。候选风格：微服务（容器 + 自动扩缩）、Serverless 函数、定时批处理任务、事件流。

(a) 旧的四类组件在云上分别换成什么？(b) 三类工作各选哪种风格，为什么？

▾ **参考答案**

项目	答案	理由
(a) 旧组件	硬件负载均衡 → 托管负载均衡（ALB）；Web 服务器 → S3 + CloudFront（假设前端改成静态单页应用，动态部分交给后端微服务）；应用服务器 → 容器化微服务 + 自动扩缩；单库加备库 → 托管数据库（RDS 多可用区）	托管服务替代自己维护的硬件
① 答题交卷	微服务 + HPA；开考前按时间表**提前扩容**；答题进度存 Redis 并定期落库	核心交互、持续使用；开考时间已知，等 HPA 被动扩容来不及
② 发邮件	交卷事件触发 Serverless 函数（Lambda + SES）	突发、短小，空闲不花钱
③ 每晚统计	定时批处理任务（CronJob）	周期性批处理

✎ 小测验｜先自己答，再看参考答案 **小测验 2：大促后缩容，用户的购物车被清空了：为什么？怎么改？**

电商把**购物车放在应用服务器的内存里**，负载均衡开了**粘性会话**（同一用户总被发到同一台）。大促时扩到 20 台，活动结束**自动缩回 5 台**，大量用户反馈购物车被清空；大促期间还出现**几台特别忙、几台很闲**。原因是什么？怎么改？

▾ 参考答案

原因：购物车只存在某一台实例的内存里，缩容时那台实例被关掉，购物车就丢了。负载不均是因为粘性会话把老用户钉在老实例上，新扩出来的实例只能接新用户。**改法：**把购物车放到共享状态存储（如 ElastiCache Redis，或直接落库），应用服务器变成无状态，任何实例都能处理任何请求，就可以随意扩缩；缩容时配合连接排空。

❝ 可用句式｜___ 处填案例内容

- Sticky sessions always route a client to the same instance, so if that instance is scaled in, the session is lost, and the load is spread unevenly. Instead, store the cart in a shared store such as ElastiCache (Redis), so any instance can handle any request.

  > 粘性会话总把客户端发到同一实例，那个实例被缩掉，会话就丢了，负载也不均匀。改为把购物车放在 ElastiCache（Redis）这样的共享存储里，任何实例都能处理任何请求。

✎ 小测验｜先自己答，再看参考答案 **小测验 3：外卖平台的四件事各用什么风格？**

候选风格：微服务（容器 + 自动扩缩）、事件驱动 + Serverless 函数、定时批处理任务、事件流。为下面四件事各选一种，并写一句理由：

1. **下单核心流程**。
2. **订单完成后给用户发优惠券短信**。
3. **每天凌晨**生成商家结算单。
4. 骑手位置**每几秒上报一次，实时汇总**给调度看板。

▾ 参考答案

1. **微服务** + 自动扩缩：核心交互、持续使用，要稳定低延迟
2. **事件驱动 + Serverless**：订阅"订单完成"事件，由函数异步发短信；突发、短小，空闲不花钱
3. **定时批处理任务**：周期性运行
4. **事件流**：持续的大量事件，要保序、实时汇总，还可能有多个程序读同一批数据

✎ 小测验｜先自己答，再看参考答案 **小测验 4：加了实例后用户时好时坏、头像时有时无：违反了十二要素哪条？**

某服务把用户上传的头像**存在本机磁盘**、登录会话**存在进程内存里**。为了扛流量**加到 4 个实例**后，用户反映"有时要重新登录""头像时有时无"。违反了十二要素的哪条（或哪几条）原则？怎么改才能水平扩展？

▾ 参考答案

违反了**第 6 条：以无状态进程运行**，以及**第 8 条：靠增加进程水平扩展**。请求被分到不同实例，而状态只存在某一台本机上。改法：会话放共享状态存储（如 ElastiCache Redis），文件放对象存储（如 S3），把它们当作外部的支撑服务（第 4 条）；进程本身不保存任何需要记住的东西，这样任意实例都能处理任意请求。

❝ 可用句式｜___ 处填案例内容

- Hybrid. The core interactive features run as microservices; the bursty notifications use serverless functions; the periodic reports use scheduled jobs; the real-time aggregation uses an event stream.
  > 混合架构：核心交互用微服务，突发的通知用 Serverless 函数，周期性报表用定时任务，实时汇总用事件流。

本章术语表 Glossary

中文	English	一句话解释
不可变基础设施	Immutable Infrastructure	不在线修改，变更时整体替换
声明式 API	Declarative API	描述期望状态，由系统负责达成
宠物与牛群	Pets vs Cattle	精心照顾的单台服务器 vs 可随时替换的实例
十二要素应用	Twelve-Factor App	构建云原生应用的十二条实践
外部化配置	External Configuration	配置与代码分离，放在环境中
无状态应用	Stateless Application	不在进程内保存状态的应用
粘性会话	Sticky Session	同一客户端总被路由到同一实例
外部状态存储	External State Store	把状态放在集群外的托管存储
六边形架构	Hexagonal Architecture (Ports & Adapters)	领域核心通过端口和适配器连接外部
事件驱动架构	Event-Driven Architecture (EDA)	组件通过事件间接通信
Lambda / Kappa 架构	Lambda / Kappa Architecture	批+流两层处理 / 只用流处理
混合架构	Hybrid Architecture	组合微服务、Serverless、定时任务、事件流
无服务器	Serverless	开发者不管理服务器的事件驱动函数计算
冷启动	Cold Start	函数首次调用时初始化导致的延迟
定时任务	Scheduled Task / CronJob	按时间周期运行的批处理
参考架构	Reference Architecture	可复用的架构模板
内容分发网络	CDN	就近缓存和分发内容的网络
Web 应用防火墙	WAF	过滤针对 Web 应用的恶意请求
渐进式 Web 应用	Progressive Web App (PWA)	可安装、可离线的 Web 应用
客户端编排	Client-side Orchestration	由客户端组合调用多个后端 API
检索增强生成	Retrieval-Augmented Generation (RAG)	先检索知识再让 LLM 生成回答

CHAPTER 09

微服务设计与服务间通信

Microservices Design & Inter-process Communication

课件来源：CNSD 07 Microservices Design · CNSD 10 Cloud Patterns · PE 04 Reusable Services · ArchSS 02（扩展立方体）· PAST Q1(a)(ii) 官方答案

对应题型 **T10、T12**，并支撑 **T5**

一句话 **微服务用"网络调用"换来了独立部署和独立扩展；所以服务怎么拆、彼此怎么通信（同步还是异步、队列还是事件流）是架构的关键决策。**

学完本章你能

- 说出微服务的特征和利弊
- 用扩展立方体解释三种扩展方式
- 为一次交互选择同步或异步通信
- 说清消息队列和事件流的区别
- 区分 API 网关与服务网格、编排与协同
- 决定哪些服务适合放进同一个容器

9.1 微服务是什么 Microservices

微服务 microservices 是一种架构风格：复杂应用由小的、独立的进程组成，它们通过与语言无关的 API 通信；每个服务围绕一项业务能力，可以独立部署。Fowler 总结的特征：

特征	含义
通过服务实现组件化	服务是可独立部署的组件。代价：远程调用比进程内调用昂贵，所以远程 API 要更粗粒度
围绕业务能力组织	康威定律：系统结构会复制组织的沟通结构。用包含全部技能的**跨职能团队**，而不是按 UI/后端/数据库分的团队
智能端点、哑管道	逻辑放在服务里，通信机制保持简单（与把大量逻辑放进 ESB 相反）
去中心化治理	每个服务可选不同技术；"谁构建谁运行" you build it, you run it
去中心化数据管理	每个服务管理自己的数据库（**多语言持久化**）；用最终一致而非分布式事务
基础设施自动化	CI/CD 流水线、自动化测试和部署
为失败而设计	任何服务调用都可能失败，客户端要尽量优雅地应对

为什么用微服务

- 多语言、技术异构，按需选技术
- **弹性且有选择地扩展**：只扩需要扩的服务
- 可替换、可并存多个版本
- 便于试验和创新，减少技术债
- 天然支持事件驱动和 DevOps

代价

- 排障复杂：服务间通信是潜在故障点
- **延迟增加：进程内通信比进程间快得多**
- 运维复杂：部署、监控、可用性、备份
- 版本管理复杂

Fowler 的提醒："除非系统复杂到无法当成单体管理，否则别考虑微服务。"从单体迁移是一段旅程：先找出最常用的功能，优先把它们改成微服务。决定是否拆分一个服务，要考虑开发者吞吐量、变更频率、复用程度、团队规模等因素。

9.2 拆分策略与扩展立方体 Decomposition & the Scale Cube

拆分微服务的三步：①找出系统操作（来自用例）；②按**业务能力**或**子领域**（DDD 的限界上下文，第4章）找出服务；③定义服务的 API 和协作方式。扩展系统有三个方向：

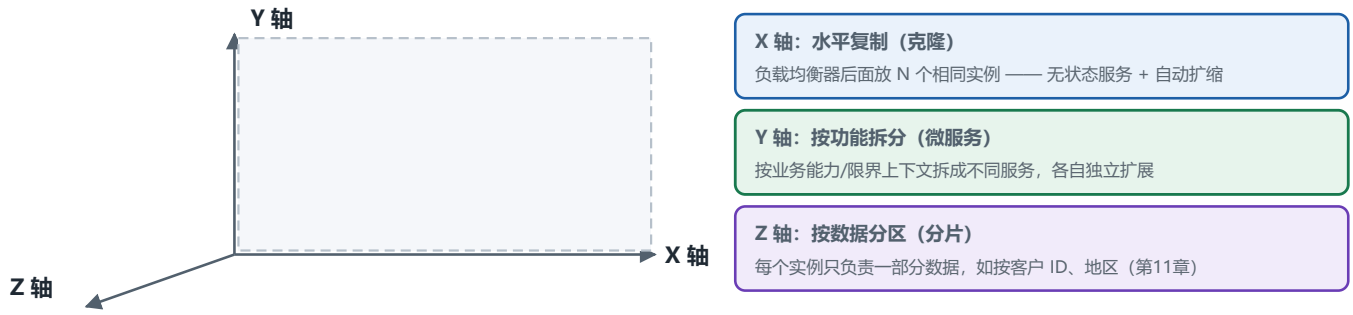


图 9-1 扩展立方体 Scale Cube：三个方向可以组合使用

9.3 服务间通信 Inter-process Communication

通信方式	含义
同步阻塞 Synchronous Blocking	调用另一个服务并阻塞等待响应。简单直观，但调用方和被调方紧耦合，下游慢就拖慢上游
异步非阻塞 Asynchronous Non-blocking	发出调用后不管对方是否收到都继续处理
请求-响应 Request-Response	请求对方做某事，并期待一个结果响应（可同步也可异步）
事件驱动 Event-Driven	服务发出事件，其他服务消费并作出反应；**发出方不知道谁在消费**
共享数据 Common Data	通过共享数据源协作（通常不被视为好的通信方式）

	一对一 one-to-one	一对多 one-to-many
同步	请求/响应	—
异步	异步请求/响应；单向通知 one-way notification	发布/订阅；发布/异步响应

实现技术：远程过程调用（SOAP、**gRPC**：用 Protocol Buffers 定义接口，不受 HTTP 动词数量限制，性能高）；REST（第5章）；消息通道（发送方通过消息代理把消息送到接收方）。**消息格式**：文本格式 JSON、XML 自描述、易读；二进制格式 Protocol Buffers、Avro 更紧凑高效。

◆ 要点 | 必背结论　怎么选同步还是异步（CNSD 07）

两种方式各有优劣，一个系统不可能只用一种。有效做法是：**先用同步请求-响应，等到确有价值时再重构为异步**。课件结论："在微服务世界里异步通常更好，但具体模式要按实际收益选择。"**需要立即拿到结果的查询用同步**；需要解耦、削峰、一个事件多个消费者、调用方不必等待的用异步。

9.4 消息队列与事件流 Message Queue vs Event Streaming

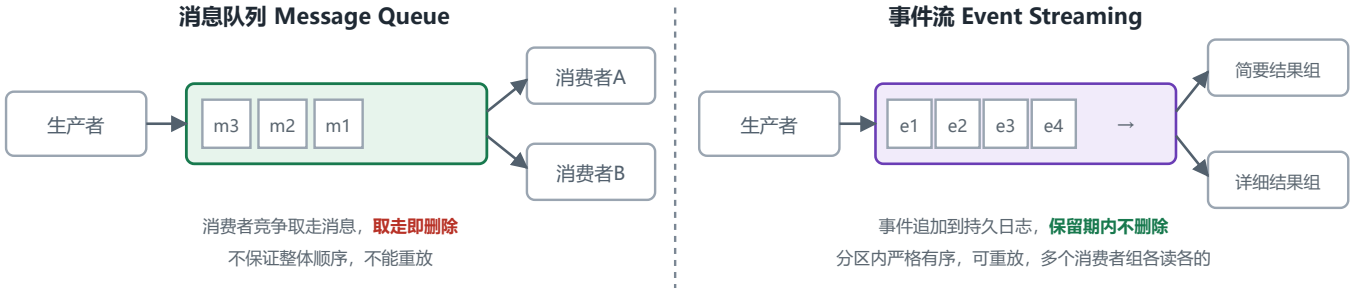


图 9-2 消息队列与事件流的本质区别

对比	消息队列	事件流
消息去向	投递后即删除 dropped upon delivery	写入持久化日志，在保留时间窗口内不删除

对比	消息队列	事件流
顺序	多个消费者竞争，不保证全局顺序	同一分区内**按时间严格有序** temporal ordering
重放	不能	能，可从任意位置重新读取
多个消费者	一条消息通常只被一个消费者处理	多个消费者组独立读取同一批事件
适合	任务分发、削峰、一次性处理（发邮件）	实时分析、结果汇总、审计、要求**结果确定且可重复**
产品	Amazon SQS、RabbitMQ、Azure Queue Storage / Service Bus	Apache Kafka / Amazon MSK、Amazon Kinesis、Azure Event Hubs、Google Pub/Sub

▶ 案例演练 | 演示怎么套用 **PAST Q1(a)(ii)（2分）：为什么用事件流而不是消息队列来触发结果更新?**

理由一：顺序。结果更新要求"确定且可重复"，选票事件必须按时间顺序处理，交换两个事件的顺序就会改变某一时刻的结果；**事件流保证事件有序。理由二：要读两遍。**用例要同时更新简要结果和详细结果，相当于要把新的选票事件处理两遍；事件流在一定时间窗口内不会丢弃事件，**而消息队列一投递就删除。**每个理由 1 分。

9.5 API 网关、服务网格与服务发现 API Gateway, Service Mesh, Discovery

组件	管哪类流量	职责	产品
API 网关	南北向（客户端 ↔ 平台）	统一入口、路由、聚合、协议转换、认证、限流；把服务提供方和消费方隔离开	Amazon API Gateway、Kong、Apigee
服务网格 Service Mesh	东西向（服务 ↔ 服务）	每个服务旁边一个 **sidecar 代理**（数据平面，如 Envoy），由**控制平面**统一配置：流量路由、重试、熔断、mTLS 加密、追踪	Istio、Linkerd、Consul
服务注册与发现	东西向	记录所有服务实例的位置；**客户端发现**（客户端查注册中心并做负载均衡，如 Eureka + Ribbon）或**服务端发现**（经负载均衡器查询转发，如 K8s Service）	Eureka、Consul、K8s DNS

9.6 编排与协同 Orchestration vs Choreography

业务逻辑一复杂，一个业务功能就会跨越多个服务，例如"新客户注册后：开积分账户、寄欢迎礼包、发欢迎邮件"。

方式	做法	优点	缺点
编排 Orchestration	Customer 服务作为"中央大脑"，依次请求积分、邮寄、邮件服务	简单，流程显式、一目了然，可借助规则引擎	中央服务过于"聪明"，其他服务变成贫血服务，耦合高
协同 Choreography	Customer 服务只发出 CustomerCreated 事件，各服务自行订阅并处理	松耦合，加新功能只需加订阅者	流程隐式，需要额外监控和追踪才能看清全局

9.7 微服务的数据 Data in Microservices

- **每服务一个数据库** database per service：服务松耦合，可按需选库，可独立扩展；代价是跨服务查询和事务变难。**共享数据库** shared database 简单、可用本地事务，但服务之间在设计和运行时都耦合在一起。
- **CQRS**（命令查询职责分离 Command Query Responsibility Segregation）：把写模型和读模型分开，读模型可以是预先算好的视图（如实时结果），读写可以独立扩展。
- **事件溯源** Event Sourcing：不保存当前状态，而是保存状态变化的事件序列，需要时重放得到当前状态。它解决了"更新数据库并发布事件"的原子性问题，还提供审计和重放能力；查询复杂，通常与 CQRS 搭配。
- **Saga**：跨服务的业务事务用一串本地事务加补偿事务实现（第6章）。
- **内容聚合模式**：由客户端聚合；在 API 网关聚合；由组合微服务聚合；在数据库层聚合。

9.8 服务共置：哪些服务放进同一个容器 Service Collocation

按理每个微服务一个容器。但有时受许可证或成本限制，必须把两个以上服务放进同一个容器实例。选择原则是**通信亲和度** communication affinity：**一段时间内彼此交换的消息数量和大小越大，越应该放在一起**。

共置的好处

- 跨服务调用变成进程内调用，延迟大幅降低
- 减少网络流量和序列化开销
- 少一个网络故障点

代价与注意

- 失去独立扩展和独立部署
- 一个出故障可能拖垮另一个
- 宜选**扩展规律相近、发布节奏相近**的服务

✎ 考试怎么考 | 题型·分值·套路

T12（约2分，PAST2 Q1f、PAST3 Q1d）：回到 §7 领域模型，找**跨框连线最多、调用最频繁**的两个服务。例：Vote Anywhere 中 PollManager 与 ObservationManager（查看结果每次都要两者协作），或 VotingManager 与 PollManager（投票时频繁读取 Poll）。答案写"选哪两个 + 它们之间交互频繁的具体用例 + **共置后变成进程内通信的好处**"。

T10 二选一题在本章常见的是"消息队列还是事件流"：先点出案例的要求（有序？要重放？多个消费者？），再给两个理由。

9.9 白话讲解与小测验 Plain-language Notes & Drills

◆ 要点 | 必背结论 **"按 X 分区"到底是什么意思**

事件流是只追加的日志，拆成几条固定的**分区**（像车道）。生产者给每个事件带一个**分区键**（如 busTripId），算哈希后对分区数取余，所以**同一个键永远进同一个分区**，很多个键共用一个分区。一个消费者组里，每个分区同一时间只交给一个实例：分区之间并行（吞吐随分区数增加），分区内部按顺序（同一班次的位置不会乱）。所以写一句"partitioned by ___ ID"就同时拿到**顺序**和**扩展**两个点。

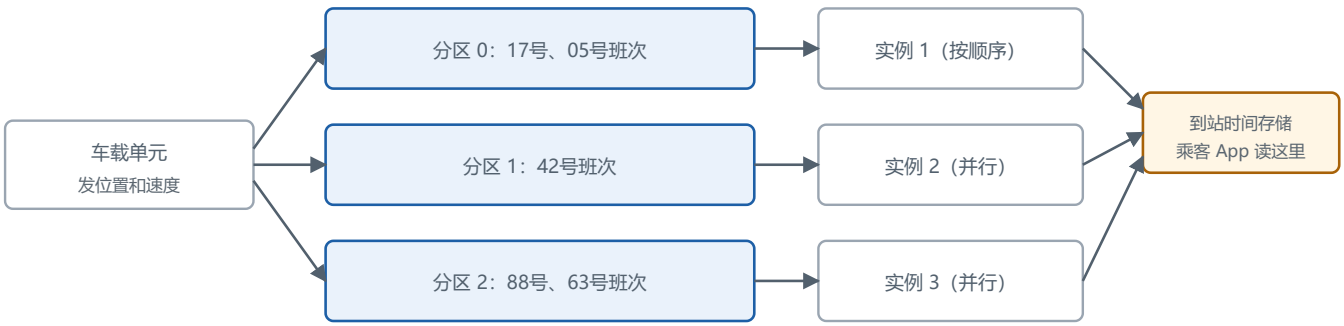


图 9-3 PAST2 公交平台：位置事件按班次号 busTripId 分区。同一班次永远进同一分区，由一个实例按顺序算；不同分区由不同实例并行算。消费者组 2（DesirableSpeedComputer）再读同一批事件，2 秒内回传理想车速

✕ 常见误区 | 这样答会丢分

- **抢票时用户是生产者，不是消费者**：用户点"抢票"是往队列里放一张订单；抢订单的是后台出票程序。一张订单只给一个实例，出完票就删。
- **实例不直接服务某个乘客**：实例 1 负责分区 0 里的所有班次，把结果写进存储；乘客 App 通过 API 读存储。
- PAST2 里第二个读同一批位置事件的是 DesirableSpeedComputer，不是报表。这正是要用事件流、不用队列的理由之一。

◆ 要点 | 必背结论 **扩展立方体：以公交平台为例**

轴	做法	公交平台例子	对应
X 轴	一模一样复制几份，放在负载均衡后面	到站计算 #1、#2、#3	无状态 + HPA
Y 轴	先按功能拆开，再只扩忙的那个	单体拆成到站计算、理想车速、线路管理；到站计算算不过来，只给它加一份"到站计算 2"，其他不动	微服务
Z 轴	按数据拆，每份只管一段	班次 1–1k、1k–2k、2k–3k 各存一个分片	分片 sharding

严格说，"加一份副本"这个动作属于 X 轴；Y 轴的作用是让你**能只扩那一个服务**，所以答题时 Y 和 X 常一起写。

✎ 小测验 | 先自己答，再看参考答案 **小测验 1：1 万辆公交的位置事件按班次分区：为什么既不乱序、又能并行?**

公交平台：**1 万辆公交车**每几秒上报一次位置。每个**班次**（一辆车跑一趟线路）的到站时间，必须**按这个班次位置发生的先后顺序**来计算。位置事件写入事件流（如 Kafka），**以班次号 busTripId 作为分区键**；一组计算实例**属于同一个消费者组**，共同消费这个事件流。为什么同一个班次的计算不会乱序？为什么还能同时处理 1 万辆车？

▾ **参考答案**

同一个班次号算出的哈希永远落在同一个分区；在同一个消费者组里，一个分区同一时间只交给一个实例，按顺序读，所以同一班次的位置按发生顺序处理。很多班次分散在多个分区里，多个实例同时处理不同分区，所以吞吐随分区数增加。前提：分区数要不少于实例数，否则多出来的实例会闲着。

✎ 小测验 | 先自己答，再看参考答案 **小测验 2：抢票系统用消息队列削峰：谁是生产者、谁是消费者?**

抢票系统开售瞬间涌入大量下单请求。**订单 API 先把每张订单放进消息队列**，然后立即回复用户"排队中"；后台有**一组出票程序**从队列里取订单出票。谁是生产者、谁是消费者？一张订单会被几个出票程序处理？如果某个出票程序**处理到一半崩溃了**，会怎样？

▾ **参考答案**

订单 API 是生产者（用户点"抢票"后由它把订单放进队列）；**出票程序**是消费者，它们竞争取订单。一张订单只交给一个出票程序处理，处理完并确认（ack）后才从队列删除。如果出票程序崩溃、没有确认，这张订单会重新投递给别的程序，所以出票要**幂等**：按订单号去重，避免重复出票。

✎ 小测验｜先自己答，再看参考答案 **小测验 3：网约车平台拆成微服务后：服务之间怎么通信?**

网约车平台拆成了乘客、订单、计价、派单、通知等微服务。乘客下单时要**马上知道下单是否成功、价格是多少**；司机接单后，要**通知乘客、更新司机统计、写入运营报表**。服务之间分别用什么方式通信？各举一个例子。

▾ **参考答案**

混合通信：要立刻拿到结果的用**同步**请求–响应（REST / gRPC，经服务发现调用），例如订单服务下单时同步调用计价服务拿价格。状态变化用**异步**事件（事件流 / 消息队列），例如派单服务发布"已接单"事件，通知、统计、报表各自订阅消费，彼此解耦，加新的下游也不用改派单服务。

❝ 可用句式｜___ 处填案例内容

- Internal communication is mixed. Queries that need an immediate answer, such as the fare, use synchronous REST calls through service discovery; state changes, such as "ride accepted", are published as events, so the notification and reporting services consume them asynchronously and stay decoupled.

  > 内部通信是混合的。要马上拿结果的查询（如车费）走同步 REST，经服务发现调用；状态变化（如"已接单"）作为事件发布，由通知、报表等服务异步消费，彼此解耦。

✎ 小测验｜先自己答，再看参考答案 **小测验 4：外卖平台的三个扩展难题：分别用扩展立方体（Scale Cube）的哪个轴解决?**

扩展立方体（AKF Scale Cube）有三个方向：X 轴复制相同实例、Y 轴按功能拆分服务、Z 轴按数据分片。下面三个问题分别用哪个轴解决？具体怎么做?

1. 晚高峰订单服务 CPU 打满，**单个实例扛不住**。
2. 订单、菜单、评价都在一个单体里，**改评价也得整体发布**；大促时只有订单模块忙，却只能整体加机器。
3. 订单表已有 **10 亿行**，单个数据库写不动。

▾ **参考答案**

1. ① → **X 轴**：订单服务做成无状态，在负载均衡后面放多个副本，配自动扩缩。
2. ② → **Y 轴**：按功能拆成订单、菜单、评价等微服务，各自发布、各自扩，只给忙的订单服务加副本。
3. ③ → **Z 轴**：按用户 ID 或城市对订单数据分片，把写入分散到多个分片。

三个方向可以组合使用；Y 轴拆开后"只给忙的服务加副本"这个动作本身属于 X 轴。

✎ 小测验 | 先自己答，再看参考答案 **小测验 5：下单后的三件事：用编排还是协同？**

电商下单成功后要做三件事：① **扣减库存：必须成功，失败要取消订单**；② **给用户发确认邮件**；③ 把这次购买记录写入推荐系统（推荐系统据此给用户推荐相关商品，**晚几分钟写入也没关系**）。

两种做法：**编排（Orchestration）**由一个协调者按顺序调用各个服务，并处理失败；**协同（Choreography）**订单服务只发出"已下单"事件，各服务自己订阅、自己处理。你会怎么设计？说明理由，并各说出编排和协同的一个优点、一个缺点。

▾ 参考答案

推荐混合：扣库存这一步用**编排**（如由订单协调者驱动的 Saga：调用库存服务扣减，失败就执行补偿、取消订单），因为它必须成功、失败要回滚，流程要显式可控；发邮件和写推荐用**协同**：订阅"已下单"事件，晚一点处理没关系，以后加新下游也不用改订单服务。

	优点	缺点
编排	流程显式、一目了然，失败和补偿集中处理	协调者知道所有步骤，容易变成中心、耦合高
协同	松耦合，加功能只需加订阅者	流程是隐式的，要靠分布式追踪和监控才能看清全局

✎ 小测验 | 先自己答，再看参考答案 **小测验 6：位置数据要同时供两套程序用，还要能重算历史：选消息队列还是事件流？**

一个物流平台的车辆位置上报**每秒上万条**，需要**同时供"实时地图"和"轨迹分析"两套程序使用**，并能在算法升级后**重算历史**。选消息队列还是事件流？给两个理由。

▾ 参考答案

事件流：① 同一批事件要被两套程序（两个消费者组）各自独立读取，消息队列投递后就删除，只能给一个消费者；② 事件持久保存、可以重放，算法升级后能从头重算历史。另外，按车辆分区可以保证每辆车的轨迹顺序正确。

❝ 可用句式 | ___ 处填案例内容

- Each onboard unit publishes its position to an event stream (Amazon Kinesis) partitioned by busTripId. ArrivalTimeComputer instances consume the partitions in parallel and write arrival times to DynamoDB, which the passenger app reads through the API.

  > 每台车载单元把位置发布到按 busTripId 分区的事件流（Amazon Kinesis）。ArrivalTimeComputer 的多个实例并行消费各分区，把到站时间写入 DynamoDB，乘客 App 通过 API 读取。

- DesirableSpeedComputer reads the same events as a second consumer group and returns the desirable speed within 2 seconds. A message queue cannot do this, because a message is deleted once one consumer has taken it.

  > DesirableSpeedComputer 作为第二个消费者组读同一批事件，2 秒内回传理想车速。消息队列做不到，因为一条消息被一个消费者取走就删除了。

本章术语表 Glossary

中文	English	一句话解释
微服务	Microservices	小的、可独立部署、围绕业务能力的服务
康威定律	Conway's Law	系统结构复制组织的沟通结构
智能端点哑管道	Smart Endpoints and Dumb Pipes	逻辑在服务里，通信保持简单
多语言持久化	Polyglot Persistence	不同数据用不同存储技术

中文	English	一句话解释
扩展立方体	Scale Cube (X/Y/Z axis)	克隆 / 功能拆分 / 数据分区三种扩展
同步阻塞	Synchronous Blocking	调用后阻塞等待响应
异步非阻塞	Asynchronous Non-blocking	调用后不等待，继续处理
单向通知	One-way Notification	发出请求，不期待响应
远程过程调用	gRPC / RPC	像调本地函数一样调远程服务
协议缓冲区	Protocol Buffers	gRPC 使用的二进制接口定义和序列化格式
消息队列	Message Queue	消息投递后删除，消费者竞争消费
事件流	Event Streaming	事件持久化、有序、可重放的日志
消息代理	Message Broker	在生产者和消费者之间转发消息的中间件
服务网格	Service Mesh	用 sidecar 代理管理服务间通信
边车代理	Sidecar Proxy	与服务部署在一起、代管其网络通信的代理
控制平面 / 数据平面	Control Plane / Data Plane	下发配置的管理层 / 实际转发流量的代理层
客户端发现 / 服务端发现	Client-side / Server-side Discovery	客户端自己查注册中心 / 由负载均衡器查
每服务一个数据库	Database per Service	每个服务独占自己的数据存储
命令查询职责分离	CQRS	读模型和写模型分开
事件溯源	Event Sourcing	以事件序列而非当前状态存储数据
通信亲和度	Communication Affinity	两个服务之间消息交换的数量和大小
共置	Collocation	把多个服务放进同一个容器实例

CHAPTER 10

Kubernetes 与托管服务

Containers, Kubernetes & Vendor Managed Services

课件来源：CNSD 03（容器与虚拟机）· CNSD 05a Kubernetes · CNSD 05b Vendor Managed Services · CNSD Discussion 6 · CNSD Workshop 02 · ArchSS 05（K8s 高可用）

对应题型 **T5**（部署计划）· **T10**

一句话 **容器把应用和依赖打包成可移植的轻量单元；Kubernetes 按你声明的"期望状态"调度、扩缩、自愈这些容器；能用托管服务的地方就别自己运维。**

学完本章你能

- 比较容器和虚拟机，回答"用 VM 还是容器"
- 说出 K8s 控制平面和节点上的组件
- 说出常用 K8s 对象的作用
- 区分 Service 与 Ingress
- 写出一份 K8s 部署计划
- 比较托管服务与自建容器

10.1 容器与虚拟机 Containers vs Virtual Machines

容器由两部分组成：**容器镜像** container image（应用及其依赖打包成的二进制包）和一组操作系统特性（命名空间 namespaces 隔离进程、控制组 cgroups 限制资源）。课件的一句话总结：**虚拟机隔离的是机器，容器隔离的是进程。**

对比	虚拟机 Virtual Machine	容器 Container
虚拟化层次	硬件层：经 Hypervisor（Type 1/2）	操作系统层：共享宿主内核（namespace、cgroups）
每个实例包含	完整的操作系统	只有应用及其依赖
大小与启动	GB 级，**分钟级**启动	MB 级，**毫秒到秒级**启动
隔离与安全	完全隔离，管理和安全工具成熟	进程级隔离，安全性相对弱
资源与密度	占内存多，单机密度低	占资源少，单机可跑很多
特点	硬件灵活	可移植，开发/生产环境一致
例子	VMware vSphere、VirtualBox、Hyper-V	Docker、Podman、LXC
适合	遗留应用、需要不同操作系统、强隔离要求	**微服务、快速扩缩容、CI/CD**

容器工作流：开发者编写应用 → 打包成容器镜像 → 推送到镜像仓库 registry → 编排平台拉取镜像并运行容器。容器的好处：创建和部署敏捷、支持持续集成和部署、开发与运维关注点分离、环境一致、跨云和操作系统可移植、以应用为中心管理、天然适合松耦合的微服务、资源隔离和利用率高。

✎ 考试怎么考 | 题型·分值·套路

T10 • VM 还是容器（PAST2 Q2b，3分）：**先引用案例 §4 的扩展要求**（用户量、峰值、更新频率），再说明选择。一般选**容器**：**秒级启动能跟上突发负载**，配合 HPA 按服务独立扩缩，资源密度高、成本低，开发和生产环境一致。最后说明扩展决策，例如"实时查询服务按 CPU/请求数水平扩展，数据更新服务按队列长度扩展"。

10.2 Kubernetes 架构 Kubernetes Architecture

Kubernetes（希腊语"舵手"，简称 K8s）的核心功能是把容器化的工作负载调度到整个基础设施上运行。你用 YAML **声明期望状态**（如"这个服务要 3 个副本"），K8s **持续把实际状态调整到期望状态**：实例挂了自动重建（**自愈**），负载高了

自动扩容。它不是 PaaS，不规定应用框架。其他编排工具有 Docker Swarm、Apache Mesos + Marathon。

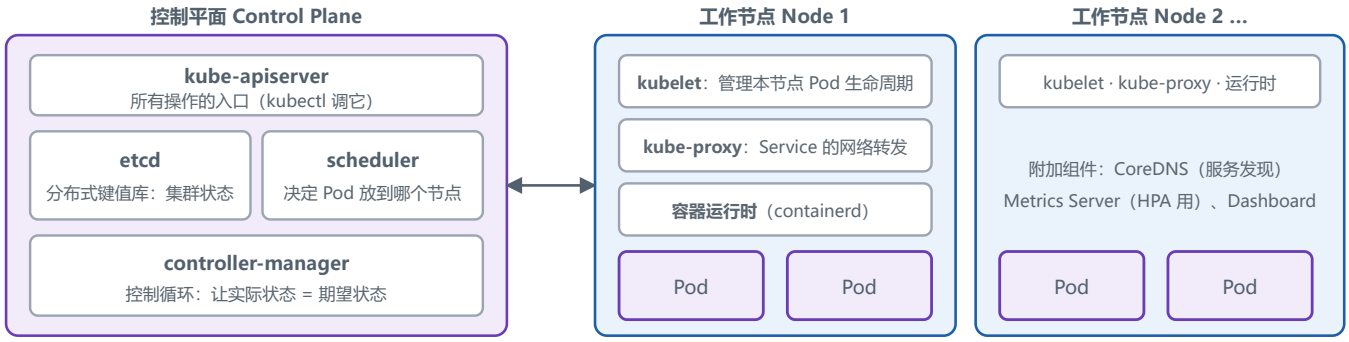


图 10-1 Kubernetes 集群：控制平面负责决策，节点负责运行 Pod

10.3 常用 Kubernetes 对象 Kubernetes Objects

对象	作用	办公楼类比 · 常见疑问
Pod	**最小部署单元**：一个或多个紧密相关的容器，共享 IP、存储和生命周期。一个 Pod 不会跨两个节点	一个工位；一个 Pod 不会跨两台节点
ReplicaSet	保证指定数量的 Pod 副本在运行，坏一个就补一个	保证工位数，坏一个补一个（Deployment 底层）
Deployment	管理**无状态**应用：声明副本数和 Pod 模板，支持滚动更新和回滚（底层管理 ReplicaSet）	经理："始终保持 N 个工位开着"。HPA 扩缩就是去改它的副本数
Service	为一组 Pod 提供**稳定的虚拟 IP 和 DNS 名**，并做负载均衡；按**标签选择器** label selector 选中 Pod。类型：ClusterIP（集群内）、NodePort、LoadBalancer（对外）	公司总机：Pod 重建后 IP 会变，客户端只记总机号；靠**标签选择器**找到通过就绪探针的 Pod
Ingress	把集群外的 HTTP/HTTPS 请求按域名、路径路由到不同 Service，一个外部 IP 暴露多个服务	大楼门口导览台：集群外的请求按路径分到不同 Service
Namespace	把集群资源分组隔离，如按环境（dev/prod）或团队	部门（逻辑分组）≠ 节点（物理机器）；加机器是加进整个集群，不是加进某个 namespace
ConfigMap / Secret	存放配置 / 敏感信息（密码、密钥），与镜像分离（十二要素的"配置"）	配置和密码与镜像分开
StatefulSet	管理**有状态**应用（数据库、Kafka、Redis）：固定名字、有序部署、每个 Pod 有自己的持久卷	有工号和储物柜的员工：重建后还是 db-0、挂回自己的盘；Deployment 的 Pod 没有身份，缩了再扩不会"接回"旧 Pod
PV / PVC	持久卷（管理员供给或按 StorageClass 动态供给的存储）/ 用户对存储的申请	一块盘 / 租盘申请单
DaemonSet	每个节点运行一个 Pod，如日志收集代理 Fluentd	每台节点一个，如 Fluentd 收集本节点日志；是部署方式，不是微服务
Job / CronJob	运行到完成的一次性任务 / 按时间表运行的定时任务	一次性任务 / 每晚统计成绩
HPA	**水平 Pod 自动扩缩** HorizontalPodAutoscaler：按 CPU、内存或自定义指标增减副本数；节点不够时由集群自动扩缩器 Cluster Autoscaler 加节点	按排队人数开关窗口；节点不够时由 Cluster Autoscaler 加机器
探针 Probes	存活探针 liveness（不健康就重启）、就绪探针 readiness（准备好才接流量）	liveness 不健康就重启；readiness 准备好才接流量

Service：集群内部连通

为构成一个逻辑服务的一组 Pod 提供内部网络连通，让集群内组件互相通信；也可以用基本的负载均衡把**单个**服务暴露到外部。

Ingress：智能的外部网关

基于 HTTP/HTTPS 管理外部对集群内**多个**服务的访问；在一个外部 IP 下暴露多个服务，并做高级路由和流量管理。

一个最小的 Deployment 声明（课件示例的简化版）：

```
apiVersion: apps/v1
kind: Deployment
metadata: { name: voting-svc, namespace: vote-prod, labels: { app: voting } }
spec:
  replicas: 3                      # 期望 3 个副本
  selector: { matchLabels: { app: voting } }
  template:
    metadata: { labels: { app: voting } }
    spec:
      containers:
      - name: voting
        image: registry.example.com/voting:1.4.2
        ports: [ { containerPort: 8080 } ]
        readinessProbe: { httpGet: { path: /health, port: 8080 } }
```

10.4 写一份 K8s 部署计划 Kubernetes Deployment Plan

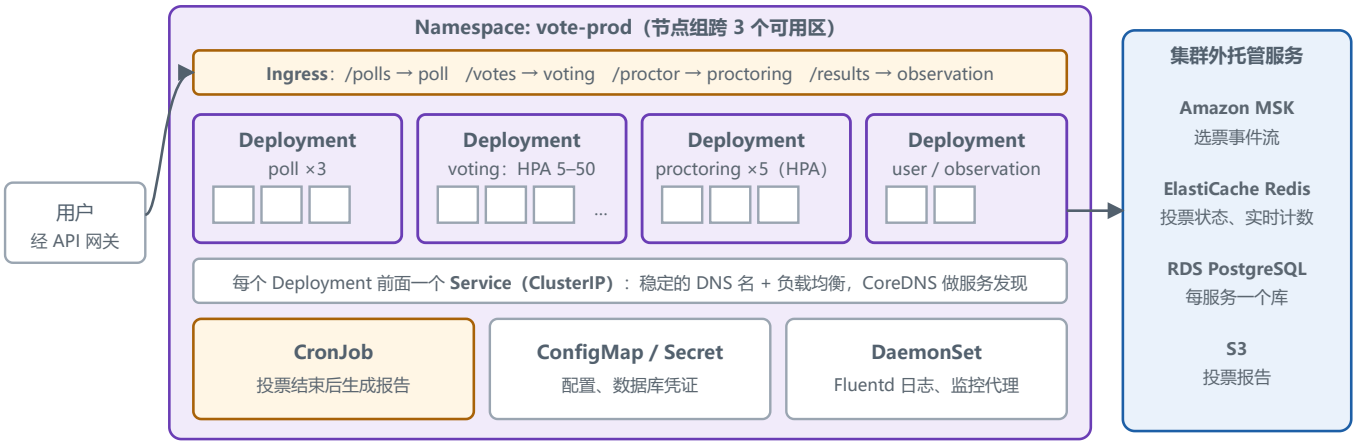


图 10-2 Vote Anywhere 的 K8s 部署计划（有状态组件放在集群外的托管服务中）

◆ 要点｜必背结论　部署计划答题模板（PAST Q2a(iv)，3分：Pod、Deployment、Service）

①用一个 **Namespace**（如 vote-prod）隔离生产环境，节点组跨多个可用区。②每个微服务打包成镜像，以 **Deployment** 部署，Pod 至少 2–3 个副本，负载大的（voting、proctoring）配 **HPA** 按 CPU 或请求数在 5–50 个副本间伸缩，节点不足时 Cluster Autoscaler 加节点。③每个 Deployment 前面一个 **ClusterIP Service** 提供稳定地址和负载均衡，服务之间经 CoreDNS 发现、经 Istio sidecar 通信。④**Ingress** 按路径把外部请求路由到各 Service（前面再接 API 网关）。⑤**有状态组件（Kafka、Redis、数据库）用托管服务或 StatefulSet + PVC**。⑥报告生成用 **CronJob**，配置和凭证用 ConfigMap/Secret，日志代理用 DaemonSet。⑦发布用滚动更新或金丝雀。

10.5 托管服务与自建容器 Managed Services vs Self-hosted Containers

托管 Kubernetes：Amazon EKS、Azure AKS、Google GKE（K8s 诞生于 Google，支持度最高）。AWS 另有专有的容器服务 ECS 和无服务器容器 Fargate。云厂商的托管服务按类型可分为：计算（VM、K8s、FaaS）、存储（对象、文

件、块）、网络（负载均衡、DNS、防火墙）、数据（关系库、NoSQL、数据仓库、机器学习）、流与消息（流处理、事件驱动、消息队列、通知）、性能与安全（IaC、身份管理、密钥加密）。完整对照表见附录 A。

CNSD 讨论题 6 以部署大模型为例，比较了"用托管服务"和"自己在 K8s 上跑容器"，结论同样适用于一般服务：

对比	托管服务 Managed Service	自建容器 Self-hosted Containers
成本模式	按用量计费（OpEx），空闲时没有基础设施成本	按预留资源计费，与利用率无关，空闲也要付钱
成本可预测性	随流量线性变化，突发时预算难控	阶梯式：资源用满前不变，之后要整台加节点
利用率风险	厂商承担	自己承担（利用率低时浪费严重）
运维责任	硬件、升级、扩缩、SLA（常见 99.9%+）由厂商负责	硬件、升级、扩缩策略（HPA）、冗余容灾都自己负责；**隐性成本常是运维人力**
性能与定制	受多租户"吵闹邻居"影响，有网络往返；定制有限	可部署在应用旁边、延迟最低；可完全定制
数据与锁定	数据离开自己的 VPC；厂商锁定明显	可完全隔离甚至离线部署，适合强监管行业；可移植
适合	**突发、不可预测或中低流量**	**高而稳定、7×24 的吞吐**，能把资源用满

10.6 白话讲解与小测验 Plain-language Notes & Drills

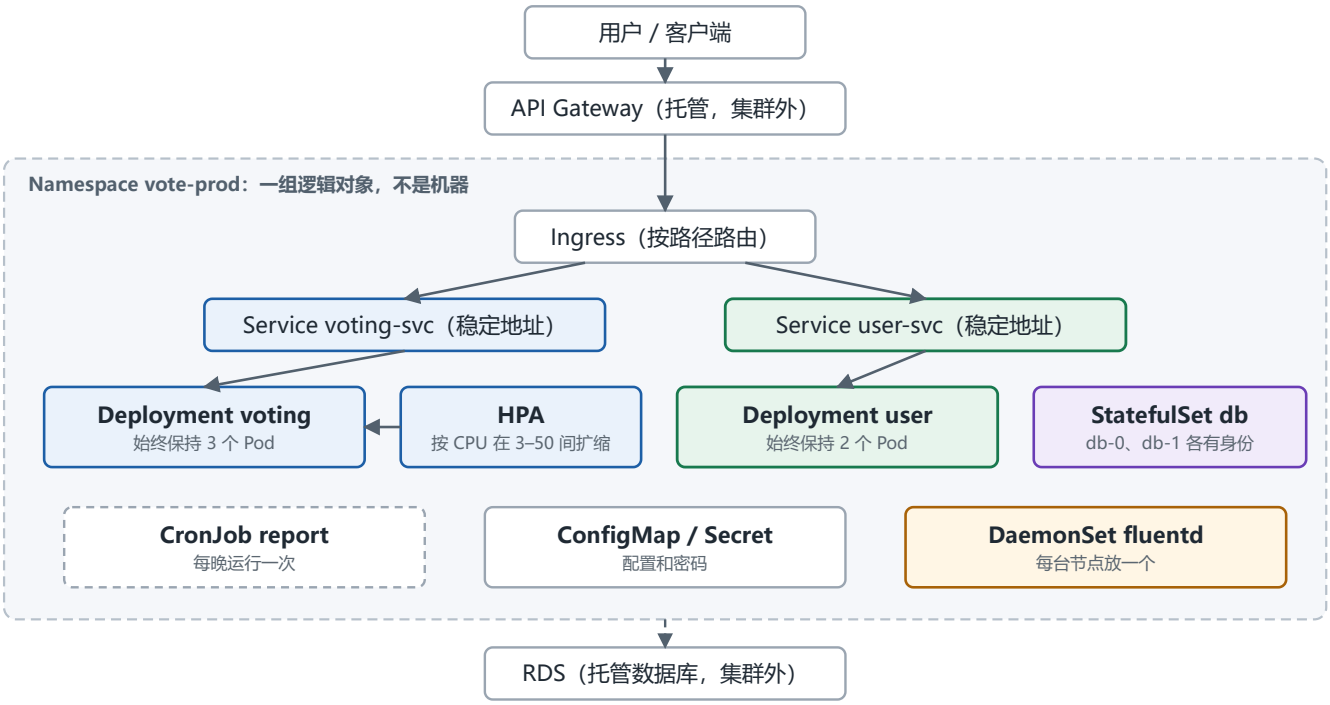


图 10-3 逻辑设计：Namespace 里放的是 K8s 对象。请求经 API Gateway → Ingress → Service，Service 靠标签找到 Deployment 管的 Pod；HPA 改 Deployment 的副本数；同色 = 同一服务

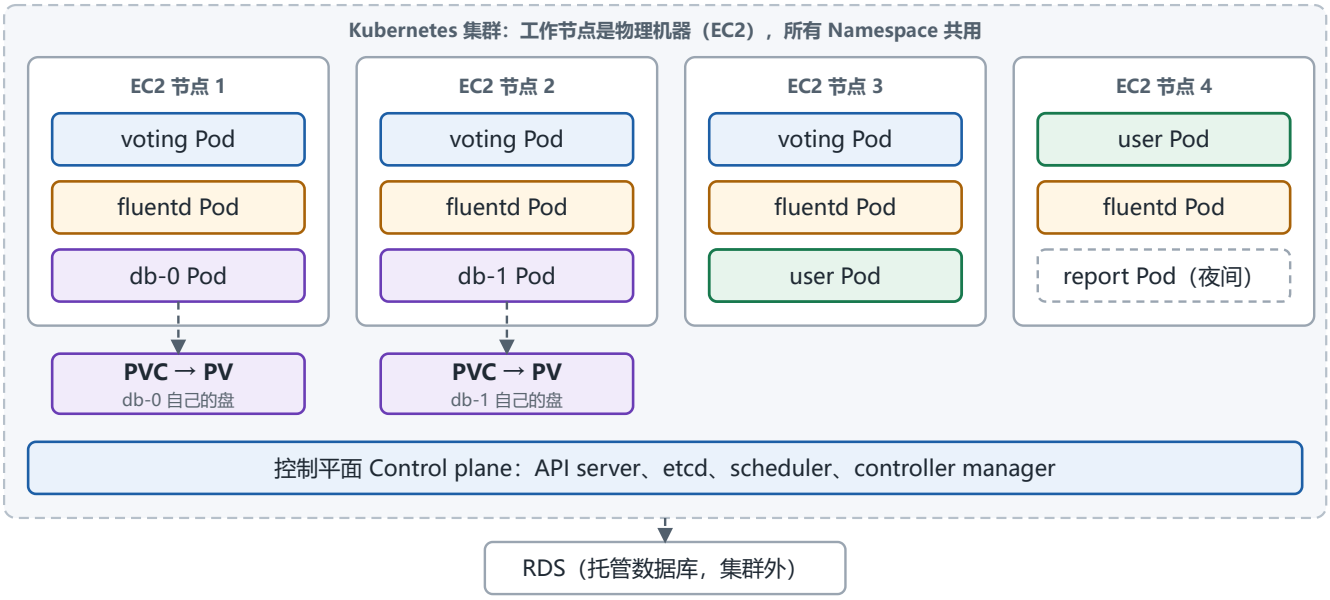


图 10-4 物理设计：Pod 真正跑在节点上。同一个 Deployment 的 Pod 分散在多台节点（一台挂了还有别的）；DaemonSet 让每台节点都有一个 fluentd；StatefulSet 的 db-0、db-1 各挂自己的盘；加节点是加进整个集群

◆ 要点｜必背结论 **副本数怎么定**

数字	依据
最小副本数	为了可用性：至少 2–3 个，分散在不同可用区
最大副本数	峰值负载 ÷ 单个 Pod 能承载的量，再留余量。PAST：10 万并发选民，假设一个 Pod 约 2,000 人 → 最多 50 个

考试里把假设写出来；HPA 只给负载大的服务配，平时负载低的服务固定副本即可。

✎ 小测验｜先自己答，再看参考答案 **小测验 1：每项需求各用哪个 Kubernetes 对象?**

为下面四项需求各选一个 Kubernetes 对象，并写一句理由：

1. 对外提供的**无状态** Web / API 服务（如商品查询）。
2. 自己在集群里运行的 **PostgreSQL 主从数据库**。
3. **每天凌晨**跑一次的汇总报表批处理任务。
4. **每台节点上都要运行**一个日志收集程序。

▾ **参考答案**

1. **Deployment**（保持副本数、支持滚动更新），前面配 Service，对外再经 Ingress 或 LoadBalancer
2. **StatefulSet + PVC**（固定身份、每个 Pod 有自己的持久卷）
3. **CronJob**（按时间表运行）
4. **DaemonSet**（每台节点正好一个 Pod）

✎ 小测验 | 先自己答，再看参考答案 **小测验 2：PostgreSQL 用 Deployment 部署，缩容再扩容后数据丢了：为什么？**

团队把 PostgreSQL 主从也**用 Deployment 部署**在 Kubernetes 上，数据写在容器本地目录（**没有挂持久卷**）。一次**缩容再扩容**后，发现**数据丢了、主从关系也乱了**。为什么？数据库应该怎么部署？无状态的 API 服务又该用什么？

▾ **参考答案**

Deployment 管的是**相同、可互换**的 Pod：名字随机，没有固定身份，也不会给每个 Pod 分一块专属的持久卷；数据写在容器本地，Pod 一删就没了，缩了再扩出来的是全新的 Pod，所以数据和主从关系都丢了。有状态的数据库应该用 **StatefulSet + PVC**：每个 Pod 有固定名字（db-0、db-1）、按顺序启动、挂自己的持久盘，重建后挂回原来的盘；生产环境更推荐直接用托管数据库（如 RDS）。无状态的 API 服务用 **Deployment**，副本随意替换，配 HPA 扩缩。

✎ 小测验 | 先自己答，再看参考答案 **小测验 3：为在线考试平台写一份 Kubernetes 部署计划**

在线考试平台有三个服务：exam（答题）、grading（判卷）、user（登录）。约 2 万名考生每晚 **19:00 同时开考登录**、**21:00 同时交卷**，**其余时间几乎没人用**。平台运行在 Amazon EKS 上。

写一份部署计划，至少包括：每个服务用的对象、副本数和可用区分布、扩缩策略、入口、配置管理。单个 Pod 能承载多少考生等数字**可以自行假设，但要写明**。

▾ **参考答案**

假设：一个 exam Pod 约承载 1,000 名在线考生，一个 user Pod 约承载 2,000 名考生的登录请求。

项目	计划
命名空间	exam-prod
对象	三个服务各一个 Deployment + ClusterIP Service；配置放 ConfigMap，密码放 Secret
副本与可用区	平时每个服务至少 3 个副本，分布在 3 个可用区
扩缩：开考	18:45 按时间表提前把 exam 扩到约 20–25 个、user 扩到约 10 个（开考时间已知，等 HPA 被动扩容来不及）；HPA 按 CPU 或请求速率继续兜底；节点由 Cluster Autoscaler / Karpenter（或 Fargate）同步扩容，避免 Pod 因没有节点而 Pending
扩缩：交卷	20:55 提前扩 grading；交卷请求先进队列，grading 按队列长度扩缩，慢慢判
入口	API Gateway → Ingress，按路径路由到三个 Service
有状态部分	数据库、缓存用托管服务（RDS、ElastiCache），不放在集群里

✎ 小测验 | 先自己答，再看参考答案 **小测验 4：订单服务把支付服务的 Pod IP 写死在配置里，支付服务滚动更新后开始报错**

订单服务的配置文件里**写死了支付服务 3 个 Pod 的 IP 地址**。支付服务做了一次**滚动更新**后，订单服务开始大量报错。为什么？应该怎么改？

▾ **参考答案**

Pod 会被销毁重建、扩缩，**重建后 IP 就变了**，写死的地址失效。应在支付服务的 Pod 前面放一个 **Service**：它提供稳定的虚拟 IP 和 DNS 名（如 `payment-svc`），靠标签选择器自动找到所有通过就绪探针的 Pod 并做负载均衡；订单服务只调用 Service 名，不关心 Pod 怎么变。

✎ 小测验｜先自己答，再看参考答案 **小测验 5：流量很不稳定、只有 3 名开发：用 Serverless 还是自建 Kubernetes?**

一家初创公司的 API **流量很不稳定**：白天偶尔有突发，**夜里几乎为零**。团队**只有 3 名开发、没有专职运维**。两个方案：A 用 API Gateway + AWS Lambda（Serverless，按调用付费，扩缩由 AWS 负责）；B 在 EC2 上自己搭 Kubernetes 集群。选哪个？给两条理由，说一个代价，并说明什么情况下应该选另一个。

▾ **参考答案**

选 **A（Serverless）**。理由：① 按调用付费，夜里几乎不花钱，适合不稳定的流量；② 扩缩、补丁、可用性都由 AWS 负责，3 个人不用养运维。代价：冷启动延迟、单次执行时长有限、对 AWS 锁定更深。什么时候选 B：流量变得高而稳定、长时间运行，需要深度定制运行环境，或者有了专职运维团队时，自建或托管 K8s 的单位成本更低、控制更多。

本章术语表 Glossary

中文	English	一句话解释
容器 / 容器镜像	Container / Container Image	隔离的进程 / 应用及依赖的打包
虚拟机监控器	Hypervisor	在硬件上运行多个虚拟机的软件
命名空间 / 控制组	Namespaces / cgroups	Linux 隔离进程 / 限制资源的内核特性
镜像仓库	Container Registry	存放和分发容器镜像的服务
容器编排	Container Orchestration	自动部署、扩缩、管理容器
期望状态	Desired State	用声明式配置描述的目标状态
控制平面	Control Plane (API Server, etcd, Scheduler)	K8s 的决策和状态管理部分
节点代理	kubelet	管理节点上 Pod 生命周期的代理
Pod	Pod	K8s 最小部署单元，含一个或多个容器
部署	Deployment	管理无状态应用副本和滚动更新
服务（K8s）	Service (ClusterIP / LoadBalancer)	一组 Pod 的稳定访问地址与负载均衡
入口	Ingress	按域名/路径把外部请求路由到 Service
有状态集	StatefulSet	管理有固定标识和持久卷的有状态应用
持久卷 / 持久卷申领	PersistentVolume / PersistentVolumeClaim	集群中的存储 / 对存储的申请
守护进程集	DaemonSet	每个节点运行一个 Pod
定时任务	CronJob	按时间表运行的 Job
水平 Pod 自动扩缩	HorizontalPodAutoscaler (HPA)	按指标自动增减 Pod 副本
集群自动扩缩器	Cluster Autoscaler	按需增减工作节点
存活 / 就绪探针	Liveness / Readiness Probe	检查是否需要重启 / 是否可接流量
标签选择器	Label Selector	按标签选中一组对象
托管服务	Managed Service	由云厂商运维的服务
吵闹邻居	Noisy Neighbour	同一硬件上其他租户抢占资源导致性能波动

CHAPTER 11

数据管理与云存储

Data Management & Cloud Persistence

课件来源：PE 06 Data Management · CNSD 04 Cloud Persistence · CNSD 10（分片模式、CQRS）· ArchSS 02（CAP、ACID/BASE）· PAST Q3(b) 官方答案

对应题型 T6、T11

一句话 **没有一种数据库适合所有数据：按读写模式和质量属性为每类数据选存储；读多加副本，写多要分片，热点数据加缓存。**

学完本章你能

- 说出 RDBMS 的优势和扩展难点
- 比较键值、文档、列族、搜索引擎、图数据库
- 解释复制与分片分别扩展什么
- 用 CAP 定理解释一致性与可用性的取舍
- 为写吞吐瓶颈设计分片方案并指出副作用
- 按质量属性选择云存储服务

11.1 平台对数据存储的要求 What We Need from Datastores

核心是能存能读；此外平台还需要：**可扩展**（垂直/水平）、**一致性**、**可用性**、**低延迟**、**可演进**（schema 能变）、能做分析、安全。可扩展性和可用性往往相互关联。数据分三类：**结构化**（固定的行和列，严格 schema，如 RDBMS 表）、**半结构化**（有模式的文本，schema 随数据走，如 JSON、XML、CSV）、**非结构化**（纯文本、PDF、图片、二进制）。大数据常用 "6V" 描述：体量 Volume、速度 Velocity、多样 Variety、真实性 Veracity、价值 Value、可变性 Variability。

11.2 关系型数据库与 NoSQL RDBMS & NoSQL

RDBMS 适合大多数场景：每个领域实体对应一张表，结构直观；可以用逻辑数据模型描述；SQL 能按任意列灵活查询；读写性能好；**强 ACID**；规范化 normalization 减少冗余、保证完整性。**难点**：在保持一致性的前提下**水平扩展很难**，常用办法是分片和复制；现代应用对读写性能的要求越来越高。

为什么需要 NoSQL：每个平台的需求不同（写密集、读密集、分析型、查询模式多样）；扩展和可用性要求各异；平台变化快，schema 经常改。NoSQL 大多能随水平扩展**线性扩展**，分区和复制是内置的；ACID 较弱、最终一致；很多支持内存存储。**NewSQL** 兼具 SQL 关系模型和 NoSQL 的水平扩展能力（如 Google Spanner、CockroachDB）。

模型	解决什么问题	擅长	短板	例子（课件的 CAP 归类）
键值 Key-Value	给定键，尽快取回整条记录	按键查找极快（点查询）；购物车、会话、缓存、限流计数、配置	范围查询代价高；需要大量内存，持久化影响性能	Redis（CP）、DynamoDB、Memcached
文档 Document	要丰富查询能力，又不想受严格 schema 约束	文档（JSON）可嵌套子实体，schema 演进平滑；按 ID 查询快、支持索引	多数不支持 join、复杂事务、约束和触发器	MongoDB（CP）、Amazon DocumentDB、Firestore
列族 Column Family / Wide Column	**写密集**（每秒数万条），schema 极灵活、值稀疏	巨大的分布式嵌套映射，线性大规模扩展；网页抓取、消息、崩溃日志、时序	无 join；二级索引慢且贵；不是通用数据库	Cassandra、HBase（CP）、Bigtable、Keyspaces

模型	解决什么问题	擅长	短板	例子（课件的 CAP 归类）
搜索引擎 Search Engine	需要排序、通配、过滤、聚合等高级查询	全文检索、日志分析	插入慢（要分词）；无事务；**不宜作主存储**	Elasticsearch / OpenSearch（AP）
图 Graph	关心事物之间的关系	社交关系、推荐、物流路线、客户关系分析	聚合要遍历全库；水平扩展难	Neo4j、Amazon Neptune
用 SQL 查文件	对存在文件里的大量数据做分析	不是数据库：SQL 解析器 + 作业生成器 + 元数据管理	不适合在线事务	Hive、Athena、BigQuery、Snowflake

11.3 多语言持久化与 CQRS Polyglot Persistence

多语言持久化 polyglot persistence：用不同的存储技术满足不同的数据需求，可以在整个企业层面，也可以在一个应用内部。现代平台大多同时使用多种数据库，例如订单用 RDBMS、会话用 Redis、商品目录用文档库、搜索用 Elasticsearch、历史分析用数据仓库。配合 **CQRS**，写模型和读模型用不同的库，读写可以独立扩展（第9章）。

11.4 扩展数据库：复制、分片、缓存 Scaling Databases

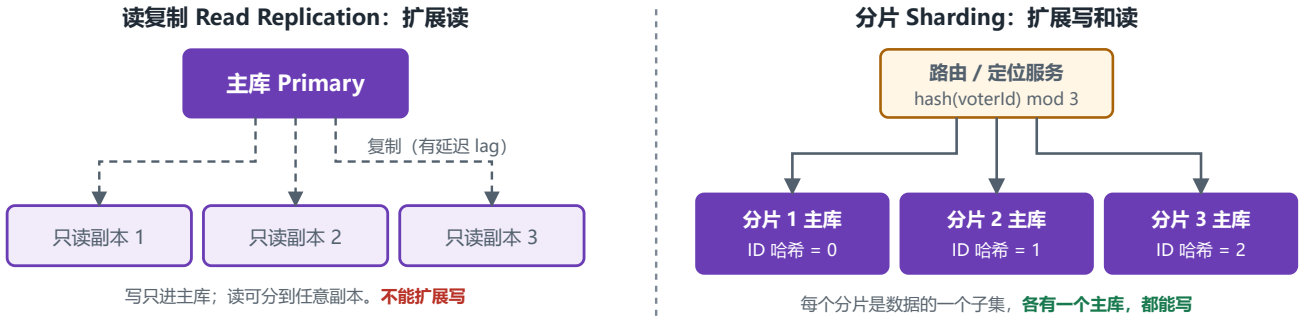


图 11-1 复制与分片：复制把同一份数据放多份，分片把不同数据放到不同节点

手段	做法	扩展什么	代价
读复制 Read Replication	只有主库接受写；只读副本有全量数据，读请求可路由到任意副本；一个副本挂了就转到其他副本	读吞吐、可用性	有**复制延迟** replication lag，副本可能读到旧数据；写仍是单点瓶颈
分片 Sharding（水平分区）	按**分片键** shard key 把数据分到多台机器，每片一个主库。用**定位服务**告诉客户端哪片负责哪些数据，或用**算法分片**（如哈希，算法在客户端，不需要定位服务）	**写吞吐**和读吞吐	固定键可能分布不均；**热点分片**；跨分片查询和聚合；跨分片事务；重新分片 rebalancing 复杂
缓存 Caching	缓存层"假装"是数据库：未命中或过期时**读穿透** read-through，写时**写穿透** write-through 保证持久	读和写（内存比磁盘快几个数量级）	缓存层在刷盘前宕机会丢数据；缓存一致性

分片策略（CNSD 10）：**哈希策略分布均匀，减少热点**；**范围**策略把相关数据放在同一片并有序，便于范围查询，但容易出现热点；**查找表**策略用映射表决定数据在哪片，灵活但多一次查询。另有**垂直分区**：按列或按功能把表拆到不同库。

11.5 CAP 定理与一致性 CAP, ACID & BASE

■ 定义｜课件正式说法 **CAP 定理（Brewer）**

一个共享数据的分布式系统，**一致性** Consistency（提交后每次读都拿到最新值）、**可用性** Availability（总能返回某个版本的数据）、**分区容错** Partition tolerance（后端网络部分故障时仍能工作）三者最多同时满足两个。分布式系统**必须**选分区容错，**所以实际是在网络故障时选 C 还是选 A**。

CP：选一致性

主库和副本之间网络断开时，保持一致，但副本可能无法接管，牺牲了可用性。**只从主库读（没有读副本）的系统就是 CP。**银行转账、库存扣减。

AP：选可用性

从副本读可能读到旧数据，系统只保证**最终一致。**课件例子："一致性被高估了"：Amazon 购物车、Twitter 时间线、Facebook 动态、Google 广告，有些结果总比没有好。

ACID（关系库事务）	BASE（很多 NoSQL）
原子性 Atomicity：事务里的操作要么全成，要么全不成；**一致性** Consistency：事务前后数据库都处于一致状态；**隔离性** Isolation：并发事务互不干扰；**持久性** Durability：提交后不会丢	**基本可用** Basically Available；**软状态** Soft state：状态可能随时间变化；**最终一致** Eventually consistent：一段时间后各副本一致。适合博客、状态更新、商品列表等不需要强一致的数据

11.6 云上的存储选项 Cloud Persistence Options

存储选项	特点	AWS	Azure	Google Cloud
本地文件系统	实例自带磁盘，实例停止或磁盘故障**数据丢失**	EC2 实例存储	VM 临时磁盘	Local SSD
网络文件系统	多个实例共享文件	EFS	Azure Files	Filestore
挂载卷（块存储）	块级卷挂到运行中的 VM，独立于实例存活	EBS	Managed Disks	Persistent Disk
对象存储	对象 = 数据 + 元数据，通过 REST API 访问；便宜、海量、高耐久；适合图片、报告、备份、数据湖	S3	Blob Storage	Cloud Storage
关系库	托管 RDBMS，自动备份、多可用区	RDS、Aurora	Azure SQL、Database for PostgreSQL	Cloud SQL、AlloyDB、Spanner
键值 / 文档	托管 NoSQL，自动扩展	DynamoDB、DocumentDB	Cosmos DB	Firestore、Bigtable
内存缓存	毫秒级读写	ElastiCache（Redis/Memcached）	Azure Cache for Redis	Memorystore
图 / 搜索	关系查询 / 全文检索	Neptune / OpenSearch	Cosmos DB Gremlin / AI Search	— / Vertex AI Search
数据仓库 / 时序	分析查询 / 按时间存指标	Redshift / Timestream	Synapse / Data Explorer	BigQuery / Bigtable

▶ 案例演练｜演示怎么套用 **PAST Q3(b)（6分）：10万选民 5 分钟内投票，数据库写吞吐成为瓶颈（现有一主一读副本）**

为什么加副本没用：读副本只能分担读，所有写仍然只进一个主库。要扩展写，必须分片。

分片方案（2分）：**按选民的唯一标识（Voter ID、NRIC、护照号）做哈希分片**，每个分片有自己的主库（可再配读副本）；还可以按选民分片分配不同的 Proctor 组。

受影响的组件及理由（4分）：**VoteController** 要知道分片规则，把选票写进正确的分片；**PollManager** 要把投票记录到正确的分片；**UserManager** 因为用户分布在不同分片里；**ObservationManager 要跨分片汇总结果**。基础设施方面：增加分片主库及其副本、引入分片路由（定位服务或客户端算法）、调整连接池；还可在前面加事件流缓冲写入峰值、批量写入。

要密切监控的副作用：①数据分布不均造成**热点分片**；②跨分片的结果聚合变慢，可能影响"1 分钟内更新简要结果"；③"每位选民只能投一票"这类约束要跨分片保证，一致性更难；④重新分片和扩容复杂；⑤每个分片的复制延迟；⑥运维成本和备份恢复复杂度上升。

11.7 按质量属性选择存储 Choosing Storage by Quality Attributes

T6 题常分两步：先说出存储最重要的两个质量属性并论证（如**写吞吐的可扩展性**和**读取的低延迟**），再推荐两个云服务并说明它们**如何满足**这两个属性。可用下表对号入座：

需求特征	存储类型	推荐服务（AWS 为例）	满足属性的理由
海量高频写入（遥测、位置、选票）	键值 / 列族 / 时序	DynamoDB、Keyspaces、Timestream	按分区键自动分片、按需容量模式自动扩展写吞吐
毫秒级读、实时计数、会话	内存缓存	ElastiCache for Redis	数据在内存中，亚毫秒延迟；支持集群分片和副本
强一致事务（订单、支付）	关系库	RDS / Aurora（Multi-AZ）	ACID 事务；多可用区自动故障转移；Aurora 最多 15 个读副本
结构多变的文档、目录	文档库	DocumentDB、MongoDB Atlas	schema 灵活、可按任意字段建索引
全文搜索、日志分析	搜索引擎	OpenSearch Service	倒排索引、聚合查询
历史数据分析、报表	数据仓库 / 数据湖	Redshift、S3 + Athena	列式存储、按查询付费，适合大批量分析
图片、报告、备份文件	对象存储	S3	几乎无限容量、11 个 9 的耐久性、成本低

✕ 常见误区｜这样答会丢分

- 写吞吐不够时答"加读副本"：副本只扩展读。
- 只写"用 NoSQL"，不写具体产品。老师说过：题目要求推荐云服务时，**必须写出具体厂商的服务名**。
- 分片方案只写"按 ID 分片"，**不说哪些组件要改、为什么**，丢掉大部分分数（6 分里占 4 分）。

11.8 T6 答法与小测验 T6 Templates & Drills

◆ 要点 | 必背结论 **逻辑节点 → AWS（画物理部署图用）**

逻辑位置	AWS	常成对出现
用户入口	Route 53 + CloudFront + S3	Route 53 解析域名，CloudFront 就近分发 S3 上的静态内容
DMZ	API Gateway + WAF（公有子网）	WAF 挡攻击，API Gateway 统一入口、限流
Application / Platform	私有子网的 EKS 或 Beanstalk + Lambda	常驻服务用容器，事件触发的用 Lambda
Data Store	RDS、DynamoDB、ElastiCache、S3	按数据特点各选一种（多语言持久化）
Integration	集成子网，经 NAT 出去	对接支付网关、短信
认证	Cognito	

画法：VPC、两个可用区、公有 / 私有子网。每个架构决策写"选了什么 + 为什么 + 满足哪个需求"。

✎ 小测验 | 先自己答，再看参考答案 **小测验 1：外卖平台上 AWS：请求经过哪些服务？三类数据各存哪里？**

外卖平台部署在 AWS，有三类场景：① 用户打开 App 浏览菜单，**菜单里有大量菜品图片**；② 用户登录、下单并支付，**订单必须可靠保存**；③ 骑手 App **每 5 秒上报一次位置**，用户在订单页**实时看骑手在哪里**。

(a) 用户下单这个请求从手机到数据库，至少写出 DNS、CDN、入口（安全防护与 API 网关）、计算、数据库各用哪个 AWS 服务，按先后顺序排列。(b) 菜品图片、订单、骑手位置各存在哪里？为什么？

▾ **参考答案**

(a) DNS：Route 53 → CDN：CloudFront（静态资源和菜品图片就近分发）→ 入口：WAF + API Gateway（统一入口、限流，Cognito 负责登录）→ VPC Link → 内部 ALB / NLB（EKS Ingress）→ 计算：私有子网里 EKS 上的订单、支付服务 → 数据库：RDS。

数据	存储	理由
菜品图片	S3（前面接 CloudFront）	大文件、读多写少，就近分发
订单	RDS（多可用区）	需要事务和强一致，不能丢
骑手位置	DynamoDB（最新位置可再放 ElastiCache）	每 5 秒写一次、写入量大，按骑手 ID 读最新值，毫秒级

✎ 小测验｜先自己答，再看参考答案 **小测验 2：投票高峰每秒数万次写入，数据库主库写不过来：怎么办？**

投票平台：**只有一个投票项目**，约 1000 万选民，选举日高峰**每秒有数万张选票写入**。现在只有一个 PostgreSQL 主库加**两个只读副本**，**主库 CPU 打满**、写入排队。请回答：

1. 再多加几个只读副本能解决吗？为什么？
2. 怎么分片？分片键怎么选？
3. 分片后，举两个会受影响的功能或组件。

▾ 参考答案

1. 不能：所有写入仍然只进主库，只读副本只能分担读。
2. 按 **voterId 哈希**分片：选民 ID 很多，写入能均匀分到各分片；"一人一票"的校验也能在单个分片内完成。不能按投票项目 ID 分（只有一个项目，全部写入会落到同一个分片）。
3. ① 计票和结果排行要跨所有分片汇总；② 以后增加分片需要迁移数据（重新分片）。另外可以先让选票进入队列削峰，再异步写库。

✕ 常见误区｜这样答会丢分

- 只写"用 NoSQL""加缓存"：老师说过必须写出具体厂商的服务名，并写满足哪个属性、靠什么机制。
- 库存、余额、订单、支付用了最终一致的存储：要强一致，选 RDS / Aurora。
- 缓存当主存储：缓存可能丢数据。

✎ 小测验｜先自己答，再看参考答案 **小测验 3：社交平台的"好友推荐"：选什么类型的数据库？**

社交平台要做"好友推荐"：给每个用户推荐"**朋友的朋友**"里还不认识的人，需要在用户关系网里**往外查两三层**。选什么类型的数据库？给出一个 AWS 服务名，并说明为什么关系型数据库不合适。

▾ 参考答案

图数据库，例如 Amazon Neptune：好友推荐就是在关系网里做多层遍历，图模型最自然、查询最快。关系型数据库要用多次自连接（JOIN）来查多层关系，层数一多查询就变得很慢、很难写。

本章术语表 Glossary

中文	English	一句话解释
结构化 / 半结构化 / 非结构化数据	Structured / Semi-structured / Unstructured Data	固定表结构 / 自带模式的文本 / 无固定结构
规范化	Normalization	拆表以减少冗余、保证完整性
键值存储	Key-Value Store	按键快速存取整条记录
文档存储	Document Store	以 JSON 等文档存数据、schema 灵活
列族存储	Column Family / Wide-Column Store	行键 + 列族的分布式映射，擅长海量写入
图数据库	Graph Database	以节点和边存储关系
新型关系库	NewSQL	兼具 SQL 与水平扩展的数据库
读副本	Read Replica	只读的数据副本，用于扩展读
复制延迟	Replication Lag	主库写入到副本可见之间的延迟
分片 / 分片键	Sharding / Shard Key	按键把数据水平分到多个节点 / 用来分片的字段

中文	English	一句话解释
热点分片	Hot Shard (Hotspot)	承受远超平均负载的分片
重新分片	Rebalancing / Resharding	调整数据在分片之间的分布
读穿透 / 写穿透	Read-through / Write-through	未命中时由缓存去读库 / 写缓存同时写库
CAP 定理	CAP Theorem	一致性、可用性、分区容错最多三选二
原子性、一致性、隔离性、持久性	ACID	关系库事务的四个特性
基本可用、软状态、最终一致	BASE	以可用性优先的 NoSQL 一致性模型
对象存储	Object Storage	通过 API 存取的"数据+元数据"对象
块存储	Block Storage	挂载到虚拟机的磁盘卷
数据仓库	Data Warehouse	面向分析查询的数据存储
数据湖	Data Lake	以原始格式存放海量数据（常用对象存储）

CHAPTER 12

性能设计与容量规划

Design for Performance & Capacity Planning

课件来源：ArchSS 06 Design for Performance · ArchSS 07 Perform Capacity Planning · PAST Q1(a)(i) 官方答案

对应题型 **T11**，并为 T5–T7 提供论据

一句话 **响应时间 = 服务时间 + 排队时间；利用率越接近 100%，排队越长、响应越慢。所以要控制需求、增加或复制资源、用缓存和预计算，并用定义清楚的负载测试来验证。**

学完本章你能

- 说出响应时间、吞吐量、利用率、延迟的关系
- 用排队论直觉解释"为什么不能把服务器跑满"
- 列出性能设计手段：负载均衡、缓存、预计算等
- 设计一次可靠的负载测试（工作负载模型）
- 说出容量规划的方法：基准测试、趋势分析、弹性扩缩

12.1 性能指标 Performance Metrics

指标	定义
响应时间 Response Time	从发出请求到收到响应的时间 = **服务时间**（真正干活）+ **等待时间**（排在前面请求后面）+ **传输时间**。Web 场景还分首字节时间 TTFB 和末字节时间 TTLB
吞吐量 Throughput	单位时间完成的工作量（请求数/秒、事务数/秒 TPS），比时钟频率等参数更能反映整体性能
利用率 Utilization	组件实际忙碌的时间占可用时间的比例 = 已用容量 / 总容量
负载 Load	可以指请求速率、利用率或吞吐量（系统未饱和且一请求一响应时，吞吐量 = 请求速率）
延迟 Latency	数据从一个组件传到另一个组件的最短时间；有些设计会用延迟换带宽
思考时间 Think Time	用户两次操作之间不使用系统的时间，影响利用率和吞吐量，负载测试必须模拟

三个组件 x、y、z	响应时间	吞吐量
串行（多层依次处理）	x + y + z（相加）	min(x, y, z)：**由最慢的瓶颈决定**
并行（同时处理 / 负载均衡）	max(x, y, z)（取最长）	x + y + z（相加）

◆ 要点 | 必背结论 **性能指标小结（ArchSS 06）**

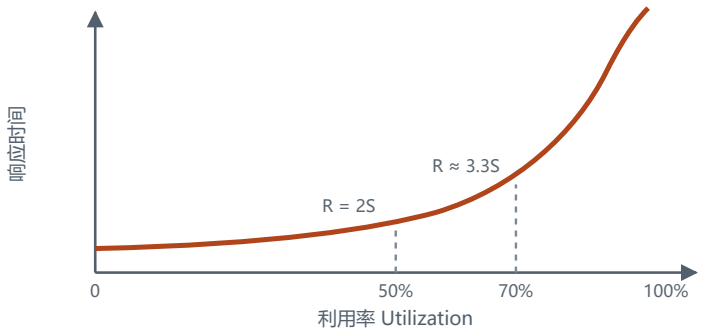
①响应时间 = 服务时间 + 排队时间；②响应时间越快，吞吐量通常越好；③负载低于饱和点（最大吞吐量）时，吞吐量与负载成正比；④负载与最大吞吐量之比，大致与利用率成正比。

12.2 排队论的直觉 Queueing Intuition

考试不需要计算，但以下几条规律能帮你写出有说服力的论证：

- **利用率定律** Utilization Law：U = λ × S（到达率 × 服务时间）。来的请求越多、每个处理越久，利用率越高。
- **Little 定律**：系统中平均请求数 n = λ × R（到达率 × 响应时间）。例：每秒 1000 个请求、每个 0.2 秒，系统里同时就有 200 个请求在处理。
- **强制流定律与瓶颈分析**：组件的负载 = 系统负载 × 该组件的访问次数；**总服务需求 D = 访问次数 × 服务时间，D 最大的组件就是瓶颈**。

- **响应时间随利用率急剧上升**：单服务器单队列（M/M/1）中 R = S / (1 − U)。负载 50% 时响应时间是服务时间的 2 倍，70% 时超过 3 倍，接近 100% 时趋于无穷。**所以要给利用率设上限**，保持在"健康"水平。
- **单队列多服务器（M/M/c）优于多个独立队列**：一条队列排给多个窗口，不会出现"一边排长队一边窗口空闲"。



读图：

· 低负载时，响应时间 ≈ 服务时间

· 超过约 70% 后，排队时间迅速增长

· 降低服务时间 S 或访问次数 V，可以降低利用率

· 扩容/自动扩缩的阈值应定在曲线拐点之前

· 案例中"90% 请求 3 秒内"要看高峰时的 P90

图 12-1 响应时间与利用率的关系（M/M/1）

12.3 性能设计手段 Performance Tactics & Heuristics

类别	战术 tactics
控制资源需求	管理到达率（限流、排队、流量控制）；输入排优先级
管理资源	增加资源；提高资源效率；**多份计算副本**（多实例负载均衡）；**多份数据副本**（缓存、读副本）；减少开销（如减少跨服务调用）；资源调度
设计启发 heuristics	**负载均衡**、**缓存**、**预计算**、降低算法复杂度、消除或减少瓶颈。这些手段都可能增加成本

- **负载均衡 load balancing**：把请求分到多台服务器，提升响应速度和可用性。分配策略：随机、按服务器负载、最短响应时间、处理量、活跃连接数、地理位置、服务器能力。**如果服务器保存客户端状态（会话），客户端就得一直连同一台**，影响可用性和负载均匀，所以要做无状态服务（第8章）。提高吞吐量：多个工人做同样的工作，或按工作类型分工；提高速度：把串行步骤改成并行。
- **缓存 cache**：命中 cache hit 时快，未命中 cache miss 时比不加缓存还慢一点。例：缓存 5ms、数据源 50ms，命中率只有 20% 时平均 44ms，比不加缓存的 50ms 好得有限，所以要看命中率。缓存位置：内存、浏览器、Web 服务器、远程服务器（CDN）。优先缓存"提速/占用空间"比值最高的数据；访问呈幂律分布时，小缓存就能有高命中率。
- **预计算 precomputation**：先算好、存起来、要用时直接取，用一次计算 + 多次读取代替多次计算。Twitter 例子：发推每秒数千次、看时间线每秒 30 万次，于是在发推时就把推文写进每个粉丝的时间线缓存（扇出），读时极快；粉丝极多的名人则仍用查询方式。
- **降低算法复杂度、把计算放对位置**：$O(n^2)$ 与 $O(n)$ 在数据量增长后差距巨大；在应用层还是数据库层计算，要同时考虑计算量和网络传输量的增长。
- **优化数据存储**：很多应用的瓶颈在数据读写，用对索引、选对存储技术效果最大（第11章）。

12.4 验证与记录性能设计 Validating & Documenting

三种验证方式：①应用战术时就做推理；②**性能建模**（排队模型、仿真工具如 JMT）；③**原型与基准测试**：如果要失败就早失败，特别是使用陌生技术或实验性设计时。步骤：定义目标 → 确定原型范围 → 构建原型（加埋点）→ 做基准测试 → 重复直到性能风险可控。在置信度和投入成本之间权衡。记录时用框图、时序图、数据流图，重点标出：主要组件、大消息、包含大量记录的调用、高频交互。

12.5 性能测试与工作负载模型 Performance Testing & Workload Model

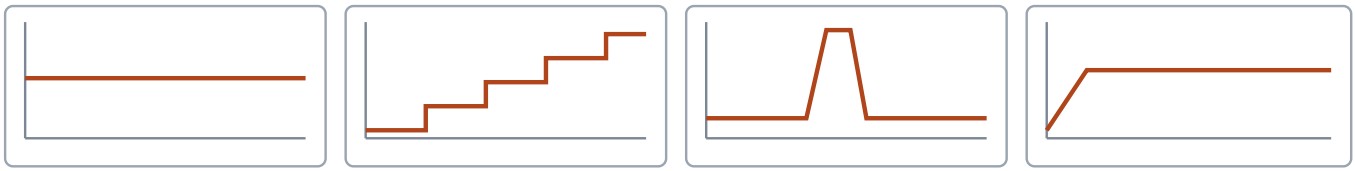


图 12-2 常见的负载曲线（横轴时间，纵轴并发用户或请求速率）

负载测试的结果**对工作负载组合非常敏感**。一次可靠的测试必须先定义**工作负载模型** workload model：

要素	说明
负载曲线	恒定、阶梯递增、突增、长时间稳定（见图 12-2），分别对应负载测试、压力/容量测试、突发测试、耐久测试
虚拟用户与到达率	多少并发用户，每秒到达多少请求（如 10万选民在 30 秒内发起）
场景组合 workload mix	各用例所占比例（如 60% 投票、30% 查结果、10% 登录）
思考时间、持续时间、测试数据	模拟真实用户行为；跑多久；用多少、什么样的数据
通过标准	用**百分位**表述，如 P90 < 3 秒、最大响应 < 5 秒、错误率 < 0.1%

工具：**JMeter**（图形界面配置）、**Artillery**（声明式）、Gatling、k6、Locust；云上有 Azure Load Testing、AWS Distributed Load Testing。这些工具都能定义场景和请求到达速率，输出响应时间和吞吐量。

▶ 案例演练 | 演示怎么套用 **PAST Q1(a)(i)（2分）：几位测试员用 Postman 测出的结果波动大、不确定，为什么？怎么改？**

原因（1分）：**负载测试最常被忽略的条件是工作负载模型**（负载是恒定、逐步增加还是变化的等）。几位测试员各自的假设和负载模型不同，结果自然不同。**更可靠的做法（1分）**：**把工作负载模型写进测试标准**，例如"30 秒内 10万次 Verify Identity 调用、按某种曲线递增、某种场景组合"，并用 JMeter 这类专门的负载测试工具从多台施压机执行，按 P90 < 3 秒、最大 < 5 秒判定。（Postman 主要用于功能测试，单机发请求，**模拟不了 10万并发**。）

12.6 容量规划 Capacity Planning

目标：确保有足够的计算资源，让预期的工作负载在可接受的服务水平下运行。新系统需要，已有系统在用户增长、数据累积、新增功能时也需要。任务：根据预期负载预测所需资源；决定买多少、什么时候买；或者让系统按负载自动扩缩（弹性）。

要预测什么	驱动因素	方法
磁盘/存储	**累积驱动**：数据只增不减	粗略估算、监控、日志。例：每月新增 100 万笔交易、每月增长 10%、每笔 1MB、保留 2 年再归档 → 按复合增长累加
服务器/计算	**峰值驱动**：按高峰负载	基准测试、负载测试、监控、日志

- **数据来源**：厂商发布的基准（如 TPC-C，Cockroach Labs 的云厂商报告）、自己做基准测试（在云上做比买本地机器容易）、历史监控数据（已有系统的最佳数据）。
- **趋势分析**：历史需求可能是随机、趋势或季节性的；用线性回归、移动平均、指数平滑预测（Excel 的 FORECAST/TREND 线性、GROWTH 指数）。要注意季节性、时区（例：Flickr 每周一是高峰）。
- **非线性**：1 台机器 17 TPS，2 台约 31，4 台约 56，**加机器不会线性提升**。**Amdahl 定律**：S(N) = 1 / ((1 − P) + P/N)，P 是可并行部分的比例，串行部分（访问同一数据、资源配额、低效算法）决定了扩展上限；**通用扩展定律**

USL 再加入"串扰"系数，解释为什么加到一定数量后性能反而下降。

- **把基准映射到业务场景**：给每个场景一个"慢速系数"（如搜索是基准的 2 倍慢），按场景组合加权得到总体吞吐量（课件例子：4 台机器约 38.8 TPS）。
- **采购提前期**：内部审批、到货、安装、集成、测试、停机窗口都要算进计划；太晚则容量不足，太早则硬件价格还在降。

◆ 要点｜必背结论 **弹性扩缩的设计准则（ArchSS 07）**

云厂商都支持自动扩缩，由策略控制：调整幅度、调整类型、冷却时间 cooldown；有历史数据时也可以**按时间表预先扩容**（如投票日开始前）。准则：①**要主动**，考虑应用启动时间；②**扩容要激进，缩容要保守**；③通过可扩展性分析确定阈值；④避免"乒乓效应"：扩容后每节点的请求量应高于缩容阈值，缩容后应低于扩容阈值。

12.7 小测验 Drills

✎ 小测验｜先自己答，再看参考答案 **小测验 1：请求依次经过四个组件，各自每秒处理 1000 / 500 / 800 / 300 个：吞吐多少？怎么提到 600?**

请求依次经过 CDN → API 网关 → 应用服务器 → 数据库，各自每秒能处理 1000 / 500 / 800 / 300 个请求。假设 CDN 不命中，**所有请求都要到达数据库，而且以写为主**。

1. 整体吞吐量是多少？瓶颈在哪?
2. 要**提高到每秒 600**，要改哪些组件、怎么改?

▾ **参考答案**

1. 串行链路的吞吐等于最慢的组件：**300**，瓶颈是数据库。
2. **每个组件都要 ≥ 600**，并留余量：API 网关（500）也不够，扩到 2 个以上实例；数据库以写为主，加读副本没用，要分片（例如分 3 片，约 900，留出余量）或把写入先进队列再批量写；应用服务器 800 暂时够用。

✎ 小测验｜先自己答，再看参考答案 **小测验 2：运维把自动扩容阈值设成 CPU 95%，高峰时响应时间突然飙升：为什么?**

运维为了"省钱、把机器用满"，把自动扩容的触发阈值设成 **CPU 95%**。结果每到高峰，响应时间都会**突然飙升好几倍**，过一阵才恢复。解释原因，并给出建议的阈值。

▾ **参考答案**

建议把阈值设在 CPU 60–70%，已知的高峰再配合提前定时扩容。原因：响应时间 R ≈ S / (1 − U)（S 为服务时间，U 为利用率），利用率 50% 时约为服务时间的 2 倍，70% 时超过 3 倍，95% 时约 20 倍；利用率一过拐点，排队急剧增长。而且新实例启动需要时间，等到 95% 才扩容，扩出来之前请求已经大量排队，所以会出现"飙升一阵才恢复"。

❝ 可用句式｜___ 处填案例内容

- Above about 70% utilization, queueing time grows sharply (R = S/(1 − U)), and at 95% the response time is about 20 times the service time. New instances also need time to start, so we scale out proactively, before the knee of the curve.

  > 利用率超过约 70% 后排队时间急剧增长（R = S/(1 − U)），95% 时响应时间约为服务时间的 20 倍。新实例启动也要时间，所以要在曲线拐点之前主动扩容。

✎ 小测验｜先自己答，再看参考答案 **小测验 3：抢票系统要做负载测试：写出工作负载模型**

抢票系统开售：**约 100 万人在开售后 5 分钟内进入**；**三类用户互不重叠**：70% 只浏览、20% 下单、10% 下单并支付。服务目标（SLA）：**P95 响应时间 ≤ 2 秒**，错误率 < 1%。

写出工作负载模型，至少包括：负载曲线、到达率、场景组合、思考时间、测试数据、通过标准。

▾ **参考答案**

要素	写法
负载曲线	**突增（Spike）**：开售瞬间陡升，5 分钟内维持高位，之后回落
到达率	100 万 ÷ 300 秒 ≈ 每秒 3,300 人进入（平均值，开售头几十秒更高）；注意这是到达人数，同时在线人数还取决于每人停留多久
场景组合	70% 浏览、20% 下单、10% 下单并支付
思考时间	页面之间停留几秒，按真实操作设定
测试数据	足够多的不同场次、座位和测试账号，避免全部抢同一张票
通过标准	P95 ≤ 2 秒，错误率 < 1%

本章术语表 Glossary

中文	English	一句话解释
响应时间	Response Time	服务时间 + 等待时间 + 传输时间
服务时间 / 等待时间	Service Time / Waiting (Queue) Time	实际处理的时间 / 排队的时间
吞吐量	Throughput	单位时间完成的工作量
利用率	Utilization	资源忙碌时间占比
延迟	Latency	数据在组件间传输的最短时间
思考时间	Think Time	用户两次操作之间的间隔
首字节时间	Time To First Byte (TTFB)	请求发出到收到第一个字节
利特尔定律	Little's Law	系统内请求数 = 到达率 × 响应时间
瓶颈	Bottleneck	总服务需求最大、限制整体吞吐的组件
负载均衡	Load Balancing	把请求分配到多台服务器
缓存命中 / 未命中	Cache Hit / Cache Miss	数据在 / 不在缓存中
预计算	Precomputation	提前计算并存储结果
基准测试	Benchmark	用标准测试评估系统相对性能
负载测试	Load Testing	按设定负载测量响应时间和吞吐量
工作负载模型	Workload Model	负载曲线、用户数、场景组合、思考时间等
工作负载组合	Workload Mix	各类请求在负载中的比例
压力 / 突发 / 耐久测试	Stress / Spike / Soak Test	测极限 / 测突增 / 测长时间稳定性
容量规划	Capacity Planning	预测并准备满足负载所需的资源
趋势分析	Trend Analysis	用历史数据预测未来需求
阿姆达尔定律	Amdahl's Law	串行部分限制并行加速的上限
通用扩展定律	Universal Scalability Law (USL)	考虑串行和串扰的扩展模型

中文	English	一句话解释
采购提前期	Procurement Lead Time	从申请到资源可用所需的时间
冷却时间	Cooldown Period	两次自动扩缩之间的最短间隔

CHAPTER 13

高可用、可维护与安全

High Availability, Maintainability & Security

课件来源：ArchSS 05 Design Highly Available Architecture · ArchSS 08 Ensure Maintainable Architecture · ArchSS 04 Design Secure Architecture · CNSD 11 Cloud Security

为 **T7** 及各题的论证提供依据

一句话 **高可用靠消除单点故障（冗余、故障检测、快速恢复）；可维护靠高内聚低耦合和可靠的变更流程；安全靠风险管理加多层控制。**

学完本章你能

- 计算和解释可用性、MTBF、MTTR、几个 9
- 说出串联和并联对可用性的影响
- 列出可用性战术和稳定性模式
- 比较 RAID 级别和三种备份策略
- 说出提高可修改性的四类战术
- 概述风险管理流程和安全设计原则

13.1 可用性的基本概念 Availability Concepts

概念	定义
可用性 Availability	系统或组件随时可用的程度，即需要时能按要求工作的概率
可靠性 Reliability	在规定条件和时间内完成预期功能的概率
容错 Fault Tolerance	在潜在错误生效前检测并纠正，出错时可降级服务而不是完全中断
MTTF / MTBF / MTTR	平均无故障时间 / 平均故障间隔 / 平均恢复（含修复）时间

$A = MTBF / (MTBF + MTTR)$　　**年停机分钟** $= (1 - A) \times 365 \times 24 \times 60$

可用性	99%（两个9）	99.9%（三个9）	99.99%（四个9）	99.999%（五个9）
每年停机约	3.65 天	8.8 小时	53 分钟	5 分钟

串联：$A = A1 \times A2 \times A3$

请求必须依次经过每个组件，**整体可用性低于最弱的组件**。三个 99% 串联只有约 97%。

并联：$A = 1 - (1-A1)(1-A2)(1-A3)$

任一副本可用即可，**整体可用性高于最强的组件**。两个 99% 并联可达 99.99%。**这就是冗余的价值**。

关注点分离能让关键组件与非关键组件松耦合，非关键部分出问题不影响关键服务。停机分**计划内**（多由部署引起）和**计划外**（故障引起），课程重点是后者。

◆ 要点 | 必背结论 **可用性算题六步**

1. 画出一次请求经过的路径，标出每个组件的可用性 A。
2. 冗余的一组（主备、多实例、多可用区）先按**并联**算成一个值：$1 - (1 - A)^n$。
3. 整条路径按**串联**相乘。
4. 与目标比较；**没达标时，先给最弱的那个组件加冗余**，它就是最大的单点约束。
5. 写出改造方案后再算一遍，给出改造后的数字。
6. 写明假设：**并联公式假设各副本独立失效**；共用电源、存储、同一个软件缺陷或同一次变更都会让实际值低于计算值，所以要做故障切换演练。

▶ 案例演练 | 演示怎么套用 **课程作业 Exercise 4（学校健康筛查系统 SHS）的可用性计算**

已知：服务器各 99%，防火墙各 99.9%，主机 Mainframe 98%，网络按 100%。实时请求路径上有 2 个防火墙和 4 类服务器（API Gateway、应用服务器、数据库服务器、集成服务器）；备份存储不在实时路径上，不计入。目标 98%。

现状（全部串联）：$0.999^2 \times 0.99^4 \times 0.98 = 0.93950 \approx$ **93.95%**，低于 98%。最弱的是 98% 的 Mainframe，四个单点服务器层又把可用性往下拉。

改造（每层做成一对，再串联）：防火墙对 $1-(1-0.999)^2 = 99.9999\%$；服务器对 $1-(1-0.99)^2 = 99.99\%$；Mainframe 对 $1-(1-0.98)^2 = 99.96\%$（要求厂商提供托管的主备，属系统外变更，需审批）。整体 $0.999999^2 \times 0.9999^4 \times 0.9996 = 0.999198 \approx$ **99.92%**，达标。

答题时三段写清：现状算式与结论 → 改造方案与新算式 → 独立性假设与剩余风险。

13.2 停机从哪里来 Causes of Downtime

视角	常见原因
软件应用	异构环境、**多个单点故障** SPOF、多个应用接口、监控不足、资源瓶颈
托管服务器	人为错误、安全漏洞、操作系统缺陷、IT 人手不足、硬件老旧不稳定
云托管	过载、吵闹邻居、重试风暴、坏的依赖、扩展边界、分片不均、"宠物"服务器、错误部署、监控缺失

分布式系统里**每个组件、每条连接都可能失败**。典型的连锁现象：**连锁反应**（所有实例运行同样的逻辑，一个 bug 让它们相继倒下）；**级联故障**（下游慢或错误沿调用链传播，**盲目重试会加剧**）；**用户冲击**（名人发帖、营销活动没打招呼、恶意用户）；**扇入放大**（被很多上游调用的公共服务，在上游扩容时首先被压垮）；**缓存踩踏** dogpile（重启后所有服务同时去数据库预热缓存）。

13.3 可用性战术与稳定性模式 Availability Tactics

类别	战术
静态分析	故障树分析：**最小割集**中只含一个事件的，就是单点故障
故障检测	Ping/Echo、**心跳** heartbeat、异常跟踪、时间戳、合理性检查、状态监控、投票表决
故障恢复·准备与修复	**主动冗余**（热备，同时处理）、**被动冗余**（温备，接收状态更新）、备用服务器（冷备）、异常处理、回滚、软件升级、重试、忽略错误行为、降级
故障恢复·重新引入	影子运行、状态重新同步、逐级重启、不停顿转发
故障预防	将组件移出服务、事务、进程监控、异常预防
动态分析	**混沌工程** chaos engineering：在生产中做实验验证系统抗动荡能力。步骤：准备（确保有追踪和监控）→ 设计实验（如实例故障时集群服务不受影响）→ 执行 → 分析改进

超时、重试、熔断

保护对外部系统或内部操作的调用。熔断器打开时：抛出不同的异常、返回上次的正确值（缓存）、返回通用而非个性化的答案，**这些降级方式要和干系人商量**。

舱壁 Bulkheads

把系统分区：服务、中间件、数据库都有冗余副本，物理隔离、可用区隔离；关键组件分成多个池，一个池出事不影响另一个。

减载、握手、背压

在系统边界限流丢弃多余请求（如 NGINX、Cloudflare 防 DDoS）；握手确认对方能接收；背压让下游告诉上游"慢一点"。

稳态与解耦中间件

定期清理累积的数据和内存缓存，保持稳态；用发布/订阅队列异步处理事件，让组件解耦（事件驱动）。

✕常见误区｜这样答会丢分

- 把可用性和可靠性混为一谈：可用性是"需要时能不能用"，可靠性是"一段时间内能不能正确工作"。
- 只写"加冗余"：冗余还要配**故障检测**（心跳、健康检查、存活探针）和**切换**（failover、ReplicaSet 补齐 Pod），否则备份节点不会自动接管。
- 只写"失败就重试"：**重试要配超时、指数退避和熔断**，否则会变成重试风暴，加剧级联故障。
- 把多个副本放在同一个可用区或同一台宿主机上，还按并联公式算：它们会一起失效，并联的收益不成立。

13.4 冗余与备份 Redundancy & Backup

避免单点故障的冗余策略：**硬件冗余**（冗余电源、风扇可热插拔，ECC 内存）、**服务器冗余**（主动/被动：用心跳监控，主节点故障时在备节点重启应用，即**故障转移** failover；主动/主动：都在处理请求）、**软件集群**（多份软件副本协同，对外呈现为一个系统；垂直或水平扩展）、**数据冗余**（RAID、SAN、备份、双数据中心）、**网络冗余**。

RAID	做法	最少磁盘	容错	空间开销
RAID 0	条带化，无容错	2	无	无
RAID 1	镜像	2	坏 1 块	50%
RAID 5	条带 + 分布式奇偶校验	3	坏 1 块	1 块盘
RAID 6	条带 + 双奇偶校验	4	坏 2 块	2 块盘
RAID 10	先镜像再条带	4	每组镜像坏 1 块	50%

备份策略	每次备份什么	备份速度/空间	恢复
完全备份 Full	全部文件	最慢、最占空间	最快：只需一份
差异备份 Differential	自上次**完全备份**以来变化的文件	中等，随时间增长	完全备份 + 最近一次差异备份
增量备份 Incremental	自上次**任意备份**以来变化的文件	最快、最省空间	最慢：完全备份 + 之后所有增量

可用性模式由简到繁（IBM）：基本 Web 应用 → 负载均衡的 Web 应用 → 负载均衡 + 热备 → 全面高可用 → 广域负载均衡（多地域）。**数据库冗余**：Oracle RAC（主动/主动，共享磁盘）、SQL Server 故障转移集群（主动/被动，共享磁盘）、数据库镜像（主动/被动，无共享）、MySQL 主从复制（主写从读，从库需"提升"为主库）、Cassandra 环形复制（主动/主动，无共享，副本数可配）。**Kubernetes** 用 ReplicaSet 自动补齐被终止的 Pod，配合存活探针自愈；AWS 参考架构把应用部署在多个**可用区** Availability Zone，防止单一地点故障。

13.5 可维护性 Maintainability

可维护性：软件能被方便地修改以纠正缺陷、提升性能或适应环境变化的程度。**维护可能占软件全生命周期成本的 80%–90%**。可维护的软件能快速修 bug 而不引入新 bug、加新功能而不改动已有功能。变化无处不在：延后的功能、需求遗漏、平台和集成变化等。改变的方式从代码修改、配置修改到用户可自定义，成本依次降低。

可修改性战术	架构层面	代码层面
拆分模块	单体的改动会波及全身；微服务按业务拆分，改动限于一个服务	封装：把变化限制在一个组件内（如折扣引擎用策略模式）
提高内聚	一个服务只负责一个限界上下文	关注点分离；抽象与分层；泛化相似场景

可修改性战术	架构层面	代码层面
降低耦合	**事件驱动**：新功能只需新增一个消费者订阅已有事件，生产者完全不用改；Kafka **schema 注册表**为消息格式做版本管理	可扩展的接口：用对象做参数，新增字段设为可选以保持向后兼容；使用标准扩展点（JDBC、REST）
推迟绑定	用工作流引擎、规则引擎在运行时定制逻辑	把可变属性参数化到配置里；让元素可替换

可分析性 analyzability：怎样发现问题、找到根因。手段：自动错误检测和上报、良好的日志、JMX/SNMP 监控；微服务中用 **EFK**（Fluentd 收集、Elasticsearch 存储、Kibana 看板）并带**关联 ID**。**可靠的变更**：配置管理（所有变更集中版本化）、自动化测试、持续集成、能回滚、**适应度函数** fitness function（自动检查架构规则，如"不允许跨层调用"）、代码质量工具（SonarQube、Checkstyle、PMD）。

◆ 要点｜必背结论 **可维护性四句话**

Split modules; increase cohesion; reduce coupling; defer binding. 答题时把它们接到前面学过的设计上：**拆分模块和提高内聚对应 DDD 的限界上下文（第4章），降低耦合对应事件驱动和 API 版本管理（第5、9章）**，推迟绑定对应外部化配置（第8章）。代码质量工具的分数只看**相对趋势**，不做绝对比较。

13.6 安全架构（概要） Secure Architecture

老师在考前辅导中说过，安全一般不是考试重点；**如果安全方面的答案论证合理也会给分**。这里只列核心框架。

- **安全目标**：保护信息的**保密性、完整性、可用性**（CIA 三元组）；另有真实性、授权、**不可否认** non-repudiation、可追责；访问控制的 AAA（认证 Authentication、授权 Authorization、审计 Accounting）。安全需求要在需求阶段就纳入，**设计即安全** secure by design。
- **风险管理流程**（ISO 31000 / 27005）：确定范围 → 风险识别（**资产**：皇冠明珠 crown jewels 即关键数据、跳板 stepping stones；**威胁**：STRIDE 分类，即仿冒、篡改、抵赖、信息泄露、拒绝服务、权限提升；**漏洞**）→ 风险分析（可能性 × 影响，可用 DREAD 打分）→ 风险评估 → 风险处置：**接受、规避、转移**（如买网络保险）、**缓解**（部署安全控制）。
- **安全控制**按功能分：**预防**（抵御攻击）、**检测**、**响应**、**恢复**；按类型分：物理、技术、管理。技术控制包括：密码学（对称加密快、适合大量数据；非对称加密用于少量数据和密钥交换；哈希、HMAC、数字签名、数字证书；长期密钥存在硬件安全模块 HSM）、传输层安全 **TLS**、VPN、访问控制（你知道的、你拥有的、你是谁三类认证因素）、**网络分区**（DMZ、独立的日志和管理网络）、入侵检测/防御 IDS/IPS、备份恢复。
- **安全设计原则**：**纵深防御**（多层保护，一层被破还有下一层）、**最小权限**（最少的权限、最短的时间）、**最少公共机制**（减少多个用户共享的保护机制）、**完全仲裁**（每次访问都要经过授权检查，不缓存授权决定）。
- **云安全**（CNSD 11）：**责任共担模型** shared responsibility model（厂商负责"云本身"的安全，用户负责"云里面"的安全，划分随 IaaS/PaaS/SaaS 变化）；身份与访问管理 IAM；传输中和静态数据加密（谁持有密钥很关键）；数据脱敏 masking、令牌化 tokenization；隔离子网做网络分区；监控数据和流量、掌握云服务使用情况。

■ 定义｜课件正式说法 **风险 = 威胁 + 漏洞（T + V = R）**

只有威胁 threat 遇上可被利用的漏洞 vulnerability 才构成风险 risk。课件给的风险场景句式，四个要素缺一不可：

<threat> exploits <vulnerability> to harm <asset> resulting in <consequence>

> 中文：<威胁> 利用 <漏洞> 损害 <资产>，造成 <后果>。

STRIDE 与安全属性一一对应：仿冒 → 真实性，篡改 → 完整性，抵赖 → 不可否认，信息泄露 → 保密性，拒绝服务 → 可用性，权限提升 → 授权。另外注意：**TLS 只保护 TLS 客户端与服务器之间这一段，不一定是端到端**。

✎ 考试怎么考 | 题型·分值·套路 **样卷没有单独考本章，但三处会用到**

- **T5 / T6 架构图与部署题**：多可用区、每个服务多副本、健康检查与自愈、托管数据库的多可用区部署，都是本章的冗余与故障检测。PAST Q2(a) 的 K8s 部署计划要写出 Pod、Deployment、Service（3 分）。
- **T11 扩展题**：PAST Q3(d) 分片方案的副作用（热点分片、跨分片汇总变慢），对应本章"扩展边界""分片不均"两种停机原因。
- **T7 质量属性题**：PAST Q1(d) 官方答案的第二个理由就是安全：**保证每位选民最多投一票**。安全论据合理也给分，不要整段删掉。

如果真的出可用性计算、安全风险或韧性设计题，按上面"可用性算题六步"作答，写法和英文句式见 **08 补充卡 T13（可用性）、T16（安全）、T17（韧性）**。

13.7 白话讲解与小测验 Plain-language Notes & Drills

✕ 常见误区 | 这样答会丢分

- 可用性（需要时能不能用）≠ 可靠性（一段时间内能否持续正确工作）。
- 只写"加冗余"不够：还要写故障检测（心跳、存活探针）和自动切换。
- 只写"失败就重试"不够：要配超时、指数退避和熔断，否则会引发重试风暴。
- 副本都在同一个可用区或同一台宿主机上，不能按并联算。

✎ 小测验 | 先自己答，再看参考答案 **小测验 1：乘客查到站时间要经过三个组件，各 99%：整体可用性多少？加一个备库后呢?**

数字仅为示意。乘客查询到站时间，请求要依次经过 API 网关、查询服务、数据库，**三者的可用性都是 99%**。

1. 整条路径的可用性是多少?
2. 给数据库在**另一个可用区加一台备库**（备库也是 99%，**两库独立失效**，主库故障时**自动切换**）。数据库层和整条路径的可用性各变成多少?

▾ **参考答案**

做法	算式	结果
串联	0.99 × 0.99 × 0.99	≈ 97%，低于最弱的一个
数据库加备库（并联）	$1 - (1 - 0.99)^2$	数据库层 ≈ 99.99%
整条路径重算	0.99 × 0.99 × 0.9999	≈ 98%；再给网关和查询服务也做多副本才能继续提高

并联公式成立的前提：两库各自独立失效（放在不同可用区），并且有健康检查和自动切换。

✎ 小测验 | 先自己答，再看参考答案 **小测验 2：增量备份和差异备份：周五早上数据坏了，恢复各需要哪几份？**

每周日 23:00 做一次完全备份，周一到周六每天 23:00 做一次备份。**周五 09:00 发现数据损坏**，损坏发生在周四 23:00 那次备份之后。

1. 如果周一到周六做的是**增量备份**，恢复需要哪几份？
2. 如果做的是**差异备份**呢？

▾ 参考答案

方案	每次备份什么	恢复需要
完全 + 每天增量	上次任意备份之后的变化（每天最快、最省空间）	周日完全 + 周一至周四 4 份增量，按顺序恢复（共 5 份，恢复最慢）
完全 + 每天差异	上次完全备份之后的全部变化（一天比一天大）	周日完全 + 周四那份差异（共 2 份）

✎ 小测验 | 先自己答，再看参考答案 **小测验 3：方案写了"每个服务部署 3 个副本"，一次可用区故障仍然全站不可用：缺了什么？**

某团队的高可用方案只写了一句"**每个服务部署 3 个副本**"。上线后**某个可用区整体故障**，全站仍然不可用。这个方案缺了什么？**至少答三点**。

▾ 参考答案

① 副本和工作节点都没有分布在**多个可用区**：3 个副本恰好都在同一可用区，会一起挂（要配节点组跨可用区和 Pod 拓扑分布约束）；② 数据库等**单点**没有做多可用区（如 RDS Multi-AZ）；③ 没有**故障检测和自动切换**：健康检查、存活和就绪探针，由负载均衡 / Service 把流量转到健康副本，ReplicaSet 补齐 Pod。

❝ 可用句式 | ___ 处填案例内容

- I would run each service in three Availability Zones and the database in Amazon RDS Multi-AZ. Health checks detect a failed instance and traffic fails over to healthy replicas automatically, so there is no single point of failure.

  > 我会把每个服务部署在三个可用区，数据库用 Amazon RDS Multi-AZ。健康检查发现实例故障后，流量自动切换到健康副本，因此没有单点故障。

✎ 小测验 | 先自己答，再看参考答案 **小测验 4：订单服务每加一个下游需求都要改代码重测：怎么改设计？**

订单服务每次下单成功后，要**直接调用发券、积分、推荐三个服务**。业务每加一个新需求（如再加一个"会员成长值"），**都得改订单服务的代码、重新测试、重新发布**。怎么改设计，让以后加功能不用动订单服务？为什么这样更易维护？代价是什么？

▾ 参考答案

改成**事件驱动**：订单服务只发布一个"已下单"（OrderPlaced）事件，发券、积分、推荐、成长值各自订阅处理。生产者不知道谁在消费，新增功能只需新增一个订阅者，订单服务不用改、不用重测，耦合降低；配合 schema 注册表管理事件格式的版本，消息格式也能平滑演进。代价：流程变成隐式的，要靠追踪和监控看清全局；要保证事件一定能发出去（如 outbox 模式）。

本章术语表 Glossary

中文	English	一句话解释
可用性	Availability	需要时能正常工作的程度
可靠性	Reliability	一段时间内正确完成功能的概率
容错	Fault Tolerance	出错时仍能提供（降级）服务
单点故障	Single Point of Failure (SPOF)	一坏就导致整个系统不可用的组件
故障转移	Failover	主节点故障时切换到备用节点
主动 / 被动冗余	Active / Passive Redundancy	热备同时工作 / 温备待命接管
心跳	Heartbeat	定期发送的存活信号
舱壁	Bulkhead	把系统分区隔离，限制故障范围
减载 / 背压	Load Shedding / Backpressure	主动丢弃超额请求 / 让上游放慢
混沌工程	Chaos Engineering	在生产中做故障实验以建立信心
缓存踩踏	Dogpile	大量请求同时回源预热缓存压垮数据库
可用区	Availability Zone (AZ)	同一地域内相互隔离的数据中心
完全 / 差异 / 增量备份	Full / Differential / Incremental Backup	全部 / 自上次完全 / 自上次任意备份后的变化
可维护性	Maintainability	修改软件的难易程度
可修改性	Modifiability	不引入缺陷地修改系统的能力
推迟绑定	Defer Binding	把决定推迟到配置或运行时
适应度函数	Fitness Function	自动检验架构规则的测试
模式注册表	Schema Registry	管理消息格式及其版本
保密性、完整性、可用性	CIA Triad	信息安全的三个核心目标
不可否认	Non-repudiation	行为发生后无法否认
威胁分类	STRIDE	仿冒、篡改、抵赖、泄露、拒绝服务、提权
纵深防御	Defence in Depth	多层安全控制
最小权限	Least Privilege	只给完成任务所需的最少权限
责任共担模型	Shared Responsibility Model	云厂商与用户分担安全责任
身份与访问管理	Identity and Access Management (IAM)	管理谁能访问什么

CHAPTER 14

样卷总览与考场策略

Sample Papers at a Glance & Exam-day Strategy

来源：PAST / PAST2 / PAST3 三套样卷 · PAST 官方答案与评分标准 · Clinic Session 老师讲解　　贯穿 **T1–T12**

一句话 **样卷的题型每套都差不多，案例每次都换：把"题型 → 思考步骤 → 得分点 → 句式"练熟，考场上只需要把新案例的名字和数字填进去。**

学完本章你能

- 说出三套样卷的结构和每种题型的分值
- 按老师的评分规则组织答案，不漏得分部分
- 用好 30 分钟阅读时间和 3 小时作答时间
- 拿到新案例时知道去附录哪一节找材料
- 用一套通用句式写出完整的英文答案

说明：本部分（第14–17章）不照抄样卷和官方答案原文，题干用中文概括、只保留关键英文指令词；原题请对照 **PM Sample Exam Questions** 目录下的 PDF，每道题都标了页码。PAST 有官方答案和评分标准；PAST2、PAST3 没有官方答案，书中的得分点是按 PAST 评分方式推断的。

14.1 三套样卷长什么样 Structure of the Papers

样卷	案例	结构与分值	官方答案	本书
PAST	Vote Anywhere 选举投票平台	Section A Q1 25 分 · Section B Q2 13 分 · Section C Q3 12 分，共 50 分	有，含评分标准	**第15章**
PAST2	MLTTP 多模式陆路交通遥测平台（公交）	Q1 25 分 · Q2 13 分 · Q3 12 分，共 50 分	无	**第16章**
PAST3	ONE Health 医疗与健身平台	只有 Section A：Q1 19 分 · Q2 15 分，共 34 分	无	**第17章**

每套题都以 "Refer to the case study in Appendix A" 开头。**附录 A 才是答题的素材库，题目只告诉你要交什么。**老师说正式考试通常有三个 section、总分 50，但各 section 分值不固定，也不保证一个 section 对应一门课。

题型 × 样卷对照

题型	PAST	PAST2	PAST3	讲解
T1 领域模型诊断	Q1(b) 9	Q1(b) 9	—	**第4章**
T2 类图与共享服务	—	—	Q1(b) 9	**第4章**
T3 API 设计	Q1(c) 6	Q1(c) 6	Q1(c)(ii)	**第5章**
T4 打包/部署找错	Q3(c) 6	Q3(a) 6	Q2(a) 6	**第2章**
T5 云原生参考架构	Q2(a) 10	Q2(a) 7	—	**第8、10章**
T6 物理部署/云服务	—	Q1(e) 4、Q2(c) 3	Q2(b) 9	**第11章、附录A**
T7 质量属性	Q1(d) 2	Q1(d) 2	—	**第1章**
T8 指标设计	Q1(e) 4	Q3(b) 6	—	**第6章**
T9 平台思维	—	Q1(a) 2	Q1(a) 2、Q1(c)(i)	**第3章**
T10 二选一决策	Q1(a)(ii) 2、Q2(b) 3	Q2(b) 3	—	**第8–10章**

题型	PAST	PAST2	PAST3	讲解
T11 性能/扩展/测试	Q1(a)(i) 2、Q3(d) 6	—	—	**第11、12章**
T12 服务共置	—	Q1(f) 2	Q1(d) 2	**第9章**

PAST 第 3 题原卷印成 c、d 两小题，老师课上按 a、b 讲。PAST3 Q1(b)(c) 的 9 分、6 分是两个小问合计。

◆ 要点｜必背结论 **每套必考的四类**

T1/T2 领域模型、T3 API、T4 打包部署找错、T5/T6 架构图，三套样卷每套都有，合计约 31 分，已经超过及格线 20 分。开考后先做这四类，再做小题。

14.2 老师讲过的评分规则 How the Lecturer Marks

老师课上说法的完整清单（附录像时间点）见开头的「老师划重点」一章。

- **答案必须贴合案例。**通用笔记照抄不得分；要点名题目里的角色、用例、实体、服务、图和数字。**同样的问法换了案例，答案很可能就变了。**
- **题目要求的每一部分都要答。**原因、措施、理由、缺点是不同的得分部分。只指出问题不给措施、给了措施不论证，都会丢分。
- **在题目指定的层面答。**领域模型题要讲限界上下文、实体归属、跨服务交互，**写"加缓存、加负载均衡"属于跑题**；题目说刻意省略的部分不能当缺陷。
- **用完整句子解释机制。**只列关键词说明不了"为什么有问题、为什么措施有效"。
- **合理的替代答案也给分。**质量属性、指标、架构取舍可以有不同答案，但必须论证。图中要求的元素、归属和连线评分较严。
- **要求两个，不代表只有两个。**"找两个不理想决策"是从多个问题里选两个答对即可。
- **推荐云服务时写具体产品名**，如 Amazon DynamoDB，而不是"NoSQL 数据库"。
- **改图只画改动部分**，不用重画整张图；但要补的连线、元素一定要画出来。
- **安全一般不是重点**，但论证合理也会给分，不要用笼统的"安全性不好"代替根因分析。

✎ 考试怎么考｜题型·分值·套路 **官方评分颗粒度（PAST）**

题	分配
T1 领域模型（5 分）	原因 2 + 对策两部分各 1.5 + 缺点 0
T1 领域模型（4 分）	原因 1.5 + 对策 1.5 + 缺点 1
T3 API（6 分）	三个小问各 2 分：解释 1 + 改进设计 1
T4 找错（6 分）	每个决策：识别 1 + 纠正 1 + 论证 1，答两个
T5 架构图（10 分）	参考架构 4 + 通信组件 2 + 缓存 1 + K8s 部署 3
T8 指标（4 分）	每个指标连同理由 2 分
T7 / T10 小题（2 分）	每个有说服力的理由 1 分
T11 分片（6 分）	分片方式 2 + 受影响组件及理由 4

"5 分写 5 点"不是通则：按题目的要求项逐项覆盖才是关键。

14.3 考场时间怎么分 Time Plan

阶段	时长	做什么
阅读时间	30 分钟	只能读、可以举手提问，**不能写字**（连草稿计划都不行）。用法见 14.4。

阶段	时长	做什么
作答	3 小时	可以按任意顺序作答，直接写在试卷空白处；不够写可以申请额外答题纸，交卷时订在对应 section 后面。
复查	预留约 20 分钟	检查每题的每个要求是否都答了、图上要求的元素是否画齐。

按分值分配时间：留 20 分钟复查后，160 分钟答 50 分，**每 1 分约 3 分钟**。以 25/13/12 的结构为例，大约 80/42/38 分钟（这是复习建议，不是老师规定）。**一道 2 分小题不要超过 7 分钟**，写不出来先跳，最后回来补，尽量不留空白。

工具：文字用笔（不要用红笔），画图可以用铅笔；不需要写代码，也不需要计算器。只能带纸质资料，不能用电子设备。

14.4 30 分钟阅读时间怎么用 Using the Reading Time

1. **先读全部题目（约 8 分钟）。**每道小题记住三件事：指令词（identify / propose / justify…）、要交几个（two reasons / three decisions）、分值。顺便判断题型 T1–T12。
2. **读附录第 4 节质量要求（约 5 分钟）。**记住并发数、响应时间百分位、实时更新时限、是否多租户，这些数字几乎每道论证题都要引用。
3. **读第 7 节领域模型（约 7 分钟）。**每个 <<service>> 框里有哪些实体、哪些连线跨框、聚合（红圈）有多大。T1、T4、T12 都靠这张图。
4. **读第 8 节打包部署图和图注（约 5 分钟）。**特别留意"incomplete""deliberately omitted"这类说明。
5. **剩下时间读用例（第 2、5 节）。**找出哪个用例在设备本地保存数据、哪个由后端发通知、哪个要实时汇总。

手册提醒：考试信只说阅读时间不能写字，没说能不能翻自带资料，可以当场举手问。按"只读题目和案例"准备，能翻资料就是额外收益。

14.5 万能答题框架 A Frame for Any Question

几乎所有论述题都能套四段：**表态/现象 → 原因（引用案例）→ 措施 → 理由与代价。**不同指令词要交的东西不同：

指令词	要交什么	最少写到
identify	点名具体的服务、实体、连线或决策	名字 + 它在图上的位置
explain / analyse	说清机制：为什么会这样	一句因果链，引用案例
propose / suggest	具体改什么、怎么改	动作 + 对象 + 结果
justify	为什么这样改能满足要求	引用第 4 节数字或用例
drawback / side effect	措施的代价	没有也要写"no major drawback"并说明
Do you agree…?	先表态，再给理由	表态句 + 两个理由
draw / illustrate	必须画图，并配文字	要求的元素和连线都画出、都标名字
recommend cloud services	具体厂商的具体产品	产品名 + 它满足哪个需求

✕常见误区｜这样答会丢分

- 小题里漏掉一半要求：例如"给一个原因、提一个更可靠的办法"只写了原因。
- 只写关键词或列表，不写成句子解释机制。
- 引用通用知识，不引用案例数字，比如只写"要高性能"不写"90% 请求 3 秒内"。
- 前一题提出的改造当成后一题的前提：每道题都以附录原图为准。
- 前两题写太久，最后一题没时间。

14.6 拿到新案例：去附录哪一节找材料 Mining Appendix A

三套样卷的附录结构几乎一样，正式考试大概率沿用。**先按下表定位，再把名字和数字填进题型卡的句式里。**

附录小节	内容	哪些题要用	抄什么
§1 背景与角色	业务目标、用户角色、外部系统	T9 平台思维、T2 类图	种子、生产者、消费者
§2 阶段与用例	Phase 1 / 2 的用例清单与流程	T4、T8、T5	本地保存、通知、实时汇总、批处理的用例
§3 可扩展性	公共服务和库、未来扩展方向	T2 共享服务、T9	"新服务/新角色"的原文要求
§4 性能与扩展	并发数、响应百分位、更新时限、多租户	T7、T6、T5、T10、T11	**所有数字原样照抄**
§5 安全	DMZ、加密、GDPR、PCI-DSS	T6、T4 的补充论据	一两句即可
§6 初始架构	分层（层名）、前后端部署层	T4、T5	层名写法要和附录一致
§7 领域模型	<<service>> 框、实体、聚合	T1、T4、T12、T5	服务名、实体名、跨框连线
§8 打包部署图	子系统、manifest、deploy、图注	T4	每条线逐一核对；读图注里的"省略"

注意：PAST、PAST2 都写明"不预期多租户"，PAST3 要求多租户与分片；附录第 6 节的层名在不同案例里也不一样（PAST2 有 Platform Service、Enterprise Integration 层）。

14.7 通用句式库 Sentence Bank

“ 可用句式 | ___ 处填案例内容 表态、引用案例、讲原因

- I disagree / I agree with the proposal, because ___.
  > 我不同意 / 同意这个提议，因为 ___。
- According to Section ___ of the case study, the platform must support ___ concurrent ___ with ___% of the requests completed within ___ seconds.
  > 根据案例第 ___ 节，平台必须支持 ___ 个并发 ___，且 ___% 的请求在 ___ 秒内完成。
- In the domain model (Figure ___), ___ is in the ___ service while ___ is in the ___ service.
  > 在领域模型（图 ___）中，___ 位于 ___ 服务，而 ___ 位于 ___ 服务。
- The most likely cause is that ___, so whenever ___, the two services must communicate across processes.
  > 最可能的原因是 ___，所以每当 ___ 时，两个服务都必须跨进程通信。

“ 可用句式 | ___ 处填案例内容 提措施、给理由、写代价

- I propose to ___ (e.g. move ___ from ___ to ___), as illustrated in the diagram below.
  > 我建议 ___（例如把 ___ 从 ___ 移到 ___），如下图所示。
- This addresses the issue because ___; as a result, ___ can be completed within ___ seconds.
  > 这能解决问题，因为 ___；结果 ___ 能在 ___ 秒内完成。
- The trade-off is that ___. It can be mitigated by ___.
  > 代价是 ___，可以通过 ___ 来缓解。
- There is no major drawback; in fact, ___ can now be scaled independently.
  > 没有大的缺点；事实上，___ 现在还能独立扩展。
- I would choose ___ instead of ___. Firstly, ___. Secondly, ___.
  > 我会选 ___ 而不是 ___。第一，___；第二，___。

❝ 可用句式｜___ 处填案例内容 **图题、指标题、收尾**

- Only the changed part of the diagram is drawn below; new elements are marked "new" and removed links are crossed out.
  > 下面只画出图中改动的部分；新增元素标 "new"，删除的连线打叉。
- Metric: ___ = ___. It is captured by ___ every ___, kept for ___ in ___.
  > 指标：___ = ___。由 ___ 每隔 ___ 采集一次，在 ___ 中保存 ___。
- Both requirements in Section 4 (___ and ___) are therefore met.
  > 因此第 4 节的两项要求（___ 和 ___）都得到满足。

CHAPTER 15

PAST 精讲：Vote Anywhere 投票平台

Sample Paper PAST — Walkthrough with the Official Marking Scheme

来源：SWE5001 Sample Exam Paper_PAST · Appendix A_PAST · Sample Solutions_PAST（官方答案与评分）· Clinic Session

11 道小题 · 50 分

一句话 **这是唯一有官方答案的样卷：先看清每道题的分是怎么给的，再把同样的思路搬到 PAST2、PAST3 和正式考试。**

每道题按同一顺序讲

- 题卡：题干要点、分值、题型、原题页码
- 思考逻辑：从读题到下笔的步骤
- 得分点：官方评分标准
- 参考答案：可照着写的英文
- 可用句式与易错点
- 换案例时怎么迁移

15.1 案例速读 Case Facts You Will Cite

业务：Vote Anywhere 让选民在任何地方用手机投票。监考员 Proctor 在线核验身份并把选民逐个请进"投票室"，观察员 Observer 实时查看结果。部署取向是云原生、服务用微服务实现。

附录 §4 数字	原文要求（照抄这些数字）
并发	10 Election Admins、1,000 Proctors、1,000 Observers、**100,000 Voters**、5 Platform Admins
交互响应	交互请求（如 Verify Identity）**90% < 3 秒**
结果更新	current simplified results 平均 **1 分钟**内；current detailed results 平均 **3 分钟**内
其他	不预期多租户；数据隔离不能牺牲实时更新和读取的一致性；安全：DMZ 反向代理、个人信息加密、支持 GDPR

§7 领域模型（Figure 5）	实体
UserManager	PlatformAdmin、User
PollManager	ElectionAdmin、Poll、PollReport、Candidate
ProctoringManager	Proctor、PollingRoom、ProofOfIdentity
ObservationManager	Observer、CurrentDetailedResults、CurrentSimplifiedResults
VotingManager	Voter、CastVote
NotificationManager	ProctoringNotif、ObservationNotif、PollingNotif、VotingCompletionNotif、RejectionNotif、VerificationNotif、NotificationEmail

§8 Figure 6 只画了投票用例在手机端的打包和部署，图注明说不完整、可能有错、**刻意省略后端部署**。

15.2 Q1(a)(i) 性能测试结果不稳定 2 marks · T11

题干要点 2 分 · T11 · 原题 p.2 · 答案 p.2

Verify Identity 接口要求 100,000 名选民下 90% 请求 < 3 秒；另给了 30 秒调用时间窗、最大响应 < 5 秒两个测试参数。几位测试员用 Postman 之类的工具测，结果各不相同、不可重复。

要求：给**一个可能原因**，再提**一个更可靠的测试办法**。

1. 题目说"结果各不相同"：问题出在**测试条件不统一**，不是系统本身。
2. 已经给了哪些参数？用户数、时间窗、响应上限。**缺的是负载怎么随时间施加**，也就是工作负载模型。
3. 办法要对准原因：把工作负载模型写进测试标准，用专门的负载测试工具按脚本重复执行。

得分点（官方）	分
一个可能原因	1
一个更可靠的办法	1

✍ **参考答案｜可照着写，名字和数字换成新案例的**

Reason: The testers did not share a common workload model. The criteria give the number of voters, the 30-second window and the maximum response time, but not how the load is applied over time (constant, stepped up, or spiky), nor the mix of requests and think time. Each tester made different assumptions, so the results differ and are not repeatable.

> 原因：测试员没有采用统一的工作负载模型。测试标准给了选民人数、30 秒时间窗和最大响应时间，却没说明负载如何随时间施加（恒定、逐步增加还是突发），也没说明请求组合和思考时间。每个测试员的假设不同，所以结果各不相同、不可重复。

More reliable way: Specify the workload model as part of the test criteria (e.g. 100,000 Verify Identity calls ramped up within 30 seconds, with a defined request mix), and run it with a load-testing tool such as JMeter from several load generators in a production-like environment. Pass if the 90th percentile is below 3 seconds and the maximum below 5 seconds; repeat the run to confirm.

> 更可靠的办法：把工作负载模型写进测试标准（例如 30 秒内逐步增加到 10 万次 Verify Identity 调用，并规定请求组合），在接近生产的环境中用 JMeter 这类负载测试工具从多台施压机执行。判定标准：第 90 百分位低于 3 秒、最大值低于 5 秒；重复运行确认结果稳定。

✕ **常见误区｜这样答会丢分**

- 只写原因不写办法，或反过来，各丢 1 分。

换案例时 只要题目说"测试结果不稳定、不可比"，第一原因都写工作负载模型；把用户数、时间窗、百分位换成新案例的数字。

15.3 Q1(a)(ii) 消息队列还是事件流 2 marks · T10

题干要点 2 分 · T10 · 原题 p.2 · 答案 p.2

"更新简要/详细结果"用例：简要结果平均 1 分钟、详细结果平均 3 分钟内更新，结果要**确定且可重复**（deterministic and repeatable）。有人想在每次投票结束时用消息队列触发它，你作为首席架构师主张用事件流。

要求：给出支持事件流的**两个理由**。

1. 圈出题干的两个关键条件："确定且可重复"和"同时更新简要和详细两种结果"。
2. "确定、可重复" → 事件必须按时间顺序处理 → 事件流在分区内保证顺序。
3. "两种结果" → 同一批投票事件要被处理两次 → 事件流在保留期内不删除事件，可多个消费者各读一遍、可重放；消息队列投递后就删除。

得分点（官方）	分
理由一：顺序（每个理由 1 分）	1
理由二：保留、可多次读取	1

✍ 参考答案｜可照着写，名字和数字换成新案例的

Reason 1 – Ordering: To produce deterministic and repeatable results, the vote events must be processed in the order in which they occurred; swapping two events would change the intermediate results. Event streaming keeps the events in order within a partition (e.g. partitioned by poll), while competing consumers on a message queue may process them out of order.

> 理由一（顺序）：要得到确定且可重复的结果，投票事件必须按发生顺序处理；交换两个事件的顺序就会改变中间结果。事件流在分区内保持事件顺序（例如按 poll 分区），而消息队列上相互竞争的多个消费者可能乱序处理。

Reason 2 – Retention and multiple consumers: The use case updates both the simplified and the detailed results, so the same sequence of vote events has to be read twice. An event stream retains the events for a configured period, so two consumer groups can read them independently and replay them if needed, whereas a message queue removes a message once it has been delivered.

> 理由二（保留、多个消费者）：这个用例要同时更新简要结果和详细结果，同一串投票事件要被读两遍。事件流会把事件保留一段配置好的时间，两个消费者组可以各自独立读取，必要时还能重放；消息队列则在消息投递后就把它删除。

❝ 可用句式｜___ 处填案例内容

- To be deterministic and repeatable, the ___ events must be processed in temporal order; event streaming guarantees ordering within a partition.
  > 要做到确定、可重复，___ 事件必须按时间顺序处理；事件流保证分区内的顺序。
- Because both ___ and ___ consume the same events, the stream's retention lets each consumer read (and replay) them independently.
  > 由于 ___ 和 ___ 消费同一批事件，事件流的保留机制让每个消费者都能独立读取（和重放）它们。

技术边界：事件流的顺序只在同一分区内保证，所以要说明按什么分区（如按 poll）。不要写"所有事件流都全局有序"。

15.4 Q1(b)(i) NotificationManager 难以复用 5 marks · T1

题干要点 5 分 · T1 · 原题 p.2 · 答案 p.3

NotificationManager 负责定义平台的所有通知，而各类通知其实属于其他服务的职责。你认为现有领域设计不利于把它复用到其他平台。

要求（每个小问都要）：*"identify the most likely cause"*、通过修改领域模型提出对策、指出对策的主要缺点（如有）。

1. 看 Figure 5 的 NotificationManager 框：7 个通知实体里，6 个是业务专属的（投票、监考、观察……），只有 NotificationEmail 是通用的。
2. 原因：通用服务里塞满了业务特有内容（specificities），换平台就得删改。
3. 对策分两半写（各 1.5 分）：① 把特有通知按类别迁回各自业务服务；② NotificationManager 只保留通用部分。
4. 缺点：没有大缺点，反而各服务可以独立扩展。0 分但要写。

得分点（官方）	分
最可能原因	2
对策①：按类别分离业务特有通知	1.5
对策②：保留通用、内聚的部分	1.5
缺点	0

✍ **参考答案｜可照着写，名字和数字换成新案例的**

Cause: The design groups every kind of notification in one service. The NotificationManager therefore contains the specificities of each business area – ProctoringNotif, PollingNotif, VotingCompletionNotif, RejectionNotif, ObservationNotif and VerificationNotif – which belong to the responsibilities of other services. Another platform would not need them, so the service cannot be reused without drastic adaptation.

> 原因：现有设计把所有类型的通知都集中在一个服务里。NotificationManager 因此包含了各业务领域的特有内容（ProctoringNotif、PollingNotif、VotingCompletionNotif、RejectionNotif、ObservationNotif、VerificationNotif），这些其实属于其他服务的职责。别的平台用不到它们，所以不做大改就无法复用这个服务。

Countermeasure: (1) Move each specific notification to the service that owns it: VerificationNotif to UserManager, ProctoringNotif to ProctoringManager, ObservationNotif to PollManager, and PollingNotif, VotingCompletionNotif and RejectionNotif to VotingManager. (2) Keep only the generic NotificationEmail in the NotificationManager, so it becomes a generic, cohesive service that other platforms can reuse. (Draw only the changed service boxes.)

> 对策：（1）把每种特有通知移到它所属的服务：VerificationNotif 到 UserManager，ProctoringNotif 到 ProctoringManager，ObservationNotif 到 PollManager，PollingNotif、VotingCompletionNotif、RejectionNotif 到 VotingManager。（2）NotificationManager 只保留通用的 NotificationEmail，成为通用、内聚、其他平台可复用的服务。（只画改动的服务框。）

Drawback: No major drawback. Separating the notifications by kind also lets the owning services scale independently.

> 缺点：没有大的缺点。按类别拆开通知后，所属的各个服务还能独立扩展。

换案例时 题干关键词是 reuse / other platforms / drastic adaptation → 找"通用服务里混了谁的专属实体"，原因和对策都点名实体。

15.5 Q1(b)(ii) Observer 看结果不及时 4 marks · T1

题干要点 4 分 · T1 · 原题 p.3 · 答案 p.3–4

你担心平台无法让 Observer 及时看到其监看的 Poll 的最新 CurrentSimplifiedResults。要求同上一题：原因、修改领域模型的对策、缺点。

1. "及时、promptly" → 问题类型是"慢"。
2. 在图上找相关实体：Observer 和 CurrentSimplifiedResults 在 ObservationManager，Poll 在 PollManager → 查看结果要两个服务协作。
3. 原因：跨进程通信（同步或事件）比进程内通信慢得多。
4. 对策：把 ObservationManager 的实体并入 PollManager。缺点：服务变大、事务范围变大，可以在服务内分多个聚合缓解。

得分点（官方）	分
最可能原因	1.5
对策	1.5
缺点	1

✍ 参考答案｜可照着写，名字和数字换成新案例的

Cause: Viewing the CurrentSimplifiedResults of a Poll requires the PollManager and the ObservationManager to collaborate, because Poll is in one service while Observer and the results are in the other. Every interaction needs inter-process communication, synchronous or event-based, which takes much longer than intra-process communication.

> 原因：查看某个 Poll 的 CurrentSimplifiedResults 需要 PollManager 和 ObservationManager 协作，因为 Poll 在一个服务里，Observer 和结果在另一个服务里。每次交互都要跨进程通信（同步或基于事件），比进程内通信慢得多。

Countermeasure: Migrate the entities of the ObservationManager (Observer, CurrentSimplifiedResults, CurrentDetailedResults) into the PollManager, as shown in the diagram. Viewing the updated results then happens within one service, so Observers see them promptly.

> 对策：把 ObservationManager 的实体（Observer、CurrentSimplifiedResults、CurrentDetailedResults）迁入 PollManager，如图所示。这样查看最新结果在一个服务内完成，观察员就能及时看到。

Drawback: The PollManager becomes a relatively large service (bounded context) with a larger transaction scope; this can be mitigated by having several aggregates inside the service.

> 缺点：PollManager 变成一个较大的服务（限界上下文），事务范围也变大；可以在服务内划分多个聚合来缓解。

✕ 常见误区｜这样答会丢分

- **写"加缓存、加负载均衡"不得分**：题目要求通过修改领域模型解决。
- 不要回答"我认为没有问题"：题目已给定现象，你要解释它。

15.6 Q1(c) API 表：RMM、版本、重构 6 marks · T3

题干要点 6 分 · T3 · 原题 p.3 · 答案 p.4–5

第一阶段设计了两个接口：Invite Next Voter（`GET /v1.1/votes/{voter-id}/invite`，输入选民标识，输出投票室内的选民列表）和 Start Vote（`GET /v1.1/votes/{voter-id}/start`，输出候选人列表和投票室内的选民列表），都属 ProctoringManager，自称 RMM Level 2。

三个小问：① 是否符合 RMM Level 2，为什么；② 提出更好的版本策略，让升级少影响调用方；③ 表中有多余或缺失的输入输出、可能错误的 HTTP 动词，重画正确的表。

1. 动词：邀请、开始投票都会**改变服务器状态**，却用了 GET → 不符合 Level 2，应改 POST。
2. 版本：版本号写在 URI 里（/v1.1），版本废弃时客户端代码会出错 → 改用请求头。
3. 输入输出逐个对用例：Invite 是"系统从队列里请下一位"，调用前不知道是谁 → 去掉输入，输出被邀请的 voter-id；Start 只需要候选人 → 删掉选民列表。

得分点（官方）	分
三个小问，每个：解释 1 + 改进设计 1	2 × 3

URI	Service	Inputs	Outputs	HTTP
/votes/invite	ProctoringManager	（无）	Voter-id of the invited voter	**POST**
/votes/{voter-id}/start	ProctoringManager	Voter identifier	List of candidates of the poll	**POST**

官方答案的改正表（版本改由请求头 Accept-version 传递）。

✍ **参考答案｜可照着写，名字和数字换成新案例的**

1. RMM Level 2? No. Level 2 requires each operation to use the HTTP verb that matches its semantics. Inviting the next voter and putting a voter into voting mode both change the state of resources on the server, but they use GET, which must be safe and free of side effects. POST should be used to submit the request and effect the state change.

> 1. 是否符合 RMM Level 2？不符合。Level 2 要求每个操作使用语义相符的 HTTP 动词。邀请下一位选民、让选民进入投票模式都会改变服务器上资源的状态，却用了 GET，而 GET 必须是安全、无副作用的。应该用 POST 提交请求并改变状态。

2. Versioning: URI versioning (/v1.1/...) breaks client code when a version is deprecated, because the endpoints themselves change. A better strategy is to pass the version in a custom request header such as `Accept-version: 1.1`, which keeps a clean, stable set of URIs and negotiates the version through headers; combined with semantic versioning, only breaking changes raise the major version.

> 2. 版本管理：URI 版本（/v1.1/...）在某个版本被废弃时会让客户端代码出错，因为端点地址本身变了。更好的策略是用自定义请求头传版本，如 Accept-version: 1.1，这样 URI 保持干净稳定，版本通过请求头协商；再配合语义化版本，只有不兼容的修改才升主版本号。

3. Corrected interfaces: see the table. The voter-id input of Invite is removed because the next voter is chosen by the platform from the queue, and the invited voter-id is returned instead; Start Vote only returns the candidates the voter needs, so the list of voters in the room is removed.

> 3. 改正后的接口见表。Invite 去掉了 voter-id 输入，因为下一位选民由平台从队列中选出，改为返回被邀请者的 voter-id；Start Vote 只返回选民需要的候选人，所以删掉投票室内的选民列表。

✕ 常见误区｜这样答会丢分

- 只改动词，忘了删多余的输入输出、忘了去掉 URI 里的版本号。
- 不看用例就假设 Invite 由调用方传入某位选民。

15.7 Q1(d) 易用性是不是最重要的非功能需求 2 marks · T7

题干要点 **2 分 · T7 · 原题 p.3 · 答案 p.6**

组员认为考虑到选民数量很大，系统（尤其是帮监考员管理投票场次）的易用性是最关键的非功能需求。*"Do you agree with this proposal"*？说明理由。

1. 先表态。官方答案是不同意，但老师说其他答案只要论证合理也给分。
2. 回第 4 节找"不满足就会失败"的要求：100,000 并发选民 → 性能/可扩展性。
3. 从用例找业务规则：每位选民最多投一票 → 安全/完整性。每个有说服力的理由 1 分。

✍ 参考答案｜可照着写，名字和数字换成新案例的

I disagree. Usability for the proctors matters, but it is not the most crucial requirement. (1) **Performance and scalability** are more critical: the platform must support 100,000 concurrent voters with 90% of interactive requests completed within 3 seconds; if it cannot, voters cannot vote during the poll no matter how easy the screens are. (2) **Security (integrity)** is also crucial: the platform must guarantee that each voter can vote at most once, otherwise the results cannot be trusted.

> 我不同意。监考员的易用性有意义，但不是最关键的需求。（1）性能和可扩展性更关键：平台必须支持 10 万并发选民，且 90% 的交互请求在 3 秒内完成；做不到的话，界面再好用，选民也无法在投票期内投票。（2）安全（完整性）同样关键：平台必须保证每位选民最多投一票，否则结果不可信。

15.8 Q1(e) "每日处理人数"适合当指标吗 4 marks · T8

题干要点 **4 分 · T8 · 原题 p.3–4 · 答案 p.6**

组员提议把**每天处理的选民数**作为应用指标，认为数字高就说明可扩展性和易用性好。是否同意？说明理由。

1. 先表态：不同意。总量只反映使用规模，受注册人数影响，不能说明高峰并发表现，也不能说明操作是否顺手。
2. 官方要求：好的指标要衡量**系统结果**（准确投票、使用情况），并说明测它为什么有助于改进。
3. 给两个替代指标，每个"指标 + 理由"2 分。

✍ 参考答案｜可照着写，名字和数字换成新案例的

I disagree. The daily number of voters processed mainly reflects how many voters are registered and turn up, not how well the system performs; a high number may even hide long queues. It says nothing about behaviour at peak load (scalability) or how smoothly voters and proctors complete their tasks (usability). Better application metrics are:

> 我不同意。每日处理的选民数主要反映注册和到场的人数，而不是系统表现得好不好；数字高甚至可能掩盖了长时间排队。它既不能说明高峰负载下的表现（可扩展性），也不能说明选民和监考员完成任务是否顺畅（易用性）。更好的应用指标是：

- **Average duration of identity verification** – shorter verification lets the platform serve more voters at peak, and shows the system supports proctors well.
  > 平均身份核验时长：核验越快，平台在高峰时能服务越多选民，也说明系统对监考员的支持好。
- **Average queueing time per voter** – measures the voter's experience and whether the platform keeps up during the voting window.
  > 每位选民的平均排队时间：衡量选民体验，以及平台在投票时间窗内是否跟得上。
- **Average time to cast a vote per voter** – tracks both the usability and the performance of the main use case.
  > 每位选民的平均投票用时：同时跟踪主用例的易用性和性能。

✕ 常见误区｜这样答会丢分

- **别因为急着给替代指标就漏了表态和理由**：题目先问的是同不同意。

15.9 Q2(a) 单体 → 混合云原生参考架构 + K8s 10 marks · T5

题干要点 10 分 · T5 · 原题 p.5 · 答案 p.7–8

团队做了一个单体 MVP 原型（题中给图）。要求把它改造成承载投票平台后端的**混合参考架构**（可考虑微服务、定时任务、事件/流驱动服务、Serverless），并回答四个小问：

(i) 画出云视角的主要组件及交互，多云环境还需要哪些额外组件或托管服务；(ii) 除了经 API Gateway 的外部通信，内部服务怎么通信，在图上标出并说明模式；(iii) 指出一个需要保存状态的服务及适合的状态管理机制；(iv) 用 K8s 部署图或一段文字描述整体部署计划。

1. 服务从 §7 领域模型来：Voting、Poll、Proctoring、Observation、User、Notification，每个一个框。
2. 判断哪些部分不适合常驻微服务：实时汇总结果 → 事件流；生成报表、汇总 → 定时任务；发通知 → Serverless。这就是"混合"。
3. 补齐支撑组件：API Gateway + 负载均衡/Ingress、服务注册中心、容器镜像仓库、GitOps 工具链、事件服务、通知服务、Redis 缓存、数据库、监控。
4. 多云额外组件：Kubernetes（可移植）、IaC、全局负载均衡/DNS、统一身份、统一监控、跨云镜像复制。
5. 四个小问逐条在图下用文字回答，**要求画图就必须画图**。

得分点（官方）	分
参考架构：服务、注册中心、状态机制、存储	4
额外通信组件：事件服务、通知服务	2
缓存或内存服务	1
部署计划：Pod、Deployment、Service	3

官方参考思路：投票先收集在内存 Redis 再持久化到 PostgreSQL；Poll 服务用内存存储跟踪状态；Observation 服务和定时任务汇总选票给出结果；配 GitOps 工具链、容器仓库、服务注册中心、负载均衡、Ingress 和 Istio 服务网格。

✍ 参考答案｜可照着写，名字和数字换成新案例的

(i) Architecture: Voters' mobile apps and the web front ends reach the platform through a CDN and an API Gateway with a load balancer/Ingress. The backend is hybrid: always-on microservices (Voting, Poll, Proctoring, Observation, User) run as containers on Kubernetes, each with its own database; result aggregation is event-stream driven; poll reports run as scheduled jobs; notifications are serverless functions. Supporting components: service registry, container registry, GitOps CI/CD, event streaming (e.g. Kafka), Redis cache, PostgreSQL and centralised monitoring. For multi-cloud we also need Kubernetes for portability, Infrastructure as Code, a global load balancer/DNS for failover, federated identity, unified monitoring and registry replication.

> (i) 架构：选民手机应用和 Web 前端经 CDN、API Gateway 加负载均衡/Ingress 进入平台。后端是混合架构：常驻的微服务（Voting、Poll、Proctoring、Observation、User）以容器运行在 Kubernetes 上，各有自己的数据库；结果汇总由事件流驱动；投票报表用定时任务；通知用 Serverless 函数。支撑组件：服务注册中心、容器镜像仓库、GitOps CI/CD、事件流（如 Kafka）、Redis 缓存、PostgreSQL、集中监控。多云环境还需要：Kubernetes 保证可移植、基础设施即代码、全局负载均衡/DNS 做故障转移、联合身份认证、统一监控、镜像仓库复制。

(ii) Internal communication: synchronous REST/gRPC through service discovery and a service mesh for queries that need an immediate answer (e.g. Proctoring checks a voter's status); asynchronous publish/subscribe through the event stream for high-volume updates – each cast vote publishes a VoteCast event consumed by the Observation service. Asynchronous is preferred for votes because it decouples services, absorbs the peak and keeps events if a consumer is down.

> (ii) 内部通信：需要立即得到结果的查询用同步 REST/gRPC，经服务发现和服务网格（例如 Proctoring 查询选民状态）；大量更新用事件流异步发布/订阅：每投出一票就发布一个 VoteCast 事件，由 Observation 服务消费。投票更适合异步，因为它解耦服务、能吸收峰值，消费者暂时宕机时事件也不会丢。

(iii) Stateful service: the Voting service keeps the in-progress votes and running counts. The state is externalised to Redis (in memory, fast) and persisted to PostgreSQL, so the service pods stay stateless and can scale out or be replaced without losing votes.

> (iii) 有状态服务：Voting 服务保存进行中的投票和实时计数。状态外置到 Redis（内存、快）并持久化到 PostgreSQL，这样服务的 Pod 保持无状态，可以随时扩容或替换而不丢票。

(iv) Kubernetes plan: one namespace for production; each microservice is a Deployment with several replica Pods spread over availability zones, exposed by a ClusterIP Service; an Ingress routes external traffic behind the API Gateway; a HorizontalPodAutoscaler scales Voting and Proctoring on CPU or request rate; report generation runs as a CronJob; Redis/Kafka/databases use managed services or StatefulSets with persistent volumes; ConfigMaps and Secrets hold configuration; releases use rolling updates.

> (iv) Kubernetes 部署计划：生产环境用一个命名空间；每个微服务是一个 Deployment，多个副本 Pod 分布在多个可用区，由 ClusterIP Service 对内暴露；Ingress 在 API Gateway 之后路由外部流量；HorizontalPodAutoscaler 按 CPU 或请求速率扩缩 Voting 和 Proctoring；报表生成用 CronJob；Redis、Kafka、数据库用托管服务或带持久卷的 StatefulSet；配置放 ConfigMap 和 Secret；发布用滚动更新。

✕ 常见误区｜这样答会丢分

- 只堆服务名、不画交互或不标通信组件，拿不到"通信组件 2 分"。
- K8s 部署计划不写 Pod、Deployment、Service 这三个词，3 分直接丢。
- 推荐云服务只写类别不写产品名。

15.10 Q2(b) 原生应用还是 PWA 3 marks · T10

题干要点 **3 分 · T10 · 原题 p.5–6 · 答案 p.8–9**

看 §6.2 逻辑架构：选民前端需要本地存储备份；平台将来要扩展到其他国家，设立配有 IoT 传感器和执行器的实体投票站。前端做原生应用还是 PWA？对后端服务有什么设计影响？

1. 先处理题干给的"本地存储"：两者都能管理客户端状态 → 从存储角度没区别（1 分）。
2. 真正的决定因素是 IoT 传感器、执行器 → 需要深度访问设备硬件 → 选原生（1 分）。
3. 后端影响：客户端编排、客户端路由和状态共享，后端服务不需要改（1 分）。

✍ **参考答案｜可照着写，名字和数字换成新案例的**

Storage: both a native app and a PWA can manage client-side state (local storage, session storage, application state stores), so local backup alone does not decide the choice.

> 存储：原生应用和 PWA 都能管理客户端状态（local storage、session storage、应用状态存储），所以仅凭本地备份不能决定选哪个。

Decision: the future polling stations will use IoT sensors and actuators for voters with special needs, which requires deep access to native device services. I would therefore build a native front end.

> 决定：未来的投票站要为有特殊需要的选民使用 IoT 传感器和执行器，需要深度访问设备的原生服务，所以我会做原生前端。

Backend implications: client-side orchestration is preferred – routing and shared state are handled in the client, which resolves its dependencies by registering multiple APIs – so no change to the backend services is required; the same APIs serve all clients.

> 对后端的影响：采用客户端编排，路由和共享状态在客户端处理，客户端通过注册多个 API 解决依赖，所以后端服务不需要修改；同一套 API 服务所有客户端。

不要背成"凡是要本地存储就选原生"：本题的结论由 IoT 需求决定。

15.11 Q3(c) 打包部署模型找错 6 marks · T4

题干要点 **6 分 · T4 · 原题 p.7 · 答案 p.10–11**

对照 §7 领域模型和 §8 Figure 6（不完整、可能有错的逻辑打包兼部署模型），**找出两个不理想的架构决策**，提出纠正措施，并解释决策如何导致这些措施。可以只画必要的部分。（原卷印作 c，老师课上称 Q3(a)。）

1. 先读图注：后端部署刻意省略 → 不算错。
2. 逐条核对 manifest：每个实体是否打包在它所属服务的 Data Access 子系统里？VotingCompletionNotif 属 NotificationManager，却在 Voting Manager Data Access。
3. 逐条核对 deploy：投票用例要在设备本地保存投票记录 → Voting Manager Data Access 应部署到手机，图上没有；通知由平台发，不由手机发 → Notification Manager Control 不应部署到手机。
4. 三个里选两个写，每个"识别 → 纠正 → 论证"三句话；改图只画受影响部分。

得分点（官方）	分
每个决策：识别 1 + 纠正 1 + 论证 1，答两个	3 × 2

✍ 参考答案｜可照着写，名字和数字换成新案例的

Decision 1: VotingCompletionNotif is packaged into the Voting Manager Data Access subsystem. **Correction:** package it into the Notification Manager Data Access subsystem. **Justification:** in the domain model it belongs to the NotificationManager service; packaging it with VotingManager breaks the bounded context and couples the two services.

> 决策 1：VotingCompletionNotif 被打包进 Voting Manager Data Access 子系统。纠正：改为打包进 Notification Manager Data Access 子系统。论证：在领域模型中它属于 NotificationManager 服务；和 VotingManager 打包在一起破坏了限界上下文，把两个服务耦合在一起。

Decision 2: the Voting Manager Data Access subsystem is not deployed to the mobile device. **Correction:** add a deploy link to the Mobile Device with Local Storage. **Justification:** Conduct Vote stores the voting record locally on the voter's device, so the device needs this subsystem to persist CastVote.

> 决策 2：Voting Manager Data Access 子系统没有部署到手机。纠正：加一条部署到 Mobile Device with Local Storage 的连线。论证：Conduct Vote 用例要把投票记录保存在选民设备本地，设备需要这个子系统来持久化 CastVote。

Decision 3 (alternative): the Notification Manager Control subsystem is deployed to the mobile device. **Correction:** remove this deployment. **Justification:** notifications are sent by the platform, not from the front end.

> 决策 3（备选）：Notification Manager Control 子系统被部署到手机。纠正：删掉这条部署。论证：通知由平台发送，不是从前端发出。

✕ 常见误区｜这样答会丢分

- **把"没画后端"当成错误**：图注说明是刻意省略。
- 把 Q1(b) 自己提出的领域模型改造当成本题前提：本题以附录原图为准。

15.12 Q3(d) 数据库写吞吐瓶颈 6 marks · T11

题干要点 6 分 · T11 · 原题 p.7 · 答案 p.11

性能测试发现：10 万选民要在 5 分钟窗口内投完票，但数据库写吞吐不够。当前是一主库加一个只读副本。要求：提出扩展写吞吐的完整方案（含软件和基础设施的改动和影响），并指出**至少两个**上线后要密切监控的副作用。

1. 只读副本只扩展读 → 写的瓶颈必须分片。
2. 分片键：选民唯一标识（Voter ID、NRIC、护照号）哈希，均匀分布；可按分片分配不同的监考员组。
3. 逐个点名受影响组件并说原因（这部分占 4 分）。
4. 题干还要求副作用：热点分片、跨分片汇总变慢、一致性、再分片和运维成本。

得分点（官方）	分
怎么分片	2
受影响组件及理由	4

✍ 参考答案｜可照着写，名字和数字换成新案例的

Plan: a read replica only scales reads, so the writes are spread by sharding the database on the voter's unique identification (voter ID / NRIC / passport number) with a hash function; each shard has its own primary (plus replicas). Optionally each voter shard is served by its own group of proctors. Votes can also be buffered in an event stream and written in batches to absorb the 5-minute peak.

> 方案：只读副本只能扩展读，所以要按选民唯一标识（选民 ID / NRIC / 护照号）用哈希函数对数据库分片，把写入分散开；每个分片有自己的主库（可再加副本）。可选：每个选民分片由自己的一组监考员负责。也可以先把选票缓冲到事件流里，再批量写入，以吸收 5 分钟的峰值。

Shard-aware components: VoteController – to record each vote in the right shard; PollManager – to record poll data in the right shard; UserManager – because users are spread over shards; ObservationManager – because it must aggregate results across all shards. Infrastructure: more database instances, a shard map/routing layer and per-shard monitoring.

> 要感知分片的组件：VoteController，把每张选票写入正确的分片；PollManager，把投票数据记录到正确的分片；UserManager，因为用户分散在不同分片；ObservationManager，因为要跨所有分片汇总结果。基础设施：更多数据库实例、分片映射/路由层、按分片的监控。

Side effects to monitor: (1) hot or uneven shards; (2) slower cross-shard aggregation, which may delay the 1-minute simplified results; (3) harder consistency for rules such as one vote per voter; (4) complex re-sharding and higher operational cost.

> 要监控的副作用：（1）热点或分布不均的分片；（2）跨分片汇总变慢，可能拖慢 1 分钟内更新的简要结果；（3）"每人一票"这类规则的一致性更难保证；（4）重新分片复杂、运维成本更高。

✕ 常见误区｜这样答会丢分

- 答"再加只读副本"：副本不扩展写。
- 只写"按 ID 分片"，不点名组件和理由，**丢掉 6 分里的 4 分**。

15.13 本卷复盘 What PAST Teaches

◆ 要点｜必背结论 **从官方答案看出的五条规律**

1. 每个得分部分都对应题干里的一个要求词：要两个理由就给两个，要缺点就写缺点。
2. 领域模型题的原因、对策都落在"哪个实体在哪个服务"上。
3. 小题的理由一定绑定 §4 数字或用例规则（10 万并发、每人一票）。
4. 图题按要素给分：服务、注册中心、状态、存储、通信组件、缓存、Pod/Deployment/Service，漏一个丢一份。
5. 找错题以附录原图为准，读图注，"刻意省略"不是错。

CHAPTER 16

PAST2 精讲：MLTTP 公交遥测平台

Sample Paper PAST2 — Walkthrough (no official solution)

来源：SWE5001 Sample Exam Paper_PAST2 · Appendix A_PAST2（Figure 5、Figure 6 已逐图核对）· 按 PAST 评分方式推断的得分点

12 道小题 · 50 分

一句话 **PAST2 的主线是"海量实时更新"：每道题都绕着 1 万台车载设备的持续上报、5 秒内展示、2 秒内回传理想车速展开。**

本卷新出现的问法

- 聚合太小、太大分别导致什么（T1）
- 大批量并发更新的 API 延迟（T3）
- 两个质量属性 → 两个云数据库（T7 + T6）
- VM 还是容器、托管 ML 架构（T10、T6）
- 找一个错 + 补全打包（T4）
- 预测准确度指标的完整设计（T8）

本卷没有官方答案。"得分点"按 PAST 官方评分的颗粒度推断，标为"推断"；参考答案是可得分的一种写法，不是唯一答案。

16.1 案例速读 Case Facts You Will Cite

业务：陆路交通管理部门建设下一代多模式遥测平台 MLTTP，先做公交，未来扩展到地铁、出租等。第一阶段做平台和两个试点应用（Web 应用、乘客移动应用）；第二阶段加历史数据分析、机器学习预测和发车调整。

附录 §4 数字	要求
并发	**100,000** Public Users、**10,000** Registered Users、5 System Admins、**10,000** Vehicle Onboard Units；车队最多 3,000 辆
响应	查看到站时间和载客量 90% < 3 秒；到站时间/载客量的变化 **5 秒内**展示给用户；理想车速 **2 秒内**送达车载设备
其他	不预期多租户；数据隔离不能牺牲一致性；第 5 节另有租户专属密钥、PCI-DSS 条款

§7 领域模型（Figure 5）	实体与聚合（红圈）
ArrivalTimeComputer	Position、Speed、ArrivalTime，**三个各自独立成聚合**
DesirableSpeedComputer	DesirableSpeed（单独聚合）
PassengerLoadComputer	PassengerEvent（子类 PassengerEntrance、PassengerExit）、PassengerLoad，各成聚合
BusNetworkManager	**BusTrip（含 passengerCount）与 BusRoute 同一聚合**；BusStop 单独聚合
UserManager	PublicUser、RegisteredUser（子类 BusCommuter）、SystemAdmin

§6 的层名与 PAST 不同：Presentation / Control / Platform Service / Data Access / Enterprise Integration。车载设备的乘客上下车事件**故意逐条上报、不批量。**

16.2 Q1(a) 平台成功的两个关键架构方面 2 marks · T9

题干要点 2 分 · T9 · 原题 p.2

两个种子是"到站时间"和"载客量"；第一阶段要支持每分钟 1 万次更新、90% 的用户更新响应 < 3 秒。交互包括在常用地点/临时地点查看到站时间和载客量；目前 10 万公众用户、1 万注册用户，未来增长；每 30 秒同步一次（约等于巴士两站之间的行驶时间）。要求：结合种子和交互，**解释两个对平台成功最关键的架构方面。**

1. 点名种子、生产者（车载设备）、消费者（乘客）、交互。
2. 交互量大且增长 → 读侧的可扩展性/弹性。
3. 种子要新鲜 → 写侧实时更新管道的性能（事件流、并行处理）。
4. 每个方面都绑一个数字。

得分点（推断）	分
每个方面 + 结合数字的理由	1 × 2

✍ 参考答案｜可照着写，名字和数字换成新案例的

(1) Scalability and elasticity of the viewing services: 100,000 public users and 10,000 registered users (and growing) view arrival times and loads at the same time, especially in peak hours. The view services must be stateless and scale out automatically, serving the latest values per bus stop from an in-memory cache, so that 90% of requests are answered within 3 seconds.

> （1）查看类服务的可扩展性和弹性：10 万公众用户和 1 万注册用户（还在增长）同时查看到站时间和载客量，高峰期尤其如此。查看服务必须无状态、能自动扩容，并从内存缓存读取每个车站的最新值，才能让 90% 的请求在 3 秒内响应。

(2) Performance of the real-time update pipeline (freshness of the seeds): 10,000 updates per minute from the onboard units must be reflected every 30 seconds. An event-driven pipeline with event streaming, partitioned by bus trip, lets the computing services process updates in parallel and in order and push new values quickly; stale seeds would drive users away from the platform.

> （2）实时更新管道的性能（种子的新鲜度）：车载设备每分钟 1 万次更新必须每 30 秒反映出来。用按班次分区的事件流做事件驱动管道，计算服务就能并行且按顺序处理更新、快速推送新值；种子过时会把用户赶走。

16.3 Q1(b)(i) 到站时间计算不够快 5 marks · T1

题干要点 5 分 · T1 · 原题 p.2

ArrivalTimeComputer 根据位置和速度计算班次的最新到站时间。你认为现有领域设计不利于快速计算。要求：原因、修改领域模型的对策、缺点。

1. 看图：Position、Speed、ArrivalTime 是**三个独立聚合**，但每次计算都要同时用到 → 一次计算要加载、保存三个聚合，三次事务。
2. 计算还要用到 BusTrip、BusStop，它们在另一个服务 BusNetworkManager → 每次更新都要跨服务通信。
3. 对策：三个合成一个聚合；在本服务内保留站点的只读副本，由领域事件同步。
4. 缺点：数据重复、最终一致、聚合变大；站点很少变，可以接受。

得分点（推断）	分
原因（点名聚合和跨服务）	2
对策（含改动图）	2

得分点（推断）	分
缺点	1

✍ 参考答案｜可照着写，名字和数字换成新案例的

Cause: Position, Speed and ArrivalTime are modelled as three separate aggregates in the ArrivalTimeComputer service, although they are always used together; each computation loads and saves three aggregates in separate transactions. The computation also needs BusTrip and BusStop, which live in the BusNetworkManager service, so every one of the 10,000 updates per minute requires inter-process communication.

> 原因：Position、Speed、ArrivalTime 在 ArrivalTimeComputer 服务里被设计成三个独立聚合，但它们总是一起使用；每次计算都要在不同事务里加载、保存三个聚合。计算还需要 BusTrip 和 BusStop，它们在 BusNetworkManager 服务里，所以每分钟 1 万次更新的每一次都要跨进程通信。

Countermeasure: (1) Merge Position, Speed and ArrivalTime into one aggregate per bus trip (e.g. TripTracking), since they change together. (2) Keep a read-only copy of each route's bus stops inside the ArrivalTimeComputer, updated by domain events such as BusRouteChanged from the BusNetworkManager, and reference BusTrip/BusStop by identity. The arrival times can then be computed within one service and one transaction.

> 对策：（1）把 Position、Speed、ArrivalTime 合并成每个班次一个聚合（如 TripTracking），因为它们一起变化。（2）在 ArrivalTimeComputer 里保留每条线路车站的只读副本，由 BusNetworkManager 发出的 BusRouteChanged 等领域事件更新，并按 ID 引用 BusTrip、BusStop。这样到站时间在一个服务、一个事务内就能算完。

Drawback: The bus-stop data is duplicated and only eventually consistent, and the merged aggregate is larger; acceptable because routes and stops change rarely.

> 缺点：车站数据重复，只能做到最终一致，合并后的聚合也更大；因为线路和车站很少变，这可以接受。

另一种可得分的对策：把 ArrivalTimeComputer 的实体并入 BusNetworkManager。缺点是那个服务更大、内存更高，正好和下一题冲突，答题时要注意两题不要自相矛盾。

16.4 Q1(b)(ii) BusNetworkManager 内存占用高　4 marks · T1

题干要点　4 分 · T1 · 原题 p.2

BusNetworkManager 管理各线路的站点和班次。你认为现有设计在运行内存上不够好，担心它的实例内存占用很高。要求同上。

1. "memory" → 问题类型是聚合太大。
2. 看红圈：BusRoute 与 BusTrip 在同一聚合里。一条线路一天有很多班次，加载一条线路就把所有班次连同频繁变化的 passengerCount 一起载入内存。
3. 对策：按"设计小聚合"拆成 BusRoute、BusTrip 两个聚合，用 ID 引用；可选地把 passengerCount 移到 PassengerLoadComputer。
4. 缺点：多一次查询；跨聚合变更最终一致。

得分点（推断）	分
原因	1.5
对策	1.5
缺点	1

✍ 参考答案｜可照着写，名字和数字换成新案例的

Cause: BusRoute and BusTrip are designed as one aggregate. A route has many trips a day (up to 3,000 buses in service), so loading a BusRoute pulls all of its BusTrip objects, including the frequently changing passengerCount, into memory; every BusNetworkManager instance therefore uses a lot of runtime memory.

> 原因：BusRoute 和 BusTrip 被设计成同一个聚合。一条线路每天有很多班次（运营中的巴士最多 3,000 辆），加载一条 BusRoute 就会把它所有的 BusTrip 对象，连同频繁变化的 passengerCount，一起载入内存；所以每个 BusNetworkManager 实例都占用大量运行内存。

Countermeasure: Following "design small aggregates", split it into a BusRoute aggregate and a BusTrip aggregate; BusTrip refers to its route by identity (busRouteId), so only the trips being updated are loaded. Optionally move passengerCount to PassengerLoad in the PassengerLoadComputer, where it is computed.

> 对策：按"设计小聚合"原则拆成 BusRoute 聚合和 BusTrip 聚合；BusTrip 按 ID（busRouteId）引用所属线路，只加载正在更新的班次。可选：把 passengerCount 移到 PassengerLoadComputer 的 PassengerLoad 里，那里才是计算它的地方。

Drawback: Listing all trips of a route needs an extra query, and changes affecting both a route and its trips become eventually consistent through domain events.

> 缺点：列出一条线路的所有班次要多一次查询；同时影响线路和班次的变更要通过领域事件实现最终一致。

16.5 Q1(c) 大批量并发更新时 API 延迟 20–30 秒 6 marks · T3

题干要点 6 分 · T3 · 原题 p.2–3

车载设备调用的两个接口：`PUT /arrivaltime`（ArrivalTimeComputer，输入班次、线路、位置、速度，输出"已处理/未处理"）和 `PUT /passengerload`（PassengerLoadComputer，输入班次、线路、一个乘客事件）。预发布环境负载测试中，多台车载设备大批量并发更新时出现约 20–30 秒的更新延迟。要求：解释可能原因，说明怎样重构 API，**用具体例子**展示新设计。（提示：API 应由外而内设计。）

1. 找原因（三条）：同步处理（回复前要算完所有到站时间和理想车速）；PUT 覆盖同一个共享资源，并发互相阻塞；URI 不面向资源、输入冗余（班次已能确定线路）。
2. 重构：面向资源 + 异步。POST 追加事件到班次下的子资源，写入事件流后立刻返回 202 Accepted，消费者按班次分区并行计算。
3. 乘客事件仍逐条上报（案例要求），但不再等待计算。
4. 写一个具体请求示例：URI、请求体、返回码。

得分点（推断）	分
延迟原因	约 2
重构方案	约 2
具体例子	约 2

URI	HTTP	Input	Output	Explanation
/bus-trips/{trip-id}/positions	POST	position, speed, timestamp	202 Accepted; eventId	Report position and speed (processed asynchronously)
/bus-trips/{trip-id}/passenger-events	POST	eventType (entrance/exit), timestamp	202 Accepted; eventId	Report one passenger entrance or exit

URI	HTTP	Input	Output	Explanation
/bus-trips/{trip-id}/passenger-count	PUT	passengerCount, timestamp	200 OK	Reset the passenger count (a real replacement)

✍ 参考答案｜可照着写，名字和数字换成新案例的

Likely reasons: (1) The APIs are synchronous: before replying "processed", the service recomputes arrival times or loads for all observed locations and the desirable speed, so thousands of concurrent requests queue up. (2) Both use PUT on one shared resource (/arrivaltime, /passengerload), so concurrent updates overwrite and block each other, although each update is really a new event. (3) The URIs are not resource-oriented and carry a redundant Bus Route (the trip already identifies its route), increasing payload.

> 可能原因：（1）API 是同步的：回复"已处理"之前，服务要为所有被关注的地点重算到站时间或载客量以及理想车速，所以成千上万的并发请求在排队。（2）两个接口都对同一个共享资源（/arrivaltime、/passengerload）用 PUT，并发更新互相覆盖、互相阻塞，而每次更新其实是一个新事件。（3）URI 不面向资源，还带了冗余的 Bus Route（班次已能确定线路），增大了请求体。

Refactoring: Make the APIs resource-oriented and asynchronous. Each onboard unit POSTs an event under its own bus trip; the API validates it, publishes it to an event stream (e.g. Amazon Kinesis) partitioned by trip ID and returns 202 Accepted immediately. ArrivalTimeComputer and PassengerLoadComputer consume the partitions in parallel and in order, and push results to users.

> 重构：把 API 改成面向资源、异步处理。每台车载设备在自己的班次下 POST 一个事件；API 校验后把它发布到按班次 ID 分区的事件流（如 Amazon Kinesis），立即返回 202 Accepted。ArrivalTimeComputer 和 PassengerLoadComputer 并行且按顺序消费各分区，再把结果推送给用户。

Example: trip 175-0832 sends `POST /bus-trips/175-0832/positions` with {"position": "1.2966,103.7764", "speed": 32, "timestamp": "08:15:30"} and receives 202 Accepted within milliseconds; the arrival times at the observed stops are updated within 5 seconds.

> 例子：班次 175-0832 发送 POST /bus-trips/175-0832/positions，请求体为 {"position": "1.2966,103.7764", "speed": 32, "timestamp": "08:15:30"}，几毫秒内收到 202 Accepted；被关注车站的到站时间在 5 秒内更新。

✕ 常见误区｜这样答会丢分

- 只说"加服务器"：题目问的是 API 设计本身为什么慢、怎么改。
- 没有给具体例子：题目明确要求 illustrate with a concrete example。

16.6 Q1(d) 存储最重要的两个质量属性 2 marks · T7

题干要点 2 分 · T7 · 原题 p.3

根据第一阶段需求，**找出存储/数据库技术最重要的两个质量属性**，作为架构指导原则的一部分，并说明它们为什么比其他属性更重要。

1. 写多：1 万台车载设备持续上报、数据不断增长 → 可扩展性（写吞吐）。
2. 读快：5 秒展示、2 秒回传、90% < 3 秒 → 性能（低延迟）。
3. 为什么比其他重要：数据简单、按键访问，复杂查询不重要；这两个不满足核心交互就失败。

✍ 参考答案｜可照着写，名字和数字换成新案例的

The two most important quality attributes are **scalability (write throughput)** and **performance (low latency)**. Up to 10,000 onboard units continuously send positions, speeds and passenger events, and the data keeps growing, so the store must scale writes horizontally. Changes must reach users within 5 seconds and the desirable speed must reach the bus within 2 seconds, so reads and writes must take milliseconds. They outweigh attributes such as rich query support because the data is simple and accessed by key (trip, stop); if the store is too slow, the core interactions fail.

> 最重要的两个质量属性是可扩展性（写吞吐）和性能（低延迟）。多达 1 万台车载设备持续发送位置、速度和乘客事件，数据不断增长，所以存储必须能水平扩展写入。变化要在 5 秒内展示给用户，理想车速要在 2 秒内送到车上，所以读写都必须是毫秒级。它们比丰富查询能力等属性更重要，因为数据简单、按键（班次、车站）访问；存储太慢，核心交互就会失败。

16.7 Q1(e) 两个云数据库服务 4 marks · T6

题干要点 **4 分 · T6 · 原题 p.3**

基于 Q1(d) 的两个质量属性，**提出两个云存储/数据库服务**，说明它们如何帮助架构满足这两个属性。

1. 写具体厂商的具体服务名。
2. 每个服务都要说清楚它满足两个属性中的哪一个、靠什么机制。

得分点（推断）	分
每个服务：产品名 + 如何满足属性	2 × 2

✍ 参考答案｜可照着写，名字和数字换成新案例的

Amazon DynamoDB – a fully managed key-value and document database with single-digit-millisecond performance at any scale. It partitions data by key (e.g. trip ID), so write throughput scales horizontally for 10,000 updates per minute (scalability), and each read or write takes milliseconds (performance). It stores positions, speeds, passenger events and computed arrival times.

> Amazon DynamoDB：全托管的键值和文档数据库，任何规模下都有个位数毫秒的性能。它按键（如班次 ID）分区，写吞吐能水平扩展到每分钟 1 万次更新（可扩展性），每次读写都是毫秒级（性能）。用来存位置、速度、乘客事件和算好的到站时间。

Amazon ElastiCache for Redis – a fully managed in-memory store. The latest arrival times and loads per bus stop are kept in Redis, so 100,000 public users get answers well within 3 seconds (performance), and nodes can be added or sharded as users grow (scalability).

> Amazon ElastiCache for Redis：全托管的内存存储。每个车站最新的到站时间和载客量放在 Redis 里，10 万公众用户的查询远低于 3 秒（性能）；用户增长时可以加节点或分片（可扩展性）。

其他可得分：Amazon Timestream（时序数据）、Kinesis + S3（历史数据湖）、Aurora。

16.8 Q1(f) 哪些服务放进同一个容器 2 marks · T12

题干要点 **2 分 · T12 · 原题 p.3**

服务用容器部署，但受容器平台许可限制，需要把两个以上服务放进同一个容器实例。**提出最适合共置的服务**，并说明共置的好处。

1. 找交互最频繁的一对：每次位置/速度更新 → 算到站时间 → 算理想车速（2 秒内送回）。
2. 它们的负载模式相同，一起扩缩容不浪费。
3. 好处：网络调用变进程内调用，延迟低、网络开销小。

✍ 参考答案｜可照着写，名字和数字换成新案例的

I propose to collocate the **ArrivalTimeComputer** and the **DesirableSpeedComputer**. Every position and speed update triggers a new arrival time, and every new arrival time triggers the desirable-speed computation, which must reach the onboard unit within 2 seconds. They exchange messages for every one of the 10,000 updates per minute and share the same load pattern. Collocating them turns these network calls into intra-process calls, cutting latency and network overhead, and they can be scaled together.

> 我建议把 ArrivalTimeComputer 和 DesirableSpeedComputer 放进同一个容器。每次位置和速度更新都会触发新的到站时间计算，每个新的到站时间又触发理想车速计算，结果必须在 2 秒内送到车载设备。每分钟 1 万次更新中的每一次它们都要交换消息，负载模式也相同。共置后这些网络调用变成进程内调用，降低延迟和网络开销，还能一起扩缩容。

另一种可得分：ArrivalTimeComputer + BusNetworkManager（领域模型中跨框连线最多）。

16.9 Q2(a) 本地部署 → 云原生架构　7 marks · T5 + T10

题干要点　　**7 分 · T5/T10 · 原题 p.4**

乘客应用 BusInfo4Me 提供交互地图、仪表盘、实时到站时间和载客量、多段行程规划和常用地点。题中给了一张传统本地部署 Web 应用的参考架构（硬件负载均衡、Web 服务器、应用服务器、数据库）。要求：重画为适合 MLTTP 的云原生架构，**标出新增、删除、修改的组件**；后端选纯微服务还是混合，并结合改造后的架构论证。

1. 逐个组件说去向：硬件负载均衡 → 托管负载均衡/Ingress；Web 服务器 → S3 + CloudFront；应用服务器 → 容器化微服务 + 自动扩缩；单库加备库 → 每服务托管库、多可用区。
2. 新增：API Gateway（REST + WebSocket 推送）、车载数据接入（IoT Core / Kinesis）、事件流、Redis、Lambda + SNS 通知、Cognito、监控、CI/CD。
3. 选混合，并按工作负载分配风格。
4. 在图上用 added / removed / changed 标签标注。

得分点（推断）	分
云原生架构图，组件标对	约 3
标出新增/删除/修改	约 2
选择混合并论证	约 2

✍ 参考答案｜可照着写，名字和数字换成新案例的

Added: CloudFront + S3 for the web app and map content; Amazon API Gateway (REST, plus WebSocket to push real-time updates); AWS IoT Core / Kinesis Data Streams to ingest telemetry from 10,000 onboard units; an EKS cluster running BusNetworkManager, ArrivalTimeComputer, PassengerLoadComputer, DesirableSpeedComputer, JourneyPlanner and UserManager; event streaming between services; ElastiCache (Redis) for the latest values; DynamoDB and RDS; Lambda + SNS for Notify User; Cognito; CloudWatch and CI/CD. **Removed:** hardware load balancers and the fixed web and application servers. **Changed:** load balancing becomes Elastic Load Balancing/Ingress; web servers become S3 + CloudFront; app servers become containerised microservices with auto-scaling; the single database with a standby becomes managed Multi-AZ databases per service.

> 新增：CloudFront + S3 承载 Web 应用和地图内容；Amazon API Gateway（REST，另加 WebSocket 推送实时更新）；AWS IoT Core / Kinesis Data Streams 接入 1 万台车载设备的遥测数据；EKS 集群运行 BusNetworkManager、ArrivalTimeComputer、PassengerLoadComputer、DesirableSpeedComputer、JourneyPlanner、UserManager；服务之间的事件流；ElastiCache（Redis）存最新值；DynamoDB 和 RDS；Lambda + SNS 实现 Notify User；Cognito；CloudWatch 和 CI/CD。删除：硬件负载均衡器和固定的 Web、应用服务器。修改：负载均衡改为 Elastic Load Balancing / Ingress；Web 服务器改为 S3 + CloudFront；应用服务器改为可自动扩缩的容器化微服务；单库加备库改为每个服务一个托管的多可用区数据库。

Hybrid, because: interactive features (journey mapping, viewing arrival times) run as microservices; telemetry is processed by event-stream consumers, ordered per trip, in near real time; notifications are serverless functions that scale with spikes and cost nothing when idle; Phase 2 historical analysis runs as scheduled jobs. Each workload uses the most suitable and cost-effective style.

> 选混合架构，因为：交互功能（行程规划、查看到站时间）用微服务；遥测数据由事件流消费者按班次有序、近实时处理；通知用 Serverless 函数，随突发量自动扩展、空闲时不花钱；第二阶段的历史分析用定时任务。每种工作负载都用最合适、最省钱的风格。

16.10 Q2(b) 虚拟机还是容器 3 marks · T10

题干要点 3 分 · T10 · 原题 p.4

根据附录第 4 节的扩展要求和上述功能，**托管用虚拟机还是容器**？简要说明这些因素的扩展设计决策。

✍ 参考答案｜可照着写，名字和数字换成新案例的

I prefer containers orchestrated by Kubernetes (Amazon EKS). (1) Load changes quickly at peak hours (100,000 public and 10,000 registered users); containers start in seconds and the HorizontalPodAutoscaler adds replicas on CPU or request rate, while VMs take minutes. (2) Each microservice scales on its own driver: view services on the number of users, ArrivalTimeComputer and PassengerLoadComputer on the update rate from 10,000 onboard units (stream lag, number of partitions). (3) Containers are lighter than VMs, so cost is lower; the Cluster Autoscaler adds nodes across availability zones when needed.

> 我选由 Kubernetes（Amazon EKS）编排的容器。（1）高峰期负载变化快（10 万公众用户、1 万注册用户）；容器秒级启动，HorizontalPodAutoscaler 按 CPU 或请求速率加副本，虚拟机则要几分钟。（2）每个微服务按自己的指标扩展：查看服务按用户数，ArrivalTimeComputer 和 PassengerLoadComputer 按 1 万台车载设备的更新速率（事件流积压、分区数）。（3）容器比虚拟机轻，成本更低；需要时 Cluster Autoscaler 跨可用区加节点。

换案例时 第三分常给在"扩展决策"上：**说清每类服务按什么指标扩**（用户数、更新速率、队列长度）。

16.11 Q2(c) 用托管 ML 服务预测到站时间 3 marks · T6

题干要点 **3 分 · T6 · 原题 p.5**

用某云厂商的全托管服务做高精度到站预测，模型把时间序列数据和其他变量结合起来。要求：为 Update Arrival Times at Observed Location 用例**画架构图**，标出推荐的云产品/托管服务，并**对每个必需组件说明可替代的选择。**

1. 按数据流画：接入 → 预处理 → 存储（历史 + 额外变量）→ 训练 → 在线推理 → 写结果 → 对外提供。
2. 每个框写产品名，旁边写替代品（其他云或自建）。

✍ 参考答案 | 可照着写，名字和数字换成新案例的

Kinesis Data Streams ingests positions and speeds → Lambda cleans them and stores history in S3 together with extra variables (weather, traffic, time of day) → Amazon SageMaker trains a time-series forecasting model and hosts it as a real-time endpoint → for each update Lambda calls the endpoint to predict arrival times at observed locations and writes them to DynamoDB → API Gateway serves them to the apps. **Alternatives:** ingestion – Amazon MSK, Azure Event Hubs, Google Pub/Sub; storage – Azure Data Lake, Google BigQuery; ML – Azure Machine Learning, Google Vertex AI; serving – containers on EKS instead of Lambda.

> Kinesis Data Streams 接入位置和速度 → Lambda 清洗后连同额外变量（天气、交通、时段）把历史数据存进 S3 → Amazon SageMaker 训练时间序列预测模型并部署为实时推理端点 → 每次更新时 Lambda 调用端点预测各关注地点的到站时间，写入 DynamoDB → API Gateway 提供给应用。替代选择：接入可用 Amazon MSK、Azure Event Hubs、Google Pub/Sub；存储可用 Azure Data Lake、Google BigQuery；机器学习可用 Azure Machine Learning、Google Vertex AI；推理服务可用 EKS 上的容器代替 Lambda。

16.12 Q3(a) 第二阶段逻辑架构：找一个错 + 补全打包 6 marks · T4

题干要点 **6 分 · T4 · 原题 p.6 · 附录 Figure 6 p.9**

参照第 2 节的第二阶段需求、第 6 节初始架构和第 8 节 Figure 6（新手画的不完整逻辑架构，未画部署节点）：**找出一个不理想的架构决策**并纠正、论证；再**补全尚未打包的功能元素**，把它们打包进子系统。

1. 核对每条 manifest：Arrival Time Data Access 打包了 Position、Speed、ArrivalTime 和 **DesirableSpeed**；而 DesirableSpeed 属于 DesirableSpeedComputer 服务 → 错。
2. 找没有 manifest 的元素：分析相关的界面、控制器、实体，发车优化相关的控制器和实体，外部 Bus Trip Planner 的接口，以及 PassengerEntrance/PassengerExit。
3. 按 §6 分层打包：boundary → Presentation（外部系统接口 → Integration）；control → Control；entity → 所属服务的 Data Access。

得分点（推断）	分
错误决策：识别 + 纠正 + 论证	3
补全打包（元素放进正确子系统）	3

✍ 参考答案｜可照着写，名字和数字换成新案例的

Undesirable decision: DesirableSpeed is packaged into the Arrival Time Data Access subsystem. **Correction:** remove this manifest and package DesirableSpeed into a new Desirable Speed Data Access subsystem. **Justification:** in the domain model DesirableSpeed belongs to the DesirableSpeedComputer service, a different bounded context; packaging it with arrival-time data couples the two services so they cannot be deployed and scaled independently.

> 不理想的决策：DesirableSpeed 被打包进 Arrival Time Data Access 子系统。纠正：删掉这条 manifest，把 DesirableSpeed 打包进新的 Desirable Speed Data Access 子系统。论证：在领域模型中 DesirableSpeed 属于 DesirableSpeedComputer 服务，是另一个限界上下文；和到站时间数据打包在一起会把两个服务耦合，无法独立部署和扩展。

子系统	打包的元素
Analytics Presentation	ConfigureAnalyticsScreen（boundary）
Analytics Control	AnalyticsConfigurationController、AnalyticsController
Analytics Data Access	AnalyticsConfiguration、AnalyticsReport
Bus Trip Planner Integration	BusTripPlannerInterface（外部系统的边界）
Bus Trip Optimisation Control	BusTripOptimiser、TripEventComputer
Bus Trip Optimisation Data Access	BusTripChangeRequest、TripEvent
Passenger Load Data Access（补）	PassengerEntrance、PassengerExit

其他可得分的决策：AnalyticsController 直接依赖另一个服务的实体 BusRoute（跨限界上下文直接访问实体），应改为经 BusNetworkManager 的 API 或事件读取。

16.13 Q3(b) 预测准确度指标 6 marks · T8

题干要点 6 分 · T8 · 原题 p.6

利益相关者希望第一阶段的预测值准确度逐步提高。在第二阶段历史数据分析中，**提出一个衡量并跟踪准确度的指标**：若能直接采集，写负责组件、采集频率、保存时长、存储位置；若要由其他数据计算，写计算方法、负责组件、计算频率、保存时长、存储位置，并对它依赖的最多两个指标做同样说明。

1. 准确度要比较"预测值"和"实际值" → 计算型指标，如 MAE（平均绝对误差）。
2. 五要素逐项写：定义/公式、负责组件、频率、保存多久、存在哪。
3. 依赖的两个指标：预测到站时间、实际到站时间，也各写五要素。

得分点（推断）	分
指标定义与计算方法	约 2
组件、频率、保存时长、位置	约 2
两个依赖指标的同样说明	约 2

✍ 参考答案｜可照着写，名字和数字换成新案例的

Metric – MAE of predicted arrival times: MAE = average of |predicted − actual arrival time| (in seconds) over all predictions for a bus service in a period; a falling MAE shows the prediction is improving. **Calculation:** a scheduled analytics job (Periodically Conduct Analysis of Historical Data, run by the AnalyticsController) matches each prediction with the actual arrival of the same trip at the same stop and computes MAE daily, per bus service and hour of day. **Retention and location:** daily MAE values kept for 3 years in the analytics store (e.g. Amazon Timestream or Redshift) to track long-term improvement and seasonality.

> 指标：预测到站时间的 MAE（平均绝对误差）。MAE = 一段时间内某条公交线路所有预测的 |预测到站时间 − 实际到站时间|（秒）的平均值；MAE 下降说明预测在变准。计算：由定时分析任务（Periodically Conduct Analysis of Historical Data，由 AnalyticsController 执行）把每个预测和同一班次在同一车站的实际到站配对，每天按线路和时段计算 MAE。保存时长和位置：每日 MAE 保存 3 年，放在分析数据库（如 Amazon Timestream 或 Redshift），用来跟踪长期改进和季节性。

Dependent metric 1 – predicted arrival time: captured by the ArrivalTimeComputer every time it computes a new arrival time (trip, stop, predicted time, prediction timestamp); kept 90 days in the data lake (Amazon S3). **Dependent metric 2 – actual arrival time:** captured when the position reported by the onboard unit reaches the stop; kept 90 days in the same data lake so both can be joined by trip and stop.

> 依赖指标 1：预测到站时间。ArrivalTimeComputer 每次算出新的到站时间时记录（班次、车站、预测时间、预测时刻），在数据湖（Amazon S3）保存 90 天。依赖指标 2：实际到站时间。车载设备上报的位置到达车站时记录，在同一个数据湖保存 90 天，两者按班次和车站关联。

❝ 可用句式｜___ 处填案例内容

- Metric: ___ = ___. It is calculated by ___ every ___, kept for ___ in ___, so that ___ can be tracked over time.
  > 指标：___ = ___。由 ___ 每隔 ___ 计算一次，在 ___ 中保存 ___，用来跟踪 ___ 随时间的变化。
- It depends on ___, captured by ___ whenever ___, and kept for ___ in ___.
  > 它依赖 ___，由 ___ 在 ___ 时采集，在 ___ 中保存 ___。

16.14 本卷复盘 What PAST2 Adds

◆ 要点｜必背结论 **PAST2 比 PAST 多练到的东西**

1. T1 不只看"实体在哪个服务"，还要看**聚合（红圈）大小**：太碎 → 慢；太大 → 吃内存。
2. T3 不只改动词，还要会**诊断 API 延迟**：同步、PUT 覆盖、冗余输入 → 异步 + POST 事件 + 202。
3. 质量属性题和云服务题常常连着出：**先定两个属性，再让每个产品对上一个属性**。
4. 指标题要写全五要素，计算型指标还要写依赖指标。
5. 两题之间不要自相矛盾（Q1(b)(i) 与 (ii) 的对策方向）。

CHAPTER 17

PAST3 精讲：ONE Health 医疗健身平台

Sample Paper PAST3 — Walkthrough (Section A only, no official solution)

来源：SWE5001 Sample Exam Paper_PAST3（Q2(a) 部署图已逐图核对）· Appendix A_PAST3 · 按 PAST 评分方式推断的得分点

8 道小题 · 34 分

一句话 **PAST3 没有现成的领域模型：要自己从用例流程里画类图、找共享服务，再设计 Level 3 API、改部署图、映射到 AWS。它最考"从零建模"的能力。**

本卷新出现的问法

- 用 DDD 从用例画 UML 类图（T2）
- 识别共享服务、决定开放给谁（T2、T9）
- RMM Level 3 API 设计（T3）
- 按通信亲和度共置（T12）
- 逻辑详细部署图找三个错（T4）
- 用题目给定的 AWS 服务画物理部署图（T6）

本卷没有官方答案；得分点为推断。附录没有第 7 节领域模型，Q1(b)(i) 必须从第 5 节用例流程自己建模。

17.1 案例速读 Case Facts You Will Cite

业务：智慧城市项目。把大医院的药房外包、整合，并扩展到小医院和诊所，让供应商与病人直接交互；同时鼓励大众健身。四个应用：PharmaMed（订药）、PharmaEquip（订医疗器材）、WeFitRun（跑步）、WeFitYoga（瑜伽）。附录开头说"two pilot applications"、第 1 节却写了四个，原文不一致，答题按四个，必要时注明。

附录要求	内容（原样引用）
扩展性（§2）	要有公共服务和库；近期以"药品/器材配送"和"新运动类型"检验扩展性；任何电脑和移动设备都能访问
性能（§3）	数据增长时保持响应；必要时按医疗机构、供应商、服务类型**分片/分区**；交互操作（如查配送状态）**90% < 1 秒**
多租户（§3）	**必须支持多租户**，租户（医疗机构、供应商、子公司）之间数据和应用隔离
安全（§4）	DMZ 反向代理；个人信息加密、GDPR；租户专属密钥加密并自动轮换；**PCI-DSS：支付和持卡信息只在尽量少的子系统里处理**

§5 用例	答题会用到的事实
Upload Prescriptions	医生开方进入 National Health System，医院管理员（HPA）手工复制到 ONE Health；**NHS 不属于平台，目前没有集成计划**
Order Medicines	病人看处方 → 按处方量下单 → 付款、确认地址 → 返回订单号和预计送达时间
Order Equipment	浏览器材目录下单，**不需要处方**
Track Delivery Orders	列出配送单 → 查看详情
Make Payment	被其他用例调用，经 Payment Gateway 支付
Generate Non-Personal Analytics	注册、注销、下单等计数，按日/月/年重置；不含个人信息

WeFitRun：跟踪跑步（路线、距离、时长、配速）、距离挑战、教练计划、成就徽章、分享到社交媒体、与 Google Fit 同步。
WeFitYoga：跟踪瑜伽、周期挑战、报名指导课程、自由练习、成就、分享、同步。

17.2 Q1(a) 平台成功的一个关键架构方面 2 marks · T9

题干要点 2 分 · T9 · 原题 p.2

两个种子是"处方药"和"器材"；交互包括上传处方、订购药品/器材、跟踪订单。结合种子和交互，**解释一个对平台成功最关键的架构方面。**

1. 点名种子、生产者（医疗机构、供应商）、消费者（病人）、交互。
2. 本案例最特别的要求是多租户和隔离 → 选"多租户 + 共享服务/API"，生态才能扩大。
3. 也可以选可扩展性（引用 90% < 1 秒和分片要求），只要论证。

✍ 参考答案 | 可照着写，名字和数字换成新案例的

The seeds are prescriptions and equipment, produced by healthcare providers and drug/equipment suppliers and consumed by patients through uploading prescriptions, ordering and tracking orders. A crucial aspect is **multitenancy with shared, exposed services**: many hospitals, clinics and suppliers must join as tenants with proper data and application isolation (tenant-specific keys, data partitioned by tenant), while sharing common services – prescription, catalogue, order, payment, delivery tracking – exposed as APIs. New producers and consumers can then join easily, which grows the ecosystem; without isolation, providers would not trust the platform with patient data.

> 种子是处方和器材，由医疗机构和药品/器材供应商产生，病人通过上传处方、下单和跟踪订单来使用。关键的架构方面是多租户加共享、对外开放的服务：许多医院、诊所和供应商要作为租户加入，并做好数据和应用隔离（租户专属密钥、按租户分区数据），同时共享以 API 形式开放的通用服务（处方、目录、订单、支付、配送跟踪）。这样新的生产者和消费者能轻松加入，生态就会扩大；没有隔离，医疗机构不会放心把病人数据交给平台。

17.3 Q1(b)(i) 用 DDD 从用例画 UML 类图 ≈5 marks · T2

题干要点 (b) 合计 9 分 · T2 · 原题 p.2

领域实体是平台可复用资产的重要部分。分析第 5 节的重要用例流程，**用 DDD 方法找出领域对象及其关系，画成 UML 类图。**

1. 逐条读用例，圈名词（候选实体）和动词（候选关系）。
2. 删掉只是属性的名词（日期、数量）和外部系统（NHS、Payment Gateway）。
3. 标关系和多重性；"是一种"用继承，"由……组成"用组合。
4. 按功能内聚分成 <<service>> 框，每个实体写 2–3 个属性。

服务（限界上下文）	实体（主要属性）
UserManager	User（userId, name）← Patient（NRIC, address）、HealthcareProviderAdministrator、SupplierUser
ProviderManager	HealthcareProvider、PreferredSupplier（order of preference）
PrescriptionManager	Prescription（id, date）、PrescribedMedicine（dosage, quantity）
CatalogueManager	Product（id, name, price）← Medicine、Equipment；Supplier ← DrugSupplier、EquipmentSupplier
OrderManager	Order（id, date, status, deliveryAddress）、OrderItem（quantity, price）
PaymentManager	Payment（id, amount, option, status）— PCI-DSS 只在这里
DeliveryManager	DeliveryOrder（no, expectedDateTime, status）

服务（限界上下文）	实体（主要属性）
AnalyticsManager	Counter（type, period, value）

关键关系与多重性：Patient places Order（1 — 0..*）；Order contains OrderItem（组合，1 — 1..*）；OrderItem refers to Product（0..* — 1）；medicine Order is based on Prescription（0..* — 0..1）；Prescription contains PrescribedMedicine（组合）；Prescription is for Patient；Order is paid by Payment（1 — 1）；Order has DeliveryOrder（1 — 1..*）；Supplier supplies Product；Medicine/Equipment is a Product（继承）。

✕ 常见误区｜这样答会丢分

- **把 NHS 画成平台里的实体或集成对象**：附录写明它不属于平台、没有集成计划。
- 让 Equipment 订单也必须关联处方：器材下单不需要处方，所以 Order — Prescription 是 0..1。
- 只画实体不分服务框：DDD 方法要按限界上下文分组。

17.4　Q1(b)(ii) 推荐的共享服务　≈4 marks · T2

题干要点　(b) 合计 9 分 · T2 · 原题 p.2

平台的重要特征是有能跨应用共享、并开放给外部应用使用的服务。**找出本平台推荐的共享服务。**

1. 列出四个应用各自的用例。
2. 两个以上应用都要用的功能 → 共享服务；§2 点名的扩展方向（配送、新运动类型）也要覆盖。
3. 每个服务写"给谁用、做什么"。

✍ 参考答案｜可照着写，名字和数字换成新案例的

(1) **User & Identity** – registration and login for all four apps; (2) **Product Catalogue** – medicines and equipment (PharmaMed, PharmaEquip); (3) **Order**; (4) **Payment** – the only PCI-DSS subsystem; (5) **Delivery Tracking** – needed by both pharmacy apps and named in Section 2 for extensibility; (6) **Notification**; (7) **Non-Personal Analytics** – all apps; (8) **Challenge Management** – distance and periodic challenges (WeFitRun, WeFitYoga); (9) **Workout Tracking** – runs and yoga sessions, and future workout types; (10) **Gamification** – dashboards and badges; (11) **Social Sharing**; (12) **Health Platform Sync** – Google Fit; (13) **Guided Programme / Coaching** – run coaching plans and yoga programmes. They are shared because more than one application needs them, so building them once speeds up future services.

> （1）用户与身份：四个应用的注册和登录；（2）商品目录：药品和器材（PharmaMed、PharmaEquip）；（3）订单；（4）支付：唯一涉及 PCI-DSS 的子系统；（5）配送跟踪：两个药房应用都需要，第 2 节也点名为扩展方向；（6）通知；（7）非个人统计分析：所有应用；（8）挑战管理：距离挑战和周期挑战（WeFitRun、WeFitYoga）；（9）运动记录：跑步、瑜伽以及未来的新运动类型；（10）游戏化：仪表盘和徽章；（11）社交分享；（12）健康平台同步：Google Fit；（13）指导课程/教练：跑步教练计划和瑜伽课程。之所以共享，是因为不止一个应用需要它们，只建一次能加快后续服务的开发。

17.5　Q1(c)(i) 开放哪两个服务的 API，给谁用　≈2 marks · T9

题干要点　(c) 合计 6 分 · T9 · 原题 p.2

从 Q1(b)(ii) 的共享服务里，**选两个作为 API 开放给任意两类客户**（生产者/消费者），点名具体客户并说明理由。

✍ 参考答案｜可照着写，名字和数字换成新案例的

Prescription Service API – for healthcare providers (producers): today the administrator copies each prescription by hand; an API lets the hospitals' own systems upload prescriptions directly, cutting manual work and errors and bringing more seeds onto the platform.

> 处方服务 API，给医疗机构（生产者）：现在由管理员手工复制每张处方；有了 API，医院自己的系统就能直接上传处方，减少人工和差错，也把更多种子带进平台。

Delivery Tracking Service API – for drug and equipment suppliers: their logistics systems receive delivery orders and update the status automatically, so patients see the correct status within 1 second and new suppliers can join easily.

> 配送跟踪服务 API，给药品和器材供应商：他们的物流系统自动接收配送单、更新状态，病人在 1 秒内看到正确的状态，新供应商也容易接入。

注意：理由不要写成"与 NHS 集成"，附录说没有这个计划；写"医疗机构自己的系统"即可。

17.6 Q1(c)(ii) 设计一个 RMM Level 3 API ≈4 marks · T3

题干要点 (c) 合计 6 分 · T3 · 原题 p.2

从上题选一个服务，按 Richardson 成熟度模型 **Level 3** 设计 API：写出服务 URI、HTTP 动作、输入输出定义，并解释每个操作的用途。

1. 资源用名词复数和层级；动作交给 HTTP 动词；状态码写清。
2. Level 3 的标志：**每个响应都带超媒体链接（rel + href）**，告诉客户端下一步能做什么。
3. 至少写 4 个操作：列表、详情、更新、追加事件。

URI	HTTP	Input	Output（含 HATEOAS 链接）	Purpose
/delivery-orders?supplierId={id}&status={s}	GET	query parameters	200 OK; list; links: self, next, each order	List a supplier's delivery orders
/delivery-orders/{no}	GET	–	200 OK; status, expected time, address; links: self, update-status, order	View one delivery order
/delivery-orders/{no}/status	PUT	status, timestamp	200 OK; links: self, delivery-order	Update the delivery status
/delivery-orders/{no}/events	POST	event (e.g. DELAYED), reason, new expected time	201 Created; links: self, delivery-order	Record a delivery event

❝ 可用句式｜___ 处填案例内容

- The API uses plural resource nouns, the correct HTTP verbs and status codes, and every response contains hypermedia links to the next possible actions, so it is at RMM Level 3.
  > 这个 API 用复数名词表示资源，使用正确的 HTTP 动词和状态码，每个响应都包含指向下一步可执行操作的超媒体链接，所以达到 RMM Level 3。
- It is versioned through a custom header (Accept-version) and secured with OAuth 2.0 per supplier tenant.
  > 它通过自定义请求头（Accept-version）做版本管理，并按供应商租户用 OAuth 2.0 保护。

17.7 Q1(d) 通信亲和度高的服务共置 2 marks · T12

题干要点 **2 分 · T12 · 原题 p.2**

Q1(b)(ii) 的可复用服务用容器部署；架构决策是：通信亲和度高（一段时间内交换的消息数量和大小大）的服务应考虑放进同一个容器。**提出两个或以上最适合共享容器的服务**并论证。

✍ 参考答案｜可照着写，名字和数字换成新案例的

I propose to collocate the **Workout Tracking Service** and the **Gamification Service**. Every tracked run or yoga session sends detailed workout data (route, distance, duration, lap times) to Gamification to update dashboards and award badges, so these two exchange the largest number and size of messages. Collocating them turns the calls into intra-process calls, reducing latency and network traffic. I would not collocate Payment with Order, because PCI-DSS requires payment handling to stay in as few subsystems as possible.

> 我建议把运动记录服务和游戏化服务放进同一个容器。每记录一次跑步或瑜伽，都要把详细运动数据（路线、距离、时长、分段用时）发给游戏化服务更新仪表盘、颁发徽章，所以这两个服务交换的消息数量和大小最大。共置后这些调用变成进程内调用，降低延迟和网络流量。我不会把支付和订单放在一起，因为 PCI-DSS 要求支付处理只在尽量少的子系统里进行。

另一个可得分：Order + Delivery Tracking（每个订单都产生配送单，跟踪时都要查订单）。

17.8 Q2(a) 逻辑详细部署图：找三个错 6 marks · T4

题干要点 **6 分 · T4 · 原题 p.3（图在试卷里）**

分析 WeFitRun 和 WeFitYoga 的逻辑详细部署图（未画功能组件），**找出三个不理想的架构决策**，逐个提出纠正措施并论证。可以画类似的图说明。

图上实际情况（核对原图）	部署到的节点
只有一个位置	<<location>> Internal Zone 1，内含 Application、Platform、Data Store、Integration 四个节点；**没有 DMZ，也没有用户设备**
Application 节点	WeFitRun Presentation、WeFitRun Control、WeFitYoga Presentation、Yoga Guided Programs Service
Platform 节点	Yoga Freestyles Service、Workout Tracking Service、Workout Gamification Service、Social Sharing Service、Health Tracking Sync Service
Data Store / Integration	一个共用的 Data Subsystem；Integration Subsystem

1. 分层是否完整：WeFitRun 有 Presentation + Control，WeFitYoga 只有 Presentation → 缺 Control。
2. 应用和平台是否分清：只给瑜伽用的 Yoga Freestyles 放在 Platform；可共享的 Guided Programs 反而放在 Application → 放反了；挑战（Challenge）这个共享服务没有出现。

3. 外部集成和安全区：同步 Google Fit 的 Health Tracking Sync 放在 Platform，而不是经 Integration；所有东西都在一个内部区，没有 DMZ。
4. 其他可选：所有服务共用一个 Data Subsystem（不利于分片和租户隔离）；Presentation 部署在内部服务器而不是用户的手机/浏览器（WeFitRun 要用 GPS）。

得分点（推断）	分
每个决策：识别 + 纠正 + 论证（共三个）	2 × 3

✍ 参考答案｜可照着写，名字和数字换成新案例的

Decision 1: there is no WeFitYoga Control subsystem. **Correction:** add one and deploy it to the Application node. **Justification:** the layering requires Presentation → Control; without it WeFitYoga Presentation must call platform services directly or reuse WeFitRun's control, coupling the two apps.

> 决策 1：没有 WeFitYoga Control 子系统。纠正：加上它并部署到 Application 节点。论证：分层要求 Presentation → Control；没有它，WeFitYoga Presentation 只能直接调用平台服务或借用 WeFitRun 的控制层，把两个应用耦合在一起。

Decision 2: the yoga-only Yoga Freestyles Service is on the Platform node, while the shareable guided-programme capability sits on the Application node. **Correction:** move the freestyle guidance into WeFitYoga's own subsystems on the Application node, and generalise guided programmes (with WeFitRun's coaching plans) into a shared Guided Programme Service on the Platform node; add the missing shared Challenge Management Service there. **Justification:** the Platform node should host only services reusable by several apps, so that new workout types can be added quickly (Section 2).

> 决策 2：只给瑜伽用的 Yoga Freestyles Service 放在 Platform 节点，而可共享的指导课程能力却放在 Application 节点。纠正：把自由练习指导移进 WeFitYoga 自己在 Application 节点上的子系统；把指导课程（连同 WeFitRun 的教练计划）泛化成 Platform 节点上的共享 Guided Programme Service；再在 Platform 上补上缺失的共享 Challenge Management Service。论证：Platform 节点只应承载多个应用都能复用的服务，这样才能快速加入新的运动类型（第 2 节）。

Decision 3: Health Tracking Sync, which talks to the external Google Fit, runs on the Platform node, and everything sits in one internal zone with no DMZ. **Correction:** route external integration through the Integration subsystem on an Integration node in a separate DMZ location, and reach the apps through a reverse proxy in the DMZ. **Justification:** Section 4 requires DMZ reverse-proxy practice; external traffic should not reach the internal zone directly, and an integration tier shields the platform from changes in external APIs.

> 决策 3：与外部 Google Fit 通信的 Health Tracking Sync 运行在 Platform 节点，而且所有东西都在一个内部区里，没有 DMZ。纠正：外部集成经 Integration 子系统，部署在单独 DMZ 位置的 Integration 节点上；应用经 DMZ 中的反向代理访问。论证：第 4 节要求遵循 DMZ 反向代理的做法；外部流量不应直接进入内部区，集成层还能让平台不受外部 API 变化的影响。

17.9 Q2(b) AWS 物理部署图 + 三个架构决策　9 marks · T6

题干要点　　**9 分 · T6 · 原题 p.3–4**

基于 Q2(a) 改正后的逻辑详细部署图，**画出物理详细部署图**，用题目列出的部分 AWS 服务说明云上部署，并**解释至少三个导出这个物理架构的重要决策。**可选服务：API Gateway、ElastiCache、EMR、Lambda、RDS、Cognito、Elastic Beanstalk、Redis、DynamoDB、S3、CloudFront。

1. 把逻辑节点换成云上的东西：用户 → Route 53 + CloudFront + S3；DMZ → API Gateway + WAF（公有子网）；Application/Platform → 私有子网的 EKS 或 Beanstalk + Lambda；Data Store → RDS、DynamoDB、ElastiCache、S3；Integration → 集成子网经 NAT 出去；认证 → Cognito；分析 → EMR。
2. 画 VPC、两个可用区、公有/私有子网。

3 写三个以上决策，每个"选了什么 + 为什么 + 满足哪个需求"。题目给的服务要用上，不必全用。

得分点（推断）	分
物理部署图（节点到 AWS 服务的映射正确）	约 3–4
每个架构决策及理由（至少三个）	约 5–6

✍ 参考答案｜可照着写，名字和数字换成新案例的

Decision 1 – API Gateway and Cognito at the edge: all app requests go through Amazon API Gateway in front of the private subnets, and Amazon Cognito handles sign-up and sign-in with social identity providers. This follows the DMZ / reverse-proxy practice and scales to millions of users.

> 决策 1：边缘用 API Gateway 和 Cognito。所有应用请求先经过私有子网前面的 Amazon API Gateway，Amazon Cognito 负责注册登录，支持社交身份提供方。这符合 DMZ / 反向代理的做法，并能扩展到数百万用户。

Decision 2 – Containers for always-on services, Lambda for event-driven ones: Workout Tracking, Challenge Management and the app control subsystems run continuously, so they run as containers (EKS or Elastic Beanstalk) across two availability zones with auto-scaling. Gamification, Social Sharing and Health Sync are triggered when a workout completes, so they are AWS Lambda functions that scale automatically and cost nothing when idle.

> 决策 2：常驻服务用容器，事件驱动的用 Lambda。运动记录、挑战管理和应用控制子系统持续运行，所以以容器（EKS 或 Elastic Beanstalk）跨两个可用区部署并自动扩缩。游戏化、社交分享和健康同步在一次运动结束时被触发，所以做成 AWS Lambda 函数，自动扩展、空闲时不花钱。

Decision 3 – Polyglot persistence: high-volume workout tracks go to DynamoDB for fast writes at any scale; users, challenges and programmes to Amazon RDS (Multi-AZ) for transactions; leaderboards and dashboards are cached in ElastiCache (Redis); route maps and media in S3. Data is partitioned by user/tenant for growth and isolation.

> 决策 3：多语言持久化。大量的运动轨迹存 DynamoDB，任何规模都能快速写入；用户、挑战和课程存 Amazon RDS（多可用区）以支持事务；排行榜和仪表盘缓存在 ElastiCache（Redis）；路线图和媒体存 S3。数据按用户/租户分区，便于增长和隔离。

Decision 4 – Delivery and analytics: the web front end is served from S3 through CloudFront for low latency; EMR processes historical workout data for dashboards. **Decision 5 – Integration:** Google Fit and social-media sync runs in an integration subnet and calls out through a NAT gateway, isolating the platform from external systems.

> 决策 4：内容分发与分析。Web 前端从 S3 经 CloudFront 分发，降低延迟；EMR 处理历史运动数据生成仪表盘。决策 5：集成。Google Fit 和社交媒体同步运行在集成子网，经 NAT 网关访问外部，把平台和外部系统隔离开。

✕ 常见误区｜这样答会丢分

- 只画图不写决策，或决策只写"用了 Lambda"不写为什么。
- 把数据库放进公有子网或 DMZ。

17.10 本卷复盘 What PAST3 Adds

◆ 要点 | 必背结论 **PAST3 比前两套多练到的东西**

1. 没有领域模型时，**用"名词 → 实体、动词 → 关系、功能内聚 → 服务框"自己建模**。
2. 共享服务要从"多个应用都用"和"§2 点名的扩展方向"两头找。
3. Level 3 和 Level 2 的区别只有一条：响应里的超媒体链接。
4. 部署找错按四个维度扫：分层完整、应用与平台分开、外部集成走集成层、安全区（DMZ）。
5. 多租户、分片、PCI-DSS 是本案例专属要求：论证时引用它们，换案例时别照搬。

APPENDIX

附录

Cloud Services Map · Glossary Index

A 云服务三厂商对照 · B 中英术语索引（按英文字母排序，数字为章号） 对应题型 **T5、T6**

A.1 云服务三厂商对照表 AWS · Azure · Google Cloud

老师说过：题目要求推荐云服务时，要写出具体厂商的服务名，熟悉一家即可。下表按架构组件列出三家的对应服务（名称以 2026 年各厂商官方命名为准，**加粗**为最常用）。

类别	组件	AWS	Azure	Google Cloud
计算	**虚拟机**	**EC2**	Virtual Machines	Compute Engine
	托管 Kubernetes	**EKS**	AKS	GKE
	无服务器容器	ECS + Fargate	Container Apps	Cloud Run
	函数 FaaS	**Lambda**	Azure Functions	Cloud Run functions
	PaaS 应用托管	Elastic Beanstalk	App Service	App Engine
网络	**API 网关**	**API Gateway**	API Management	API Gateway / Apigee
	负载均衡	Elastic Load Balancing（ALB/NLB）	Load Balancer / Application Gateway	Cloud Load Balancing
	CDN	**CloudFront**	Front Door	Cloud CDN
	DNS	Route 53	Azure DNS	Cloud DNS
	WAF / 防 DDoS	AWS WAF / Shield	Azure WAF / DDoS Protection	Cloud Armor
	服务网格	EKS 上自装 Istio / VPC Lattice	AKS Istio 插件	Cloud Service Mesh
存储	**对象存储**	**S3**	Blob Storage	Cloud Storage
	块存储 / 文件存储	EBS / EFS	Managed Disks / Azure Files	Persistent Disk / Filestore
数据库	**关系型**	**RDS、Aurora**	Azure SQL Database、Database for PostgreSQL	Cloud SQL、AlloyDB、Spanner
	键值 / 文档	**DynamoDB**、DocumentDB	Cosmos DB	Firestore、Bigtable
	宽列（Cassandra）	Keyspaces	Managed Instance for Apache Cassandra	Bigtable
	内存缓存	**ElastiCache**（Redis/Valkey）、MemoryDB	Azure Managed Redis	Memorystore
	图 / 搜索 / 时序	Neptune / OpenSearch / Timestream	Cosmos DB Gremlin / AI Search / Data Explorer	Spanner Graph / Vertex AI Search / Bigtable
分析	**数据仓库**	Redshift	Synapse Analytics / Fabric	**BigQuery**
	用 SQL 查文件	Athena	Synapse serverless SQL	BigQuery 外部表

类别	组件	AWS	Azure	Google Cloud
	流处理	Managed Service for Apache Flink	Stream Analytics	Dataflow
	ETL / 大数据集群	Glue / EMR	Data Factory / HDInsight、Databricks	Dataflow / Dataproc
消息	**消息队列**	**SQS**、Amazon MQ	Service Bus、Queue Storage	Pub/Sub、Cloud Tasks
	发布订阅 / 事件总线 / 推送	SNS、EventBridge	Event Grid、Notification Hubs	Pub/Sub、Eventarc、Firebase Cloud Messaging
	事件流	**Kinesis Data Streams、MSK**	Event Hubs	Pub/Sub、Managed Service for Apache Kafka
编排	**工作流 / 定时调度**	Step Functions / EventBridge Scheduler	Logic Apps、Durable Functions / 定时触发器	Workflows / Cloud Scheduler
AI	**机器学习平台**	SageMaker AI	Azure Machine Learning	Vertex AI
	生成式 AI	Bedrock	Azure AI Foundry（Azure OpenAI）	Vertex AI（Gemini）
安全	**用户注册登录**	**Cognito**	Entra External ID	Identity Platform
	访问控制 / 密钥	IAM / Secrets Manager、KMS	Entra ID + RBAC / Key Vault	Cloud IAM / Secret Manager、Cloud KMS
运维	**指标 / 日志 / 追踪**	**CloudWatch**、CloudWatch Logs、X-Ray	Azure Monitor、Log Analytics、Application Insights	Cloud Monitoring、Cloud Logging、Cloud Trace
	IaC	CloudFormation、CDK	ARM / Bicep	Infrastructure Manager（Terraform）
	CI/CD / 镜像仓库	CodePipeline、CodeBuild / ECR	Azure DevOps / ACR	Cloud Build、Cloud Deploy / Artifact Registry
IoT	**设备接入**	IoT Core	IoT Hub	无原生服务（Google IoT Core 已于 2023 年停用）

跨云通用的开源选择：Kubernetes、Istio、Kafka、Redis、PostgreSQL、Prometheus + Grafana、Elasticsearch、Terraform。多云架构里用它们能降低厂商锁定。

B.1 中英术语索引 Glossary Index A–Z

按英文排序，汇总各章术语表；末尾数字是出现的章号，可回到该章术语表和正文查看详细解释。

4

4+1 View Model 4+1 视图 1

A

ACID 原子性、一致性、隔离性、持久性 11
Active / Passive Redundancy 主动 / 被动冗余 13
Aggregate / Aggregate Root 聚合 / 聚合根 4
Amdahl's Law 阿姆达尔定律 12
Anemic Domain Model 贫血模型 4
Anticorruption Layer 防腐层 4
API Gateway API 网关 5
Architecturally Significant Requirement 架构上重要的需求 2
Architecture 架构 1
Architecture Decision 架构决策 2
Asynchronous Non-blocking 异步非阻塞 9
At-Least-Once Delivery 至少一次投递 4
Availability 可用性 13
Availability Zone (AZ) 可用区 13

B

Backend for Frontend (BFF) 面向前端的后端 5
Backward Compatible 向后兼容 5
BASE 基本可用、软状态、最终一致 11
Benchmark 基准测试 12
Big Ball of Mud 大泥球 4
Block Storage 块存储 11
Blue/Green Deployment 蓝绿部署 6
Bottleneck 瓶颈 12
Boundary / Control / Entity 边界/控制/实体 2
Bounded Context (BC) 限界上下文 4
Bulkhead 舱壁 13
Business Ecosystem 商业生态系统 3
Business Invariant 业务不变量 4

C

CaaS 容器即服务 7
Cache Hit / Cache Miss 缓存命中 / 未命中 12
Canary Release 金丝雀发布 6
Capacity Planning 容量规划 12

CAP Theorem CAP 定理 11
Cascading Failure 级联故障 6
CDN 内容分发网络 8
Centralized Logging (ELK/EFK) 集中式日志 6
Chaos Engineering 混沌工程 13
CIA Triad 保密性、完整性、可用性 13
Circuit Breaker 熔断器 6
Client-side Orchestration 客户端编排 8
Client-side / Server-side Discovery 客户端发现 / 服务端发现 9
Cloud Native 云原生 7
Cluster Autoscaler 集群自动扩缩器 10
Cohesion 内聚 1
Cold Start 冷启动 8
Collocation 共置 9
Column Family / Wide-Column Store 列族存储 11
Communication Affinity 通信亲和度 9
Compensating Transaction 补偿事务 6
Container / Container Image 容器 / 容器镜像 10
Container Orchestration 容器编排 10
Container Registry 镜像仓库 10
Context Mapping 上下文映射 4
Control Plane (API Server, etcd, Scheduler) 控制平面 10
Control Plane / Data Plane 控制平面 / 数据平面 9
Conway's Law 康威定律 9
Cooldown Period 冷却时间 12
Correlation ID 关联 ID 6
CORS 跨域资源共享 5
Coupling 耦合 1
CQRS 命令查询职责分离 9
CronJob 定时任务 10

D

DaemonSet 守护进程集 10
Database per Service 每服务一个数据库 9
Data Lake 数据湖 11
Data Warehouse 数据仓库 11
Declarative API 声明式 API 8
Defence in Depth 纵深防御 13
Defer Binding 推迟绑定 13
Deploy 部署 2
Deployment 部署 10
Deployment Unit 部署单元 2
Desired State 期望状态 10
Developer Empathy 开发者同理心 5
Developer Experience (DevEx) 开发者体验 3
Distributed Tracing 分布式追踪 6
DMZ (Demilitarized Zone) 非军事区 1
Document Store 文档存储 11
Dogpile 缓存踩踏 13
Domain-Driven Design (DDD) 领域驱动设计 4
Domain Event 领域事件 4
DORA Metrics DORA 指标 3

E

East-West / North-South Traffic 东西向 / 南北向流量 6
Economy of Scale 规模经济 7
Elasticity 弹性 1
Enterprise Architecture (EA) 企业架构 1
Entity 实体 4
Event-Driven Architecture (EDA) 事件驱动架构 8
Event Sourcing 事件溯源 9
Event Streaming 事件流 9
Eventual Consistency 最终一致性 4
Evolvability 可演进性 1
External Configuration 外部化配置 8
External State Store 外部状态存储 8

F

FaaS 函数即服务 7
Failover 故障转移 13
Fault Tolerance 容错 13
Fitness Function 适应度函数 13
Full / Differential / Incremental Backup 完全 / 差异 / 增量备份 13
Functional Cohesion 功能内聚 4

G

Golden Path 黄金路径 3
Granularity 粒度 5
Graph Database 图数据库 11
gRPC / RPC 远程过程调用 9

H

Harvest 收获率 6
HATEOAS 超媒体作为应用状态引擎 5
Heartbeat 心跳 13
Hexagonal Architecture (Ports & Adapters) 六边形架构 8
HorizontalPodAutoscaler (HPA) 水平 Pod 自动扩缩 10
Hot Shard (Hotspot) 热点分片 11
Hybrid Architecture 混合架构 8
Hybrid Cloud / Multi-cloud 混合云 / 多云 7
Hypervisor 虚拟机监控器 10

I

IaaS 基础设施即服务 7
Idempotency Key 幂等键 5
Idempotent 幂等 5
Idempotent Receiver 幂等接收者 4
Identity and Access Management (IAM) 身份与访问管理 13
Immutable Infrastructure 不可变基础设施 8
Infrastructure as Code (IaC) 基础设施即代码 7
Ingress 入口 10
Interaction 交互 3
Internal Developer Platform (IDP) 内部开发者平台 3

J

JWT (JSON Web Token) JSON 网络令牌 5

K

Key-Value Store 键值存储 11
kubelet 节点代理 10

L

Label Selector 标签选择器 10
Lambda / Kappa Architecture Lambda / Kappa 架构 8
Landing Zone 着陆区 7
Latency 延迟 12
Layer / Tier 层 / 层级 2
Least Privilege 最小权限 13
Lift and Shift (Rehost) 直接搬迁 7
Little's Law 利特尔定律 12
Liveness / Readiness Probe 存活 / 就绪探针 10
Load Balancing 负载均衡 12
Load Shedding / Backpressure 减载 / 背压 13
Load Testing 负载测试 12
Location 位置 2
Logical Architecture 逻辑架构 2

M

Magnet 磁铁 3
Maintainability 可维护性 13
Managed Service 托管服务 10
Manifest 体现 2
Matchmaker 撮合者 3
Measured Service 可计量服务 7
Message Broker 消息代理 9
Message Queue 消息队列 9
Microservices 微服务 9
Modifiability 可修改性 13
MTBF / MTTR 平均故障间隔 / 平均恢复时间 6
Multiplicity 多重性 4

N

Namespaces / cgroups 命名空间 / 控制组 10
Network Business Model 网络业务模式 3
NewSQL 新型关系库 11
Node 节点 2
Noisy Neighbour 吵闹邻居 10
Non-Functional Requirement (NFR) 非功能需求 1
Non-repudiation 不可否认 13
Normalization 规范化 11

O

OAuth 2.0 授权框架 5
Object Storage 对象存储 11
On-demand Self-service 按需自助 7
One-way Notification 单向通知 9
OpenAPI Specification (OAS) OpenAPI 规范 5
Open Host Service 开放主机服务 4

Orchestration / Choreography 编排 / 协同 6
Outside-in Design 由外而内 5

P

PaaS 平台即服务 7
Partial Response / PATCH 部分响应 / 部分更新 5
Percentile (P90/P99) 百分位 6
PersistentVolume / PersistentVolumeClaim 持久卷 / 持久卷申领 10
Pets vs Cattle 宠物与牛群 8
Physical Architecture 物理架构 2
Platform 平台 3
Platform Engineering 平台工程 3
Pod Pod 10
Polyglot Persistence 多语言持久化 9
Precomputation 预计算 12
Procurement Lead Time 采购提前期 12
Producer / Consumer 生产者 / 消费者 3
Progressive Web App (PWA) 渐进式 Web 应用 8
Proof-of-Concept (PoC) 概念验证 2
Protocol Buffers 协议缓冲区 9
Publish/Subscribe 发布/订阅 1

Q

Quality Attribute 质量属性 1
Quality Attribute Scenario 质量属性场景 1

R

Rapid Elasticity 快速弹性 7
Rate Limiting 限流 6
Read Replica 读副本 11
Read-through / Write-through 读穿透 / 写穿透 11
Rearchitect / Refactor 重新架构 7
Rebalancing / Resharding 重新分片 11
Reference Architecture 参考架构 1,8
Reliability 可靠性 13
Replace / Repurchase 替换 7
Replatform 更换平台 7
Replication Lag 复制延迟 11
Repository 仓储 4
Resource Pooling 资源池化 7
Response Time 响应时间 12
Retrieval-Augmented Generation (RAG) 检索增强生成 8
Retry with Exponential Backoff 指数退避重试 5
Reverse Proxy 反向代理 1
Richardson Maturity Model (RMM) 理查森成熟度模型 5
Rolling Update 滚动更新 6

S

SaaS 软件即服务 7
Safe Method 安全方法 5
Saga Pattern Saga 模式 6
Scalability 可扩展性 1
Scale Cube (X/Y/Z axis) 扩展立方体 9
Scheduled Task / CronJob 定时任务 8
Schema Registry 模式注册表 13
SDK 软件开发工具包 5
Seed 种子 3
Semantic Versioning 语义化版本 5
Serverless 无服务器 8
Service (ClusterIP / LoadBalancer) 服务（K8s） 10
Service Discovery 服务发现 6
Service Mesh 服务网格 9
Service Registry 服务注册中心 6
Service Time / Waiting (Queue) Time 服务时间 / 等待时间 12
Shadowing 影子流量 6
Sharding / Shard Key 分片 / 分片键 11
Shared Responsibility Model 责任共担模型 13
Shared Service 共享服务 4
Sidecar Proxy 边车代理 9
Single Point of Failure (SPOF) 单点故障 13
Sizing 规模估算 2
Smart Endpoints and Dumb Pipes 智能端点哑管道 9
Solution Architecture 解决方案架构 1
Stakeholder 干系人 1
StatefulSet 有状态集 10
Stateless 无状态 5
Stateless Application 无状态应用 8
Sticky Session 粘性会话 8
Stress / Spike / Soak Test 压力 / 突发 / 耐久测试 12
STRIDE 威胁分类 13
Structured / Semi-structured / Unstructured Data 结构化 / 半结构化 / 非结构化数据 11
Subdomain (Core / Supporting / Generic) 子领域 4
Subsystem 子系统 2
Synchronous Blocking 同步阻塞 9
System Context 系统上下文 2

T

Think Time 思考时间 12
Throughput 吞吐量 12
Time-Series Database (TSDB) 时序数据库 6
Time To First Byte (TTFB) 首字节时间 12
Toolbox 工具箱 3
Total Cost of Ownership (TCO) 总拥有成本 7
Trace / Span 跟踪 / 跨度 6
Trade-off 权衡 1
Trend Analysis 趋势分析 12
Twelve-Factor App 十二要素应用 8
Two-Phase Commit (2PC) 两阶段提交 6

U

Ubiquitous Language (UL) 通用语言 4
Unified Process (UP) 统一过程 2
Uniform Interface 统一接口 5
Universal Scalability Law (USL) 通用扩展定律 12
Utility Tree 效用树 1
Utilization 利用率 12

V

Value Object 值对象 4
Vendor Lock-in 厂商锁定 7
Virtual Private Cloud (VPC) 虚拟私有云 7

W

WAF Web 应用防火墙 8
Workload Mix 工作负载组合 12
Workload Model 工作负载模型 12
Work Product 工作产品 2

Y

Yield 产出率 6